



ACITM Make

Release 5 User Guide

© 2001 by ACI Worldwide Inc. All rights reserved.

All information contained in this document is confidential and proprietary to ACI Worldwide Inc. This material is a trade secret and its confidentiality is strictly maintained. Use of any copyright notice does not imply unrestricted or public access to these materials. No part of this document may be photocopied, electronically transferred, modified, or reproduced in any manner without the prior written consent of ACI Worldwide Inc.

ACI, BASE24, BASE24 Remote Banking, BASE24-atm, BASE24-atm self service banking, BASE24-billpay, BASE24-card, BASE24-check auth, BASE24-check disbursement, BASE24-frequent shopper, BASE24-from host maintenance, BASE24-mail, BASE24-pos, BASE24-refund auth, BASE24-telebanking, BASE24-teller, and NET24-XPNET are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Print Record

06/2001, Version 1 (New)

Contents

Preface	ix
Conventions Used in This Manual	xiii
Section 1	
Introduction	1-1
Overview	1-2
Make Utility Releases	1-4
Section 2	
Make Hierarchy	2-1
Macro Files and Makefiles	2-2
Macro Files	2-3
Naming Macro Files	2-4
Naming File Defines	2-5
Naming Make Flags	2-6
Global Macro Files	2-7
ALLMACS Define	2-7
CUSTMACS File	2-7
Flag Documentation	2-15
ALLMACS File	2-18
BASE24-retail Products	2-21
Personal Overrides	2-22
Channel-Specific Enhancements	2-24
CSMMACS File	2-24
BASE24 Procedural Product Macro Files	2-25
Product Group Macro Files	2-25
Product Macro Files	2-27
Process Macro Files	2-32

Make Hierarchy *continued*

BASE24 ITS Product Macro Files	2-35
Object Group Macro Files	2-35
Subsystem Macro Files	2-44
Object Macro Files	2-47
Makefiles	2-50
ALLMACS Macro	2-53
Master Makefiles	2-54
B24M Makefile	2-54
CSMM Makefile	2-55
Product Group or Object Group Makefiles	2-56
CLSCM Makefile	2-56
ITSM Makefile	2-56
CSMM Makefile	2-56
Product or Subsystem Makefiles	2-57
Product Makefiles	2-57
Subsystem Makefiles	2-57
Process or Object Makefiles	2-58
Process Makefiles	2-58
Object Makefiles	2-58
Hierarchy Charts	2-59

Section 3

Make Syntax	3-1
Makefile Components	3-2
Target Files and Dependency Files	3-5
Target Specification Syntax	3-5
Timestamps	3-6
Dependency File as a Target File	3-7
Source and Object Files	3-8
Dependency Lists	3-10
Include Statements	3-11
Conditional Text	3-12
Comments	3-13

Make Syntax *continued*

Directives	3-15
Warning	3-15
Abend	3-16
Make Macros	3-17
Macro Definition.....	3-17
Macro Use.....	3-18
Predefined Macros	3-18
TACL Defines	3-20
Additional Defines	3-21
Additional Targets.....	3-23
Inline Code	3-25
TACL Macros	3-29
ACI_SCOBOLX_MAKE Macro	3-30
CHECK_COMPLETION Macro	3-32
CHECK_SCOBOL_COMPLETION Macro	3-33
CHK_PERUSE_JOB Macro.....	3-33
DDL TAL_UNDERBAR_CONV Macro	3-34
DEL Macro	3-35
DEL_PERUSE_JOB Macro	3-35
DISPLAY_COMPLETION_INFO Macro.....	3-36
DUP_DICT Macro	3-36
DUP_EXEC_FILE Macro	3-37
EXPANDDEF Macro	3-38
EXPANDSSV Macro	3-39
EXPANDSSV_DICT_NOSAVE Macro	3-39
GET_VOL_SUBVOL Macro.....	3-40
PURGE_DICT Macro.....	3-40
SCUP_ROUTINE Macro	3-40
SECURE Macro	3-41
TACLMACS_VERSION Macro	3-41
TEMPLMAC Macro.....	3-42
TOTAL_SIZE Macro	3-43
UPDATE_DICT_ALT_FILES Macro	3-43

Section 4

Using the ACI Make Utility	4-1
Running the ACI Make Utility	4-2
Setting the ALLMACS Define	4-2
Setting the Radix Table Size	4-3
Starting the ACI Make Utility	4-6
Using Obey Files to Start the ACI Make Utility	4-7
Command Line Options	4-15
Printing Command Line Options	4-19
Default Printing	4-19
Available Printing Options	4-23
Other Command Line Options	4-33
Maintaining File Name References	4-52
EQUIV Report Examples	4-53
EQUIV File Example	4-55
Installing BASE24 with the ACI Make Utility	4-57
Performing Specific Tasks with the ACI Make Utility	4-59
Obtaining Dependency Lists for a Specific Executable Module	4-59
Compiling a Subset of Your System	4-63
Compiling DDLs	4-65
Compiling Pathway Requester Processes	4-66
Using Multiple Makefiles	4-71

Appendix A

ACI Make Utility Summary	A-1
Viewing Files	A-2
CUSTMACS File	A-3
ALLMACS File	A-4
BASE24 Procedural Product Make Files	A-5
Using the AUTHM Makefile	A-15
Files Used	A-15
Personal Override File	A-16
Identifying Target Files	A-18
Testing the Personal Override File	A-21
Generating Target Files	A-23

Appendix B

Implementation of Make in BASE24	B-1
Common Source File Modifications	B-2
External File References	B-2
Compile and Bind TACL Command Files	B-2
Product Level DDL Modifications	B-3
Base Global Definitions	B-3
DDL Command File	B-3
DDL Dictionary Command	B-3
Language Output Files	B-3
TAL Output File ITS Support	B-4
Example DDL Makefile	B-4
Pathway Requester Modifications	B-7
Pathway Screen Source	B-7
POBJ Management	B-7
Example Requester Makefile	B-7
COBOL Modifications	B-9
Example COBOL Makefile	B-9
TAL Modifications	B-10
General Modifications	B-10
BASE24-atm Device Handler Processes	B-10
ATM DCT Server	B-15

Appendix C

Make for Personalized BASE24 Software	C-1
Code Location and Naming Conventions	C-2
Makefile Modifications for Customized Code	C-5
File Name Macros	C-5
CSMM Makefile	C-5
CSMMACS File	C-6
ALLMACS Define	C-7
Makefile Hierarchy	C-8

Appendix D

Upgrading BASE24 Sync Point 2 Products	D-1
Tandem Operating System Requirements.....	D-2
Modifying the ALLMACS File	D-3
TACLINIT File.....	D-3
TACLMACS File	D-4
Modifying the B24C File	D-5
Modifying the BASE24-mail Makefile.....	D-6
Modifying the BASE24-telebanking Customer Service Representative Interface Makefile	D-7
 Glossary	 Glossary-1
 Index.....	 Index-1

Preface

This manual provides an explanation of the ACI Make utility and an overview of its functionality. It also provides information for using the utility, including the command syntax and input files involved. It includes guidelines for modifying the input files for customized development (i.e., CSEs and CSMs) and contains a demonstration of the utility.

While the ACI Make utility is independent of any specific ACI software product, this manual presents the manner in which it is used with BASE24 procedural products (e.g., BASE24-atm and BASE24-pos) and BASE24 ITS products (e.g., BASE24-telebanking).

Audience

This manual is written for application programmers and system administrators who must be familiar with the ACI Make utility to perform their jobs.

Additional Documentation

The ACI documentation set is arranged so that each manual presents a topic or group of related topics in detail. When one manual presents a topic that has already been covered in detail in another manual, the topic is summarized and the reader is directed to the other manual for additional information. Information has been arranged in this manner to be more efficient for readers who do not need the additional detail and at the same time provide the source for readers who require the additional information. This manual contains references to the following ACI publications:

- The ***Installation of BASE24 Classic 6.0 Systems*** contains detailed installation steps for using the ACI Make utility with BASE24 release 6.0 procedural products.
- The ***BASE24 Classic 5.3 Installation Guide*** contains detailed installation steps for using the ACI Make utility with BASE24 release 5.3 procedural products.

- The ***BASE24-telebanking/billpay Installation Guide release 1.3*** contains detailed installation steps for using the ACI Make utility with the BASE24 release 1.3 and release 6.0 ITS products.
- The informal ***ACI Make CSM Standards Manual*** contains additional information about using the ACI Make utility with BASE24 custom software modifications (CSMs).

Software

This manual documents standard processing for ACI Make utility release 5.01, which is compatible with the following software:

- BASE24 release 1.1.2
- BASE24 release 1.3
- BASE24 release 5.1.2
- BASE24 release 5.3
- BASE24 release 6.0

Although the ACI Make utility is distributed in object form that cannot be modified, custom software modifications (CSMs) may result in software that differs from the examples presented in this manual. The customer is responsible for identifying and noting these changes.

Manual Summary

The following is a summary of the contents of this manual.

“Conventions Used in This Manual” follows this preface and describes notation and documentation conventions necessary to understand the information in the manual.

Section 1, “Introduction,” contains an overview of the Make hierarchy and the function of the ACI Make utility. It also explains the product releases.

Section 2, “Make Hierarchy,” describes each level of the Make hierarchy, including examples of macro files, and contains charts that illustrate the Make hierarchy.

Section 3, “Make Syntax,” describes the syntax for building a makefile.

Section 4, “Using the ACI Make Utility,” contains procedures for using the ACI Make utility, including the TACL command syntax for starting the utility, the available reporting options, and procedures for performing specific tasks with the utility.

Appendix A, “ACI Make Utility Summary,” summarizes the Make macro files and makefiles used by the ACI Make utility with BASE24 products, and provides a demonstration of the ACI Make utility being used to compile the BASE24-atm Authorization process.

Appendix B, “Implementation of Make in BASE24,” describes the changes made to BASE24 software when the ACI Make utility was used for the first time.

Appendix C, “Make for Customized BASE24 Software,” describes how the ACI Make utility is used with BASE24 software that has custom software modifications (CSMs).

Appendix D, “Upgrading BASE24 Sync Point 2 Products,” contains the procedures needed to upgrade BASE24 sync point 2 products from release 1.02 to release 5.01 of the ACI Make utility.

A glossary that contains definitions of terms used in this manual follows the last appendix.

Publication Identification

Two entries appearing at the bottom of each page uniquely identify this ACI Make publication. The publication number (for example, TO-MK000-06 for the ***ACI Make Release 5 User Guide***) appears on every page to assist readers in applying updates to the appropriate manual. The publication date (for example, 06/2001 for June, 2001) identifies the last time material on a particular page has been revised. This date can be used to verify that the reader has the most current information and is particularly helpful when dealing with HELP24 to answer operating questions.

The print record, located on the page immediately following the title page, is also helpful when verifying that a manual contains the most current information. An entry is added to the print record each time this publication is modified in any way. Each print record entry includes the date of the change, a unique version number, and the type of change (new, update, reissue, or preliminary). Version 1 is assigned to a *new* manual. Later versions can be updates or reissues, depending on the changes being made. An *update* typically involves changes on a small number

of pages throughout the manual. A *reissue* occurs when all pages of the manual are replaced because changes affect a large number of pages. A manual is identified as *preliminary* when it is released to customers on a limited basis, but it has not yet reached final form.

Conventions Used in This Manual

This section explains how syntax notation is documented in this manual.

Syntax Notation

Syntax descriptions in this manual use several symbols and conventions to denote how commands must be entered. These conventions are described in the table below.

Notation	Meaning
UPPERCASE LETTERS	Indicate key words and literals that you must enter exactly as shown.
<i>lowercase italics</i>	Identify variables that you must provide.
Brackets []	Enclose optional syntax items that you can enter in the command. In a target file specification, brackets identify additional defines.
Braces { }	Enclose a group of items from which you are required to choose one item. The items are separated within the braces by a vertical bar (). In a target file specification, braces identify additional targets and the vertical bar is not used.
Vertical bar	Separates alternatives in a list of items.
Ellipsis ...	Indicates the immediately preceding syntactical item can occur one or more times.
Punctuation	Must be entered as shown. This includes parentheses, commas, semicolons, and other symbols not previously described.

Notation	Meaning
Spaces	Are required as delimiters wherever it would be otherwise impossible to discriminate between adjacent words or symbols. You can insert additional spaces between any adjacent words or symbols, but not within a word or multiple character symbol.
Indented lines	Indicates a continuation of the preceding line of syntax. If the syntax of a command is too long to fit on a single line, each continuation line is indented.

Section 1

Introduction

The ACI Make utility is a tool that can be used to assist with the installation and maintenance of ACI software products. This section introduces you to the ACI Make utility, including its functionality and the ACI software products with which it is used.

Overview

The ACI Make utility configures and compiles ACI software. ACI implemented this utility to enable our customers, as well as our own developers, to maintain our software more effectively. ACI software includes hundreds of source files that must be compiled in a specific order to create the desired object files. The ACI Make utility automates this process, ensuring that object files are created in the correct order with all the necessary information. This utility also saves time and resources by automating the work of compiling, binding, moving, and securing object code.

The ACI Make utility effectively directs program compilation by automatically regenerating files that are older than the files from which they were created. Without this utility, operators must manually initiate compile commands for each module in the proper sequence, ensuring that each compilation finishes properly before they continue with the next module.

The ACI Make utility uses a hierarchy, known as the Make hierarchy, to establish the relationships among all of the source and object files in a configuration. By building this hierarchy from information in multiple source files, the ACI Make utility can ensure that the following occurs.

- Proper file names are used throughout the configuration
- Object files are compiled in the correct order
- Only the necessary files are recompiled when a change is made

The ACI Make utility first establishes the Make hierarchy, then uses it to determine which object files must be regenerated and in what order. The ACI Make utility automatically compiles each module, if necessary, and checks the compilation completion status. If a creation step fails, the ACI Make utility can stop processing or continue processing just the modules that do not depend on information from the unsuccessful step.

The ACI Make utility enforces module dependencies and automatically compiles modules if any of these dependencies have changed. For example, when a DDL changes, the ACI Make utility ensures all the modules that use that DDL are recompiled.

The ACI Make utility simplifies BASE24 system installation by using conditional language support (i.e., IF, THEN, and ELSE statements). Within the ACI Make framework, flags define the products (e.g., BASE24-atm, BASE24-pos, BASE24-telebanking) and modules within the product (e.g., NCR 50xx and Diebold 910) that the BASE24 user installs on site.

The ACI Make utility normally runs at a high process identification number (PIN), which is a PIN greater than 255. The ACI Make utility also starts its TACL Server process at a high PIN. A command line option is available to have the ACI Make utility start its TACL Server process at a low PIN (0–254).

The ACI Make utility is available in two versions, as follows.

- The MAKEV501 process is a native object (file type 700) for Tandem NonStop Reduced Instruction Set Computing (RISC) processors, also called TNS/R processors. It requires a NonStop Kernel operating system of D45 or greater.
- The MAKET501 process is a regular object (file type 100) for Tandem NonStop processors, also called TNS processors, that do not support native object files, as follows:
 - Processors without Reduced Instruction Set Computing must have a NonStop Kernel operating system of at least D38.
 - Processors with Reduced Instruction Set Computing and a NonStop Kernel operating system of at least D38 but less than D45.

The MAKET501 process can be used on TNS/R processors with a NonStop Kernel operating system of D45 or greater. However, the MAKEV501 process operates considerably faster than the MAKET501 process on these operating systems.

Prior to release 5.00, functions now performed by the ACI Make utility required two utilities—ACI Make and Makescan. The Makescan utility was required to perform the file location maintenance associated with installing software in a new location. Effective with release 5.00, the ACI Make utility is enhanced to perform the file location maintenance, and the Makescan utility is no longer available.

Make Utility Releases

While the ACI Make utility is not product specific, the macro files and makefiles that it uses must be customized according to the product with which they are being used.

The ACI Make utility was first used with release 1.0 of the BASE24 ITS products (e.g., BASE24-telebanking) and with release 5.1 of BASE24 procedural software products (e.g., BASE24-atm, BASE24-pos). Appendix B explains the transition of the BASE24 procedural software products to the ACI Make utility.

Two releases of the ACI Make utility are currently being distributed, as shown in the following table:

ACI Make Utility	BASE24 Procedural	BASE24-telebanking
Release 4.0	Release 5.1.2* Release 5.3*	Release 1.1.2* Release 1.3*
Release 5.01	Release 5.1.2* Release 5.3* Release 6.0	Release 1.1.2* Release 1.3* Release 6.0

* BASE24 release 5.1.2, BASE24 release 5.3, and BASE24 release 1.3 customers continue to receive release 4.0 of the ACI Make utility, with an option to upgrade to release 5.01 of the ACI Make utility.

This manual is based on release 5.01 of the ACI Make utility. Appendix D contains the procedures needed to upgrade BASE24 sync point 2 products still being used with release 1.02 of the ACI Make utility so they can be used with either of the currently distributed releases. No other upgrades are necessary to use the newest release of the ACI Make utility with previous releases of BASE24.

Section 2

Make Hierarchy

The Make hierarchy is the key to the success of the ACI Make utility. The ACI Make utility uses this hierarchy to establish the relationships among all of the source and object files in a configuration. This section presents all of the components of the Make hierarchy.

Macro Files and Makefiles

The ACI Make utility uses two groups of files—macro files and makefiles. Macro files contain configuration information and file definitions that must be established for use in the makefiles. Makefiles contain the commands, macros, dependency information, and inline code needed to create a target file.

See section 3 for information regarding the syntax used in macro files and makefiles.

While the ACI Make utility is not product specific, the macro files and makefiles that this utility uses must be customized according to the product with which it is being used. The macro files and makefiles discussed in this section are located on subvolumes associated with BASE24 release 6.0, while the ACI Make utility is located on a subvolume named MAKEV500.

The remainder of this section contains detailed descriptions, with examples, of macro files and makefiles.

Macro Files

The ACI Make utility uses macro files to build the Make hierarchy. Macro files contain configuration information and file definitions.

Configuration macros control such things as what products to build, what modules within the product to build, what options to pass to the compilers (e.g., TAL, DDL, SCOBOL), and what general process run options to use.

File definition macros map the logical file names used in source code and makefiles to the actual physical location of the files.

The ordering of the configuration information and file definitions contained in macro files is a key to the way the ACI Make utility operates. The utility uses the first definition received for a macro, ignoring any later definitions for the same macro. This concept is particularly important when you want to temporarily change (override) macro definitions for personal work, a project, or testing purposes.

There are separate macro file hierarchies for BASE24 procedural products and BASE24 ITS products. Separate hierarchies exist because of differences in the way BASE24 procedural products and BASE24 ITS products are designed; and configuration information is defined earlier in the hierarchy for object-oriented products such as BASE24 ITS. The following table illustrates the levels of each hierarchy.

Level	BASE24 Procedural	BASE24 ITS
1	Global	Global
2	Product group	Object group
3	Product	Subsystem
4	Process	Object

Naming Macro Files

Macro files are named for easy identification. At the global level, files with names ending with *MACS*, such as the CUSTMACS File, ALLMACS File, and CSMMACS File, contain configuration and file definition macros. Below the global level, files with names ending with *FM* contain configuration macros, and files with names ending with *MM* contain file definition macros.

Macro file names ending with the *FM* or *MM* suffix usually match the name of the subvolume on which they are located. However, macro files at the product group and object group level use names that identify the group, plus the *FM* or *MM* suffix. Some examples of macro file names are shown in the following table.

Product	Subvolume	Configuration Macro File	File Definition Macro File
BASE24-atm	AT60SRC	ATMFM	ATMMM
NCR 50XX Device Handler	AT60N50	ATN50FM	ATN50MM
Base AFT	BA60AFT	BAAFTFM	BAAFTMM
ISO Host Interface	BA60HISO	BAHISOFM	BAHISOMM
BASE24 ITS core subsystem	OC60SRC	CORFM	CORMM
Core AFM	OC60AFM	CAFMFM	CAFMMM
Core Transaction History object	OC60HIST	HISTFM	HISTMM
Kernel EMT Build utility	OK60EMTB	KEMTBFM	KEMTBMM

Naming File Defines

A naming convention exists for the file defines established in the macro files and used in the makefiles. The name is made up of the subvolume and file name without the version number, as illustrated below for some of the BASE24 procedural and BASE24 ITS products. The third column of the following table contains the subvolume and name of the macro file in which the define is assigned.

File Name	File Define	Macro File
ATM Authorization Source File	atauth_auths	AT60AUTH. AUTHMM
ATM Authorization Externals File	atauth_auth	
ISO Host Interface Executable File	bahiso_hiso	BA60HISO. BAHISOMM
Base Host Interface Source File	bahiso_bahisos	
Core Transaction History TAL Source File	ochist_histts	OC60HIST. HISTMM
Core Transaction History TAL Externals File	ochist_hist	
Core EMT Build TAL Object File	ocembt_cembt	OC60EMTB. CEMTBMM
Core EMT Build TAL Source File	ocembt_cembts	

The following example shows how the define entries listed above might appear in a macro file. Entries in a macro file usually are arranged alphabetically.

```

atauth_auths      = \r.$earl.at60auth.auths
atauth_auth       = \r.$earl.at60auth.auth
bahiso_bahisos    = \r.$earl.ba60hiso.bahisos
bahiso_hiso       = \r.$earl.ba60hiso.hiso
ocembt_cembt      = \r.$earl.oc60obj.cembt
ocembt_cembts     = \r.$earl.oc60emt.cembts
ochist_hist       = \r.$earl.oc60hist.hist
ochist_histts     = \r.$earl.oc60hist.histts

```

Naming Make Flags

The following naming convention is used for the flags contained in the macro files. Some older flags may not exactly follow this naming convention.

Syntax

[product-ID_] [process-ID_] [enhancement-ID_] on

product-ID — The product that uses the flag. This ID is used when a distinction must be made between products that use the flag, and is omitted when multiple products share the flag.

process-ID — The process that uses the flag. This ID is used when a distinction must be made between processes that use the flag, and is omitted when multiple processes share the flag.

enhancement-ID — The enhancement that uses the flag. This ID is used when a distinction must be made between enhancements that use the flag, and is omitted when multiple enhancements share the flag.

Examples

atm_dh_c1000_emv_on

This flag is for the BASE24-atm product, the Diebold 10XX/478X Device Handler process (dh_c1000), and the Europay, MasterCard, and Visa (EMV) enhancement. The product, process, and enhancement are included because the presence of the EMV enhancement must be identified for each BASE24-atm Device Handler process.

pos_emv_on

This flag is for the BASE24-pos product and the EMV enhancement. The product and enhancement are included, but the process is not because multiple processes can use the flag.

refresh_on

This flag is for the Refresh process (refresh). Multiple products can use the enhancement-free version of the process.

atm_on

This flag is for the BASE24-atm product. BASE24-atm processes and enhancements have their own flags.

Global Macro Files

Global macro files provide the ACI Make utility with customer-specific configuration parameters plus system-level information needed by the ACI Make utility. The ACI Make utility uses the information in these files as the starting point each time it runs. BASE24 procedural products and BASE24 ITS products share the same global macro files.

ALLMACS Define

The TACL define named ALLMACS (=allmacs) identifies which global macro file the ACI Make utility uses first when it builds the Make tree or hierarchy. For example, in a configuration that has a CUSTMACS File and an ALLMACS File, the CUSTMACS File is specified in the ALLMACS define because the Make utility must read the CUSTMACS File first.

See the “Running the ACI Make Utility” topic in section 4 for the syntax of the ALLMACS define and procedures for using the ALLMACS define.

CUSTMACS File

The CUSTMACS File contains customer-specific configuration information for standard BASE24 software, and usually serves as the top level global macro file.

The CUSTMACS File provides customers a single location to define which BASE24 products to include in the system. The true/false flags in the CUSTMACS File are copied from other files throughout the Make hierarchy. By having all true/false flags contained in the CUSTMACS File, customers can set all flags, as well as some of the configuration options, in a single location instead of modifying multiple files located throughout the Make hierarchy.

True/false flags in the CUSTMACS File are arranged by BASE24 product (with and without enhancements), channel-specific product (with and without enhancements), and configuration options. The flags are arranged alphabetically within each product group. In some cases there is an assumed introductory qualifier to each flag. For example, atm_ is assumed to be present at the beginning of the flags for all ATM Enhancements. Flags in the CUSTMACS File are set to false when ACI delivers it.

When a true/false flag or configuration option is set multiple times, the Make utility uses only the first setting it receives for that flag or configuration option. When a true/false flag is referenced before it has been set, its default value is false.

The CUSTMACS File contains logic that checks for the presence of personal override files and channel-specific enhancement (CSE) macro files. This logic is located at the beginning of the CUSTMACS File, meaning that any personal overrides or CSE macros override the flags and configuration options contained in the CUSTMACS File. See the “Personal Overrides” and “Channel-Specific Enhancements” topics in this section for more detailed information about using personal overrides and CSE macros to override the CUSTMACS File settings.

The last statement in the CUSTMACS File incorporates the ALLMACS File, which is the next global macro file, by reference. The process of incorporating another file by reference is often called *including* the file because the INCLUDE command is used to perform this process.

The CSMMACS File, described later in this section, also contains customer-specific configuration information. However, the CSMMACS File contains information only for BASE24 software modules that have been created or customized to meet the specific requirements of a customer. These software modules are called custom software modifications (CSMs).

Example CUSTMACS File

This example includes several portions of the CUSTMACS File provided with BASE24. The following table describes the portions of the CUSTMACS File in the example, and gives the file sequence numbers that identify each portion of the file. The file sequence numbers appear at the right of each line in the file. For improved readability, lines separate the portions.

Portion	Sequence Numbers
Global configuration macros <i>true</i> and <i>false</i> . Note: These macros are ignored because they are defined within the ACI Make utility and cannot be changed. These macros are included in this CUSTMACS File because they were required with previous releases of the ACI Make utility.	83–87
Logic that checks for the presence of personal override files.	89–105

Portion	Sequence Numbers
Logic that checks for the presence of channel-specific enhancement (CSE) macro files.	107–123
Flags for BASE24 release 6.0. This example contains several groups of the flags contained in the CUSTMACS File. The ACI Make utility uses all of the flags in the CUSTMACS File to determine which BASE24 procedural (classic) products and BASE24 ITS objects are installed.	125–143 145–151 154–185 187–198 199–212 309–339
Flags for channel-specific products and enhancements can be added to this portion of the CUSTMACS File.	444–472
Flag for custom software modifications (CSMs). This flag is always set to false in the CUSTMACS File. When CSMs are involved, this flag is set to true in the CSMMACS File.	475–479
Configuration options copied from the BASEFM File. These options must be uncommented if they are changed.	482–516
Object file destinations. The MOVE_OBJ_SUBVOL_ON flag controls whether object files are moved from the subvolume that contains the Make files used to create them. This flag must be uncommented and set to true for any object files to be moved. The locations need to be uncommented to be changed.	518–544
INCLUDE command that incorporates the ALLMACS File.	546–549

Note: Comment lines in the illustrations presented throughout this manual are identified by a pound sign (#) in the first column. See the “Comments” topic in section 3 for additional information.

#		# 0083
#	Since CUSTMACS uses TRUE and FALSE, they should be defined first.	# 0084
#		# 0085
true	= 1	# 0086
false	= 0	# 0087

```
#----- # 0089
# Personal Override File - # 0090
#----- # 0091
# 0092
mymacs = \r.$earl.ba60src.ztmpmacs # 0093
# 0094
# This file intentionally points to a non-existent file. To use # 0095
# this hook for overrides, use MAKE's -D option to point the MAKE # 0096
# Macro define name MYMACS to an existing file. # 0097
# e.g. # 0098
# MAKE MAKEFILE -Dmymacs=$volume.subvol.mymacs # 0099
# # 0100
!IF $F($(mymacs)) = TRUE |THEN| # 0101
    # 0102
    !include $(mymacs) # 0103
# 0104
!ENDIF # 0105
```

```
#----- # 0107
# CSE Macro File - # 0108
#----- # 0109
# 0110
bacse_csemacs = \r.$earl.ba60cse.ztmpmacs # 0111
# 0112
# This file intentionally points to a non-existent file. To use # 0113
# this hook for overrides, use MAKE's -D option to point the MAKE # 0114
# Macro define name BACSE_CSEMACS to an existing file. # 0115
# e.g. # 0116
# MAKE MAKEFILE -Dbacse_csemacs=$volume.subvol.csemacs # 0117
# # 0118
!IF $F($(bacse_csemacs)) = TRUE |THEN| # 0119
    # 0120
    !include $(bacse_csemacs) # 0121
# 0122
!ENDIF # 0123
```

```
##### # 0125
# BASE24 Products # 0126
##### # 0127
# 0128
classic_on = $(TRUE) # 0129
# 0130
base_on = $(TRUE) # 0131
atm_on = $(FALSE) # 0132
card_on = $(FALSE) # 0133
cor_on = $(FALSE) # 0134
# 0134B00
cs_on = $(FALSE) # 0134B01
```

```

fhm_on                = $(FALSE)                # 0134B02
iep_on                = $(FALSE)                # 0135
its_on                = $(FALSE)                # 0136
mail_on              = $(FALSE)                # 0137
pos_on               = $(FALSE)                # 0138
prm_on               = $(FALSE)                # 0139
retail_on            = $(FALSE)                # 0140
switch_on            = $(FALSE)                # 0141
                                # 0142
                                # 0142B00
tb_on                 = $(FALSE)                # 0142B01
                                # 0142B02
tlr_on                = $(FALSE)                # 0143

#####                                # 0145
# BASE Enhancements                                # 0146
#####                                # 0147
                                # 0148
emv_on                = $(FALSE)                # 0149
mcu_on                = $(FALSE)                # 0150
fortify_on            = $(FALSE)                # 0151

#                                # 0154
# BASE Modules                                # 0155
#                                # 0156
ansi_host_interface_on = $(FALSE)                # 0157
ansi_host_interface_atm_on = $(FALSE)            # 0158
ansi_host_interface_fhm_on = $(FALSE)            # 0159
ansi_host_interface_mail_on = $(FALSE)            # 0160
ansi_host_interface_pos_on = $(FALSE)            # 0161
                                # 0162
atalla_cntl_util_on   = $(FALSE)                # 0163
dct_on                 = $(FALSE)                # 0164
extract_on             = $(FALSE)                # 0165
iso_92_on              = $(FALSE)                # 0166
                                # 0167
iso_host_interface_on = $(FALSE)                # 0168
iso_host_interface_atm_on = $(FALSE)            # 0169
iso_host_interface_fhm_on = $(FALSE)            # 0170
iso_host_interface_mail_on = $(FALSE)            # 0171
iso_host_interface_pos_on = $(FALSE)            # 0172
iso_host_interface_ssb_on = $(FALSE)            # 0173
iso_host_interface_tlr_on = $(FALSE)            # 0174
                                # 0175
perf_utils_on          = $(FALSE)                # 0176
perf_utils_atm_on      = $(FALSE)                # 0177
perf_utils_pos_on      = $(FALSE)                # 0178
perf_utils_rb_on       = $(FALSE)                # 0179
perf_utils_tlr_on      = $(FALSE)                # 0180
                                # 0181
racal_cntl_util_on     = $(FALSE)                # 0182
refresh_on             = $(FALSE)                # 0183

```

```

sdf_on                = $(FALSE)                # 0184
tbhiso_on             = $(FALSE)                # 0185

#####                                              # 0187
# ATM Enhancements                                # 0188
#####                                              # 0189
#####                                              # 0190
# assumes 'atm_' at the beginning of flags for alphabetic ordering # 0191
#####                                              # 0192
atm_emv_on            = $(FALSE)                # 0193
#####                                              # 0193B00
cse_mac_mult_acct_on  = $(FALSE)                # 0193B01
#####                                              # 0193B02
atm_mcu_on            = $(FALSE)                # 0194
ncd_on                = $(FALSE)                # 0195
ssb_base_on           = $(FALSE)                # 0196
ssb_chk_on            = $(FALSE)                # 0197
surcharge_on          = $(FALSE)                # 0198

#                                                    # 0199
# ATM Device Handlers                              # 0200
#                                                    # 0201
atm_dh_b650_on        = $(FALSE)                # 0202
atm_dh_c910_on        = $(FALSE)                # 0203
#####                                              # 0203B00
atm_dh_c910_cse_mac_mas_on = $(FALSE)          # 0203B01
#####                                              # 0203B02
atm_dh_c912_on        = $(FALSE)                # 0204
#####                                              # 0205
atm_dh_c1000_on       = $(FALSE)                # 0206
#####                                              # 0206B00
atm_dh_c1000_cse_mac_mas_on = $(FALSE)          # 0206B01
#####                                              # 0206B02
atm_dh_c1000_dlp_on   = $(FALSE)                # 0207
atm_dh_c1000_emv_on   = $(FALSE)                # 0208
atm_dh_c1000_ncd_on   = $(FALSE)                # 0209
atm_dh_c1000_ssb_base_on = $(FALSE)            # 0210
atm_dh_c1000_ssb_chk_on = $(FALSE)             # 0211
atm_dh_c1000_ssb_dh_on = $(FALSE)             # 0212

#####                                              # 0309
# ITS                                              # 0310
#####                                              # 0311
#####                                              # 0312
knl_lrg_mem_md1_on    = $(FALSE)                # 0313
#####                                              # 0314
acct_on               = $(FALSE)                # 0315
acpc_on              = $(FALSE)                # 0316
acu_on               = $(FALSE)                # 0317
bpdu_on              = $(FALSE)                # 0318
cemtb_on             = $(FALSE)                # 0319
#####                                              # 0320B00

```

		# 0320B01
csi_on	= \$(FALSE)	# 0321
cust_on	= \$(FALSE)	# 0322
eopp_on	= \$(FALSE)	# 0323
fi_on	= \$(FALSE)	# 0324
hist_on	= \$(FALSE)	# 0325
imgr_on	= \$(FALSE)	# 0326
ims_on	= \$(FALSE)	# 0327
imt_on	= \$(FALSE)	# 0328
intf_on	= \$(FALSE)	# 0329
mc_on	= \$(FALSE)	# 0330
psnl_on	= \$(FALSE)	# 0331
rfsq_on	= \$(FALSE)	# 0332
rte_on	= \$(FALSE)	# 0333
tsec_on	= \$(FALSE)	# 0336
txn_on	= \$(FALSE)	# 0337
vru_on	= \$(FALSE)	# 0338
yang_on	= \$(FALSE)	# 0339

#####	# 0444	
# Channel Specific Products - America	# 0445	
#####	# 0446	
	# 0447	
	# 0448B00	
	# 0448B01	
	# 0448B02	
#####	# 0455	
# Channel Specific Products - ACIP	# 0456	
#####	# 0457	
	# 0458	
#	# 0459	
# Base Channel Specific enhancement - ACIP	# 0460	
#	# 0461	
	# 0462B00	
	# 0462B01	
blocker_compile_on	= \$(FALSE)	# 0464
vpro_on	= \$(FALSE)	# 0465
		# 0466
		# 0467
#####	# 0468	
# Channel Specific Products - EMEA	# 0469	
#####	# 0470	
	# 0471	
	# 0472	

```
#####                                # 0475
# CSM                                # 0476
#####                                # 0477
                                # 0478
csm_on                                = $(FALSE)          # 0479
```

```
#-----                                # 0482
# Configuration Options                -                # 0483
#-----                                # 0484
                                # 0485
#                                # 0486
# The following macros are defined in the BASEFM file.  If the # 0487
# user needs to override one of these values, it is suggested # 0488
# that the corresponding macro be uncommented and modified   # 0489
# here, rather than in the BASEFM.                            # 0490
#                                # 0491
# For more information on the use of these macros, refer to the # 0492
# comments in the BASEFM file.                                # 0493
#                                # 0494
                                # 0495
#base_tal_opts                        = symbols, suppress    # 0496
#base_scobolx_opts                    = nolist, symbols      # 0497
#base_t6520_on                        = $(TRUE)               # 0498
#base_t3270_on                        = $(TRUE)               # 0499
#base_cobol_opts                      = suppress, symbols    # 0500
#base_ddl_opts                        = setcobol74            # 0501
#base_compile_opts                    =                      # 0502
#base_secure_opt                      = "NNNU"                # 0503
#base_purge_spooler_on                = $(TRUE)               # 0504
#base_axcel_on                        = $(TRUE)               # 0505
#base_axcel_opts                      = ,CPU 1, PRI 125       # 0506
#base_bind_steps                      =                        # 0507
                                # 0508
#p1                                  = InheritsCC_On operator^dump, # 0509
#p2                                  = InheritsCC_On operator_dump, # 0510
#p3                                  = SuppressMsg operator^dump 34, # 0511
#p4                                  = SuppressMsg operator_dump 34, # 0512
#p5                                  = SuppressMsg programmatic^dump 34, # 0513
#p6                                  = SuppressMsg programmatic_dump 34 # 0514
                                # 0515
#base_axcel_directive                 = $(p1) $(p2) $(p3) $(p4) $(p5) $(p6) # 0516
```

```
#-----                                # 0518
# Object file destination.              -                # 0519
#-----                                # 0520
                                # 0521
#move_obj_subvol_on                  = $(FALSE)              # 0522
                                # 0523
```

```

#                                                    # 0524
# The following macros are commented out. They define the object # 0525
# locations for each of these products. The user may override the # 0526
# location for the objects by uncommenting the corresponding macro # 0527
# and setting the desired location. # 0528
#                                                    # 0529
#                                                    # 0530
#atm_obj_loc      = \R.$EARL.AT60obj # 0531
#base_obj_loc     = \R.$EARL.BA60obj # 0532
#bp_obj_loc       = \R.$EARL.BP60obj # 0533
#fhm_obj_loc      = \R.$EARL.FH60obj # 0534
#mail_obj_loc     = \R.$EARL.MA60obj # 0535
#pos_obj_loc      = \R.$EARL.PS60obj # 0536
#swi_obj_loc      = \R.$EARL.SW60obj # 0537
#tlr_obj_loc      = \R.$EARL.TR60obj # 0538
#                                                    # 0539
#----- # 0540
# All Requesters are automatically SCUPped into the - # 0541
# following pobj if they are compiled O.K. - # 0542
#----- # 0543
dest_pobj          = \r.$earl.ba60obj.pobj # 0544

```

```

#                                                    # 0546
# Include the ALLMACS file containing all other configuration # 0547
#                                                    # 0548
#include \r.$earl.ba60MAKE.ALLMACS # 0549

```

Flag Documentation

ACI provides a text file that briefly explains the use of all product and module flags that appear in the CUSTMACS File. This file, called the FLGSDOC File, is located on the BA60MAKE subvolume. The flags appear in the FLGSDOC File in the order in which they appear in the CUSTMACS File. The FLGSDOC File contains TGAL formatting commands for printing a paper copy of the documentation.

Example FLGSDOC File

The following illustration shows a portion of the printed FLGSDOC File.

BASE24 Products

The following set of flags represents BASE24 products that may be present in a system. These flags control the highest level of decisions in MAKE.

The following two flags must always be set to TRUE

classic_on

base_on

The following flags must be set to true if a product exists in the system being created.

atm_on

Set this flag to TRUE if BASE24-atm is present.

card_on

Set this flag to TRUE if BASE24-card is present.

cor_on

Set this flag to TRUE if any of the following products are present:

- BASE24-billpay
- BASE24-telebanking
- Customer Service

cs_on

Set this flag to TRUE if Customer Service is present.

fhm_on

Set this flag to TRUE if BASE24-from host maintenance is present.

iep_on

Set this flag to TRUE if any of the following products are present:

- BASE24-billpay
- BASE24-telebanking
- Customer Service
- NET24/IEP

its_on

Set this flag to TRUE if any of the following products are present:

- BASE24-billpay
- BASE24-telebanking
- Customer Service
- NET24/IEP

mail_on

Set this flag to TRUE if BASE24-mail is present.

pos_on

Set this flag to TRUE if BASE24-pos is present.

prm_on

Set this flag to TRUE if Prism Components are present.

retail_on

Set this flag to TRUE if any of the following products are present:

- Check Authorization
- Refunds Authorization
- Check Disbursements
- Frequent Shopper

switch_on

Set this flag to TRUE if any interchange interfaces are present.

tb_on

Set this flag to TRUE if BASE24-telebanking or BASE24-billpay are present.

tlr_on

Set this flag to TRUE if BASE24-teller is present.

ALLMACS File

The ALLMACS File contains the following information that controls Make processing.

- General configuration macros. The presence of the ALLMACS macro indicates the Make macro file hierarchy has been built.

The True and False macros are ignored because they are defined within the ACI Make utility and cannot be changed. These macros are included in this ALLMACS File because they were required with previous releases of the ACI Make utility.

The following macros are optional because the ACI Make utility assigns the default values shown below if these macros are not in the ALLMACS File.

Macro	Default Value
MAKE_TACL_MACROS	TACLMACS, located on the same subvolume as the Make utility.
TACL	\$SYSTEM.SYSTEM.TACL
TACL_INIT	TACLINIT, located on the same subvolume as the Make utility.

- Flags used to identify whether BASE24-retail products (RETAIL_ON) or BASE24 Classic products (CLASSIC_ON) are included during Make processing. When the ACI Make utility reads the CUSTMACS File first, the values assigned to these flags in the CUSTMACS File takes precedence over the values assigned in the ALLMACS File.
- Locations of some of the executables used during Make processing.
- Locations of some of the system declarations, definitions, and libraries used during Make processing.
- Locations of files containing configuration macros and file definition macros associated with channel-specific enhancements (CSEs). Because this logic is located ahead of the INCLUDE commands associated with the CLASSIC_ON flag, the CSE macro files take precedence over the classic macro files. See the “Channel-Specific Enhancements” topic in this section for more detailed information about using CSE macros to override the macros normally used to create the Make macro file hierarchy.
- Locations of files containing configuration macros and file definition macros associated with BASE24 Classic products.

Example ALLMACS File

This example includes several portions of the ALLMACS File for BASE24 release 6.0. The following table describes the portions of the ALLMACS File in the example, and gives the file sequence numbers that identify each portion of the file. The file sequence numbers appear at the right of each line in the file. For improved readability, lines separate the portions.

Portion	Sequence Numbers
General configuration macros	172–179
Executables macro locations	181–199
Tandem macro locations	205–213
RETAIL_ON flag	329–332
CSE macro locations	334–357
CLASSIC_ON flag	359–363
Classic macro locations	364–369

```

#                                     # 0172
#  General Config and Macros         # 0173
#                                     # 0174
allmacs                             = allmacs          # 0175
true                                = 1                  # 0176
false                               = 0                  # 0177
make_tacl_macros                     = $sysaci.makev500.taclmacs # 0178
tacl_init                            = $sysaci.makev500.taclinit # 0179

```

```

#                                     # 0181
#  Executables Macros                # 0182
#                                     # 0183
aciaid                              = $sysaci.acilibi.aciaid # 0184
axcel                                = $system.system.axcel    # 0185
bind                                 = $system.system.bind     # 0186
c                                    = $system.system.c        # 0187
cobol                                = $system.system.cobol85  # 0188
ddl                                  = $system.system.ddl       # 0189
edit                                 = $system.system.edit      # 0190

```

enable	= \$system.system.enable	# 0191
fup	= \$system.system.fup	# 0192
genfwd	= \$sysaci.acilibi.genfwd2	# 0193
genpro	= \$sysaci.acilibi.genpro	# 0194
peruse	= \$system.system.peruse	# 0195
scobolx	= \$system.system.scobolx	# 0196
scup	= scup_routine \$system.system.scup	# 0197
sqlci	= \$system.system.sqlci	# 0198
tacl	= \$system.system.tacl	# 0199

#		# 0205
# Tandem Macros		# 0206
#		# 0207
system_cextdecs	= \r.\$system.system.cextdecs	# 0208
system_cwide	= \r.\$system.system.cwide	# 0209
system_extdecs	= \r.\$system.system.extdecs	# 0210
system_extdecs0	= \r.\$system.system.extdecs0	# 0211
system_cobollib	= \r.\$system.system.cobollib	# 0212
system_gpldefs	= \r.\$system.system.gpldefs	# 0213

#		# 0329
# Retail		# 0330
#		# 0331
retail_on = \$(FALSE)		# 0332
#		# 0334
# CSE		# 0335
#		# 0336
bacse_csefm = \r.\$earl.ba60cse.zztempfm		# 0337
bacse_csemm = \r.\$earl.ba60cse.zztempmm		# 0338
#		# 0339
# These MAKE Macro Define Names, BACSE_CSEFM and BACSE_CSEMM		# 0340
# intentionally point to files that don't exist. They are used to		# 0341
# include CSE 'fm' and 'mm' files by overriding them to files that		# 0342
# do exist, prior to the execution of the !include statements below.		# 0343
#		# 0344
# The override can be done on the command line using MAKE's -D command		# 0345
# i.e. MAKE MAKEFILE -Dbacse_csefm=\$volume.subvol.csefm &		# 0346
# -Dbacse_csemm=\$volume.subvol.csemm ...		# 0347
#		# 0348
# The override can also be done by defining them in a file that is		# 0349
# included before this point. For example, using the BACSE_CSEMACS		# 0350
# file that gets included in CUSTMACS using the -D override		# 0351
# technique.		# 0352
#		# 0353

```

!IF $(bacse_csefm) = $(TRUE) |THEN|                                # 0354
    !include $(bacse_csefm)                                         # 0355
    !include $(bacse_csemm)                                         # 0356
!ENDIF                                                             # 0357

```

```

#                                                                    # 0359
# Classic                                                            # 0360
#                                                                    # 0361
classic_on = $(TRUE)                                              # 0362
                                                                    # 0363

```

```

!IF $(classic_on) = $(TRUE) |THEN|                                # 0364
    !include \r.$earl.ok60src.itsfm                                # 0365
    !include \r.$earl.ok60src.itsmm                                # 0366
    !include \r.$earl.ba60src.clscfm                                # 0367
    !include \r.$earl.ba60src.clscmm                                # 0368
!ENDIF                                                             # 0369

```

BASE24-retail Products

The RETAIL_ON flag, which is the first product flag in the CUSTMACS File and ALLMACS File, identifies the presence of one or more BASE24-retail products in the BASE24 installation. These products include BASE24-check auth, BASE24-frequent shopper, and BASE24-refund.

The ACI Make utility uses the RETAIL_ON flag in a slightly different manner than it uses the CLASSIC_ON flag. The ACI Make utility uses the CLASSIC_ON flag and product flags in the CUSTMACS File and ALLMACS File to determine which macro files and makefiles need to be used. The same product file definition macro files (BAAFTMM and BASRCMM) and product makefile (RQMEGAM) get used, regardless of the RETAIL_ON flag setting. When the RETAIL_ON flag is set to true, the file definitions point to BASE24-retail subvolumes, and the makefile includes commands to build BASE24-retail requester screens. When the RETAIL_ON flag is set to false, the file definitions point to dummy files contained on the BA60AFT and BA60SRC subvolumes, and the makefile excludes the commands to build BASE24-retail requester screens.

Personal Overrides

You can use personal override files when you want to temporarily change macro definitions for personal work, a project, or testing purposes. One or more personal override files can be called from the CUSTMACS File. Since the CUSTMACS File starts with a test for personal override files, the values contained in any personal override files override the values in the CUSTMACS File and ALLMACS File.

For example, if you are part of a project team that is working on a group of product modules, you can have a project macro file with defines for all of the file names, file locations, and product or module flags that are affected by the project. You can also have a personal macro file with defines for the file names, file locations, and product or module flags for the product modules assigned to you personally.

When you start the ACI Make utility, set the CUSTMACS File as the global level macro file in the ALLMACS define and use a command line option to point the MYMACS macro to your personal override file. See the “ALLMACS Define” topic earlier in this section for more information about that TACL define.

The ACI Make utility performs the following steps:

1. Opens the makefile (e.g., B24M or AUTHM) specified in the command line to start the ACI Make utility.
2. Opens the CUSTMACS File.
3. Checks for the presence of your personal override file. When your personal override file is present, the ACI Make utility uses all of the definitions in your personal override file. The last statement of your personal override file can incorporate an additional project macro file by reference.
4. Uses the definitions in the project macro file that were not in your personal override file. The project macro file does not incorporate any files by reference.
5. Returns to the CUSTMACS File and uses the definitions in the CUSTMACS File that were not defined in your personal override file or the project macro file. The last statement of the CUSTMACS File incorporates the ALLMACS File by reference.
6. Uses the definitions in the ALLMACS File that were not in your personal override file, the project macro file, or the CUSTMACS File.
7. Uses the INCLUDE statements in the ALLMACS File to continue processing.

The ACI Make utility supports a maximum nesting depth of ten files. You reduce the number of nesting levels by starting with the CUSTMACS File, processing your personal macros file and any additional project macros files, and then returning to the CUSTMACS File before processing the ALLMACS File. By returning to the CUSTMACS File, the ACI Make utility places the first INCLUDE statement from the ALLMACS File in the fourth nesting level, as shown in the first two columns of the following table.

Returning to CUSTMACS		Using Only INCLUDE Statements	
File	Level	File	Level
Makefile	1	Makefile	1
CUSTMACS	2	Personal overrides	2
Personal overrides	3	Project macros	3
Project macros	4	CSEMACS	4
CUSTMACS	2	CUSTMACS	5
CSEMACS	3	ALLMACS	6
CUSTMACS	2	ALLMACS INCLUDE	7
ALLMACS	3		
ALLMACS INCLUDE	4		

By comparison, if you do not point the MYMACS macro to your personal override file, you have to set your personal override file as the global level macro file in the ALLMACS define, then use an INCLUDE statement at the end of each override or macro file to pass control to the next file. The ACI Make utility places the first INCLUDE statements from the ALLMACS File in the seventh nesting level, as shown in the last two columns of the table.

Channel-Specific Enhancements

Some ACI software is enhanced to meet the requirements of specific market areas, called channels. The ACI Make utility must process these channel-specific enhancements (CSEs) in addition to the regular software products. The CUSTMACS File contains a test for the CSEMACS File right after the test for personal overrides. The CSEMACS File is a global level file that contains flags for CSEs, much like the CUSTMACS File does for regular software products.

When you start the ACI Make utility, you use the CUSTMACS File as the global level macro file, and use a command line option to point the BACSE_CSEMACS macro to the CSEMACS File. You also use command line options to point the BACSE_CSEFM and BACSE_CSEMM macros to the files containing configuration macros (CSEFM File) and file definition macros (CSEMM File) associated with the CSEs.

In the ALLMACS File, the INCLUDE commands for the CSEFM File and CSEMM File appear before the INCLUDE commands for any other products.

CSMMACS File

The CSMMACS File contains configuration information for custom software modifications (CSMs), and usually serves as the top level global macro file when a BASE24 installation includes CSMs.

The CSMMACS File contains only the macros that are required for the CSMs. The last statement of the CSMMACS File incorporates the CUSTMACS File, which contains configuration information for the standard BASE24 code, by reference.

Two runs of the ACI Make utility are required when you have CSMs installed. The first run uses the CUSTMACS File as the top level global macro file and compiles the standard BASE24 software. The second run uses the CSMMACS File as the top level global macro file and compiles the CSMs.

See appendix C for additional information about using the ACI Make utility to install CSMs.

BASE24 Procedural Product Macro Files

The BASE24 procedural products use three levels of macro files that are located below the global macro files and contain configuration macros and file definition macros. These levels are called product group, product, and process.

Product Group Macro Files

The product group macro files are located directly below the global macro files in the Make hierarchy, and contain configuration macros and file definition macros for the product groups. The first four characters of these file names (CLSC) represent *classic*, which is another name given to BASE24 procedural products.

The BASE24 procedural product group macro files are BASE24 Classic Configuration Macro File (CLSCFM) and BASE24 Classic File Definition Macro File (CLSCMM). The CLSCFM File contains the product flags (e.g., ATM_ON, POS_ON), the location of the product level macro files (e.g., BASEFM, BASEMM, ATMMFM, ATMMM), and the location of the product makefiles (e.g., BASEM, ATMM). You can set flags in the CUSTMACS File for the appropriate BASE24 procedural products, and these flags override the flags in the CLSCFM File.

Example Product Group Configuration Macro File

A portion of the BASE24 Classic Configuration Macro File (CLSCFM) is shown below. The example CLSCFM File contains product-specific text for BASE24-atm on lines with sequence numbers 65–79. The actual CLSCFM File contains similar text for each of the other BASE24 products.

```

basrc_clscm      = \r.$earl.ba60src.clscm                                # 0037
                                                         # 0038
#-----# 0039A02
# All Requesters are automatically SCUPped into the                # 0039A03
# following pobj if they are compiled O.K.                          # 0039A04
#-----# 0039A05
dest_pobj        = \r.$earl.ba60obj.pobj                                # 0043
                                                         # 0044
#-----# 0045A02
# This flag controls whether a copy of an object file is          # 0045A03
# moved to a common execution subvol for a given product          # 0045A04
#-----# 0045A05

```

```
move_obj_subvol_on = $(TRUE)                                # 0049
                                                            # 0050
#-----                                                    # 0051A02
#   The product level DDLs are compiled by the following    # 0051A03
#   makefile                                                # 0051A04
#-----                                                    # 0051A05
basrc_clscddlml = \r.$earl.ba60src.clscddlml                # 0055
                                                            # 0056
#-----                                                    # 0057A02
# Global/Common Config and Macros for the BASE24-base product # 0057A03
#-----                                                    # 0057A04
base_on          = $(FALSE)                                # 0060
basrc_basefm     = \r.$earl.ba60src.basefm                  # 0061
basrc_basemm     = \r.$earl.ba60src.basemm                  # 0062
basrc_basem      = \r.$earl.ba60src.basem                   # 0063
                                                            # 0064
#-----                                                    # 0065A02
# BASE24-atm                                                # 0065A03
#-----                                                    # 0065A04
atm_on           = $(FALSE)                                # 0068
                                                            # 0069
!IF $(atm_on) = TRUE |THEN|                                  # 0070
                                                            # 0071
#-----                                                    # 0072A02
# Global/Common Config and Macros for the BASE24-atm        # 0072A03
#-----                                                    # 0072A04
atsrc_atmfm      = \R.$EARL.AT60src.atmfm                   # 0075
atsrc_atmmm      = \R.$EARL.AT60src.atmmm                   # 0076
atsrc_atmm       = \R.$EARL.AT60src.atmm                    # 0077
                                                            # 0078
!ENDIF                                                      # 0079
```

Example Product Group File Definition Macro File

The BASE24 Classic File Definition Macro File (CLSCMM) incorporates the appropriate product level configuration and file definition macro files by reference. A portion of the CLSCMM File is shown below. The example CLSCMM File contains product-specific text for BASE24-atm on lines with sequence numbers 42–48. The actual CLSCMM File contains similar text for each of the other BASE24 products.

```
!include $(basrc_basefm)                                     # 0039
!include $(basrc_basemm)                                     # 0040
                                                            # 0041
```

```
#----- # 0042A02
# BASE24-atm # 0042A03
#----- # 0042A04
!IF $(atm_on) = TRUE |THEN| # 0045
    !include $(atsrc_atmfm) # 0046
    !include $(atsrc_atmmm) # 0047
!ENDIF # 0048
```

Product Macro Files

The product macro files are located directly beneath the product group level files. The product level configuration macro files (e.g., BASEFM, ATMFm, POSFM) contain the following information:

- Configuration macros that are passed to the various compilers and can contain any options supported by the compiler.

You can set compiler options in the CUSTMACS File to override the standard values contained in the BASEFM File. The remaining product level configuration files (e.g., ATMFm, POSFM) use the compiler options contained in the BASEFM File as default values.

- Flags identifying which of the optional modules associated with a product are included in the system.

For example, the ATM product is composed of various optional Device Handler processes. The ATMFm File contains flags that are used to specify which Device Handler processes to build.

You can set flags in the CUSTMACS File to turn on the appropriate product modules, and these flags override the product module flags in the product level macro file.

- Flags identifying which channel-specific enhancements (CSEs) are used.
- File defines that locate configuration macro files (file names ending with *FM*) and file definition macro files (file names ending with *MM*) for the modules associated with the product. Some defines are always set. See lines with sequence numbers 414–447 in the following example. Other defines are set only when certain flags are set to true. See lines with sequence numbers 449–463 in the following example.

The product level file definition macro files (e.g., BASEMM, ATMMM, POSMM) incorporate configuration macro files and file definition macro files for the appropriate product modules.

Example Product Configuration Macro File

Portions of the configuration macro, module flag, and file define sections of the BASE24-atm Configuration Macro File (ATMFM) are shown below. The following table describes the portions of the ATMFM File in the example, and gives the file sequence numbers that identify each portion of the file. The file sequence numbers appear at the right of each line in the file. For improved readability, lines separate the portions.

Portion	Sequence Numbers
Configuration macro locations	251–291
Flags for BASE24-atm modules	346–362
Flags for device support module features	369–374
Flags for channel-specific enhancements (CSEs)	399–408
File locations that are always set	414–447
File locations set only when certain conditions are met	449–463

```

#-----                                # 0251
# Configuration Macros                      -    # 0252
#-----                                # 0253
                                           # 0254
# This macro definition is passed to the check_completion      # 0255
# macro and used to allow warnings as a successful completion. # 0256
# $(TRUE) - Warnings are considered a successful completion.   # 0257
# $(FALSE) - Warnings are considered an unsuccessful completion. # 0258
#                                                              # 0259
atm_ddl_warnings_allowed = $(base_ddl_warnings_allowed)        # 0260
                                                              # 0261
# DDL Allowed Warnings Text                                     # 0262
#                                                              # 0263
atm_ddl_warning_text    = $(base_ddl_warning_text)             # 0264
atm_ddl_alwd_warnings   = $(base_ddl_alwd_warnings)            # 0265
                                                              # 0266

```

```

#  TAL options                                     # 0267
#                                                  # 0268
atm_tal_opts          = $(base_tal_opts)          # 0269
                                                    # 0270
#  SCOBOL options                                 # 0271
#                                                  # 0272
atm_scobolx_opts      = $(base_scobolx_opts)      # 0273
                                                    # 0274
#  $(TRUE) - If you wish to SCOBOL compile for 6520 # 0275
#  $(FALSE) - If you don't want to SCOBOL compile for 6520 # 0276
#                                                  # 0277
atm_t6520_on          = $(base_t6520_on)          # 0278
                                                    # 0279
#  $(TRUE) - If you wish to SCOBOL compile for 3270 # 0280
#  $(FALSE) - If you don't want to SCOBOL compile for 3270 # 0281
#                                                  # 0282
atm_t3270_on          = $(base_t3270_on)          # 0283
                                                    # 0284
#  COBOL options                                 # 0285
#                                                  # 0286
atm_cobol_opts        = $(base_cobol_opts)        # 0287
                                                    # 0288
#  DDL options                                   # 0289
#                                                  # 0290
atm_ddl_opts          = $(base_ddl_opts)          # 0291

```

```

#----- # 0346
#  BASE24-atm modules - # 0347
#    Set the appropriate option to $(TRUE) - # 0348
#    to include the module. - # 0349
#----- # 0350
                                                    # 0351
iso_host_interface_atm_on = $(FALSE) # 0352
iso_host_interface_ssb_on = $(FALSE) # 0353
                                                    # 0354
                                                    # 0355
atm_emv_on                = $(FALSE) # 0356
atm_mcu_on                = $(FALSE) # 0357
                                                    # 0358
ncd_on                    = $(FALSE) # 0359
ssb_dh_on                 = $(FALSE) # 0360
ssb_base_on               = $(FALSE) # 0361
ssb_chk_on                = $(FALSE) # 0362

```

```

atm_dh_c1000_on          = $(FALSE) # 0369
atm_dh_c1000_ssb_dh_on   = $(FALSE) # 0370
atm_dh_c1000_ssb_base_on = $(FALSE) # 0371
atm_dh_c1000_ssb_chk_on  = $(FALSE) # 0372

```

atm_dh_c1000_ncd_on	= \$(FALSE)	# 0373
atm_dh_c5100_on	= \$(FALSE)	# 0374

#-----		# 0399
# Channel Specific Enhancements	-	# 0400
# Set the appropriate option to \$(TRUE)	-	# 0401
# to include the CSE.	-	# 0402
#-----		# 0403
atm_dh_c1000_cse_mac_mas_on	= \$(FALSE)	# 0404
atm_dh_fuj7_cse_mac_mas_on	= \$(FALSE)	# 0405
atm_dh_c910_cse_mac_mas_on	= \$(FALSE)	# 0406
atm_dh_n50_cse_mac_mas_on	= \$(FALSE)	# 0407
atm_dh_i3624_v8_cse_mac_mas_on	= \$(FALSE)	# 0408

#		# 0414
# DEFS		# 0415
#		# 0416
atdefs_atdefsmm	= \r.\$earl.at60defs.atdefsmm	# 0417
		# 0418
#		# 0419
# DDL		# 0420
#		# 0421
atddl_atddl_fm	= \r.\$earl.at60ddl.atddl_fm	# 0422
atddl_atddlmm	= \r.\$earl.at60ddl.atddlmm	# 0423
		# 0424
#		# 0425
# Libraries		# 0426
#		# 0427
atlib_atlib_fm	= \r.\$earl.at60lib.atlib_fm	# 0428
atlib_atlibmm	= \r.\$earl.at60lib.atlibmm	# 0429
		# 0430
#		# 0431
# EMT Build		# 0432
#		# 0433
atemtb_aemb_fm	= \r.\$earl.at60emb.aemb_fm	# 0434
atemtb_aembmm	= \r.\$earl.at60emb.aembmm	# 0435
		# 0436
#		# 0437
# SRC		# 0438
#		# 0439
atsrc_atsrc_fm	= \r.\$earl.at60src.atsrc_fm	# 0440
atsrc_atsrcmm	= \r.\$earl.at60src.atsrcmm	# 0441
		# 0442
#		# 0443
# AFT		# 0444
#		# 0445
ataft_ataft_fm	= \r.\$earl.at60aft.ataft_fm	# 0446
ataft_ataftmm	= \r.\$earl.at60aft.ataftmm	# 0447

```

#                                                    # 0449
# ATALLA Security Module Utility Program          # 0450
#                                                    # 0451
!IF $(atalla_cntl_util_on) = TRUE |THEN|          # 0452
    atatal_atatalfm          = \r.$earl.at60atal.atatalfm    # 0453
    atatal_atatalmm          = \r.$earl.at60atal.atatalmm    # 0454
!ENDIF                                              # 0455
                                                    # 0456
#                                                    # 0457
# RACAL Security Module Utility Program          # 0458
#                                                    # 0459
!IF $(racal_cntl_util_on) = TRUE |THEN|          # 0460
    atracl_atraclfm          = \r.$earl.at60racl.atraclfm    # 0461
    atracl_atraclmm          = \r.$earl.at60racl.atraclmm    # 0462
!ENDIF                                              # 0463

```

Example Product File Definition Macro File

A portion of the BASE24-atm File Definition Macro File (ATMMM) is shown below. The ATMMM File contains an INCLUDE statement for each file macro that was defined in the ATMFM File. Like the ATMFM File, the ATMMM File contains some INCLUDE statements that are always set and some INCLUDE statements that are set only when certain flags are set to true.

```

#-----                                                    # 0257
# Macro Information -                                         # 0258
#-----                                                    # 0259
                                                    # 0260
#                                                    # 0261
# DEFS                                                    # 0262
#                                                    # 0263
!include $(atdefs_atdefsmm)                               # 0264
                                                    # 0265
#                                                    # 0266
# DDL                                                    # 0267
#                                                    # 0268
!include $(atddl_atddl_fm)                                 # 0269
!include $(atddl_atddlmm)                                  # 0270
                                                    # 0271
#                                                    # 0272
# Libraries                                              # 0273
#                                                    # 0274
!include $(atlib_atlib_fm)                                 # 0275
!include $(atlib_atlibmm)                                  # 0276

```

#	# 0277
#	# 0278
# EMT Build	# 0279
#	# 0280
!include \$(atemb_aembfm)	# 0281
!include \$(atemb_aembmm)	# 0282
	# 0283
#	# 0284
# SRC	# 0285
#	# 0286
!include \$(atsrc_atsrcfm)	# 0287
!include \$(atsrc_atsrcmm)	# 0288
	# 0289
#	# 0290
# AFT	# 0291
#	# 0292
!include \$(ataft_ataftfm)	# 0293
!include \$(ataft_ataftmm)	# 0294
	# 0294B00
#	# 0294B01
# CSE	# 0294B02
#	# 0294B03
!include \$(atcse_atcsemm)	# 0294B04
	# 0294B05
	# 0295
#	# 0296
# ATALLA Security Module Utility Program	# 0297
#	# 0298
!IF \$(atalla_cntl_util_on) = TRUE THEN	# 0299
!include \$(atatal_atatalfm)	# 0300
!include \$(atatal_atatalmm)	# 0301
!ENDIF	# 0302
	# 0303
#	# 0304
# RACAL Security Module Utility Program	# 0305
#	# 0306
!IF \$(racal_cntl_util_on) = TRUE THEN	# 0307
!include \$(atracl_atraclfm)	# 0308
!include \$(atracl_atraclmm)	# 0309
!ENDIF	# 0310

Process Macro Files

The process level macro files are located directly beneath the product level files. The BASE24 procedural process level configuration macro files (e.g., BADDLFM and AUTHFM) contain process level compiler option macros. By default, the

process level macros refer to the product level macros, which allows all the modules of a product to use the same options. Optionally, process level macro files can contain settings that are customized for the process.

Example Process Configuration Macro File

A portion of the BASE24-atm Authorization Process Configuration Macro File (AUTHFM) is shown below. The configuration macros in the AUTHFM File refer to macros that are defined in the ATMFM File.

```

#-----# 0013
# Configuration Macros -# 0014
#-----# 0015
# 0016
# 0017
# TAL options# 0018
# 0019
atauth_tal_opts = $(atm_tal_opts)# 0020
# 0021
# 0022
# SCOBOL options# 0023
# 0024
atauth_scobolx_opts = $(atm_scobolx_opts)# 0025
# 0026
# 0027
# $(TRUE) - If you wish to SCOBOL compile for 6520# 0028
# $(FALSE) - If you don't want to SCOBOL compile for 6520# 0029
# 0030
atauth_t6520_on = $(atm_t6520_on)# 0031
# 0032
# 0033
# $(TRUE) - If you wish to SCOBOL compile for 3270# 0034
# $(FALSE) - If you don't want to SCOBOL compile for 3270# 0035
# 0036
atauth_t3270_on = $(atm_t3270_on)# 0037
# 0038
# 0039
# COBOL options# 0040
# 0041
atauth_cobol_opts = $(atm_cobol_opts)# 0042
# 0043
# 0044
# DDL options# 0045
# 0046
atauth_ddl_opts = $(atm_ddl_opts)# 0047

```

Example Process File Definition Macro File

The BASE24 procedural process level file definition macro files (e.g., BADDLMM and AUTHMM) contain definitions for other files used by the module. All of the definitions contained in the BASE24-atm Authorization Process File Definition Macro File (AUTHMM) are shown below.

#-----	# 0067
# File Macros (defines)	# 0068
#-----	# 0069
atauth_auth = \r.\$earl.at60auth.auth	# 0070
atauth_authcsee = \r.\$earl.at60auth.authcsee	# 0071
atauth_authcseg = \r.\$earl.at60auth.authcseg	# 0072
atauth_authcses = \r.\$earl.at60auth.authcses	# 0073
atauth_authc = \r.\$earl.at60auth.authc	# 0074
atauth_authg = \r.\$earl.at60auth.authg	# 0075
atauth_authlibe = \r.\$earl.at60auth.authlibe	# 0076
atauth_authlibo = \r.\$earl.at60auth.authlibo	# 0077
atauth_authlibs = \r.\$earl.at60auth.authlibs	# 0078
atauth_authm = \r.\$earl.at60auth.authm	# 0079
atauth_autho = \r.\$earl.at60auth.autho	# 0080
atauth_auths = \r.\$earl.at60auth.auths	# 0081
atauth_authtbl = \r.\$earl.at60auth.authtbl	# 0082
	# 0083
atauth_authemvd = \r.\$earl.at60auth.authemvd	# 0084
atauth_authmcud = \r.\$earl.at60auth.authmcud	# 0085
	# 0085A00
atauth_authpid = \r.\$earl.at60auth.authpid	# 0085A01

BASE24 ITS Product Macro Files

The BASE24 ITS products use three levels of macro files that are located below the global macro files and contain configuration macros and file definition macros. These levels are called object group, subsystem, and object.

Object Group Macro Files

The object group macro files are located directly below the global macro files in the Make hierarchy, and contain configuration macros and file definition macros for all objects.

The BASE24 ITS object group macro files are BASE24 ITS Configuration Macro File (ITSFM) and BASE24 ITS Configuration File Definition File (ITSMM). The ITSFM File contains flags that identify which of the subsystems and objects are to be included in the system, and the location of the subsystem and object level macro files (e.g., CORFM, CORMM, TBRPTFM, TBRPTMM). The ITSFM File also contains the location of the subsystem makefiles (e.g., CORM, TBM). The Core subsystem is divided into standard functionality and optional functionality. You can set flags in the CUSTMACS File for the appropriate BASE24 ITS subsystems and objects, and these flags override the flags in the ITSFM File.

Example Object Group Configuration Macro File

Portions of the BASE24 ITS Configuration Macro File (ITSFM) are shown below. Several macros have been omitted to simplify the example. The following table describes the portions of the ITSFM File in the example, and gives the file sequence numbers that identify each portion of the file. The file sequence numbers appear at the right of each line in the file. For improved readability, lines separate the portions.

Portion	Sequence Numbers
Kernel subsystem locations.	66–115
IEP subsystem locations. The IEP_ON flag controls whether any of these location defines are used.	189–220
End of IEP subsystem locations. This statement identifies the end of the conditional text controlled by the IEP_ON flag.	270

Portion	Sequence Numbers
Required Core subsystem locations. The CORE_ON flag controls whether any of these location defines are used.	272–301
Optional Core subsystem locations. Each optional Core subsystem has its own flag that controls whether its location defines are used. These flags are in addition to the CORE_ON flag.	333–361 397–410
End of Core subsystem locations. This statement identifies the end of the conditional text controlled by the CORE_ON flag.	524

```

#----- # 0066
# ITS Configuration - # 0067
#----- # 0068
oksrc_itsm = \R.$EARL.ok60src.itsm # 0069
#----- # 0070
#----- # 0071
# KNL - # 0072
#----- # 0073
#----- # 0074
oksrc_knlm = \R.$EARL.ok60src.knlm # 0075
#----- # 0076
#----- # 0077
# Global/Common Config and Macros for the Kernel Subsystem # 0078
#----- # 0079
oksrc_knlfm = \R.$EARL.ok60src.knlfm # 0080
oksrc_knlmm = \R.$EARL.ok60src.knlmm # 0081
#----- # 0082
#----- # 0083
# Individual Modules/Objects for the Kernel Subsystem - # 0084
#----- # 0085
#----- # 0086
# # 0087
# AFM # 0088
# # 0089
okafm_kafmf = \R.$EARL.ok60afm.kafmf # 0090
okafm_kafmm = \R.$EARL.ok60afm.kafmm # 0091
#----- # 0092
#----- # 0093
# ANS # 0094
# # 0095
okans_ansfm = \R.$EARL.ok60ans.ansfm # 0096
okans_ansmm = \R.$EARL.ok60ans.ansmm # 0097
#----- # 0098

```

```

#                                                    # 0099
#   APL                                                    # 0100
#                                                    # 0101
okapl_apl_fm      = \R.$EARL.ok60apl.apl_fm          # 0102
okapl_apl_mm      = \R.$EARL.ok60apl.apl_mm          # 0103
#                                                    # 0104
#                                                    # 0105
#   CLAS                                                    # 0106
#                                                    # 0107
okclas_clas_fm    = \R.$EARL.ok60clas.clas_fm        # 0108
okclas_clas_mm    = \R.$EARL.ok60clas.clas_mm        # 0109
#                                                    # 0110
#                                                    # 0111
#   DDL                                                    # 0112
#                                                    # 0113
okddl_kddl_fm     = \R.$EARL.ok60ddl.kddl_fm         # 0114
okddl_kddl_mm     = \R.$EARL.ok60ddl.kddl_mm         # 0115

```

```

#-----                                                    # 0189
# IEP                                                    - # 0190
#-----                                                    # 0191
iep_on            = $(FALSE)                            # 0192
#-----                                                    # 0193
!IF $(iep_on)     = TRUE |THEN|                          # 0194
#-----                                                    # 0195
    oesrc_iep_fm  = \R.$EARL.oe60src.iep_fm            # 0196
#-----                                                    # 0197
#-----                                                    # 0198
# Global/Common Config and Macros for the IEP Subsystem - # 0199
#-----                                                    # 0200
oesrc_iep_fm      = \R.$EARL.oe60src.iep_fm            # 0201
oesrc_iep_mm      = \R.$EARL.oe60src.iep_mm            # 0202
#-----                                                    # 0203
#                                                    # 0204
#   AFM                                                    # 0205
#                                                    # 0206
oeafm_eaf_fm      = \R.$EARL.oe60afm.eaf_fm           # 0207
oeafm_eaf_mm      = \R.$EARL.oe60afm.eaf_mm           # 0208
#-----                                                    # 0209
#                                                    # 0210
#   CTXM                                                    # 0211
#                                                    # 0212
oectxm_ctxm_fm    = \R.$EARL.oe60ctxm.ctxm_fm         # 0213
oectxm_ctxm_mm    = \R.$EARL.oe60ctxm.ctxm_mm         # 0214
#-----                                                    # 0215
#                                                    # 0216
#   DDL                                                    # 0217
#                                                    # 0218

```

oeddl_eddlfm	= \R.\$EARL.oe60ddl.eddlfm	# 0219
oeddl_eddlmm	= \R.\$EARL.oe60ddl.eddlmm	# 0220

!ENDIF # if iep	# 0270
-----------------	--------

#-----	# 0272
# COR -	# 0273
#-----	# 0274
cor_on = \$(FALSE)	# 0275
	# 0276
!IF \$(cor_on) = TRUE THEN	# 0277
	# 0278
ocsrc_corm = \R.\$EARL.oc60src.corm	# 0279
	# 0280
#-----	# 0281
# Global/Common Config and Macros for the COR Subsystem -	# 0282
#-----	# 0283
ocsrc_corfm = \R.\$EARL.oc60src.corfm	# 0284
ocsrc_cormm = \R.\$EARL.oc60src.cormm	# 0285
	# 0286
#-----	# 0287
# Standard Core Information -	# 0288
#-----	# 0289
	# 0290
#	# 0291
# AFM	# 0292
#	# 0293
ocafm_cafmfmm = \R.\$EARL.oc60afm.cafmfmm	# 0294
ocafm_cafmmmm = \R.\$EARL.oc60afm.cafmmmm	# 0295
	# 0296
#	# 0297
# DDL	# 0298
#	# 0299
ocddl_cddl_fm = \R.\$EARL.oc60ddl.cddl_fm	# 0300
ocddl_cddl_mm = \R.\$EARL.oc60ddl.cddl_mm	# 0301

#-----	# 0333
# Optional Core Information -	# 0334
#-----	# 0335
	# 0336
acct_on = \$(FALSE)	# 0337
acpc_on = \$(FALSE)	# 0338
acu_on = \$(FALSE)	# 0339
	# 0340

```
# Telebanking # 0341
# Note: Billpay requires Telebanking; but Telebanking does NOT # 0342
# require Billpay. Therefore, BP is included within the # 0343
# TB condition. # 0344
# The bpdu_on flag is for Billpay Database Utilities # 0345
# # 0346
tb_on = $(FALSE) # 0347
# # 0348
!IF $(tb_on) = TRUE |THEN| # 0349
    bpdu_on = $(FALSE) # 0350
    tbsrc_tbm = \R.$EARL.tb60src.tbm # 0351
    tbsrc_tbfm = \R.$EARL.tb60src.tbfm # 0352
    tbsrc_tbmm = \R.$EARL.tb60src.tbmm # 0353
    # 0354
    tbafm_tbfmfm = \R.$EARL.tb60afm.tbafmfm # 0355
    tbafm_tbfmmm = \R.$EARL.tb60afm.tbafmmm # 0356
    # 0357
    tbrpt_tbrptfm = \R.$EARL.tb60rpt.tbrptfm # 0358
    tbrpt_tbrptmm = \R.$EARL.tb60rpt.tbrptmm # 0359
    # 0360
!ENDIF # 0361
```

```
!IF $(acct_on) = TRUE |THEN| # 0397
    ocacct_acctfm = \R.$EARL.oc60acct.acctfm # 0398
    ocacct_acctmm = \R.$EARL.oc60acct.acctmm # 0399
!ENDIF # 0400
# 0401
!IF $(acpc_on) = TRUE |THEN| # 0402
    ocacpc_acpcfm = \R.$EARL.oc60acpc.acpcfm # 0403
    ocacpc_acpcmm = \R.$EARL.oc60acpc.acpcmm # 0404
!ENDIF # 0405
# 0406
!IF $(acu_on) = TRUE |THEN| # 0407
    ocacu_acufm = \R.$EARL.oc60acu.acufm # 0408
    ocacu_acumm = \R.$EARL.oc60acu.acumm # 0409
!ENDIF # 0410
```

```
!ENDIF # if cor # 0524
```

Example Object Group File Definition Macro File

The BASE24 ITS Configuration File Definition File (ITSMM) incorporates the appropriate subsystem level configuration and file definition macro files by reference. A portion of the ITSMM File is shown below. This portion corresponds to the portion of the ITSFM File shown above, with the following exceptions:

- In the ITSMM File, the Kernel subsystems are divided into a group that is always used and a group that is used based upon the ITS_ON flag setting. Kernel subsystems are arranged in a single group in the ITSFM File.
- The ITSM File location is defined in the ITSFM File, but the INCLUDE statement for the ITSM File appears in the B24M makefile.
- The KNLM File, IEPM File, and CORM File locations are defined in the ITSFM File, but the INCLUDE statements for them appear in the ITSM File.

The following table describes the portions of the ITSMM File in the example, and gives the file sequence numbers that identify each portion of the file. The file sequence numbers appear at the right of each line in the file. For improved readability, lines separate the portions.

Portion	Sequence Numbers
INCLUDE statements for Kernel subsystems that are always used.	70–94
INCLUDE statements for Kernel subsystems used by BASE24 IEP, but not BASE24 Classic. The ITS_ON flag controls whether any of these statements are used.	121–138
End of INCLUDE statements for Kernel subsystems used by BASE24 IEP, but not BASE24 Classic. This statement identifies the end of the conditional text controlled by the ITS_ON flag.	188
INCLUDE statements for IEP subsystems. The IEP_ON flag controls whether any of these statements are used.	190–217
End of INCLUDE statements for IEP subsystems. This statement identifies the end of the conditional text controlled by the IEP_ON flag.	267

#	# 0122
# ANS	# 0123
#	# 0124
!include \$(okans_ansfm)	# 0125
!include \$(okans_ansmm)	# 0126
	# 0127
#	# 0128
# APL	# 0129
#	# 0130
!include \$(okapl_aplfm)	# 0131
!include \$(okapl_aplmm)	# 0132
	# 0133
#	# 0134
# CLAS	# 0135
#	# 0136
!include \$(okclas_clasfm)	# 0137
!include \$(okclas_clasmm)	# 0138

!ENDIF	# 0188
--------	--------

#-----	# 0190
# IEP -	# 0191
#-----	# 0192
!IF \$(iep_on) = TRUE THEN	# 0193
	# 0194
#	# 0195
# Global Info	# 0196
#	# 0197
!include \$(oesrc_iepfm)	# 0198
!include \$(oesrc_iepmm)	# 0199
	# 0200
#	# 0201
# AFM	# 0202
#	# 0203
!include \$(oeafm_eafmfmm)	# 0204
!include \$(oeafm_eafmmmm)	# 0205
	# 0206
#	# 0207
# CTXM	# 0208
#	# 0209
!include \$(oectxm_ctxmfmm)	# 0210
!include \$(oectxm_ctxmmmm)	# 0211
	# 0212
#	# 0213
# DDL	# 0214
#	# 0215

```

!include $(ocddl_eddlfm)                                # 0216
!include $(ocddl_eddlmm)                                # 0217

```

```

!ENDIF # if iep is on                                    # 0267

```

```

#-----                                                # 0269
# COR                                                    -   # 0270
#-----                                                # 0271
!IF $(cor_on) = TRUE |THEN|                               # 0272
#                                                         # 0273
#   Global Info                                           # 0274
#                                                         # 0275
#                                                         # 0276
!include $(ocsrc_corfm)                                   # 0277
!include $(ocsrc_cormm)                                   # 0278
#                                                         # 0279
#-----                                                # 0280
#   Standard Core Information                             -   # 0281
#-----                                                # 0282
#                                                         # 0283
#                                                         # 0284
#   AFM                                                   # 0285
#                                                         # 0286
!include $(ocafm_cafmfmm)                                # 0287
!include $(ocafm_cafmmm)                                # 0288
#                                                         # 0289
#                                                         # 0290
#   DDL                                                   # 0291
#                                                         # 0292
!include $(ocddl_cddlfm)                                  # 0293
!include $(ocddl_cddlmm)                                  # 0294

```

```

#-----                                                # 0327
#   Optional Core Information                             -   # 0328
#-----                                                # 0329
#                                                         # 0330
#   Account                                               # 0331
#                                                         # 0332
#                                                         # 0333
!IF $(acct_on) = TRUE |THEN|                               # 0334
!include $(ocacct_acctfm)                                # 0335
!include $(ocacct_acctmm)                                # 0336
!ENDIF                                                    # 0337
#                                                         # 0338

```

#	# 0339
# ACPC	# 0340
#	# 0341
!IF \$(acpc_on) = TRUE THEN	# 0342
!include \$(ocacpc_acpcfm)	# 0343
!include \$(ocacpc_acpcmm)	# 0344
!ENDIF	# 0345
	# 0346
#	# 0347
# ACU	# 0348
#	# 0349
!IF \$(acu_on) = TRUE THEN	# 0350
!include \$(ocacu_acufm)	# 0351
!include \$(ocacu_acumm)	# 0352
!ENDIF	# 0353
	# 0354
#	# 0355
# Telebanking/Billpay	# 0356
# Note: Billpay requires Telebanking; but Telebanking does	# 0357
# NOT require Billpay. Therefore, BP is included	# 0358
# within the TB condition.	# 0359
#	# 0360
!IF \$(tb_on) = TRUE THEN	# 0361
!include \$(tbsrc_tbfm)	# 0362
!include \$(tbsrc_tmmm)	# 0363
	# 0364
!include \$(tbafm_tbfmfm)	# 0365
!include \$(tbafm_tbfmmm)	# 0366
	# 0367
!include \$(tbrpt_tbrptfm)	# 0368
!include \$(tbrpt_tbrptmm)	# 0369
	# 0370
	# 0371
!ENDIF	# 0372

!ENDIF # if cor is on	# 0552
-----------------------	--------

Subsystem Macro Files

The subsystem macro files are located directly beneath the object group files. The three subsystem level configuration macro files (i.e., KNLFM, CORFM, IEPFM) contain compiler option macros that are passed to the various compilers and can contain any options supported by the compiler. The subsystem level macro files

use the compiler options contained in the BASEFM File as default values. You can set compiler options in the CUSTMACS File to override the standard values contained in the BASEFM File.

The three subsystem level file definition macro files (i.e., KNLMM, CORMM, IEPMM) contain global file definitions for each subsystem (i.e., OKTG, OCTG, and OETG).

Example Subsystem Configuration Macro File

The BASE24 ITS Kernel Configuration Macro File (KNLFM) is shown below in its entirety.

```

#-----# 0087
# -# 0088
# Configuration Macros -# 0089
# -# 0090
#-----# 0091
# 0092
# The ANS Object, which performs the functions of the Space# 0093
# Object, allows for a Large Memory Model or a Small Memory# 0094
# Model. The Large Memory Model requires the use of flat# 0095
# segments. Flat segments require a D30 operating system and# 0096
# a TNS/R machine.# 0097
# # 0098
# $(TRUE) - Compile objects using the Large Memory Model# 0099
# $(FALSE) - Compile objects using the Small Memory Model# 0100
# # 0101
knl_lrg_mem_mdl_on = $(TRUE)# 0102
# 0103
# This macro definition is passed to the check_completion# 0104
# macro and used to allow warnings as a successful completion.# 0105
# $(TRUE) - Warnings are considered a successful completion.# 0106
# $(FALSE) - Warnings are considered an unsuccessful completion.# 0107
# # 0108
knl_ddl_warnings_allowed = $(base_ddl_warnings_allowed)# 0109
# 0110
# DDL Allowed Warnings Text# 0111
# # 0112
knl_ddl_warning_text = $(base_ddl_warning_text)# 0113
knl_ddl_alwd_warnings = $(base_ddl_alwd_warnings)# 0114
# 0115
# TAL options# 0116
# # 0117
knl_tal_opts = $(base_tal_opts)# 0118
# 0119

```

```
# SCOBOL options                                # 0120
#                                                # 0121
knl_scobolx_opts      = $(base_scobolx_opts)    # 0122
#                                                # 0123
# COBOL options                                # 0124
#                                                # 0125
knl_cobol_opts        = $(base_cobol_opts)      # 0126
#                                                # 0127
# $(TRUE) - If you wish to SCOBOL compile Kernel info for 6520 # 0128
# $(FALSE) - If you don't want to SCOBOL compile Kernel info for 6520 # 0129
#                                                # 0130
knl_t6520_on          = $(base_t6520_on)        # 0131
#                                                # 0132
# $(TRUE) - If you wish to SCOBOL compile Kernel info for 3270 # 0133
# $(FALSE) - If you don't want to SCOBOL compile Kernel info for 3270 # 0134
#                                                # 0135
knl_t3270_on          = $(base_t3270_on)        # 0136
#                                                # 0137
# DDL Options                                # 0138
#                                                # 0139
knl_ddl_opts          = $(base_ddl_opts)        # 0140
#                                                # 0141
# Compile Options                            # 0142
#   For example: PRI, SWAP, CPU                # 0143
#   This string must start with a "," and must not end with one # 0144
#   For example: ,PRI 100, SWAP \ACI2.$ABC, CPU 7 # 0145
#                                                # 0146
# IF nothing is supplied, these options will not be set by MAKE. # 0147
#                                                # 0148
knl_compile_opts      = $(base_compile_opts)    # 0149
#                                                # 0150
# Secure Option                              # 0151
# Determines the security of created Targets, e.g., anstto.      # 0152
# If the keyword "default" is supplied, the security will not    # 0153
# be changed and will be the security of the user running the    # 0154
# Make.                                                           # 0155
#                                                # 0156
knl_secure_opt         = $(base_secure_opt)     # 0157
#                                                # 0158
# This flag indicates if the Spooler output should be purged if  # 0159
# the particular step completes successfully. Set this flag to   # 0160
# $(FALSE) if you do NOT want the Spooler output to be purged.   # 0161
# Note that regardless of the setting of this flag, if a build   # 0162
# step fails then the Spooler job will be left intact.           # 0163
#                                                                    # 0164
knl_purge_spooler_on = $(base_purge_spooler_on) # 0165
#                                                                    # 0166
# This macro definition is used to accelerate an object or not.  # 0167
#   $(TRUE) - Accelerate the object                      # 0168
#   $(FALSE) - Do NOT Accelerate the object              # 0169
#                                                         # 0170
knl_axcel_on          = $(base_axcel_on)        # 0171
```

```

# Additional Binder Steps                                # 0172
#   For example: SET HIGHPIN ON                          # 0173
# This string must be a semicolon separated list        # 0174
#   For example: SET HIGHPIN ON ; SHOW SELECT            # 0175
#                                                         # 0176
#                                                         # 0177
knl_bind_steps      =                                     # 0178

```

Example Subsystem File Definition Macro File

The BASE24 ITS Kernel File Definition Macro File (KNLMM) is shown below in its entirety.

```

#-----                                                # 0001
#   KNL Macros                                           -      # 0002
#-----                                                # 0003
oksrc_oktg      = \R.$EARL.ok60src.oktg                 # 0004

```

Object Macro Files

The object macro files are located directly beneath the subsystem level files. The BASE24 ITS object level configuration macro files (e.g., KISMTFM and CSIFM) contain object level compiler option macros. By default, the object level macros refer back to the subsystem level macros, which allows all the objects of a subsystem to use the same options. Optionally, object level macros can contain settings that are customized for the object.

The BASE24 ITS object level file definition macro files (e.g., KISMTMM and CSIMM) contain definitions for other files used by the module.

Example Object Configuration Macro File

The BASE24 ITS Integrated Server Module Type Configuration Macro File (KISMTFM), which is shown below in its entirety.

```

#-----# 0046
# -# 0047
# Configuration Macros -# 0048
# -# 0049
#-----# 0050
# This macro definition is passed to the check_completion# 0051
# macro and used to allow warnings as a successful completion.# 0052
# $(TRUE) - Warnings are considered a successful completion.# 0053
# $(FALSE) - Warnings are considered an unsuccessful completion.# 0054
# # 0055
okismt_ddl_warnings_allowed = $(knl_ddl_warnings_allowed)# 0056
# # 0057
# # 0058
# DDL Allowed Warnings Text# 0059
# # 0060
okismt_ddl_warning_text = $(knl_ddl_warning_text)# 0061
okismt_ddl_alwd_warnings = $(knl_ddl_alwd_warnings)# 0062
# # 0063
# # 0064
# DDL Options# 0065
# # 0066
okismt_ddl_opts = $(knl_ddl_opts)# 0067
# # 0068
# # 0069
# # 0070
# Compile Options# 0071
# For example: PRI, SWAP, CPU# 0072
# This string must start with a "," and must not end with one# 0073
# For example: ,PRI 100, SWAP \ACI2.$ABC, CPU 7# 0074
# # 0075
# IF nothing is supplied, these options will not be set by MAKE.# 0076
# # 0077
okismt_compile_opts = $(knl_compile_opts)# 0078
# # 0079
# # 0080
# Secure Option# 0081
# Determines the security of created Targets, e.g., ismtet.# 0082
# If the keyword "default" is supplied, the security will not# 0083
# be changed and will be the security of the user running the# 0084
# Make.# 0085
# # 0086
okismt_secure_opt = $(knl_secure_opt)# 0087
# # 0088

```

```

#                                                    # 0089
# This flag indicates if the Spooler should be cleaned up if      # 0090
# the particular step completes successfully. Set this flag to   # 0091
# $(FALSE) if you do NOT want the Spooler output to be purged.   # 0092
# Note that regardless of the setting of this flag, if a build   # 0093
# step fails then the Spooler job will be left intact.           # 0094
#                                                    # 0095
okismt_purge_spooler_on = $(knl_purge_spooler_on)                # 0096

```

Example Object File Definition Macro File

The BASE24 ITS object level file definition macro files (e.g., KISMTMM and CSIMM) contain definitions for other files used by the module. All of the definitions contained in the BASE24 ITS Integrated Server Module Type File Definition Macro File (KISMTMM) are shown below.

```

#-----                                                    # 0044
#                                                    -      # 0045
# File Macros (defines)                                     -      # 0046
#                                                    -      # 0047
#-----                                                    # 0048
okismt_dict          = \r.$earl.ok60ismt.dictver            # 0049
                                                              # 0050
okismt_kismtm         = \r.$earl.ok60ismt.kismtm            # 0051
okismt_kismtm         = \r.$earl.ok60ismt.kismtm            # 0052
                                                              # 0053
okismt_kismtea        = \r.$earl.ok60ismt.kismtea           # 0054
okismt_kismtes        = \r.$earl.ok60ismt.kismtes           # 0055
okismt_kismtet        = \r.$earl.ok60ismt.kismtet           # 0056
okismt_kismtps        = \r.$earl.ok60ismt.kismtps           # 0057
okismt_kismtts        = \r.$earl.ok60ismt.kismtts           # 0058
okismt_klbprops       = \r.$earl.ok60ismt.klbprops          # 0059
okismt_klbstaps       = \r.$earl.ok60ismt.klbstaps          # 0060
okismt_kobstaps       = \r.$earl.ok60ismt.kobstaps          # 0061

```

Makefiles

A makefile contains the commands, macros, dependency information, and inline code needed to create a target file. The makefiles, like the configuration macro files and file definition macro files, are organized into a hierarchy. Separate hierarchies exist because of differences in the way BASE24 procedural products and BASE24 ITS products are designed; and configuration information is defined earlier in the hierarchy for object-oriented products. The following table illustrates the levels of each hierarchy.

Level	BASE24 Procedural	BASE24 ITS
1	Master	Master
2	Product group	Object group
3	Product	Subsystem
4	Process	Object

Makefile names include suffixes that identify the type of information in the makefile, as shown in the following table.

M = Makefile
DM = DDL makefile used to compile DDLs
LM = Link makefile used to link BASE24 ITS objects
RM = Relational makefile used to create SQL tables and indexes
TM = TAL makefile
XM = SCOBOLX makefile

Makefile names usually match the name of the subvolume on which they are located, plus the appropriate suffix. However, variations from the subvolume name occur when one subvolume contains multiple makefiles. Some examples of macro file names are shown in the following table.

Product	Subvolume	Makefile Name	Makefile Function/Description
BASE24	BA60MAKE	B24M	Include all BASE24 procedural and BASE24 ITS makefiles
BASE24 procedural	BA60SRC	CLSCM	Include all BASE24 procedural makefiles

Product	Subvolume	Makefile Name	Makefile Function/Description
BASE24 procedural	BA60SRC	BASRCM	Include all makefiles on the BA60SRC subvolume except the CLSCM
BASE24-atm	AT60SRC	ATMM	Include all BASE24-atm makefiles
BASE24-atm	AT60SRC	ATSRCM	Include the BASE24-atm utilities makefile on the AT60SRC subvolume
NCR 50XX Device Handler	AT60N50	ATN50M	Include all NCR 50XX Device Handler makefiles
Base AFT	BA60AFT	BAAFTM	Include all base AFT makefiles
		RQCPFM	Control the creation of the CPF Requestor process
ISO Host Interface	BA60HISO	BAHISOM	Compile the base ISO Host Interface process module
		HISOM	Compile, bind, and accelerate all ISO Host Interface process modules
BASE24 ITS Core subsystem	OC60SRC	CORM	Combined makefile for the core subsystem
Core AFM	OC60AFM	CAFMM	Combined makefile for the core AFM
		RQPCDFXM	SCOBOLX makefile for the PCDF Requester process
Core EMT Build utility	OC60LINK	CEMTBLM	Link makefile for the core EMT Build utility

Product	Subvolume	Makefile Name	Makefile Function/Description
Core SQL compiler	OC60CMP	ALLTBRM	Relational makefile for the entire BASE24-telebanking product
		CACTRM	Relational makefile for the CACT
		CATTRM	Relational makefile for the CATT
		CPITRM	Relational makefile for the CPIT
		CSTTRM	Relational makefile for the CSTT
		PITRM	Relational makefile for the PIT
		TTSCRM	Relational makefile for the TTSC
Core Transaction History object	OC60HIST	HISTDM	DDL makefile for the History object
		HISTM	Combined makefile for the History object
		HISTTM	TAL makefile for the History object
		RQTHCFXM	SCOBOLX makefile for the THCF Requester process
		SVTHCFTM	TAL makefile for the THCF Server process
Kernel EMT Build utility	OK60EMTB	KEMTBM	Combined makefile for the Kernel EMT Build utility
		KEMBTBM	TAL makefile for the Kernel EMT Build utility

ALLMACS Macro

The first statement in each makefile performs a test for the presence of the ALLMACS macro, as shown below.

```

!IF $D(allmacs) = FALSE |THEN|                                # 0153
#                                                            # 0154
#   Define any macros you want to change from their standard # 0155
#   values here                                              # 0156
#                                                            # 0157
#                                                            # 0158
!include =allmacs                                             # 0159
!ENDIF                                                        # 0160

```

The commands shown above appear at the beginning of each makefile to ensure that all necessary configuration and file definition information is established before any makefile commands are executed, regardless of which makefile is used to start the ACI Make utility. The range of sequence numbers assigned to these commands varies according to the length of the history section at the start of the makefile.

The ALLMACS File contains commands that create the ALLMACS macro and incorporate the remaining files in the Make macro file hierarchy.

If the ALLMACS macro does not exist, the ACI Make utility adds any macro definitions specified between the IF statement and the INCLUDE command, then executes the INCLUDE command to incorporate the global macro files, which are designated by the ALLMACS define (a TACL file define called =allmacs). See the “Running the ACI Make Utility” topic in section 4 for the syntax of the ALLMACS define and procedures for using the ALLMACS define.

Once the ALLMACS macro has been created, commands between the IF and ENDIF statements shown above are ignored when encountered in all other makefiles. For example, if you put macros between the initial IF and ENDIF statements in a process makefile (e.g., BASE24-pos Device Handler makefile) and use a product makefile (e.g., BASE24-pos makefile) in the RUN command for the ACI Make utility, the macros you added in the BASE24-pos Device Handler makefile are ignored because the BASE24-pos makefile has already created the ALLMACS macro.

Master Makefiles

There are currently two master makefiles at the top of the Make hierarchy or tree—the B24M Makefile for standard BASE24 procedural and BASE24 ITS software, and the CSMM Makefile for customized BASE24 procedural and BASE24 ITS software. There are two master makefiles so that standard software and custom software modifications (CSMs) are always maintained separately.

This is the level normally used to build a BASE24 system.

When new software is added to the ACI Make structure, new makefiles can be included in one of the existing master makefiles or in a new master makefile, whichever is appropriate.

B24M Makefile

The B24M Makefile, the current master makefile for standard BASE24 procedural and BASE24 ITS code, is located on the BA60SRC subvolume.

The B24M Makefile incorporates the product group and object group level makefiles (CLSCM Makefile for BASE24 procedural code and ITSM Makefile for BASE24 ITS code) based on the CLASSIC_ON flag in the CUSTMACS File.

The B24M Makefile incorporates ACI channel-specific enhancements (CSEs) by testing for the presence of the CSEM Makefile, and using the INCLUDE command to process the CSEM Makefile if it is present. See the “Channel-Specific Enhancements” topic in this section for more detailed information about using CSE macros to override the CUSTMACS File settings.

The current B24M Makefile is shown below.

```
!IF $D(allmacs) = FALSE |THEN|                                # 0061
#                                                            # 0062
#   Define any macros you want to change from their standard # 0063
#   values here                                              # 0064
#                                                            # 0065
#include =allmacs                                             # 0066
!ENDIF                                                       # 0067
                                                            # 0068
```



```
#----- # 0069
# MAKE the Classic level # 0070
#----- # 0071
!IF $(CLASSIC_ON) = TRUE |THEN| # 0072
    !include $(basrc_clscm) # 0073
    !include $(oksrc_itsm) # 0074
!ENDIF # 0075
#----- # 0076
#----- # 0077
# MAKE the CSM files # 0078A00
#----- # 0078A01
#----- # 0078A02
#----- # 0079
#----- # 0080A00
#----- # 0080A01
!IF $(CSM_ON) = TRUE |THEN| # 0080A02
    !include $(custmake_csmm) # 0080A03
#----- # 0080A04
!ENDIF # 0082
#----- # 0083
#----- # 0084
# MAKE the CSE level # 0085
#----- # 0086
!IF $(bacse_csem) = TRUE |THEN| # 0087
    !include $(bacse_csem) # 0088
!ENDIF # 0089
```

Note: The B24M Makefile contains an INCLUDE statement for the CSMM Makefile. However, the CSMM Makefile now serves as a master makefile for custom software modifications at the same level as the B24M Makefile serves for standard BASE24 procedural and ITS software. The CSM_ON flag in the CUSTMACS File should always be set to false.

CSMM Makefile

Each customer with customized BASE24 procedural or BASE24 ITS code has a master makefile named CSMM. Thus, the CSMM Makefile incorporates all the individual CSM makefiles for a customer. There is a makefile for each customized module.

Product Group or Object Group Makefiles

The product group and object group makefiles are specifically named in the B24M Makefile. These makefiles contain only commands that incorporate the makefiles for individual products or subsystems.

CLSCM Makefile

The CLSCM Makefile contains statements that incorporate the product level makefiles for the following BASE24 procedural products:

- Base
- BASE24-atm
- BASE24-card
- BASE24-from host maintenance
- BASE24 Interchange Interfaces
- BASE24-mail
- BASE24-pos
- BASE24 PRISM Interface
- BASE24-teller

The CLSCM Makefile also contains a statement that incorporates the CLSCDDL Makefile, which is for product level DDLs.

ITSM Makefile

The ITSM Makefile contains statements that incorporate the subsystem level makefiles needed to build the kernel, integrated endpoint, and core subsystems for the BASE24 Remote Banking products.

CSMM Makefile

The B24M Makefile still contains an include statement for the CSMM Makefile. However, the CSMM Makefile now serves as a master makefile at the same level as the B24M Makefile. Thus, the flag for the CSMM Makefile should always be set to false in the CUSTMACS File.

Product or Subsystem Makefiles

The product and subsystem makefiles are located directly beneath the product group and object group makefiles, respectively.

Product Makefiles

The following files are product level makefiles for BASE24 procedural products:

- ATMM Makefile for the BASE24-atm product
- BASEM Makefile for the code shared by all BASE24 products
- CARDM Makefile for the BASE24-card product
- CLSCDDL Makefile for the DDLs used by BASE24 products
- FHMM Makefile for the BASE24-from host maintenance product
- MAILM Makefile for the BASE24-mail product
- POSM Makefile for the BASE24-pos product
- SWIM Makefile for the BASE24 Interchange Interfaces (switches)
- PRMM Makefile for the BASE24 PRISM Interface
- TLRM Makefile for the BASE24-teller product

Each of these makefiles contains statements to include makefiles for each of the various modules comprising the product.

Subsystem Makefiles

The following files are subsystem level makefiles for the BASE24 ITS subsystems:

- KNLM Makefile for the kernel subsystem
- IEPM Makefile for the integrated endpoint subsystem
- CORM Makefile for the core subsystem

Each of these makefiles contains statements to include makefiles for each of the various objects comprising the subsystem.

Process or Object Makefiles

The process and object makefiles are located directly beneath the product and subsystem makefiles, respectively.

Process Makefiles

Each process level makefile either contains the commands to create the process, or it includes the makefiles needed to create the various pieces of code that comprise the process. An example of the former is the HISOM Makefile on the BA60HISO subvolume, which creates the ISO Host Interface process. Examples of the latter are the various AFT makefiles.

Object Makefiles

The object level makefile includes the makefiles needed to create the various pieces of code that comprise the object. An example is the ACCTM Makefile on the OC60ACCT subvolume, which creates the Account object.

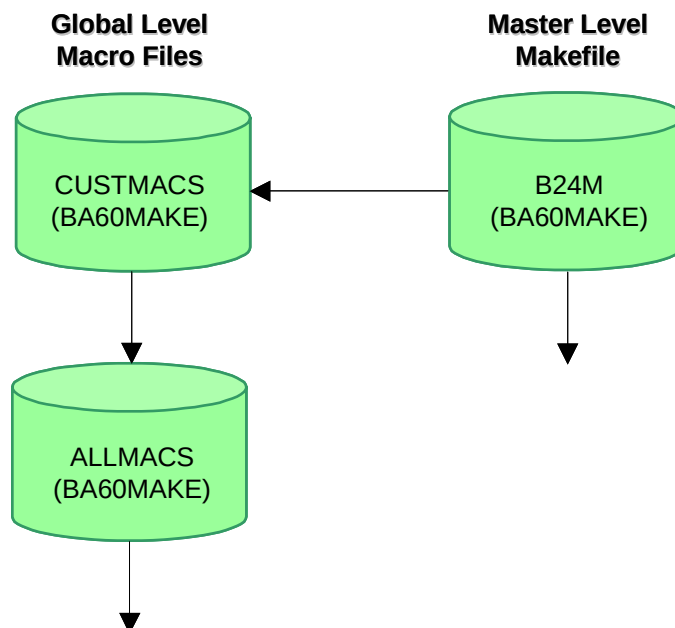
Hierarchy Charts

The following charts illustrate the Make hierarchy, including configuration macro files (file names ending with *FM*), file definition macro files (file names ending with *MM*), and makefiles (file names ending with *M*). The first level, shown below, includes the global level macro files CUSTMACS and ALLMACS, and the master level makefile B24M. Macro files and the makefile at this level are used by both the BASE24 procedural products and the BASE24 ITS products.

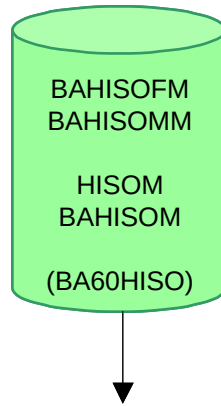
The B24M Makefile pulls in the macro files, starting with the CUSTMACS File, then proceeds with the makefiles.

Note: The hierarchy charts in this section illustrate a BASE24 configuration that does not have any customized BASE24 software. See appendix C for a hierarchy chart that includes customized BASE24 software.

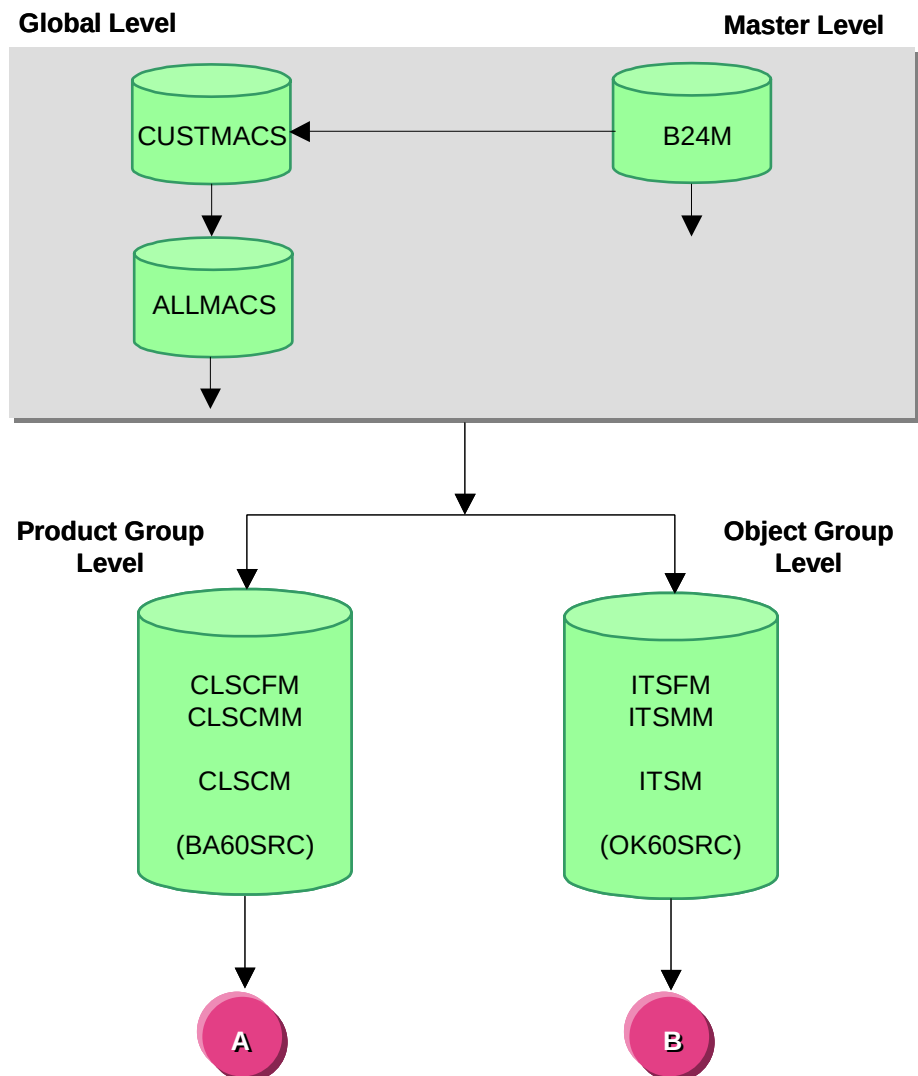
The subvolume on which each file is located is shown in parentheses below the file name. The CUSTMACS File, ALLMACS File, and B24M Makefile are located on the BA60MAKE subvolume. Macro files and makefiles are product specific, so they are located on BASE24 product subvolumes, not the Make utility subvolume of MAKEV500.



In the remaining illustrations, the macro files and makefiles are combined according to the subvolume on which they are located. The BA60HISO subvolume is shown as an example. The macro files are listed in the first group, the makefiles are listed in the second group, and the subvolume is shown last.



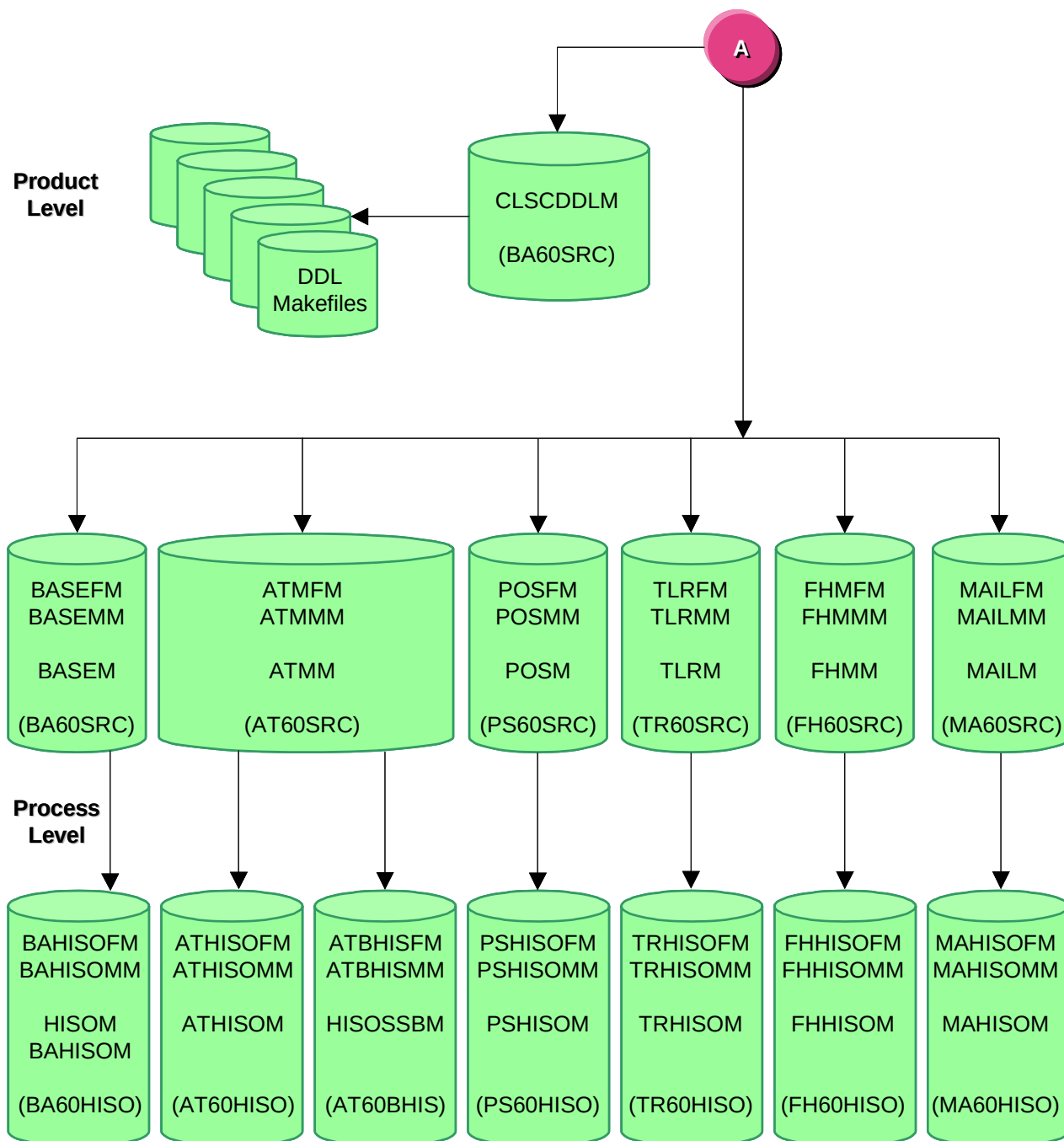
The second level, shown below, includes the BASE24 procedural product group level files and the BASE24 ITS object group level files. This is the first level at which a distinction is made between the BASE24 procedural products and the BASE24 ITS products. The following illustration also contains the global level macro files and master level makefiles to show the relationship between levels.



The CLSCFM File, CLSCMM File, and CLSCM Makefile are used to create the BASE24 procedural software. This branch is taken if the CLASSIC_ON flag is set to true at the global level. The remaining levels of this branch are shown in the following illustration. The *A* shown in this illustration matches the *A* at the top of the BASE24 procedural branch illustration.

The ITSFM File, ITSMM File, and ITSM Makefile are used to create the BASE24 ITS software. Part of this branch is taken if the CLASSIC_ON flag is set to true at the global level, but the majority of this branch is taken only if the ITS_ON flag is set to true at the global level. These flags are set in the CUSTMACS File. The *B* shown in this illustration matches the *B* at the top of the BASE24 ITS branch illustration.

The following illustration shows the product level and process level for BASE24 procedural software, using the ISO Host Interface process as an example.



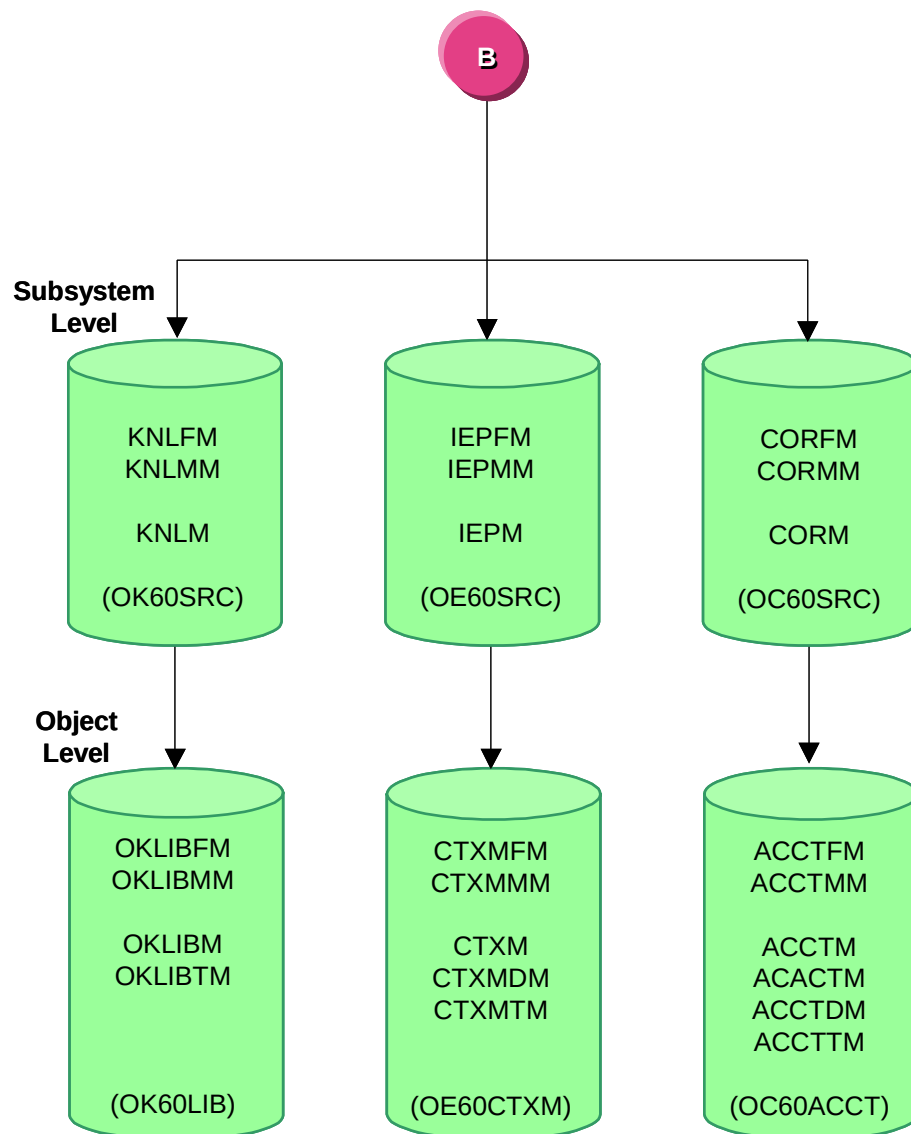
The CLSCDDL Makefile and BASEM Makefile are both product level makefiles located on the BA60SRC subvolume. However, the CLSCDDL Makefile is shown at a separate level in the illustration because the DDLs for all applicable BASE24 procedural products (indicated by the settings for the product flags in the CLSCM Makefile) are compiled first so they can be used when the other product level makefiles are executed.

The product level files for the base product are BASEFM File, BASEM Makefile, and BASEMM File. The base product files are always required for BASE24 procedural and BASE24 ITS products. The other product branches are taken depending on the settings for the product flags in the CLSCM Makefile. The products shown are BASE24-atm, BASE24-pos, BASE24-teller (TLR), BASE24-from host maintenance (FHM), and BASE24-mail. Other products include BASE24-card and BASE24 interchange interfaces.

Each product has process level files for the ISO Host Interface module. The base process level subvolume (BA60HISO) contains the BAHISOFM File, BAHISOM Makefile, and BAHISOMM File. There are two process level subvolumes for the BASE24-atm product. The AT60SRC subvolume is for the standard product, and the AT60BHIS subvolume is for the BASE24-atm Self Service Banking (SSB) add-on product.

The BA60HISO subvolume also contains the HISOM Makefile, which is used to build the entire ISO Host Interface module.

The following illustration shows the subsystem level and object level for BASE24 ITS software.



There are three subsystems: kernel, integrated endpoint, and core.

The kernel subsystem is always required. The example shows the kernel library routines. The subvolume contains the macro files and makefiles used by the BASE24 procedural product, plus a TAL makefile (OKLIBTM) that is ITS specific.

The integrated endpoint subsystem example is the Context Manager object. The subvolume contains the macro files and makefiles used by the kernel subsystem, plus a DDL makefile (CTXMDM) that is ITS specific.

The core subsystem example is the Account object. The subvolume contains the macro files and makefiles used by the Context Manager object, and an additional TAL makefile (ACACTM) for the Account Access object.

Section 3

Make Syntax

This section describes the syntax contained in the macro files and makefiles used by the ACI Make utility.

Makefile Components

A makefile provides instructions for the ACI Make utility to follow, and contains the following components:

- Macro definitions to create the environment needed by the ACI Make utility
- Parameters for determining whether to execute the build commands
- Build commands to be executed

The environment needed by the ACI Make utility must be created before any other processing can occur. The ALLMACS macro serves as a flag that indicates whether this environment has been created, and the first statement in each makefile is an IF statement that performs a test for the presence of the macro. If the ALLMACS macro does not exist, the ACI Make utility adds any macro definitions that follow the IF statement, then incorporates the global macro files, the last of which is always the ALLMACS File. The ALLMACS File contains commands that create the ALLMACS macro and incorporate the remaining files in the Make macro file hierarchy. Once the ALLMACS macro exists, no changes are made to the environment. See the ALLMACS File and ALLMACS macro descriptions in section 2 for more information.

Once the needed environment has been created, the ACI Make utility checks the parameters in the makefile and determines whether to execute the build commands in the makefile. These parameters can consist of a single IF statement or a complex combination of target files, dependency files, additional defines, and additional targets that must be evaluated before the determination can be made. All of these possibilities are presented in this section.

If determined necessary based on the parameters, the ACI Make utility executes the build commands contained in the makefile. These commands can consist of a single INCLUDE statement or a combination of commands that you can enter at a TACL prompt.

The B24M Makefile, shown below, is an example of the simplest makefile. The following table describes the portions of the B24M Makefile in the example, and gives the file sequence numbers that identify each portion of the file. The file sequence numbers appear at the right of each line in the file. For improved readability, lines separate the portions.

Portion	Sequence Numbers
Check whether the ALLMACS macro has been defined. If the macro has not been defined, create the Make environment.	62–68
Check the CLASSIC_ON flag and include the CLSCM Makefile and ITSM Makefile, if necessary.	70–76
Check the CSM_ON flag and include the CSMM Makefile, if necessary. Note: This flag should always be set to false because the CSMM Makefile is not run from within the B24M Makefile.	78–83
Check whether the customer-specific enhancements macro (BACSE_CSEM) has been defined. If this macro has been defined, include the CSEM Makefile.	85–90

The following makefile example also shows the use of an exclamation point (!) immediately preceding each command (e.g., IF, ENDIF, and INCLUDE), and a pound character (#) in the first column of each comment line.

```

!IF $D(allmacs) = FALSE |THEN|                                # 0061
#                                                             # 0062
#   Define any macros you want to change from their standard # 0063
#   values here                                              # 0064
#                                                             # 0065
#include =allmacs                                             # 0066
!ENDIF                                                       # 0067
#                                                             # 0068
#-----                                                    # 0069
#   MAKE the Classic level                                  # 0070
#-----                                                    # 0071
!IF $(CLASSIC_ON) = TRUE |THEN|                              # 0072
    !include $(basrc_clscm)                                  # 0073
    !include $(oksrc_itsm)                                    # 0074
!ENDIF                                                       # 0075
#                                                             # 0076

```

```
#----- # 0077
#  MAKE the CSM files # 0078A01
#----- # 0079
!IF $(CSM_ON) = TRUE |THEN| # 0080A02
    !include $(custmake_csmm) # 0080A03
!ENDIF # 0082
#----- # 0083
#  MAKE the CSE level # 0084
#----- # 0085
!IF $(bacse_csem) = TRUE |THEN| # 0086
    !include $(bacse_csem) # 0087
!ENDIF # 0088
#----- # 0089
```

The remainder of this section presents the syntax of makefiles that have more complex parameters and build commands.

Target Files and Dependency Files

The parameters used most often for determining whether to execute the build commands are called target specifications, which identify specific target files and dependency files. Target specifications can also contain additional defines and targets, which are presented later in this section.

A target is a file to be created or updated by the ACI Make utility. A dependency is a file from which the target file is created or updated, and includes source code and DDLs.

The ACI Make utility executes the build commands associated with a target specification in the makefile if the target file does not exist, if any of the dependency files is newer than the target file, or if the FORCE option is specified when the utility is started.

In addition to the dependency files identified in makefiles, you can use either or both of the following command line options to create additional dependencies.

- The `MAKE_FILE_DEP_CHK` option ensures that a target file is rebuilt in situations where the makefile that defines the target is revised, even if none of the dependency files specified in that makefile are revised. The makefile is considered a dependency for every target defined in the makefile.
- The `TIMESTAMP_FILE` option identifies a file from which the ACI Make utility obtains timestamp information to determine whether a target file must be built or rebuilt. This file is considered a dependency for every target file. The ACI Make utility builds or rebuilds every target file that has an older timestamp than the one on the file identified by this option.

Target Specification Syntax

The following examples show the syntax of the same target specification.

```
file1 : file2 file3 file4
```

```
file1 : file2 \  
       file3 \  
       file4
```

The colon (:) separates the target files and dependency files. File 1 is the target file, indicated by the FILE1 entry being on the left side of the colon. Files 2, 3, and 4 are dependency files, indicated by entries for all three files being on the right side of the colon. The second example uses the backslash (\) character at the end of the

first two lines to indicate each line is continued. The entries for files 3 and 4 are aligned with the entry for file 2 to improve readability. The second example is the preferred format.

The target specification is followed immediately by the build commands used to build the target file. The ACI Make utility executes the build commands to create or replace file 1 in any of the following situations:

- File 1 does not exist.
- File 2, file 3, or file 4 is newer than file 1.
- The utility is started with the `FORCE` option.
- The utility is started with the `MAKE_FILE_DEP_CHK` option, and the makefile defining file 1 is newer than file 1.
- The utility is started with the `TIMESTAMP_FILE` option, and the timestamp of the file specified in the `TIMESTAMP_FILE` option is newer than file 1.

Timestamps

The ACI Make utility uses the modification timestamps assigned to the target files and dependency files by the Tandem NonStop Kernel operating system when determining whether the target file needs to be created or updated.

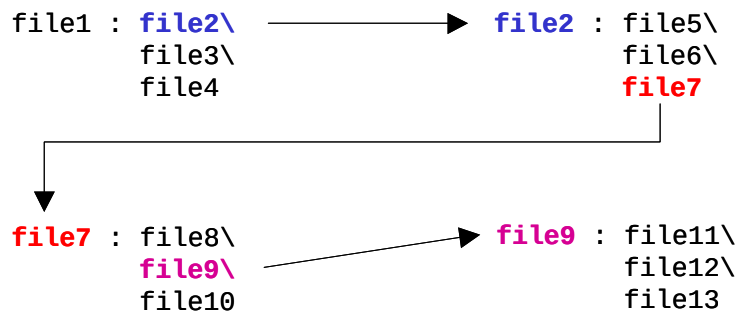
The ACI Make utility uses a special procedure to determine the modification timestamp for Pathway requester processes. The Tandem NonStop Kernel operating system sets the modification timestamp for these processes on a Screen COBOL object library (POBJ) file, instead of the individual requester files. Makefiles for Pathway requester processes create a pseudo target file for each compiled requester. The pseudo target file can be empty or contain a copy of the requester process source code, depending on whether the `SYMBOLS` option is included in the `TAL Options` macro (e.g., `BASE_TAL_OPTS` in the `CUSTOMACS File` or `BASEFM File`). The requester process can be debugged with the Tandem `INSPECT` utility if the pseudo target file contains the source code.

The ACI Make utility uses the modification timestamps from these pseudo target files when determining whether an object file needs to be rebuilt. Without these pseudo target files, the ACI Make utility compiles the requester processes every time it is run, regardless of whether their dependency files have changed, because it cannot find a modification timestamp for the target file.

Dependency File as a Target File

A file identified as a dependency in one target specification can also be identified as a target in another target specification. This relationship of a dependency also being a target can occur any number of times, effectively creating a hierarchy of files that must be reviewed to determine whether a target file must be rebuilt.

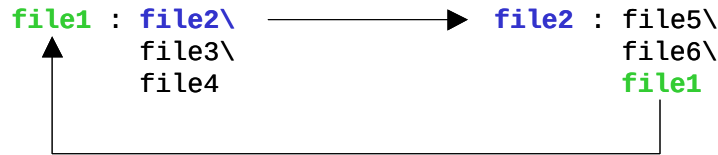
For example, consider the four target specifications shown below.



In this example, file 2, file 7, and file 9 are identified as dependencies in one target specification at the same time they are identified as target files in other target specifications. None of the remaining dependency files are identified as targets in other target specifications. The ACI Make utility checks the modification timestamps of all 13 files when determining whether to rebuild file 1, as follows:

1. Rebuild file 9 if its modification timestamp is older than the modification timestamps of file 11, file 12, or file 13.
2. Rebuild file 7 if its modification timestamp is older than the modification timestamps of file 8, file 9 or file 10. The file 9 modification timestamp used in this comparison is the updated modification timestamp if file 9 is rebuilt in step 1.
3. Rebuild file 2 if its modification timestamp is older than the modification timestamps of file 5, file 6, or file 7. The file 7 modification timestamp used in this comparison is the updated modification timestamp if file7 is rebuilt in step 2.
4. Rebuild file 1 if its modification timestamp is older than the modification timestamps of file 2, file 3, or file 4. The file 2 modification timestamp used in this comparison is the updated modification timestamp if file 2 is rebuilt in step 3.

Two target file specifications cannot use the target file from one specification as a dependency file in the other specification. This situation is known as a circular file specification, as shown below.



This condition is not permitted because it creates an endless loop for the ACI Make utility. If an endless loop exists, the ACI Make utility displays an error message and aborts.

Source and Object Files

The following example uses source and object files in the target specifications and adds build commands. The build commands are shown in *italics* because they are just summaries of the actual commands involved.

```
file1 : file2o \
        file3o \
        file4o
BIND commands to create file1o
file2o : file2s
TAL commands to create file2o
file3o : file3s
TAL commands to create file3o
file4o : file4s
TAL commands to create file4o
```

In this example, the final target file is named file1 and its dependency files are named file2o, file3o, and file4o because all of them are object files. File1 is created by using the Tandem Binder program to combine file2o, file3o, and file4o. These dependency object files are created by compiling their corresponding TAL source files, named file2s, file3s, and file4s, respectively. This is accomplished by defining a target specification for each object file with its source file as the dependency. For example, file3s is the dependency for file3o.

If file3s changes (e.g., you make a change to the file and save it), file3s has a newer modification timestamp than file3o. When the ACI Make utility is run with the makefile shown above, it first checks the dependency files for file1. Next, it checks the dependency files for file2o, file3o, and file4o. Since the modification timestamp for file3s is newer than the modification timestamp for file3o, the ACI

Make utility executes the commands specified to create an updated file3o. The updated file3o now has a newer modification timestamp than the modification timestamp for the file1. Therefore, the ACI Make utility executes the BIND commands to recreate file1.

If both file2s and file3s are modified, the ACI Make utility recompiles each one before performing the BIND commands.

BASE24 Procedural Example

The following is an example of multiple target specifications for the ISO Host Interface process. The build commands are shown in *italics* because they are just summaries of the actual commands involved.

```
HISO :    BAHIS00 \
        ATHIS00 \
        PSHIS00
```

BIND commands to create the HISO File

```
BAHIS00 : BAHIS0S \
```

Other dependency files and TAL commands to create the BAHIS00 File

```
ATHIS00 : ATHIS0S \
```

Other dependency files and TAL commands to create the ATHIS00 File

```
PSHIS00 : PSHIS0S \
```

Other dependency files and TAL commands to create the PSHIS00 File

BASE24 ITS Example

The following is an example of a target specification for the Integrated Endpoint process. The build commands are shown in *italics* because they are just summaries of the actual commands involved.

```
$(oclink_cieplo) : $(oklib_oklibto) \
                  $(oelink_ieplo) \
                  $(oclib_oclibto) \
                  !IF $(vru_on) = $(TRUE) |THEN| \
                    $(ocvru_vrueto) \
                  !ENDIF \
                  !IF $(ebms_on) = $(TRUE) |THEN| \
                    $(ocebms_ebmeto) \
                  !ENDIF \
                  !IF $(imgr_on) = $(TRUE) |THEN| \
                    $(ocimgr_imgrto) \
                  !ENDIF \
```

```
!IF $(imt_on) = $(TRUE) |THEN| \
    $(ocimt_imteto) \
!ENDIF
```

TAL commands to create the CIEPLO File

Dependency Lists

Target specifications usually appear in a makefile in the following order.

1. Tandem system
2. ACI and XPNET
3. Product
4. Module

However, the order in which target specifications appear in makefiles does not affect the order the ACI Make utility executes them. The execution order is based on the dependency file hierarchy for all target files involved. In the BASE24 procedural example above, the ACI Make utility executes the build commands for the BAHISOO, ATHISOO, and PSHISOO files before it executes the build commands for the HISO.

As presented in section 2, a specific order exists in which macro files and makefiles are added to a system. For BASE24 procedural and BASE24 ITS products, the CUSTMACS File is at the top of the macro file hierarchy and the B24M Makefile is at the top of the makefile hierarchy. The ACI Make utility uses both hierarchies in processing. The macro file hierarchy provides the configuration information and file definitions that must be established for use in the makefiles. The makefile hierarchy provides the target specifications needed to develop a dependency list.

The ACI Make utility reads the macro files and makefiles into memory, combining all of the target specifications from the makefiles into a single dependency list. It arranges all of the target specifications in the dependency list so that each target file can be created or updated only once during the ACI Make utility run, then uses the modification timestamps for each target file and dependency file to identify which target files must be created or updated.

After determining which target files must be created or updated, the ACI Make utility uses the makefiles already in memory to obtain the commands, macros, and inline code needed to create or update the target files.

Include Statements

The ACI Make utility supports the ability to include other files, similar to the SOURCE directive in TAL. The INCLUDE statement can be used in Make macro files and makefiles. The entire file specified in the INCLUDE statement is incorporated into the macro file or makefile immediately following the INCLUDE statement. Files can be nested to a maximum depth of ten levels.

Syntax

`!include file-name`

file-name — The name of the Make macro file or makefile to be incorporated into the macro file or makefile. This name is usually expressed as a Make macro (e.g., \$(basrc_clscm)). However, it can also be expressed as a TACL define (i.e., =ALLMACS appears in makefiles) or a disk file name (e.g., \system.\$volume.ba60make.allmacs appears in the CUSTMACS File).

Note: All files specified in an INCLUDE statement must exist, except when the ACI Make utility is building an EQUIV File. See the “Maintaining File Name References” topic in section 4 for additional information about building an EQUIV File.

Conditional Text

Commands that are executed only when a specified condition is met are called conditional text. The use of conditional text by the ACI Make utility can simplify BASE24 installation because the same macro file or makefile can be used with several different product configurations. For example, you can set flags to identify which products, features, or options are to be installed.

The following example illustrates conditional text used in target specifications from the makefile for the ISO Host Interface process. If the \$(ATM_ON) flag is set to true, the makefile includes the ATHISOO File as a dependency in the HISO target specification and includes the ATHISOO target specification. If the \$(POS_ON) flag is set to true, the makefile includes the PSHISOO File as a dependency in the HISO target specification and includes the PSHISOO target specification. The build commands are shown in *italics* because they are just summaries of the actual commands involved. If the \$(ATM_ON) macro is set to false instead of true, the IF statements that appear in bold in the following example fail, and the statements for the ATHISOO File as a dependency in the HISO target specification and for the ATHISOO target specification are ignored because they are between the IF statement and its corresponding ENDIF statement. This illustration also shows the proper syntax for the IF and ENDIF statements.

```
HISO    : BAHISOO                                \
    !IF $(ATM_ON) = TRUE |THEN|                  \
        ATHISOO                                \
    !ENDIF                                        \
    !IF $(POS_ON) = TRUE |THEN|                  \
        POSHISOO                               \
    !ENDIF                                        \
Other dependency files and BIND commands to create the HISO File

BAHISOO : BAHISOS                                \
Other dependency files and TAL commands to create the BAHISOO File

!IF$(ATM_ON)=TRUE |THEN|
    ATHISOO : ATHISOS                            \
Other dependency files and TAL commands to create the ATHISOO File
!ENDIF

!IF$(POS_ON)=TRUE |THEN|
    PSHISOO : PSHISOS                            \
Other dependency files and TAL commands to create the PSHISOO File
!ENDIF
```


Comments

Comments provide additional information to the reader of a macro file or makefile. The ACI Make utility does not use this information. The following table describes the various types of comments used in a makefile, and gives the file sequence numbers that identify each type in the AUTHM Makefile example on the following page. The file sequence numbers appear at the right of each line in the file. For readability, the #, &, and \ characters appear in bold in the example.

Comment	Sequence Numbers
A # character in the first column. Any text entered on this line is a comment because of the # character in the first column. Additional # characters are ignored.	221–223
A # character that is not in the first column and not preceded by a space. The text immediately following this # character is not a comment.	287
A # character preceded by a space. The file sequence numbers are comments. On lines other than inline input lines (lines that start with an = character in the first column), the space following the # character with the file sequence number is used to effectively handle a mix of lines with and without inline input commands.	221–228 283
A # character preceded and followed by a space. The file sequence numbers are comments. On inline input lines (lines that start with the = character in the first column), a space is required before and after the # character.	284–290
A # character that follows a Make continuation character (\). The comments that follow the \ character on the line are ignored.	225–227
A # character that follows a TACL continuation character (&). The comments that follow the & character on the line are ignored.	286–289

```

#----- # 0221
# AUTH Library Object # 0222
#----- # 0223
# 0224
$(atauth_authlibo) : [ \ # 0225
    $(basrc_baddltal) \ # 0226
    $(atsrc_atddltal) ] $(aciutils_utildefs) \ # 0227
    $(aciutils_utilexts) # 0228
$(TACL)/INLINE/ # 0283
= $(tacl_init) $(make_tacl_macros) # 0284
# 0285
= $(TAL)/in $(atauth_authlibs), & # 0286
= out $s.#atauthl.t & # 0287
= $(atauth_compile_opts)/ & # 0288
= $(atauth_authlibo); & # 0289
= $(atauth_tal_opts) # 0290

```

Directives

You can use directives in makefiles to provide additional operator information when certain error conditions occur. Two directive commands are available, depending on whether you want processing to continue or terminate when the condition occurs.

Warning

The warning directive causes the ACI Make utility to display error message 2340 with the text that you provide, if any, then continue processing. The utility also displays the name and line number of the makefile where the error is encountered.

Syntax

`!WARNING [ text ]`

text — Specifies the text to be displayed with error message 2340.

Example

```
100      !if $(highpin_on) = TRUE |THEN|
101      !if $(toc_c30_on) = TRUE |THEN|
102      !warning HIGHPIN IS NOT SUPPORTED IN TOS C30
103      !endif
104      !endif
```

In this example, the ACI Make utility displays the following error message if HIGHPIN_ON and TOS_C30_ON are both set to true.

```
MAKE ERROR: 2340
-----
!
! WARNING
!
Text:  HIGHPIN IS NOT SUPPORTED IN TOS C30
```

Suggestions:

```
-----
A WARNING directive was encountered.
MAKEFILE: filename, Line number: 103
Review the MAKE file to determine the purpose of this warning.
```

If no text is provided with the warning directive in the makefile, error message 2340 displays WARNING (no text) instead of WARNING, and the Text line is omitted from the display.

Abend

The `abend` directive causes the ACI Make utility to display error message 2350 with the exit code and text that you provide, if any, plus the name and line number of the makefile containing the `abend` directive, then terminate processing.

Syntax

`!ABEND [ exit-code ] [ text ]`

exit-code — Specifies the completion code the ACI Make utility reports to the TACL process. Valid values are positive numbers and zero. The default value is 2.

text — Specifies the text to be displayed with error message 2350.

Example

```
100      !if $(atm_on) = TRUE |THEN|
101          !if $(atsrc_atmm) = FALSE |THEN|
102              !abend 2 ATM_ON IS TRUE, BUT $(atsrc_atmm) DOESN'T EXIST
103          !endif
104      !endif
```

In this example, the ACI Make utility displays the following error message if `ATM_ON` is set to true and the file defined by `ATSRC_ATMM` does not exist.

```
MAKE ERROR: 2350
-----
!
! ABEND 2
!
Text:  ATM_ON IS TRUE, BUT $volume.subvolume.ATMM DOESN'T EXIST

Suggestions:
-----
An ABEND directive was encountered.
MAKEFILE: filename, Line number: 102
Review the MAKE file to determine the purpose of this abend.
```

The error message contains the expanded name of the file defined by `$(ATSRC_ATMM)`. The error message also contains the name and line number of the makefile containing the `abend` directive. After displaying the error message, the ACI Make utility terminates processing and passes the completion code to the TACL process. If no text is provided with the `abend` directive in the makefile, error message 2350 displays `ABEND 2 (no text)` instead of `ABEND 2`, and the Text line is omitted from the display.

Make Macros

The ACI Make utility uses a macro facility that is similar to that contained in the TAL and C programming languages.

Make macros typically are defined in one of the macro files described in section 2 before they are used in makefiles or in other macro files. The following options are also available for use in test environments.

- Macros can be placed in the makefile that is used to start the ACI Make utility, just ahead of the INCLUDE statement for the Make macro file hierarchy, because the first definition assigned to a macro takes precedence over any later definitions given to the same macro.
- The TEST option or TESTCHK option can be used when running the ACI Make utility. When the current subvolume contains files that are defined by macros, these options cause the ACI Make utility to use the files on the current subvolume instead of the files originally defined by the macros.
- The EXCEPT_FILE and EXCEPT_LOC options can be used to create a new global level macro file that takes precedence over any later definitions given to the same macros.
- Fully qualified file names can be used instead of file definition macros in makefiles. Prior to release 4.0 of the Make utilities, this option could not be used in target specifications.

Note: The use of file names instead of file definition macros in makefiles is discouraged at any time, and is not permitted by ACI developers once testing is complete.

Macro Definition

The format of a macro definition is as follows.

Note: The syntax examples in this subsection use braces ({}) to identify optional items because parentheses () are part of the macro definition and use commands.

Syntax

macro-name = replacement-text

macro-name = The name assigned to the macro. Macro names can contain from 1 to 31 alphanumeric, underscore (_), and circumflex (^) characters. When appearing in a dependency list, file name macros are limited to 23 characters and must start with the letter A through Z.

replacement-text = Text inserted into the input stream when the macro is expanded. Multiple line replacement text (replacement text that consists of multiple source lines) is possible. Each line of replacement text in the macro definition except the last is terminated with a continuation character (\).

Note: If the replacement text is a file name, the file name must be fully qualified, beginning with either the system name (e.g., \b24) or the volume location (e.g., \$data).

Macro Use

A macro can be used anytime after it has been defined. The syntax for using a macro is as follows:

Syntax

\$(macro-name)

macro-name = The name of a previously defined macro.

Predefined Macros

Below is a list of the predefined macros available with the ACI Make utility.

\$C(*file-name*) — Is replaced by the file code of the file identified by the *file-name* variable. For example, an object file has a file code of 100 and an edit file has a file code of 101.

\$D(*macro-name*) — Is replaced by true or false to indicate whether the macro identified by the *macro-name* variable has been defined. The replacement value is true if the specified macro has been defined or false if the specified macro has not been defined.

\$E(*file-name*) — Is replaced by the size of the file identified by the *file-name* variable. The replacement value is the file size expressed in bytes if the specified file exists or -1 if the specified file does not exist.

\$F(*file-name*) — Is replaced by true or false to indicate whether the file identified by the *file-name* variable exists. The replacement value is true if the file exists or false if the file does not exist. This macro also returns false if the *file-name* variable is missing or is an invalid file name.

\$(CREATENAME) — Is replaced by a system-generated process pair descriptor name (e.g., \$X123).

\$(DATE) — Is replaced by the current date, in ddmmmyy format

\$(FALSE) — Is replaced by the value 0.

\$(HOMETERM) — Is replaced by the name of the home terminal used by the Make utility.

\$(SUBVOL) — Is replaced by the current subvolume name.

\$(SYS) — Is replaced by the current default system name.

\$(TARGET) — Is replaced by the name of the target file.

\$(TIMESTAMP) — Is replaced by the current time, in ddmmmyyyy hh:mm:ss.tt format.

\$(TRUE) — Is replaced by the value 1.

\$(VOL) — Is replaced by the current volume name.

TACL Defines

The ACI Make utility dynamically adds Tandem NonStop Kernel defines to the TACL environment for use as variables when a target is built. These TACL defines are for file macros referenced in the target specification for the target being built. The ACI Make utility removes these defines from the TACL environment after the execution statements have been completed, unless the utility is started with the NO_DELETE_DEFINES option. With this option, the ACI Make utility accepts any TACL defines in effect when it is started and does not delete the TACL defines that it adds.

TACL defines use the names established by Make file definition macros. See section 2 for the naming convention used for file definition macros. For example, the following target specification is one of two contained in the BAHISOM.

```
$(bahiso_bahisoe) : $(bahiso_bahisos)
```

If the target file must be built, the ACI Make utility adds defines to the TACL environment as shown in the following entries from an ACI Make utility listing, executes the commands to build the target file, and removes the defines.

Target Define:

```
DEFINE ADDED: BAHISO_BAHISOE  VALUE: \R.$EARL.BA60HISO.BAHISOE
```

Dependency Define:

```
DEFINE ADDED: BAHISO_BAHISOS  VALUE: \R.$EARL.BA60HISO.BAHISOS
```

A TACL define can be added for each file named in the Make macro file hierarchy. However, the TACL defines that exist at any point during an ACI Make utility run can represent only a small subset of the total number of file macros. There are limitations in the number of defines that TACL or any other process can pass to another process. This limit is approximately 600 defines passed between processes on the same Tandem system and approximately 300 defines passed between processes on remote Tandem systems.

Additional Defines

Situations exist where the ACI Make utility must use files other than the dependency files for input to compile a target file, yet the modification timestamps of these additional defines do not need to be considered when determining whether the target file is to be compiled. Additional defines are also called additional define files or additional input files.

Typically, the product-level language targets (e.g., TAL, C, or COBOL) for DDLs are the only additional defines specified. All DDL source files for a product are compiled into a single language target for that product. For example, BASE24 has approximately 50 base DDL source files compiled into one target for TAL (BADDLTAL) and one target for COBOL (BADDLCOB). Any time even one of the 50 source files is updated, the targets must be recompiled.

A module obtains its DDL information from the language targets. However, the definitions needed by the module usually originate from only a small number of the DDL source files used to build the language targets. If the language target is considered to be a dependency, the module is recompiled every time any of the DDL source files is modified, even though the module needs to be recompiled only when DDL information that it uses is modified. If the DDL source files containing the definitions the module actually uses are considered to be dependencies, the module is recompiled only when these DDLs have changed, but the presence of the language target is not ensured.

Specifying additional defines in a target file specification addresses the need to have a file present without considering its modification timestamp. In a target file specification that uses DDLs, the DDL target is specified as an additional define and the DDL source files that contain the required definitions are specified as dependencies. This way, the ACI Make utility uses the information from the DDL target and uses the modification timestamps of the appropriate DDL source files.

The ACI Make utility adds Tandem NonStop Kernel defines for the additional defines to the TACL environment for use as variables while the commands associated with the target file specification are executed.

The syntax for specifying additional defines in a target file specification is shown on the following page. The additional defines appear following the colon that separates the target file and the dependency files, and are surrounded by brackets ([]). For readability, additional defines usually are indented from the target file.

```
target          :[                \
    addl_input_1                \
    addl_input_2    ] source_1    \
                        source_2    \
                        source_3
```

All additional defines must appear before the dependency files. If a target file specification also contains additional targets, which are discussed later in this section, the additional targets must also appear before the dependency files. However, additional defines can appear before or after the additional targets. The following example shows the additional targets before the additional defines.

```
target          :{
    addl_output_1                \
    addl_output_2    }[          \
    addl_input_1                \
    addl_input_2    ] source_1    \
                        source_2    \
                        source_3
```

The following partial listing of a target file specification from the AUTHM Makefile for the BASE24-atm product illustrates the use of additional defines.

```
$(atauth_authlibo) : [                \                                # 0225
    $(basrc_baddltal)                \                                # 0226
    $(atsrc_atddltal) ] $(aciutils_utildefs) \                            # 0227
                        $(aciutils_utilexts) \                            # 0228
                        $(atauth_authcsee) \                               # 0229
                        $(atauth_authcseg) \                               # 0230
                        $(atauth_authcses) \                               # 0231
                        \                                                    # 0232
                        $(atauth_authemvd) \                               # 0233
                        $(atauth_authg) \                                  # 0234
                        $(atauth_authlibe) \                               # 0235
                        $(atauth_authlibs) \                               # 0236
                        $(atauth_authmcud) \                               # 0237
                        $(atauth_authtbl) \                                # 0238
                        $(atddl_ddlftlf) \                                 # 0239
                        $(bacse_attkcid) \                                 # 0240
                        $(bacse_batkccve) \                                # 0241
                        $(bacse_batkcid) \                                 # 0242
                        $(bacse_ddlattkc) \                                # 0243
                        $(bacse_ddlbatkc) \                                # 0244
                        $(baddl_ddlattkn) \                                # 0245
                        $(baddl_ddlbatkn) \                                # 0246
                        $(baddl_ddlbcnst) \                                # 0247
                        $(baddl_ddlfcaf) \                                 # 0248
```

Other dependency files and TAL commands to create the AUTHLIBO File

Additional Targets

Situations exist where, as part of the build commands associated with a target file specification, the ACI Make utility must generate files other than the target file. The modification timestamps of these additional targets do not need to be considered when determining whether the build commands should be performed. Additional targets are also called additional target files or additional output files. Additional targets are used in the following situations.

- The target file is a TAL product-level language target (e.g., BADDLTAL, ATDDLTLAL). The product-level targets for other languages (e.g., BADDLCOB, BADDLFUP, ATDDLCOB, ATDDLUFUP) must be specified as additional targets so they can be created also.
- The target file is a Pathway requester process object file (e.g., RQPBFO for the PBF Requester process). The Screen COBOL object library (POBJ) file must be specified as an additional target so the Tandem Screen COBOL Utility Program (SCUP) can copy all requester process object files into a single POBJ.

A file that is used as an additional target in one target file specification can also be used as a dependency in other target file specifications, as long as it does not result in a circular target file specification.

The ACI Make utility adds Tandem NonStop Kernel defines for the additional targets to the TACL environment for use as variables while the commands associated with the target file specification are executed.

The syntax for specifying additional targets in a target file specification is shown below. The additional targets appear following the colon that separates the target file and the dependency files, and are surrounded by braces ({ }). For readability, additional targets usually are indented from the target file.

```
target           :{                \  
    addl_output_1      \  
    addl_output_2 }    source_1    \  
                        source_2    \  
                        source_3
```

All additional targets must appear before the dependency files. If a target file specification also contains additional defines, which are discussed earlier in this section, the additional defines must also appear before the dependency files. However, additional targets can appear before or after the additional defines. The following example shows the additional targets after the additional defines.

```
target      :[
    addl_input_1      \
    addl_input_2      ]{      \
    addl_output_1      \
    addl_output_2      }  source_1  \
                        source_2  \
                        source_3
```

The following makefile example for BASE24-pos DDLs illustrates the use of additional targets.

```
$(pssrc_psddlta1) :{      \      # 0067
    $(psddl_ddldict)      \      # 0068
    $(pssrc_psddlcob)      \      # 0069
    $(pssrc_psddlufup)      \      # 0070
    $(pssrc_psddlc)  }  $(badefts_ddlgdefs) \      # 0071
                        $(badefts_defsdict) \      # 0072
                        $(psddl_ddlfcart) \      # 0073
                        $(psddl_ddlfpirf) \      # 0074
                        $(psddl_ddlfpddf) \      # 0075
                        $(psddl_ddlfprrf) \      # 0076
                        $(psddl_ddlfpdtd) \      # 0077
                        $(psddl_ddlfpdtdf) \      # 0078
                        $(psddl_ddlfpdtdf) \      # 0079
                        $(psddl_ddlgpdd) \      # 0080
                        $(psddl_ddlgpddfx) \      # 0081A01
                        $(psddlm_deps)      \      # 0081A02
```

Commands to build the product-level language targets

The following makefile example for a Pathway requester process illustrates the use of an additional target and an additional define.

```
$(baaft_rqpbfo) :{      \      # 0063
    $(dest_pobj)  }[      \      # 0064
    $(basrc_baddlcob) ] $(baaft_rqpbfs)      \      # 0065
                        $(baaft_scrnpbf)      \      # 0066
                        $(baddl_ddlfpbf)      \      # 0067
                        $(baddl_ddlfpsec)      \      # 0068
                        $(baddl_ddlgpth)      \      # 0069
                        $(basrc_bacoutls)      \      # 0070
                        $(basrc_cobnames)      \      # 0071
                        $(basrc_hlpscrn)      \      # 0072
```

Commands to build the PBF requester and the POBJ File

Inline Code

The ACI Make utility supports an inline code facility to define the steps performed to build a target. With the inline code facility, sets of commands can be embedded within the makefile.

The ACI Make utility starts a single TACL Server process when it has read all of the macro files and makefiles, then passes the commands for creating each target file to the TACL Server process. The ACI Make utility does not start a new TACL process for each target specification.

If a makefile contains the `INLINE` command as a run option with a command to start a process, the ACI Make utility passes all lines in the makefile that contain an equal sign (=) in the first column to its TACL Server process. Blank lines, comment lines containing a # character in the first position, and lines that start with `IF` or `ENDIF` commands are not tested for the equal sign. All inline commands must be valid TACL commands. The ACI Make utility continues passing process lines from the makefile until it reaches one of the following:

- A new target file in the current makefile.
- A new target file resulting from an `INCLUDE` statement in the current makefile.
- The end of the makefile containing the process lines currently being passed.

Comments in an `INLINE` input line must be identified by a # character that is preceded and followed by a space. See the “Comments” topic earlier in this section for additional information and examples for comments.

In an `INLINE` input line, a # character that is not followed by a space identifies the built-in functions and variables used for programming in TACL, as well as spooler locations. The `#PUSH` and `#APPEND` commands in the makefile presented with this discussion are examples of built-in functions.

Inline commands use file defines established in Make macro files to reference the appropriate files, and the ACI Make utility expands the file defines before passing them on to the inline process.

Inline commands use TACL macros developed by ACI to simplify makefiles. See the “TACL Macros” topic later in this section for more information about TACL macros.

The following table describes the various inline code components, using part of the HISOM as an example, and gives the file sequence numbers that identify each type in the example on the following page. The file sequence numbers appear at the right of each line in the file.

Component	Sequence Numbers
The \$(bahiso_hiso) macro names the target file.	94
The list of dependency files is shown in the example as a variable named <i>dependency-list</i> .	94
The \$(TACL)/INLINE/ command identifies the start of commands performed by the TACL Server process.	126
The \$(<i>tacl_init</i>) \$(<i>make_tacl_macros</i>) inline command executes a TACL macro identified by the TACL_INIT define. This macro initializes the new TACL environment and loads the TACL macros required by the makefiles.	127
The # <i>PUSH</i> : <i>var</i> inline command creates a temporary TACL variable.	129
The # <i>APPEND</i> inline commands take the text contained in the file identified by the macro in each of the commands and add it to the temporary TACL variable.	130
IF and ENDIF commands selectively include inline commands based on flag settings.	132–165 173–177
Once all of the # <i>APPEND</i> inline commands are completed, the \$(<i>BIND</i>) inline command starts the Tandem Binder program, passing the TACL variable containing the BIND commands previously added as input.	183
A TACL macro checks for the successful completion of the BIND operation. This macro abends the TACL Server process if the BIND operations is not successful.	184
A TACL macro deletes the spooler job for the BIND operation.	185
The \$AXCEL command starts the RISC Accelerator process, if required.	191–197

Component	Sequence Numbers
A TACL macro checks for the successful completion of the RISC Accelerator process.	198
A TACL macro deletes the spooler job for the RISC Accelerator process.	199
A TACL macro moves and secures the resulting executable file to the appropriate Base object location.	204–205

```

$(bahiso_hiso) :    dependency-list                                \    # 0094
                                                              # 0125
    $(TACL)/INLINE/                                           # 0126
= $(tacl_init) $(make_tacl_macros)                             # 0127
                                                              # 0128
= #PUSH    :var                                              # 0129
= #APPEND :var add *    from $(bahiso_bahisoo)                # 0130
                                                              # 0131
!IF $(ISO_HOST_INTERFACE_ATM_ON) = TRUE |THEN|                # 0132
= #APPEND :var add code *    from $(athiso_athisoo), delete   # 0133
= #APPEND :var add data (atm^globals) from $(athiso_athisoo), delete # 0134
= #APPEND :var add data (attkn^globals) from $(athiso_athisoo), delete # 0135
!IF $(ISO_HOST_INTERFACE_SSB_ON) = TRUE |THEN|                # 0136
= #APPEND :var add code *    from $(atbhis_hisossbo), delete  # 0137
!ENDIF                                                         # 0138
!ENDIF                                                         # 0139
                                                              # 0140
!IF $(ISO_HOST_INTERFACE_POS_ON) = TRUE |THEN|                # 0141
= #APPEND :var add code *    from $(pshiso_pshisoo), delete   # 0142
= #APPEND :var add data (pos^globals) from $(pshiso_pshisoo), delete # 0143
= #APPEND :var add data (pstkn^globals) from $(pshiso_pshisoo), delete # 0144
!ENDIF                                                         # 0145
                                                              # 0146
!IF $(ISO_HOST_INTERFACE_TLR_ON) = TRUE |THEN|                # 0147
= #APPEND :var add code *    from $(trhiso_trhisoo), delete   # 0148
= #APPEND :var add data (tlr^globals) from $(trhiso_trhisoo), delete # 0149
= #APPEND :var add data (trtkn^globals) from $(trhiso_trhisoo), delete # 0150
!ENDIF                                                         # 0151
                                                              # 0152
!IF $(ISO_HOST_INTERFACE_FHM_ON) = TRUE |THEN|                # 0153
= #APPEND :var add code *    from $(fhhiso_fhhisoo), delete   # 0154
= #APPEND :var add data (upd^globals) from $(fhhiso_fhhisoo), delete # 0155
!ENDIF                                                         # 0156
                                                              # 0157
!IF $(ISO_HOST_INTERFACE_MAIL_ON) = TRUE |THEN|              # 0158
= #APPEND :var add code *    from $(mahiso_mahisoo), delete   # 0159

```

```

= #APPEND :var add data (mail^globals) from $(mahiso_mahisoo), delete # 0160
!ENDIF # 0161
# 0162
!IF $(TBHISO_ON) = TRUE |THEN| # 0163
= #APPEND :var add code * from $(tbhiso_tbhiso), delete # 0164
!ENDIF # 0165
# 0166
= #APPEND :var select search $(BASRC_TKNCV) # 0167
= #APPEND :var select search $(BASRC_BAUTIL) # 0168
= #APPEND :var select search $(BASRC_HISWUTL) # 0169
= #APPEND :var select search $(BASRC_SECUTIL) # 0170
= #APPEND :var select search $(BASRC_KEYUTIL) # 0171
= #APPEND :var select search $(ACIUTILS_UTILB) # 0172
!IF $(TBHISO_ON) = TRUE |THEN| # 0173
= #APPEND :var select search $(OKLIB_OKLIBTO) # 0174
= #APPEND :var select search $(OCLIB_OCLIBTO) # 0175
= #APPEND :var select search $(OCTXN_TXNLIBTO) # 0176
!ENDIF # 0177
= #APPEND :var set data 64 # 0178
= #APPEND :var select list * off # 0179
= #APPEND :var $(bahiso_bind_steps) # 0180
= #APPEND :var build $(bahiso_hiso) ! # 0181
# 0182
= $(BIND)/ inv :var, out $s.#hiso/ # 0183
= check_completion $(bahiso_hiso) # 0184
= del_peruse_job $(PERUSE) $(bahiso_purge_spooler_on) #hiso # 0185
= #POP :var # 0186
# 0187
#----- # 0188
# HISO AXCEL # 0189
#----- # 0190
= [#IF $(BAHISO_AXCEL_ON) = $(TRUE) |THEN| # 0191
= #PUSH :var # 0192
= #EOF :var # 0193
= $(AXCEL)/inv :var, OUT $S.#rhiso & # 0194
= $(bahiso_axcel_opts) / & # 0195
= $(bahiso_hiso), $(bahiso_hiso); & # 0196
= USERLIB $(spannet_hskelb) # 0197
= check_completion $(bahiso_hiso) # 0198
= del_peruse_job $(PERUSE) $(bahiso_purge_spooler_on) #rhiso # 0199
= #POP :var # 0200
= ] == end if AXCEL On # 0201
# 0203
= dup_exec_file $(move_obj_subvol_on) $(bahiso_secure_opt) & # 0204
= $(bahiso_hiso) $(base_obj_loc) # 0205

```


TACL Macros

The ACI Make utility starts a single TACL Server process when it has read all of the macro files and makefiles, then passes the commands for creating each target file to the TACL Server process. ACI has developed a set of TACL macros to simplify the commands in the makefiles. These macros are located in the TACLMACS File, which is located on the same subvolume as the ACI Make utility (MAKEV500).

The TACLMACS File contains the following groups of macros.

- Independent macros that can be used in makefiles for BASE24 procedural and BASE24 ITS products. These macros are documented in this subsection.
- Independent macros for other ACI products (e.g., C++ development projects). These macros are not documented at this time.
- Helper macros that are used by the independent macros, but are not designed to be used by themselves. These macros are not documented at this time.

The following table contains all of the TACLMACS File macros.

Macro	Description
ACI_SCOBOLX	SCOBOL compile, TACL only
ACI_SCOBOLX_MAKE	SCOBOL compile
CHECK_COMPLETION	Completion check
CHECK_SCOBOL_COMPLETION	SCOBOL completion check
CHK_PERUSE_JOB	Completion check
CONVERT_SCOBOL_FILE	Helper only
CPATHEQ_BUILD	C++ development only
CPATHEQ_CREATE	C++ development only
CPP_COMPILE	C++ development only
DDLTAL_UNDERBAR_CONV	DDLTAL underbar conversion
DEL	File deletion

Macro	Description
DEL_PERUSE_JOB	Spooler job deletion
DISPLAY_COMPLETION_INFO	TACL completion information
DUP_DICT	DDL dictionary duplication
DUP_EXEC_FILE	Executable file duplication and security
EXPANDDEF	Define name
EXPANDSSV	Volume and subvolume names
EXPANDSSV_DICT_NOSAVE	Temporary subvolume creation
GET_VOL_SUBVOL	Volume and subvolume names
PRCSS_SCOBOL_CMD_LINE	Helper only
PRCSS_SCOBOL_CMD_LINE_MAKE	Helper only
PRODUCE_TIME_STR	Helper only
PURGE_DICT	Dictionary file purge
SCUP_ROUTINE	SCUP with completion check
SECURE	Secure file
TACLMACS_VERSION	TACLMACS version
TEMPLMAC	EMS template information creation
TOTAL_SIZE	Total size of set of files
UPDATE_DICT_ALT_FILES	DDL dictionary files update

ACI_SCOBOLX_MAKE Macro

The ACI Make utility dynamically adds Tandem NonStop Kernel defines to the TACL environment for use as variables when a target is built. The ACI_SCOBOLX_MAKE macro is required so TACL defines can be used within the SCOBOL source files that are used for Pathway requesters. Prior to executing the

SCOBOL compiler, this macro reads the define into a TACL variable and replaces TACL defines in the source file with the actual text of the TACL define. The macro then passes this modified code to the compiler.

The ACI_SCOBOLX_MAKE macro creates the pseudo target file that is maintained for each requester process. The ACI Make utility uses the modification timestamps from this pseudo target file when determining whether an object file needs to be rebuilt. A pseudo target file can be empty or contain a copy of the requester process source code, depending on whether the SYMBOLS option is included in the TAL Options macro (e.g., BASE_TAL_OPTS in the CUSTMACS File or BASEFM File). The requester process can be debugged with the Tandem INSPECT utility if the pseudo target file contains the source code.

The ACI_SCOBOLX_MAKE macro also checks whether any errors or warnings occur when the source file is compiled, and purges the target if any errors or warnings occur.

Syntax

ACI_SCOBOLX_MAKE *compiler output-file commands*

compiler — The fully qualified name or the Make macro for the SCOBOL compiler. The SCOBOLX macro is used for this compiler name.

output-file — The name of the pseudo target file that is empty or contains only source information for use with the Tandem INSPECT utility. The predefined TARGET macro can be used for this file name.

commands — The commands required by the SCOBOL Compile macro, including the SCOBOL source file name, spooler location, compiler options, pseudo target file name, a settog value that distinguishes between IBM 3270 terminals (settog 1) and Tandem 6530 terminals (settog 3), and SCOBOL compiler options. The following typical usage example is for IBM 3270 terminals.

Typical Usage

```
aci_scobolx_make $(SCOBOLX) $(TARGET) / in $(baaft_rqecfs),      &
                                                         out $.#rqecf      &
                                                         $(baaft_compile_opts) / &
                                                         [temp_pobj]; settog 1; &
                                                         $(baaft_scobolx_opts)
```

CHECK_COMPLETION Macro

The CHECK_COMPLETION macro checks the completion code in the TACL environment, and is typically used immediately after a compilation statement (e.g., TAL, COBOL, DDL). The CHECK_SCOBOL_COMPLETION macro is used following SCOBOL compilation statements.

The following table shows whether the specified file is purged based on compilation results and the optional warning flag setting. If the specified file is purged, the TACL Server process is abended.

Compilation Results	Warning Allowed Flag Value	Purge File
No warnings or errors	0 or 1	No
Errors with or without warnings	0 or 1	Yes
Warnings without errors	0	Yes
Warnings without errors	1	No

If version 5.00 or greater of the TACLMACS File is installed, the CHECK_COMPLETION macro calls the DISPLAY_COMPLETION_INFO macro when a process failure occurs. This way, ACI Make utility output contains completion information that would otherwise be lost. The CHECK_COMPLETION macro in earlier versions of the TACLMACS File does not call the DISPLAY_COMPLETION_INFO macro if a process failure occurs.

An ACI Make utility command line option, NO_STOP_ON_ERRORS, controls how the utility responds when errors occur or when warnings occur and are not allowed, as follows:

- With the NO_STOP_ON_ERRORS option, the utility stops building the target being built when the problem occurs and does not build any other targets that depend upon the target that cannot be built. However, the utility continues processing all targets that do not depend upon the target that cannot be built.
- Without the NO_STOP_ON_ERRORS option, the utility stops processing the target being built when the problem occurs and does not process any other targets.

Syntax

CHECK_COMPLETION *purge-file* [*warning-allowed-flag*]

purge-file — The fully qualified name of the file to be purged. This value is usually the Make macro for the target file name.

warning-allowed-flag — A code indicating whether warnings are allowed. Valid values are as follows:

0 = No, warnings are not allowed. This is the default value if the code is not specified.

1 = Yes, warnings are allowed.

Typical Usage

check_completion \$(target)

CHECK_SCOBOL_COMPLETION Macro

The CHECK_SCOBOL_COMPLETION macro does not perform any processing because the ACI_SCOBOLX_MAKE macro performs its own completion checking. However, the CHECK_SCOBOL_COMPLETION macro still follows the ACI_SCOBOLX_MAKE macro in makefiles in case processing must be added to the CHECK_SCOBOL_COMPLETION macro in the future.

Syntax

CHECK_SCOBOL_COMPLETION *purge-file*

purge-file — The fully qualified name of the file to be purged. This value is usually the Make macro for the target file name.

Typical Usage

check_scobol_completion \$(TARGET)

CHK_PERUSE_JOB Macro

The CHK_PERUSE_JOB macro performs a completion check when warnings are allowed. This macro checks a specific peruse job for a specific search text string. Any text string that matches this search text string and does not match an allowed text string is printed to the output file. If a disallowed text string is found, the TACL Server process abends.

Syntax

CHK_PERUSE_JOB *target* *abend-flag* *peruse-executable* *peruse-job*
search-text *allowed-text* [...]

target — The fully qualified name of the file to be purged. This value is usually the Make macro for the target file name.

abend-flag — A flag that specifies whether the TACL Server process abends when disallowed text is found. Valid values are \$(TRUE) or \$(FALSE).

peruse-executable — The fully qualified name of the Tandem Peruse executable.

peruse-job — The name of the peruse job, without the spooler (\$s.) prefix.

search-text — The text to find in the peruse job.

allowed-text — A string of allowed text. You can define as many allowed text strings as necessary. The following example has two allowed text strings.

Typical Usage

```
= chk_peruse_job $(TARGET) $(TRUE) $(PERUSE) #ba.ddl &  
= $(baddl_ddl_warning_text) &  
= $(baddl_ddl_alwd_warnings)
```

DDL TAL_UNDERBAR_CONV Macro

The DDL TAL_UNDERBAR_CONV macro converts a BASE24 procedural DDL TAL file for use with BASE24 ITS modules. It performs the following steps.

1. Converts all circumflex (^) characters to underbar characters (_).
2. Appends an underbar character (_) to each section name that still does not have one (after all circumflex characters have been converted).

Syntax

DDL TAL_UNDERBAR_CONV *ddl-tal-file*

ddl-tal-file — The name of the DDL file processed by this macro, usually expressed as a Make macro.

Typical Usage

DDLTLAL_UNDERBAR_CONV \$(baddl_baddlital)

DEL Macro

The DEL macro attempts to purge all files with names matching the file template provided. If no files are deleted, the text “no matching file(s)” appears in the ACI Make utility output.

Syntax

DEL *file-template* [[,*file-template*] [...]

file-template — The fully qualified name or wildcard template of the file or files to be deleted. Wildcard characters can be used to identify groups of files, and commas or spaces can be used to separate multiple template entries.

Typical Usage

= SINK [del [temp_pobj]*]

DEL_PERUSE_JOB Macro

The DEL_PERUSE_JOB macro deletes a specified spooler job if the makefile steps preceding the macro are completed successfully, subject to the setting of the specified delete flag.

In the event that the PURGE_SPOOLER_ON flag for a module is set to false, this macro sets the report name to PASSED for all spooler jobs concerning successful completions. This assists in differentiating between failed and passed spooler jobs when they are deliberately left in the spooler.

Spooler jobs are not deleted if TACL errors occur during any of the preceding makefile steps. The CHECK_COMPLETION macro causes the TACL Server process to abend before the DEL_PERUSE_JOB macro executes.

Syntax

DEL_PERUSE_JOB *executable delete-flag job-name*

executable — The Make macro for the Tandem Peruse program.

delete-flag — The name of the Make macro that contains a code indicating whether the spooler job is to be deleted if the makefile steps preceding the DEL_PERUSE_JOB macro are completed successfully. Valid values for the code are as follows:

- True = Delete the specified spooler job.
- False = Do not delete the specified spooler job, but change the report name to PASSED.

job-name — The name of the file to be deleted, without the spooler (\$s.) prefix.

Typical Usage

```
del_peruse_job $(PERUSE) $(atauth_purge_spooler_on) #authl
```

DISPLAY_COMPLETION_INFO Macro

The DISPLAY_COMPLETION_INFO macro displays information from the TACL completion structure.

TACL saves completion code information in the variable :_COMPLETION, which is defined by Tandem code in the standard TACLSEGF. TACL sets this variable whenever you attempt to start a process, whenever you successfully start a process, and whenever a process you started terminates (successfully or otherwise). Whenever TACL sets this variable, its previous contents are lost.

The CHECK_COMPLETION macro calls the DISPLAY_COMPLETION_INFO macro if a process failure occurs. This way, ACI Make utility output contains completion information that would otherwise be lost.

Syntax

```
DISPLAY_COMPLETION_INFO
```

DUP_DICT Macro

The DUP_DICT macro duplicates the DDL dictionaries (e.g., DICTALT, DICTCDF, DICTDDF) from one location to another, and updates the alternate key attributes.

Syntax

```
DUP_DICT src-loc dest-loc
```


src-loc — The source location (system, volume, and subvolume) from which the DDL dictionaries are duplicated. The EXPANDSSV TACL macro is usually used to obtain the source location from a DDL file Make macro.

dest-loc — The destination location (system, volume, and subvolume) to which the DDL dictionaries are duplicated. The EXPANDSSV TACL macro is usually used to obtain the destination location from a DDL file Make macro.

Typical Usage

`dup_dict [expandssv $(badefts_defsdct)] [expandssv $(baddl_badict)]`

DUP_EXEC_FILE Macro

The DUP_EXEC_FILE macro copies an executable file to a destination subvolume and secures it.

The main purpose for this macro is to move executable files from the module subvolume to the execution subvolume. An example is moving the BASE24-atm Authorization executable file from the AT60AUTH subvolume to the AT60OBJ subvolume.

The DUP_EXEC_FILE macro uses the security value specified in the *fup-secure-string* to set the security of the destination file. The macro also ensures that the security of the source file matches this security value. It changes the source file security if necessary.

Syntax

`DUP_EXEC_FILE move-flag fup-secure-string src-file dest-loc`

move-flag — A flag to indicate if the source file is actually to be duplicated. The MOVE_OBJ_SUBVOL_ON macro from the CUSTMACS File is used for this value. Valid values are as follows:

- True = Duplicate the source file at the destination location. Secure the source and destination files with the FUP security string.
- False = Do not copy the source file to the destination location. Secure the source file with the FUP security string.

fup-secure-string — The TACL security assigned to the destination file. The BASE_SECURE_OPT macro in the CUSTMACS File and the product-specific SECURE-OPT macro in the appropriate configuration macro files are used for this value. If the keyword “default” is supplied, the security of the destination file is set to that of the user running the ACI Make utility.

If a TACL security value is specified (e.g., “NNNU”), it must be a valid value. Valid values are A, C, G, N, O, and U. Super user (-) security is not allowed.

The DUP_EXEC_FILE macro uses this security value to set the security of the destination file. The macro also ensures that the security of the source file matches this security value. It changes the source file security if necessary.

src-file — The fully qualified name of the source file. The specific Make macro for the source file is used for this value.

dest-loc — The destination volume and subvolume. The appropriate product-specific OBJ_LOC macro in the CUSTMACS File is used for this value. The destination file name is the same as the source file; however, if the destination file is open, the DUP_EXEC_FILE macro attempts to rename the file and then save it. For the first attempt, the macro uses the destination file name with a 1 added to the end. If the renamed destination file does not exist or exists but is not open, the macro saves the renamed file and processing is complete. Otherwise, the macro repeats this procedure with the numbers 2 through 9. If the original file name contains eight characters, the macro replaces the last character with the numbers.

Typical Usage

```
dup_exec_file $(move_obj_subvol_on) $(baaft_secure_opt) &  
$(baaft_svcaf) $(base_obj_loc)
```

EXPANDDEF Macro

The EXPANDDEF macro returns the fully qualified file name of a define. For example, it returns *\$volume.subvolume.test* for the *=test* define.

Syntax

EXPANDDEF *define-name*

define-name — An active define for a fully qualified file name.

Typical Usage

expanddef =test

EXPANDSSV Macro

The EXPANDSSV macro returns the volume and subvolume names from a fully qualified file name. It returns the *\$volume.subvolume* or *\system.\$volume.subvolume* portion of the file name, depending on whether the original file name included the system name. The EXPANDSSV macro is often used within another command or macro when just the volume and subvolume for a file are required.

Syntax

EXPANDSSV *file-name*

file-name — The fully qualified file name or the Make macro for a fully qualified file name.

Typical Usage

#APPEND :var ?DICTN [expandssv \$(baddl_badict)]

EXPANDSSV_DICT_NOSAVE Macro

The EXPANDSSV_DICT_NOSAVE macro parses the subvolume name and places the letters DICT over the first four letters of the subvolume name. This macro creates a separate subvolume for temporary DDL dictionary files.

Syntax

EXPANDSSV_DICT_NOSAVE *file-name*

file-name — The fully qualified file name or the Make macro for a fully qualified file name.

Typical Usage

#APPEND :var ?DICTN [expandssv_dict_nosave \$(baddl_badict_nosav)]

GET_VOL_SUBVOL Macro

The GET_VOL_SUBVOL macro returns the volume and subvolume names from a fully qualified file name obtained from a line of a macro file such as the ALLMACS File. It searches for the equal sign (=) in the text received, then returns the \$*volume.subvolume* portion of the file name that follows the equal sign. The GET_VOL_SUBVOL macro is often used within another command or macro when just the volume and subvolume for a file are required.

Syntax

GET_VOL_SUBVOL *file-name*

file-name — The fully qualified file name or the Make macro for a fully qualified file name.

Typical Usage

#SET :vol_subvol [get_vol_subvol [:allmacs_line]]

PURGE_DICT Macro

The PURGE_DICT macro purges the dictionary files (e.g., DICTALT, DICTCDF, DICTDDF) that are located on the current subvolume. These dictionary files are created during a DDL compilation and can be purged.

Syntax

PURGE_DICT

SCUP_ROUTINE Macro

The SCUP_ROUTINE macro serves the special purpose of obtaining the completion code when the Tandem Screen COBOL Utility Program (SCUP) program is called with an input variable, as is done in makefiles for Pathway requester processes. Without this macro, the SCUP program does not properly report any errors that occur while it is updating the Screen COBOL object library (POBJ) files, and the ACI Make utility does not stop processing when it encounters a requester process that does not compile properly.

This macro appears in the ALLMACS File instead of individual makefiles. The Make macro named SCUP is expanded from just the location of the SCUP program to include the name of this macro, as shown in the following syntax.

Syntax in the ALLMACS File

```
scup    =    $system.system.scup (without macro)
scup    =    scup_routine $system.system.scup (with macro)
```

The macro is invoked from a makefile with the following syntax.

Typical Usage in Makefiles

```
= $(SCUP)/inv :var/
```

SECURE Macro

The SECURE macro secures a file with the supplied security.

Syntax

SECURE *executable file options*

executable — The Make macro for the Tandem File Utility Program (FUP).

file — The Make macro for the file being secured.

options — The option used to secure the file. If a valid security sequence is provided within quotes, the security of the file is changed accordingly. If the value 'default' is used, the security remains set to its current sequence.

Typical Usage

```
secure $(FUP) $(basrc_bautil) "NCNC"
```

```
secure $(FUP) $(basrc_bautil) default
```

TACLMACS_VERSION Macro

The TACLMACS_VERSION macro displays the version number of the current TACLMACS File. The version number of the TACLMACS File must match the release number of the ACI Make utility being used so that all ACI Make utility functionality operates correctly. For example, while version 400 of the

TACLMACS File can be used with release 5.01 of the ACI Make utility, version 501 of the TACLMACS File is required for all release 5.01 ACI Make utility functionality to operate correctly.

Syntax

TACLMACS_VERSION_501

The version number is included in the name of this macro for quick reference and can be displayed at any time by loading the TACLMACS File. Enter the following command at a TACL prompt to load the TACLMACS File without keeping the macros in memory.

```
load/keep 0/$volume.subvolume.tacmacros
```

The following information is displayed on your terminal, with the TACLMACS Version macro listed first.

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD
CPATHEQ_CREATE	CPP_COMPILE
DDLTAL_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

TEMPLMAC Macro

The TEMPLMAC macro purges the NEWNRES and NEWRES files associated with compiling EMS templates. The TEMPLMAC macro identifies a file to purge by adding a digit (0 through 9) as the last character (e.g, NEWNRES0 or NEWRES1), then purges the file if it is not open. The macro identifies a maximum of 10 NEWNRES and 10 NEWRES files.

Syntax

TEMPLMAC

TOTAL_SIZE Macro

The TOTAL_SIZE macro returns the total number of bytes used by the specified file template.

Syntax

TOTAL_SIZE *file-template*

file-template — The fully qualified name of the file or files to be included in the size calculation. Wildcard characters can be used to identify groups of files.

Typical Usage

total_size \$test.ba6*.*

total_size \$(atauth_authcsee)

UPDATE_DICT_ALT_FILES Macro

The UPDATE_DICT_ALT_FILES macro updates the alternate key files associated with the DICTKDF, DICTOBL, DICTODF, DICTOUF, and DICTRDF dictionary files. The DICTOUF uses the DICTOUK as the alternate key, and the remaining files listed use the DICTALT as the alternate key.

This macro is used after an existing set of dictionaries are moved from one volume to another, and must be run from the volume to which the dictionaries have been moved.

Syntax

UPDATE_DICT_ALT_FILES

ACI Worldwide Inc.

Section 4

Using the ACI Make Utility

This section describes the general procedures required for using the ACI Make utility. It also provides the procedures for performing specific tasks with the ACI Make utility.

The ACI Make utility is available in two versions, as follows.

- The MAKEV501 process is a native object (file type 700) for Tandem NonStop Reduced Instruction Set Computing (RISC) processors, also called TNS/R processors. It requires a NonStop Kernel operating system of D45 or greater.
- The MAKET501 process is a regular object (file type 100) for Tandem NonStop processors, also called TNS processors, that do not support native object files, as follows:
 - Processors without Reduced Instruction Set Computing must have a NonStop Kernel operating system of at least D38.
 - Processors with Reduced Instruction Set Computing and a NonStop Kernel operating system of at least D38 but less than D45.

The MAKET501 process can be used on TNS/R processors with a NonStop Kernel operating system of D45 or greater. However, the MAKEV501 process operates considerably faster than the MAKET501 process on these operating systems.

Running the ACI Make Utility

The ACI Make utility effectively directs program compilation by automatically regenerating files that are older than the files from which they are created. Three steps are required to run the ACI Make utility.

- Set the ALLMACS define in your TACL environment to identify which global level macro file you want to use first.
- Set the Radix table size to control how much space will be available for the internal table that holds the file macros.
- Start the ACI Make utility.

You can perform each of these steps individually or you can combine them in an obey file. The order in which you set the ALLMACS define and Radix table size does not matter, as long as you perform both steps before you start the ACI Make utility. Each step is described in greater detail below and on the following pages.

Setting the ALLMACS Define

All makefiles developed for BASE24 load the global level macro file identified by the ALLMACS define. This means that you must add a TACL file define called ALLMACS to your TACL environment before you run the ACI Make utility. Once you add the ALLMACS define, it exists until you alter it, delete it, or end your TACL session.

The global level macro file is the CUSTMACS File or the CSMMACS File, depending on whether you are using the ACI Make utility to compile a standard system or a customized system, respectively. The CUSTMACS File contains tests for files containing macros for a project team or individual developer. See “Global Macro Files” in section 2 for additional information about each of these macro files.

Syntax to add the ALLMACS define

delete define =allmacs

add define =allmacs, file *first-file*

Syntax to alter an existing ALLMACS define

alter define =allmacs, file *first-file*

first-file — Specifies the global level macro file to be read first. This variable can be qualified if necessary to identify the file.

Note: Spaces cannot be entered between the equal sign (=) and *allmacs*.

You can use the syntax shown above to add the ALLMACS define in any of the following ways:

- Enter the commands at the TACL prompt. The DELETE DEFINE command removes an existing ALLMACS define. At the TACL prompt, you have the option of using the INFO DEFINE command to check for an existing ALLMACS define. If an ALLMACS define exists, you can use the ALTER DEFINE command to adjust the existing value instead of using the DELETE DEFINE and ADD DEFINE commands to replace it.
- Include the DELETE DEFINE and ADD DEFINE commands in your TACLCSTM file. Include the DELETE DEFINE command as a precaution to remove an existing ALLMACS define.
- Include the DELETE DEFINE and ADD DEFINE commands in the obey file that you use to start the ACI Make utility. Include the DELETE DEFINE command as a precaution to remove an existing ALLMACS define. Refer to the “Using Obey Files to Start the ACI Make Utility” topic later in this section for additional information.

Setting the Radix Table Size

The ACI Make utility places all configuration information and file definitions from Make macro files into two internal tables as it builds the Make hierarchy and dependency tree. One table contains one entry for each Make macro and associated text, and the other table contains the fully qualified file name referenced in the ACI Make utility and ties each file name to the appropriate Make macro. The Radix table size controls how much space will be available for each of these internal tables. Since there are two tables, the ACI Make utility allocates twice the specified Radix table size for extended memory.

The Radix table size can be set using the TACL parameter called MAX-RADIX-TBL-SIZE or using the ACI Make utility option called MAX_RADIX_TBL_SIZE. However, the parameter and option do not work the same. The Radix table size you set with the TACL parameter does not change until you modify it or end your TACL session, regardless of how many times you run the ACI Make utility. The ACI Make utility option overrides the TACL parameter for a single run of the utility.

For example, if you usually need a maximum Radix table size of only 10 million bytes, you can use the MAX-RADIX-TBL-SIZE parameter to set the table to 10 million bytes. If you need a maximum table size of 15 million bytes for one run of the ACI Make utility, you can use the MAX_RADIX_TBL_SIZE option to temporarily override the TACL parameter for the single run. Otherwise, you must use the parameter twice—once to enlarge the Radix table to 15 million bytes and again to reset it to 10 million bytes.

The size requirement is based on the number of products being built by a single execution of the ACI Make utility. Specifying a Radix table size that is too large wastes system resources. Specifying a size that is too small can cause the ACI Make utility to exit prematurely while building its macro tables. The default Radix table size, if this option and parameter are not specified, is 5 million bytes.

The following information in the summary at the end of each run of the ACI Make utility helps you determine the correct Radix table size. This summary is not provided when the DUMPMACS option is used.

Information on RADIX Tables:

2 Tables Allocated	28000000 bytes each.
- Table 1 Used	6977774 (24.9%)
- Table 2 Used	18207258 (65.0%)

If the Radix table size is greater than 10 million bytes, the following warning message follows the Radix table information when the larger of the two tables uses less than 70 percent of the available Radix table size. The warning provides the current Radix table setting and a suggested Radix table setting that is based on the size of the larger Radix table.

```
**** WARNING ****
*
* The MAX-RADIX-TBL-SIZE is specified too high.
*
* This message is generated when more than 10000000
* is requested, and the required space is
* less than 70% of what was requested.
*
* Current setting is 28000000
*
* Try 'PARAM MAX-RADIX-TBL-SIZE 26010300'
*
*****
```

You can use the ACI Make utility to determine the necessary Radix table size. The ACI Make utility, when run with the NOBUILD option and an oversized Radix table parameter, provides a Radix table recommendation without actually building any files. You can then use the table information and modify the Radix table size setting before you build any files. With a table 2 size of 18,207,258 bytes, as shown in this example, no warning message appears for a Radix table size setting between 18,207,258 bytes and 26,010,300 bytes, because the percentage used is between 70 percent and 100 percent. However, a higher percentage indicates a greater possibility of the ACI Make utility failing due to insufficient space for the Radix tables. The Radix tables grow due to additions to the Make macros.

Syntax when entered as a TACL parameter

PARAM MAX-RADIX-TBL-SIZE *size*

Syntax when entered as an ACI Make utility option

-MAX_RADIX_TBL_SIZE=*size*

size — The size of memory, in bytes, allocated to hold each of the internal tables created by the ACI Make utility.

You can set the Radix table size in any of the following ways:

- Use the default Radix table size (5 million bytes).
- Enter the PARAM MAX-RADIX-TBL-SIZE command at the TACL prompt.
- Include the MAX_RADIX_TBL_SIZE option when you start the ACI Make utility.
- Include the MAX-RADIX-TBL-SIZE parameter or the MAX_RADIX_TBL_SIZE option in the obey file that you use to start the Make utility. See the “Using Obey Files to Start the ACI Make Utility” topic later in this section for additional information.

Starting the ACI Make Utility

After you have added the ALLMACS define and set the Radix table size, start the ACI Make utility by entering the following command at a TACL prompt.

Syntax

```
RUN [$volume.subvolume.] { MAKEV501 | MAKET501 } [ / params / ]  
    [ makefile ] [ options ]
```

MAKEV501 — The file name of the TNS/R version of the ACI Make executable file.

MAKET501 — The file name of the TNS version of the ACI Make executable file.

params — Any of the standard Tandem run options except for the input file option (e.g., OUT, NOWAIT, CPU, HIGHPIN OFF). An input file (IN option) cannot be specified because the makefile serves as the input file.

makefile — The name of the first makefile that the ACI Make utility is to use. This makefile can be at the master level or at a lower level, depending on what portion of a system you want to build. See sections 1 and 2 for detailed descriptions of makefiles.

The makefile specified with the EQUIV, EQUIVALL, and UPDATE options must match the global level macro file specified in the ALLMACS define. This file name usually identifies the CUSTMACS File, and can be fully qualified if necessary.

The makefile parameter is required unless you use one of the following ACI Make command line options: ERROR, HELP, or VERSION. If a makefile parameter is not provided when it is expected, the ACI Make utility searches the current subvolume for a file named MAKEFILE.

options — Any ACI Make command line options. Each option must be preceded by a hyphen (-), and multiple options must be separated by spaces.

The B24C File portion of the “Using Obey Files to Start the ACI Make Utility” topic presents several combinations of command line options you can use as you install BASE24.

The “Command Line Options” topic provided later in this section presents descriptions of all available options.

Using Obey Files to Start the ACI Make Utility

An obey file is a convenient tool for starting the ACI Make utility because of the ALLMACS define and the number of command line options that are often involved. Using an obey file to start the ACI Make utility can ensure consistency of the options used and accuracy in entering the commands, as well as reduce time spent at the keyboard. The easiest way to explain obey files is to provide three examples. The first example, the B24C File, also serves as a starting point for customized obey files such as the RUNMKNB File and RUNMAKE File.

B24C Obey File

The B24C File is located on the BA60MAKE subvolume provided with BASE24 procedural and BASE24 ITS products. The example of this file contains the commands required to do the following:

1. Set the TACL MAX-RADIX-TBL-SIZE parameter. As an alternative, you can use the MAX_RADIX_TBL_SIZE option with the RUN command to set this parameter.
2. Remove the ALLMACS define, if one exists.
3. Set a new ALLMACS define.
4. Run the ACI Make utility. The B24C File contains several variations of the RUN command. Each RUN command is accompanied by a brief explanation of the options used and the expected result. The ACI Make utility performs the following:
 - a. Uses the UPDATE and DETAIL options with the B24M Makefile to update the macro files and makefiles for your BASE24 system.
 - b. Uses the NOBUILD option with the B24M Makefile to check the dependencies.
 - c. Uses the FORCE and PURGE_TARGETS options with the B24M Makefile to purge all target files that were included on your BASE24 software tape.
 - d. Uses the LIST, LISTREPLACE, NO_STOP_ON_ERRORS, and TIMESTAMP_FILE options with the B24M Makefile to rebuild targets. With this variation, the ACI Make utility uses a timestamp file to determine whether each target needs rebuilding, and generates the list of files read while it builds the dependency tree.

- e. Uses the LIST, LISTREPLACE, NO_STOP_ON_ERRORS, SUPPRESS_UNTIL_SERVER, and TIMESTAMP_FILE options with the B24M Makefile to rebuild targets. With this variation, the ACI Make utility uses a timestamp file to determine whether each target needs rebuilding, and suppresses the list of files read while it builds the dependency tree.
- f. Uses the LIST, LISTREPLACE, FORCE, and SUPPRESS_UNTIL_SERVER options with a specific makefile (e.g., RQMEGAM) to rebuild a single BASE24 module.

You must customize the file names and locations and select one variation of the RUN command before you use the B24C File. Select a variation by replacing the comment (==) entry at the beginning of the appropriate RUN command line with spaces. Select a different variation by reentering the comment (==) entry on the previous RUN command and removing them from the appropriate RUN command.

- 5. Remove the ALLMACS define set in step 3.

```
Comment *****
Comment *   This file starts a MAKE run that compiles all the      *
Comment *   products and modules in your BASE24 system based on the *
Comment *   flags set in BA60MAKE.CUSTOMACS.                      *
Comment *                                                         *
Comment *   Uncomment the RUN statement that corresponds to the type *
Comment *   of MAKE run you wish to start. To uncomment the RUN   *
Comment *   statement, replace the '==' for the lines of the RUN   *
Comment *   statement with spaces.                                 *
Comment *                                                         *
Comment *   Usage:  TACL/IN B24C, NAME/                            *
Comment *****
```

```
Comment  Set TACL Defines for MAKE
```

```
param max-radix-tbl-size 20000000
```

```
delete define =allmacs
add    define =allmacs,      file $volume.ba60make.custmacs
```

```
Comment *****
Comment *                                                         *
Comment *   If you have just restored the BASE24 files received from *
Comment *   ACI for your system, MAKE must be run to update the BASE24 *
Comment *   MAKE files to point to the correct system, volumes, and   *
Comment *   subvolumes.                                              *
Comment *                                                         *
```



```

Comment * Ensure that you have updated your ALLMACS, CUSTMACS, and *
Comment * EQUIV files on the BA60MAKE subvolume first to reflect *
Comment * your system's configuration. *
Comment * * *
Comment * The syntax to UPDATE your BASE24 files is: *
Comment * * *
Comment * MAKEVxxx /out $s.#make.b24m, nowait/ CUSTMACS -UPDATE & *
Comment * -DETAIL *
Comment * * *
Comment * where: xxx is the version of MAKE. *
Comment * * *
Comment * CUSTMACS is the CUSTMACS file location. *
Comment * * *
Comment * -DETAIL generates a detail listing of *
Comment * all of the files to be updated. *
Comment * * *
Comment * -UPDATE will update all of the MAKE files *
Comment * based on your EQUIV file settings. *
Comment * *****

```

```

== run $volume.makev500.makev501 /out $s.#make.b24m, nowait/ &
== $volume.ba60make.custmacs -detail -update

```

```

Comment *****
Comment * *
Comment * The following is the syntax to run MAKE to check whether *
Comment * all of the MAKE dependencies are in place. *
Comment * * *
Comment * MAKEVxxx /out $s.#make.b24m, nowait/ B24M -FORCE -NOBUILD *
Comment * * *
Comment * where: xxx is the version of MAKE. *
Comment * * *
Comment * B24M is the top-level MAKE file. *
Comment * * *
Comment * -FORCE forces MAKE to need all targets *
Comment * rebuilt. *
Comment * * *
Comment * -NOBUILD do not generate the targets. Just *
Comment * check the MAKE dependencies. *
Comment * *****

```

```

== run $volume.makev500.makev501 /out $s.#make.b24m, nowait/ &
== $volume.ba60make.b24m -force -nobuild

```

```

Comment *****
Comment * *
Comment * The following is the syntax to run MAKE to purge all *
Comment * targets that were included on your BASE24 software tape. *
Comment * * *
Comment * MAKEVxxx /out $s.#make.b24m, nowait/ B24M -FORCE *
Comment * -PURGE_TARGETS *
Comment * *

```

```
Comment *      where:      xxx      is the version of MAKE.      *
Comment *
Comment *      B24M      is the top-level MAKE file.      *
Comment *
Comment *      -FORCE      forces MAKE to need all targets      *
Comment *                  rebuilt.      *
Comment *
Comment *      -PURGE_TARGETS      *
Comment *                  attempts to purge all targets      *
Comment *                  that need to be rebuilt.      *
Comment *                  MAKE assumes a -NOBUILD.      *
Comment * *****
```

```
== run $volume.makev500.makev501 /out $s.#make.b24m, nowait/ &
==      $volume.ba60make.b24m -force -purge_targets
```

```
Comment *****
Comment *
Comment * The following is the syntax to run MAKE to use a timestamp *
Comment * file. The modification timestamp of the timestamp file *
Comment * will be used in determining whether a target needs *
Comment * rebuilding. Using this syntax, MAKE will rebuild targets. *
Comment *
Comment * MAKEVxxx /out $s.#make.b24m, nowait/ B24M -LIST *
Comment * -LISTREPLACE -NO_STOP_ON_ERRORS *
Comment * -TIMESTAMP_FILE=$volume.BA60MAKE.TSTEMP *
Comment *
Comment *      where:      xxx      is the version of MAKE.      *
Comment *
Comment *      B24M      is the top-level MAKE file.      *
Comment *
Comment *      -LIST      will turn MAKE file listing on.      *
Comment *
Comment *      -LISTREPLACE *
Comment *                  will list all macro replacement *
Comment *                  text. *
Comment *
Comment *      -NO_STOP_ON_ERRORS *
Comment *                  indicates that MAKE should not *
Comment *                  terminate upon the first compile *
Comment *                  error it encounters. Instead, it *
Comment *                  will continue and build the *
Comment *                  targets that are not dependent on *
Comment *                  targets that have already failed. *
Comment *
Comment *      -TIMESTAMP_FILE=$volume.BA60MAKE.TSTEMP *
Comment *                  indicates a file whose *
Comment *                  modification timestamp will be *
Comment *                  used to determine which targets *
Comment *                  need to be rebuilt. If a Target *
Comment *                  has an older timestamp than the *
```

```

Comment *                               TSTEMP file, the target file will *
Comment *                               be rebuilt. *
Comment *****
== run $volume.makev500.makev501 /out $s.#make.b24m, nowait/ &
==     $volume.ba60make.b24m -list -listreplace -no_stop_on_errors &
==     -timestamp_file=$volume.ba60make.tstemp

Comment *****
Comment *
Comment * The following is the syntax to run MAKE using a timestamp *
Comment * file (see above), suppressing the listing of the files it *
Comment * reads while building its dependency tree, and rebuilding *
Comment * the targets. *
Comment *
Comment * MAKEVxxx /out $s.#make.b24m, nowait/ B24M -LIST & *
Comment * -LISTREPLACE -NO_STOP_ON_ERRORS -SUPPRESS_UNTIL_SERVER & *
Comment * -TIMESTAMP_FILE=$volume.BA60MAKE.TSTEMP *
Comment *
Comment * where:      xxx      is the version of MAKE. *
Comment *
Comment *           B24M      is the top-level MAKE file. *
Comment *
Comment *           -LIST      will turn MAKE file listing on. *
Comment *
Comment *           -LISTREPLACE *
Comment *                   will list all macro replacement *
Comment *                   text. *
Comment *
Comment *           -NO_STOP_ON_ERRORS *
Comment *                   indicates that MAKE should not *
Comment *                   terminate upon the first compile *
Comment *                   error it encounters. Instead, it *
Comment *                   will continue and build the *
Comment *                   targets that are not dependent on *
Comment *                   targets that have already failed. *
Comment *
Comment *           -SUPPRESS_UNTIL_SERVER *
Comment *                   directs MAKE to begin the listing *
Comment *                   after the dependency tree has been *
Comment *                   built, and the TACL Server is *
Comment *                   being started. *
Comment *
Comment *           -TIMESTAMP_FILE=$volume.BA60MAKE.TSTEMP *
Comment *                   indicates a file whose *
Comment *                   modification timestamp will be *
Comment *                   used to determine which targets *
Comment *                   need to be rebuilt. If a Target *
Comment *                   has an older timestamp than the *
Comment *                   TSTEMP file, the target file will *
Comment *                   be rebuilt. *
Comment *****

```

```
== run $volume.makev500.makev501 /out $s.#make.b24m, nowait/ &
==    $volume.ba60make.b24m -list -listreplace -no_stop_on_errors &
==    -suppress_until_server -timestamp_file=$volume.ba60make.tstemp
```

```
Comment *****
Comment *
Comment * The following is the syntax to run MAKE if you wish to
Comment * run MAKE against one BASE24 module.
Comment *
Comment * This example is for rqmega. Please note that on each
Comment * BASE24 subvolume, there will be a MAKE file for each
Comment * BASE24 module.
Comment *
Comment * MAKEVxxx /out $s.#make.b24m, nowait/BA60AFT.RQMEGAM &
Comment * -LIST -LISTREPLACE -FORCE -SUPPRESS_UNTIL_SERVER
Comment *
Comment * where:      xxx      is the version of MAKE.
Comment *
Comment *      BA60AFT.RQMEGAM
Comment *      is the MAKE file for RQMEGA.
Comment *
Comment * -LIST      will turn MAKE file listing on.
Comment *
Comment * -LISTREPLACE
Comment *      will list all macro replacement
Comment *      text.
Comment *
Comment * -FORCE      will force MAKE to reMAKE RQMEGA
Comment *      without checking for any
Comment *      timestamps or targets.
Comment *
Comment * -SUPPRESS_UNTIL_SERVER
Comment *      directs MAKE to begin the listing
Comment *      after the dependency tree has been
Comment *      built, and the TACL Server is
Comment *      being started.
Comment *****
```

```
== run $volume.makev500.makev501 /out $s.#make.b24m, nowait/ &
==    $volume.ba60aft.rqmegam -list -listreplace -force &
==    -suppress_until_server
```

```
delete define =allmacs
```

RUNMKNB Obey File

The name of this obey file, RUNMKNB, is an abbreviation for “Run the ACI Make utility with the NOBUILD option.” You can create this obey file on a subvolume with the other files needed for testing purposes, and create additional RUNMKNB files on other subvolumes if you have other projects in progress, with each file customized to meet the needs of the project.

The RUNMKNB File is similar to the final variation given in the B24C File. The final B24C variation uses the SUPPRESS_UNTIL_SERVER option to reduce the report output. The RUNMKNB File uses the NOBUILD option for testing purposes and uses a D option for the mymacros macro.

The combination of FORCE, LIST, and LISTREPLACE options in this obey file causes the utility to build or rebuild all target files, list all of the macro files and makefiles, and list the values used for all macros during processing. Using the NOBUILD option with the other options provides an opportunity to review the listings for accuracy and review which targets need rebuilding before the target files are built for the first time.

In this example, the ACI Make utility is starting with the BASE24-atm DDL makefile (ATDDL). The Tandem OUT parameter (\$\$.#ATDDL) is similar to the makefile name. The D option with the mymacros macro points to a customized global level macro file (TESTMACS).

```
param max-radix-tbl-size 20000000

delete define =allmacs
add define =allmacs, file $volume.ba60make.custmacs

run $sysaci.makev500.makev501 /out $$.#atddl/ &
$volume.subvolume.atddl -force -list -listreplace -nobuild &
-Dmymacs=$volume.subvolume.testmacs

delete define =allmacs
```

RUNMAKE Obey File

The name of this obey file, RUNMAKE, is an abbreviation for “Run the ACI Make utility.” You can create this obey file on the same subvolume as the RUNMKNB File shown previously. The only difference between these files is the removal of the NOBUILD option. After using the RUNMKNB File to run the utility with the NOBUILD option, and reviewing the results of that run, you can use the RUNMAKE File to run the ACI Make utility without the NOBUILD option. The FORCE option remains active in a test environment to ensure that all coding changes are included.

```
param max-radix-tbl-size 20000000

delete define =allmacs
add define =allmacs, file $volume.ba60make.custmacs

run $sysaci.makev500.makev501 /out $s.#atddl/ &
$volume.subvolume.& atddlm -force -list -listreplace &
-Dmymacs=$volume.subvolume.testmacs

delete define =allmacs
```

Note: You can use the `TIMESTAMP_FILE` option instead of the `FORCE` option to ensure that all target files get rebuilt. By using the `TIMESTAMP_FILE` option, which was added with release 5.00 of the ACI Make utility, you can restart the ACI Make process when an error occurs without also having to rebuild the target files that were built successfully. See the `TIMESTAMP_FILE` option description in the “Other Command Line Options” topic for additional information.

Command Line Options

The ACI Make utility provides an extensive set of options that you can enter with the Run command when you start the utility. The following groups of command line options are available.

- Printing command line options control printing when the ACI Make utility is run. You can specify one or more of these options to tailor a run of the ACI Make utility to meet your needs. See the “Printing Command Line Options” topic for detailed descriptions of these options.
- Other command line options control operations other than printing when the ACI Make utility is run. You can specify one or more of these options to tailor a run of the ACI Make utility to meet your needs. The file location maintenance options (previously part of the Makescan utility) are included in this group. See the “Other Command Line Options” topic for detailed descriptions of these options.
- SQL command line options were used by the ACI development organization before the ACI Make utility was available as a native object (file type 700) for Tandem NonStop Reduced Instruction Set Computing (RISC) processors. These command line options are not documented.

The following table contains all of the command line options, arranged in alphabetical order.

Option	Group	Description
ABEND_ON_ERROR	Other	Creates SAVEABEND file in certain error conditions
COMMIT_COUNT	SQL	Not documented
CUST	SQL	Not documented
D	Other	Defines a Make macro
DEL_CUST	SQL	Not documented
DETAIL	Print	Provides detailed file location maintenance reports
DUMPMACS	Other	Copies macro information to edit file
ERROR	Other	Enters Make error message help mode

Option	Group	Description
EQUIV	Other	Creates edit file for file location maintenance
EQUIVALL	Other	Creates edit file for file location maintenance
EXCEPT_FILE	Other	Creates edit file for exception Make macro entries
EXCEPT_LOC	Other	Creates edit file for exception Make macro entries
EXT_BUILD	Other	Disabled command
EXT_USE	Other	Disabled command
FORCE	Other	Rebuilds all targets
HELP	Other	Enters Make operations help mode
HIGHPIN_OFF	Other	Starts the Make TACL server process with a low process identification number (PIN)
INFO_NAME	SQL	Not documented
INFO_MAC	SQL	Not documented
LIST	Print	Adds source file information
LISTBS	Print	Adds build steps as dependency files are read
LISTDEP	Print	Adds file names and timestamps
LISTINLINE	Print	Adds execution statements and inline commands
LISTREPLACE	Print	Adds replacement text for macros
LINE_SIZE_MAX	Other	Adjusts memory allowed for macros
MAKE_FILE_DEP_CHK	Other	Expands file dependency checks
MAX_RADIX_TBL_SIZE	Other	Specifies size of Radix table

Option	Group	Description
NOBUILD	Other	Checks file dependency information without building target files
NOCHECK	SQL	Not documented
NO_DELETE_DEFINES	Other	Accepts existing TACL defines
NO_EXT_CHECK	Other	Disabled command
NO_INCLUDES	SQL	Not documented
NOLIST	Print	Does not include source file listings
NOSQL	SQL	Undocumented default value that turns off SQL processing
NO_STOP_ON_ERRORS	Other	Continues processing if build errors occur
PURGE_TARGETS	Other	Purges selected targets without rebuilding
SQL	SQL	Not documented
SQL_LOAD_MACS	SQL	Not documented
SQL_UNLOAD_MACS	SQL	Not documented
SUMMARY	SQL	Not documented
SUPPRESS_COMMENTS	Print	Suppresses comments
SUPPRESS_UNTIL_SERVER	Print	Suppresses output generated before starting the TACL server process
SYNTAX	Other	Flags macros that appear to be in error
TEST	Other	Points to files that match Make file macros
TESTCHK	Other	Points to files that match Make file macros
TIMESTAMP_FILE	Other	Adds a timestamp to target file dependency files

Option	Group	Description
UPDATE	Other	Performs file location maintenance
VALIDATE_DB	SQL	Not documented
VERSION	Other	Displays the current version of the Make executable

Printing Command Line Options

The ACI Make utility accepts several command line options to control printing when it is run. One or more of these options can be specified to tailor a run of the ACI Make utility to meet your needs.

Each option appearing on the command line must be preceded by a hyphen (-), and multiple options must be separated by spaces. All examples are taken from the same makefile used for the demonstration presented in appendix A.

Default Printing

The ACI Make utility provides the following information regardless of which printing command line options are selected.

- The RUN command used to start the Make process.
- The value of the ALLMACS define.
- The dialog for starting the TACL server. The TACL macros are loaded twice during this dialog because one set serves as a backup during processing.
- The following information for each file built:
 - ❑ File name, expressed as a fully qualified file name
 - ❑ Reason for building the file
 - ❑ Defines established to build the file, expressed as fully qualified file names
 - ❑ Name of the makefile used
 - ❑ First execution statement, including fully qualified file names
- A list of the files that were built successfully or that need to be rebuilt, depending on whether the NOBUILD option was used.
- A list of files not built because of errors or warnings.
- The following processing information:
 - ❑ *Files rebuilt* shows the number of files successfully rebuilt or the number of files that need to be rebuilt.
 - ❑ *Build errors* shows the number of files that could not be rebuilt because of errors.

- ❑ *Build warnings* shows the number of files that the ACI Make utility did not attempt to rebuild because they had dependency files that could not be rebuilt because of errors.
- ❑ *Elapsed time* shows the total length of time, in hours, minutes, seconds, and hundredths of seconds, from the start of the run to the end of the run.
- ❑ *Time before starting first build* is the length of time, in hours, minutes, seconds, and hundredths of seconds, used to read the Make macro files and makefiles, create the Make hierarchy, and determine which target files must be rebuilt.
- ❑ *Longest Line Length* is the length, in bytes, of the longest individual line contained in a makefile.
- ❑ *Line Size Max value* is the length, in bytes, of the longest individual line that can be contained in a makefile.
- ❑ *Number of SQL macro hits* is the number of times the ACI Make utility uses SQL macros, which is now zero.
- ❑ *Number of Memory macro hits* is the number of times the ACI Make utility uses macros defined in memory.
- ❑ *Bytes in build steps* is the size, in bytes, of the build steps placed in extended memory during the run. A build step is any execution statement in a makefile that results in building a target. The LISTBS option adds these statements to the report output.
- ❑ *Build Steps Table Size* is the size, in bytes, of the extended memory allocated for the build steps. The Kernel Managed Swap Facility (KMSF) allocates resources for this file, as the ACI Make utility requests them.
- ❑ *Information on RADIX Tables* is the size, in bytes, allocated for each Radix table, followed by the space used, in bytes and as a percentage of the space allocated, by each table. This item also includes a recommended allocation amount if the allocation is greater than 10,000,000 bytes and the larger of the two tables uses less than 70 percent of the space allocated. See the “Setting the Radix Table Size” topic for additional information.

The following is an example of the TACL server dialog, the information provided for one of four files that were built by the ACI Make utility, and the file lists and processing summary for the run.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

RUN CMD > \r.\$sysaci.makev500.makev501 \r.\$earl.at60auth.authm &
> -DMYMACS=\$BRIDGE.XYZTEST.XYZMACS

Define =ALLMACS equals: \R.\$EARL.BA60MAKE.CUSTMACS

Starting the TACL Server...

TACL (T9205D46 - 30JUL1999), Operating System D45, Release D48.00
COPYRIGHT COMPAQ COMPUTER CORPORATION 1985,1987-1999
CPU 1, process has no backup
May 5, 2001 21:07:31
(Invoking \$SYSTEM.SYSTEM.TACLLOCL)

Loaded from \$SYSTEM.SYSTEM.RTACLDEF:

(Invoking \$BRIDGE.subvolume.TACLCSTM)
Current volume is \$BRIDGE.XYZTEST
\$SYSACI.MAKEV500.TACLINIT \$SYSACI.MAKEV500.TACLMACS

TACLINIT - Version 5.01

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD
CPATHEQ_CREATE	CPP_COMPILE
DDLTAL_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD

CPATHEQ_CREATE	CPP_COMPILE
DDLTA_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

TACL Server Started...

Build file:

\R.\$BRIDGE.XYZTEST.AUTH

Reason for Building File:

Timestamp: 2001 05:05:20:44:006:008 Target: \R.\$BRIDGE.XYZTEST.AUTH
Timestamp: 2001 05:05:21:09:009:084 Dependency: \R.\$BRIDGE.XYZTEST.AUTHO

Target Define:

DEFINE ADDED: ATAUTH_AUTH VALUE: \R.\$BRIDGE.XYZTEST.AUTH

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILB	VALUE: \R.\$SYSACI.ACIUTILS.UTILB
DEFINE ADDED: ATAUTH_AUTHO	VALUE: \R.\$BRIDGE.XYZTEST.AUTHO
DEFINE ADDED: ATAUTH_AUTHLIBO	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBO
DEFINE ADDED: ATLIB_ATLIBLO	VALUE: \R.\$EARL.AT60LIB.ATLIBLO
DEFINE ADDED: BASRC_BAUTIL	VALUE: \R.\$EARL.BA60SRC.BAUTIL
DEFINE ADDED: BASRC_SECUTIL	VALUE: \R.\$EARL.BA60SRC.SECUTIL
DEFINE ADDED: BASRC_BATKNCVO	VALUE: \R.\$EARL.BA60SRC.BATKNCVO
DEFINE ADDED: OKLIB_OKLIBTO	VALUE: \R.\$EARL.OK600BJ.OKLIBTO

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/
File Utility Program - T6553D45 - (09NOV98) System \R
Copyright Tandem Computers Incorporated 1981, 1983, 1985-1998
SECURE \R.\$BRIDGE.XYZTEST.AUTH, "NNNU"

*** MAKE complete 5 MAY 2001 21:09:34.79 ***

Files Rebuilt:

\R.\$BRIDGE.XYZTEST.AUTHLIBE
\R.\$BRIDGE.XYZTEST.AUTHLIBO
\R.\$BRIDGE.XYZTEST.AUTHO
\R.\$BRIDGE.XYZTEST.AUTH

Files NOT Rebuilt:

NONE

Summary:

Files rebuilt : 4

Build errors : 0

Build warnings : 0

Elapsed Time : 00:02:13.38 (h:m:s.hs)

Time before starting first build: 00:00:11.47 (h:m:s.hs)

Longest Line Length (bytes): 1726

Line Size Max value (bytes): 250000

Number of SQL macro hits: 0

Number of Memory macro hits: 645

Bytes in build steps: 19490

Build Steps Table Size: 120000000

Information on RADIX Tables:

2 Tables Allocated 10000000 bytes each.

- Table 1 Used 1780514 (17.8%)

- Table 2 Used 3858728 (38.6%)

Available Printing Options

The ACI Make utility accepts the following command line options to control printing when it is run. One or more of these options can be specified to expand the default information. Options are arranged alphabetically.

DETAIL Option

The DETAIL option is one of the options used with file name maintenance (part of the Makescan utility prior to release 5.00). This option is used with one of the other file name maintenance options (EQUIV, EQUIVALL, or UPDATE) to control the amount of information provided in the file name maintenance report.

Without this option, the report contains entries for macro files only. With this option, the report contains the same macro file entries, plus entries for all of the macros contained in the macro files.

Note: ACI recommends always using the DETAIL option with the UPDATE option.

Refer to the “Maintaining File Name References” topic for additional information about using this option, including examples of reports generated with and without the DETAIL option.

The DETAIL option first became available in release 5.00 of the ACI Make utility.

LIST Option

The LIST option lists the following additional information, focusing primarily on the source files for the ACI Make utility run. The line numbers provided with the LIST option are the actual line numbers (not sequence numbers) from each source file.

- Entries from all Make macro files read during the run, expressed as macros
- Commands from all makefiles read during the run, expressed as macros
- A statement indicating when the dependency list was created
- All execution statements (including inline commands), for each file built, expressed as macros

See the LISTINLINE option description for an example of how execution statements are listed with the LIST option.

The LIST and NOLIST options are mutually exclusive, and the NOLIST option is the default value if neither option is specified.

Note: The ACI Make utility can fill the printer spooler if the LIST option is used when an entire BASE24 system is being built. As an alternative, use the SUPPRESS_UNTIL_SERVER option, the SUPPRESS_COMMENTS option, or both, with the LIST option to reduce the output, or use other list options instead of the LIST option.

LISTBS Option

The LISTBS option lists the build steps information while the ACI Make utility reads through each dependency list. A build step is any execution statement in a makefile that results in building a target. The build step lines contain information about each target, its makefile, and the steps defining how to build it, as the ACI Make utility finishes reading each build step definition. The line numbers provided with the LISTBS option are the actual line numbers (not sequence numbers) from each makefile.

The LISTBS option first became available in release 5.00 of the ACI Make utility.

The following is an example of the information the LISTBS option adds to the default printing example presented earlier. This example contains three sets of build steps because it has three targets. The first two sets contain only one execution statement, while the third set contains a group of execution statements.

```

BS: Start  ----- Build Steps Printout */
BS: TARGET : \R.$BRIDGE.XYZTEST.AUTHCSEE
BS:
BS: Build Steps
BS:
BS: \R.$EARL.AT60AUTH.AUTHM
BS: 214      $(GENFWD)/in $(atauth_authcses)/ ext $(atauth_authcsee)      # 0213
BS: End      ----- Build Steps Printout */

BS: Start  ----- Build Steps Printout */
BS: TARGET : \R.$BRIDGE.XYZTEST.AUTHLIBE
BS:
BS: Build Steps
BS:
BS: \R.$EARL.AT60AUTH.AUTHM
BS: 217      $(GENFWD)/in $(atauth_authlibs)/ ext $(atauth_authlibe)      # 0216
BS: End      ----- Build Steps Printout */

BS: Start  ----- Build Steps Printout */
BS: TARGET : \R.$BRIDGE.XYZTEST.AUTHLIBO
BS:
BS: Build Steps
BS:
BS: \R.$EARL.AT60AUTH.AUTHM
BS: 283                                           # 0282
BS: 284      $(TACL)/INLINE/                      # 0283
BS: 285      = $(tacl_init) $(make_tacl_macros)      # 0284
BS: 286                                           # 0285

```

```
BS: 287 = $(TAL)/in $(atauth_authlibs), & # 0286
BS: 288 = out $s.#atauthl.t & # 0287
BS: 289 = $(atauth_compile_opts)/ & # 0288
BS: 290 = $(atauth_authlibo); & # 0289
BS: 291 = $(atauth_tal_opts) # 0290
BS: 292 = check_completion $(TARGET) # 0291
BS: 293 = del_peruse_job $(PERUSE) $(atauth_purge_spooler_on) #atauthl.t # 0292
BS: 294 # 0293
BS: 295 = secure $(FUP) $(atauth_authlibo) $(atauth_secure_opt) # 0294
BS: 296 = check_completion $(TARGET) # 0295
BS: End ----- Build Steps Printout */
```

LISTDEP Option

The LISTDEP option adds a list of the fully qualified file names and the modification timestamp information used to determine which files are rebuilt during the ACI Make utility run.

The following is an example of the information the LISTDEP option adds to the default printing example presented earlier. The dependency information for each file rebuilt is presented as sources between the name of the target file and the status of the target file. This report also identifies when the ACI Make utility determines that a target must be rebuilt. The source file named UTILB (shown in bold) does not have an associated target, so that target must be rebuilt. While the remaining source files also do not have an associated target, the rebuild message appears only the first time the need is identified.

```
TACL Server Started...
*** TARGET: 5MAY2001 21:17:16.83 - \R.$BRIDGE.XYZTEST.AUTHCSEE
*** SOURCE: 22MAR2001 08:59:01.80 - \R.$EARL.AT60AUTH.AUTHCSES
*** File (\R.$BRIDGE.XYZTEST.AUTHCSEE) did not require rebuilding
*** TARGET: 5MAY2001 21:17:30.85 - \R.$BRIDGE.XYZTEST.AUTHE
*** SOURCE: 2MAY2001 08:00:57.56 - \R.$BRIDGE.XYZTEST.AUTHS
*** File (\R.$BRIDGE.XYZTEST.AUTHE) did not require rebuilding
*** TARGET: File not present -
*** SOURCE: 3JAN2001 22:19:51.64 - \R.$SYSACI.ACIUTILS.UTILB
*** Rebuild set - Target doesn't exist
*** SOURCE: 5MAY2001 21:19:01.56 - \R.$BRIDGE.XYZTEST.AUTHO
*** SOURCE: 5MAY2001 21:18:16.22 - \R.$BRIDGE.XYZTEST.AUTHLIBO
*** SOURCE: 5MAY2001 17:43:11.60 - \R.$EARL.AT60LIB.ATLIBLO
*** SOURCE: 3APR2001 20:15:12.50 - \R.$EARL.BA60CSE.BATKCCVO
*** SOURCE: 2MAY2001 15:02:00.99 - \R.$EARL.BA60SRC.BAUTIL
```

```
*** SOURCE: 4MAY2001 17:31:56.34 - \R.$EARL.BA60SRC.SECUTIL
*** SOURCE: 3APR2001 20:15:54.44 - \R.$EARL.BA60SRC.BATKNCVO
*** SOURCE: 4MAY2001 19:17:35.02 - \R.$EARL.OK600BJ.OKLIBTO
```

```
*****
```

Build file:

\R.\$BRIDGE.AT60DCT.RQDSTANO

Reason for Building File:

Timestamp: 1999 05:13:21:18:033:007 Target: \R.\$BRIDGE.AT60DCT.RQDSTANO

Timestamp: 1999 05:20:15:11:035:054 Dependency: \R.\$SYSACI.XPNET.DDLCOB

LISTINLINE Option

The LISTINLINE option lists the following information for an ACI Make utility run.

- Execution statements for each file built, including fully qualified file names
- Inline commands for each file built, including fully qualified file names

The following examples illustrate the impact of the LISTINLINE option. The first example shows the execution statement without any print options. Only the first execution statement is printed.

Execution Statement:

```
403                                                    # 0402
404    $(TACL)/INLINE/                                # 0403
$SYSTEM.SYSTEM.TACL/INLINE/
```

The second example shows the execution statements for the same makefile with the LIST option. All of the execution statements are shown with macros.

Execution Statement:

```
403                                     # 0402
404     $(TACL)/INLINE/                 # 0403
$SYSTEM.SYSTEM.TACL/INLINE/
405 = $(tacl_init) $(make_tacl_macros) # 0404
406                                     # 0405
407 = #PUSH      :var                  # 0406
408 = #APPEND    :var add * from $(atauth_autho),      delete # 0407
409 = #APPEND    :var add code * from $(atauth_authlibo), delete # 0408
410                                     # 0409
411                                     # 0410
412 !IF $(atm_emv_on) = TRUE |THEN|    # 0411
415                                     # 0414
416 !IF $(atm_mcu_on) = TRUE |THEN|    # 0415
419                                     # 0418
420                                     # 0419
421                                     # 0420
422                                     # 0421
423 !IF $(NCD_ON) = TRUE |THEN|        # 0422
426                                     # 0425
427 !IF $(SSB_BASE_ON) = TRUE |THEN|   # 0426
433                                     # 0432
434                                     # 0433
435 !IF $(atm_emv_on) = TRUE |THEN|    # 0434
438                                     # 0437
439 !IF $(atm_mcu_on) = TRUE |THEN|    # 0438
442                                     # 0441
443 = #APPEND    :var select search $(ATLIB_ATLIBLO)    # 0442
444 = #APPEND    :var select search $(OKLIB_OKLIBTO)   # 0443
445 = #APPEND    :var select search $(ACIUTILS_UTILB)  # 0444
446                                     # 0445
447                                     # 0446
448                                     # 0447
449                                     # 0448
450                                     # 0449
451 = #APPEND    :var select search $(BASRC_BAUTIL)    # 0450
452 = #APPEND    :var select search $(BASRC_BATKNCVO)  # 0451
453 = #APPEND    :var select search $(BASRC_CVUTIL)    # 0452
454 = #APPEND    :var select search $(BASRC_SECUTIL)   # 0453
455 = #APPEND    :var select search $(BACSE_BATKCCVO)  # 0454
456 = #APPEND    :var set data 64                    # 0455
457 = #APPEND    :var select list * off                # 0456
458 =                                                  # 0457
459                                     # 0458
460                                     # 0459
461                                     # 0460
462                                     # 0461
463 = #APPEND    :var $(atauth_bind_steps)             # 0462
464 = #APPEND    :var build $(atauth_auth) !          # 0463
465                                     # 0464
```

```

466 = $(BIND)/ inv :var, out $s.#atauth.b $(atauth_compile_opts)/      # 0465
467 = check_completion $(TARGET)                                       # 0466
468 = del_peruse_job $(PERUSE) $(atauth_purge_spooler_on) #atauth.b     # 0467
469 = #POP      :var                                                    # 0468
470                                                    # 0469
471 #----- # 0470
472 #   AUTH Accelerate # 0471
473 #----- # 0472
474 # 0473
475 !IF $(atauth_axcel_on) = TRUE |THEN| # 0474
486 # 0485
487 = dup_exec_file $(move_obj_subvol_on) $(atauth_secure_opt) & # 0486
488 =                $(atauth_auth) $(atm_obj_loc) # 0487

```

The third example shows the execution statements for the same makefile with only the LISTINLINE option. All of the execution statements are shown with fully qualified file names. However, a report with just the LISTINLINE option does not include the lines with the original execution statements. For example, the following lines appear in a report with only the LISTINLINE option.

Execution Statement:

```

$SYSTEM.SYSTEM.TACL/INLINE/
>> $SYSACI.MAKEV500.TACLINIT $SYSACI.MAKEV500.TACLMACS
>> #PUSH      :var
>> #APPEND    :var add * from \R.$BRIDGE.XYZTEST.AUTH0,          delete
>> #APPEND    :var add code * from \R.$BRIDGE.XYZTEST.AUTHLIB0, delete
>> #APPEND    :var select search \R.$EARL.AT60LIB.ATLIBLO
>> #APPEND    :var select search \R.$EARL.OK600BJ.OKLIBTO
>> #APPEND    :var select search \R.$SYSACI.ACIUTILS.UTILB
>> #APPEND    :var select search \R.$EARL.BA60SRC.BAUTIL
>> #APPEND    :var select search \R.$EARL.BA60SRC.BATKNCVO
>> #APPEND    :var select search \R.$EARL.BA60SRC.CVUTIL
>> #APPEND    :var select search \R.$EARL.BA60SRC.SECUTIL
>> #APPEND    :var select search \R.$EARL.BA60CSE.BATKCCVO
>> #APPEND    :var set data 64
>> #APPEND    :var select list * off
>> #APPEND    :var
>> #APPEND    :var build \R.$BRIDGE.XYZTEST.AUTH !
>> $SYSTEM.SYSTEM.BIND/ inv :var, out $s.#atauth.b ,NAME/
>> check_completion \R.$BRIDGE.XYZTEST.AUTH
>> del_peruse_job $SYSTEM.SYSTEM.PERUSE 1 #atauth.b
>> #POP      :var
>> dup_exec_file 0 "NNU" &
>> \R.$BRIDGE.XYZTEST.AUTH \R.$EARL.AT600BJ

```

The fourth example shows a few of the execution statements for the same makefile with the LISTINLINE option and LIST option. The replacement text can be difficult to use without the execution statements that it replaces. However, when the LIST option and LISTINLINE option are used together, the execution statements and corresponding replacement text appear as follows, with the replacement text appearing immediately below each execution statement.

```
451 = #APPEND :var select search $(BASRC_BAUTIL) # 0450
>> #APPEND :var select search \R.$EARL.BA60SRC.BAUTIL
452 = #APPEND :var select search $(BASRC_BATKNCVO) # 0451
>> #APPEND :var select search \R.$EARL.BA60SRC.BATKNCVO
453 = #APPEND :var select search $(BASRC_CVUTIL) # 0452
>> #APPEND :var select search \R.$EARL.BA60SRC.CVUTIL
```

LISTREPLACE Option

The LISTREPLACE option lists the replacement text used for every line of a Make macro file or makefile that uses a macro once the macro has been defined.

A report with just the LISTREPLACE option does not include the lines where the macros are defined initially, only the lines where existing macros are used. For example, the following lines appear in a report with the LISTREPLACE option.

```
^^^^^.^^^ !IF FALSE = FALSE |THEN|
^^^^^.^^^ !IF TRUE = TRUE |THEN|
^^^^^.^^^ !include $BRIDGE.XYZTEST.XYZMACS
^^^^^.^^^ !IF FALSE = TRUE |THEN|
^^^^^.^^^ !IF FALSE = 1 |THEN|
```

The replacement text can be difficult to use without the statements that it replaces, especially in IF commands. However, when the LIST option and LISTREPLACE option are used together, the macros and corresponding replacement text appear as follows, with the replacement text appearing immediately below each macro.

```
172      !IF $(allmacs) = FALSE |THEN|                                # 0171
^^^^^^.^^^ !IF FALSE = FALSE |THEN|
145      !IF $(mymacs)) = TRUE |THEN|                                # 0101
^^^^^^.^^^ !IF TRUE = TRUE |THEN|
147          !include $(mymacs)                                       # 0103
^^^^^^.^^^ !include $BRIDGE.XYZTEST.XYZMACS
163      !IF $(bacse_csemacs)) = TRUE |THEN|                          # 0119
^^^^^^.^^^ !IF FALSE = TRUE |THEN|
355      !IF $(bacse_csefm)) = $(TRUE) |THEN|                        # 0354
^^^^^^.^^^ !IF FALSE = 1 |THEN|
```

NOLIST Option

The NOLIST option excludes the source file listings from the ACI Make utility output. The LIST and NOLIST options are mutually exclusive, and the NOLIST option is the default value if neither option is specified.

SUPPRESS_COMMENTS Option

The SUPPRESS_COMMENTS option suppresses all comment entries from the ACI Make utility output. A comment entry is any line that has the comment character (#) as the first nonspace character. This option significantly reduces the spooler output created by a run of the ACI Make utility.

The SUPPRESS_COMMENTS option first became available in release 5.00 of the ACI Make utility.

SUPPRESS_UNTIL_SERVER Option

The SUPPRESS_UNTIL_SERVER option suppresses all ACI Make utility output that is generated prior to the dialog for starting the TACL server process. When used with the LIST and LISTREPLACE options, this option causes the ACI Make utility to report the makefile text and replacement text for all generated target files without reporting all of the macro files and makefiles that were read to build the Make hierarchy, significantly reducing the report output.

Note: If an error occurs while the ACI Make utility is building its dependency list, you will need to run the utility without the SUPPRESS_UNTIL_SERVER option to obtain information you need to locate the cause of the error.

The SUPPRESS_UNTIL_SERVER option does not affect the default printing or the LISTDEP and LISTINLINE options because their output does not occur until after the TACL Server process starts.

The SUPPRESS_UNTIL_SERVER option first became available in release 4.00 of the ACI Make utility.

Other Command Line Options

The ACI Make utility accepts the following command line options to control operations other than printing when it is run. One or more of these options can be specified to tailor a run of the ACI Make utility to meet your needs. Options are arranged alphabetically.

Each option appearing on the command line must be preceded by a hyphen (-), and multiple options must each be separated by a space.

ABEND_ON_ERROR Option

The ABEND_ON_ERROR option causes the Inspect utility to create the appropriate SAVEABEND file for debugging an unusual condition where the ACI Make utility stops itself. The ABEND_ON_ERROR option only causes SAVEABEND files for conditions that cause the ACI Make utility to print a “MAKE ERROR” message and terminate processing.

When the ACI Make utility stops itself for any of its known termination conditions, it returns completion information to TACL, but does not automatically create a SAVEABEND file. This is for system performance reasons. Usually the ACI Make utility error conditions can be determined from the messages displayed in the output, or can be resolved using LIST and LISTREPLACE options.

If the ACI Make utility fails for unexpected reasons (i.e., invalid address reference or illegal instruction), the Inspect utility creates a SAVEABEND file that can be used by HELP24 to better determine the cause of the unexpected termination, regardless of the ABEND_ON_ERROR option.

The ABEND_ON_ERROR option first became available in release 5.00 of the ACI Make utility.

D Option

The D option defines a Make macro with the associated text. Macro definitions placed in the command line with the D option take precedence over definitions for the same macros contained in Make macro files. Up to 25 Make macros can be defined in one command line.

Syntax

-Dmacro=text

macro — The name of the Make macro being defined or redefined.

text — The replacement text associated with the Make macro.

No spaces can be entered between the *D* and the end of the replacement text.
The following example contains definitions for four macros.

Example

```
-DBASE_PURGE_SPOOLER_ON=$(TRUE) &  
-DBASE_AXCEL_ON=$(FALSE) &  
-DBASE_TAL_OPTS=SYMBOLS,INSPECT &  
-DMYMACS=$volume.subvolume.XYZMACS
```

DUMPMACS Option

The DUMPMACS option takes all of the ACI Make macro information used when the ACI Make utility is run and copies it into a specified edit file. The utility creates the edit file if it does not exist, and deletes all previous information from an edit file that already exists.

Syntax

-DUMPMACS=file

file — The fully qualified name of the edit file into which the Make macro information is copied when the ACI Make utility is run.

Example

```
-DUMPMACS=$volume.subvolume.MACSFIL
```

A small portion of the edit file resulting from this option is shown on the next page. The listing also provides the names of the files from which the macro information is obtained.

For example, the first group of macros is defined in the file identified by the ALLMACS define, which is the CUSTMACS File.

```
__INCL_1 = =ALLMACS
CLASSIC_ON          = $(TRUE)
BASE_ON             = $(TRUE)
ATM_ON              = $(FALSE)
CARD_ON             = $(FALSE)
COR_ON              = $(FALSE)
CS_ON               = $(FALSE)
FHM_ON              = $(FALSE)
IEP_ON              = $(FALSE)
ITS_ON              = $(FALSE)
MAIL_ON             = $(FALSE)
POS_ON              = $(FALSE)
PRM_ON              = $(FALSE)
RETAIL_ON           = $(FALSE)
SWITCH_ON           = $(FALSE)
TB_ON               = $(FALSE)
TLR_ON              = $(FALSE)
__INCL_2 = \R.$EARL.BA60MAKE.ALLMACS
ALLMACS             = ALLMACS
TRUE                 = 1
FALSE                = 0
MAKE_TACL_MACROS     = $SYSACI.MAKEV500.TACLMACS
TACL_INIT            = $SYSACI.MAKEV500.TACLINIT
ACIAID               = $SYSTEM.SYSTEM.ACIAID
AXCEL                = $SYSTEM.SYSTEM.AXCEL
```

ERROR Option

The ERROR option accesses a file named MAKERROR, which is located on the Make object subvolume. The MAKERROR File contains information about each error message the ACI Make utility can generate. The description of each error message includes the text that is displayed for that particular error message, including any parameters that are replaced by the display. In some cases, additional information is given concerning the particular error. Error message help is available in interactive mode and in display and terminate mode.

Note: The ERROR option does not use a makefile in the RUN command.

See the HELP option for operations help.

The ERROR option first became available in release 5.01 of the ACI Make utility.

Syntax for interactive mode

-ERROR

Syntax for display and terminate mode

-ERROR={HELP | LIST | *nnnn*}

HELP — Displays the first page of the Makerror File, which provides some instructions on using the ERROR facility.

LIST — Displays, in column form, the numbers of the error messages that are currently contained in the Makerror File

nnnn — Displays the section that describes the error message (where *nnnn* represents the four-digit number assigned to the error).

Example

RUN \$SYSACI.MAKEV500.MAKEV501 -ERROR=1000

To enter the interactive mode, specify the ERROR option without the equal sign. The ACI Make utility displays the first section of the MAKERROR File and prompts you for interactive commands, which are the same ones available in the display and terminate mode.

The information available for error message 1000 is shown below.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

Text:

```
> MAKE ERROR: 1000
> -----
> Unable to purge %s
>
> Suggestions:
> -----
> None Available
>
```

Variables:

%s - File being purged.

Additional Information: None.

EQUIV Option

The EQUIV option is used to maintain file name references. This option creates an edit file to be used with the UPDATE option in a later run of the ACI Make utility. The utility places the edit file on the current subvolume. The default name for this file is EQUIV. However, you can specify a different file name. The ACI Make utility creates a new edit file if one does not exist and replaces the edit file if it does exist.

The file created with the EQUIV option contains entries for all files that need to be mapped to a new system. This option requires the third and fourth positions of the subvolume name in each product entry to be numeric. Under ACI naming conventions, the third and fourth positions identify the product release number (e.g., 60 in AT60SRC), and only the first four characters of the subvolume name are used for entries in the file created by this option. The release number restriction applies only to product subvolumes. Subvolumes such as \$system.system are not tested for this restriction.

Note: With the EQUIV option, the makefile specified when starting the ACI Make utility matches the global level macro file specified in the ALLMACS define. This file name usually identifies the ALLMACS File or the CUSTMACS File, and can be fully qualified if necessary.

See the “Maintaining File Name References” topic for additional information about using this option.

The EQUIV option first became available in release 5.00 of the ACI Make utility.

Syntax

-EQUIV [=*file*]

file — The fully qualified name of the edit file into which the file names are copied when the ACI Make utility is run. The edit file is named EQUIV if this parameter is not specified.

Examples

RUN \$SYSACI.MAKEV500.MAKEV501 *makefile* -EQUIV

**RUN \$SYSACI.MAKEV500.MAKEV501 *makefile* &
-EQUIV=\$volume.subvolume.MYEQUIV**

EQUIVALL Option

The EQUIVALL option is used to maintain file name references. This option creates an edit file to be used with the UPDATE option in a later run of the ACI Make utility. The utility places the edit file on the current subvolume.

The file created with the EQUIVALL option contains entries for all files that need to be mapped to the new system. This option **does not** require the third and fourth positions of the subvolume name to be numeric. The ACI Make utility captures **all** subvolume names and adds them to the file created by this option. The default name for this file is EQUIV. However, you can specify a different file name. The ACI Make utility creates a new edit file if one does not exist and replaces the edit file if it does exist.

Note: With the EQUIVALL option, the makefile specified when starting the ACI Make utility matches the global level macro file specified in the ALLMACS define. This file name usually identifies the ALLMACS File or the CUSTMACS File, and can be fully qualified if necessary.

See the “Maintaining File Name References” topic for additional information about using this option.

The EQUIVALL option first became available in release 5.00 of the ACI Make utility.

Syntax

-EQUIVALL [=*file*]

file — The fully qualified name of the edit file into which the file names are copied when the ACI Make utility is run. The edit file is named EQUIV if this parameter is not specified.

Examples

```
RUN $SYSACI.MAKEV500.MAKEV501 makefile -EQUIVALL
```

```
RUN $SYSACI.MAKEV500.MAKEV501 makefile &  
-EQUIVALL=$volume.subvolume.MYEQUIV
```

EXCEPT_FILE and EXCEPT_LOC Options

The EXCEPT_FILE and EXCEPT_LOC options are used together to create an edit file that contains exception Make macro entries. Entries are made only for macros that name targets and additional targets that need to be built. Use the NOBUILD and FORCE options with the EXCEPT_FILE and EXCEPT_LOC options to ensure that no target files are built and all needed entries are included. The FORCE option produces an edit file with all target overrides, for building targets on a location apart from the source. Without the FORCE option, the edit file contains only entries for outdated targets.

The EXCEPT_LOC option contains the temporary location (system, volume, and subvolume) for files identified by macros in an existing makefile, and the EXCEPT_FILE option provides the name of the edit file that contains the exception Make macro entries. The ACI Make utility reads all Make macro entries from the makefile specified in the RUN command, applies the temporary location from the EXCEPT_LOC option to each entry, and adds each modified entry to the edit file identified in the EXCEPT_FILE option. The ACI Make utility creates the edit file if it does not exist, and deletes all previous information from the edit file that already exists.

Syntax

-EXCEPT_FILE=*file*

file — The fully qualified name of the edit file that is to contain the exception Make macro entries.

Syntax

-EXCEPT_LOC=*location*

location — The temporary location, expressed as `\system.$volume` or `\system.$volume.subvolume`, for the files identified by the Make macro entries.

The NOBUILD and FORCE options are also used when creating the exception edit file to ensure that all entries are included.

Example

```
RUN $SYSACI.MAKEV500.MAKEV501 makefile -NOBUILD -FORCE &  
-EXCEPT_FILE=\system.$volume.MYSUBVOL.XYZMACS &  
-EXCEPT_LOC=\system.$volume
```

In response to the preceding command, the ACI Make utility creates a modified Make macro entry in the XYZMACS File for each target file and additional target entry in the specified *makefile*. Each entry in the XYZMACS File looks like the following example.

```
basrc_baddltal      = \system.$volume.BA60SRC.BADDLTAL
```

You must then review the XYZMACS File to ensure that it contains entries for all necessary target files and additional targets. You can add entries for any source files that you have copied to a separate location and any flags that you change during development or testing.

When running the ACI Make utility, you must add a D option that redefines the MYMACS macro to point to the XYZMACS File that you created. Use the CUSTMACS File as the global level macro file in the ALLMACS define. With the MYMACS macro pointing at your XYZMACS File instead of a dummy file, the ACI Make utility reads your XYZMACS File as the first step of processing the CUSTMACS File.

The EXCEPT_FILE and EXCEPT_LOC options first became available in release 4.00 of the ACI Make utility.

EXT_BUILD and EXT_USE Options

The EXT_BUILD option defines the extended segment file that is to contain all of the macro and file dependency information for a given run of the ACI Make utility, and the EXT_USE option identifies the extended segment file from which the ACI Make utility is to obtain macro information and the file dependency list.

Both of these options were available only in release 4.00 of the ACI Make utility. They were disabled in release 5.00 because, starting with release 5.00, the ACI Make utility always uses extended memory instead of temporary files.

FORCE Option

The FORCE option causes the ACI Make utility to omit modification timestamp comparisons when determining whether to rebuild target files. You must use the FORCE option in one of the following ways the first time the ACI Make utility is run for a new system.

- The FORCE option with the PURGE_TARGETS option removes all target files without rebuilding them. Run the ACI Make utility once with this combination to ensure that all target files are removed, then run the utility as many times as required to build all target files. If an error occurs during processing, the ACI Make utility leaves the target files that have already been built successfully, and builds only the target files that still need to be built.
- The FORCE option without the PURGE_TARGETS option rebuilds all target files. If an error occurs during processing, the ACI Make utility must rebuild all target files instead of building only the target files that still need to be built.

HELP Option

The HELP option accesses a file named MAKEHELP, which is located on the Make object subvolume. You can then select from the topics contained in the MAKEHELP File, with the information for each topic displayed on your terminal or sent to a designated spooler location. Operations help is available in interactive mode only.

Note: The HELP option does not use a makefile in the RUN command.

See the ERROR Option for help associated with error messages.

Syntax

-HELP

Example

RUN \$SYSACI.MAKEV500.MAKEV501 -HELP

The menu of available topics displayed in response to this command is shown below. From each page displayed, you can stop, display the next page of the MAKEHELP File, or type one of the topics shown on this menu.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

Help is available for the following MAKE topics.

GENERAL: General Information about what MAKE is and how to run it

HISTORY: History of the changes to MAKE

CURRENT_RELEASE_INFO: Displays only the info about the current release.

OPTIONS: Run time options for MAKE

INCLUDE: Include files

COMMENTS: Comment syntax

CONTINUATION: Line continuation syntax

MAKEFILE: General format of the makefile.

DEPEND: Dependency rule syntax. Specifies which source files affect target files and how to rebuild target (object) files.

NON_ACTIVE_DEPENDENCIES: Those dependencies that may or may not be available for a specific MAKE file.

ADDITIONAL_DEFINES: Files that are necessary to rebuild the target, but aren't desired to be used as dependencies.

ADDITIONAL_TARGETS: Files that are created while creating the "main" target. For example, DDL compiles often create ?tal, ?tacl, ?cobol all from one single DDL compile. The ?tacl and ?cobol would be considered additional targets.

COMMANDS: ASSIGN, CLEAR, PARAM, PURGE, and RUN.

SSVS: Search path subvolumes.

INLINE: Interactively passing launched processes source input.

MACRODEFS: Macro definition

MACROUSAGE: Macro invocation

PREDEFINED: Predefined macros

DSP00L: Using DSP00L (or other nowait processes) with MAKE.

CONDITIONS: Conditional execution directives

EXPRESSIONS: Expression evaluation for conditional execution

EXAMPLE1: The MAKEFILE used to build make

EXAMPLE2: A MAKEFILE using DSP00L for listings

LIMITS: Comparison with UNIX make, enhancements, changes, unsupported features.

HELP: Redisplays this menu.

Type <ctl Y> to stop, <CR> to continue, or 'Topic'<CR> >

HIGHPIN_OFF Option

The HIGHPIN_OFF option controls the process identification number (PIN) assigned to the TACL server process started by the ACI Make utility. Without this option, the ACI Make utility requests a high PIN (greater than 256) for the process. With this option, the utility requests a low PIN (less than 256) for the process. With or without this option, the ACI Make utility runs with a high PIN.

Syntax

-HIGHPIN_OFF

If it is necessary to run the ACI Make utility itself with a low PIN, the TACL HIGHPIN OFF startup option can be entered with the RUN command.

The HIGHPIN_OFF option first became available in release 5.01 of the ACI Make utility.

LINE_SIZE_MAX Option

The LINE_SIZE_MAX option controls the maximum space provided for dependency lists in makefiles. Without this option, the maximum line size is 250,000 bytes. The Longest Line Length value at the end of the ACI Make standard report provides the maximum line size being used, and Make error message 1800 appears if the line size is insufficient.

The LINE_SIZE_MAX option first became available in release 5.00 of the ACI Make utility.

Syntax

-LINE_SIZE_MAX=*size*

size — The maximum space, in bytes, provided for dependency lists in makefiles.

Example

-LINE_SIZE_MAX=270000

MAKE_FILE_DEP_CHK Option

The MAKE_FILE_DEP_CHK option expands the dependency checks performed by the ACI Make utility. This option ensures that a target file is rebuilt in situations where the makefile that defines the target is revised, even if none of the dependency files specified in that makefile are revised.

With this option, the ACI Make utility includes the modification timestamp of the makefile itself with the modification timestamps assigned to the dependencies contained in each target specification. The ACI Make utility rebuilds each target file in the makefile whenever the makefile is newer than the target file, as well as whenever a dependency file is newer than the target file.

Without this option, the ACI Make utility does not consider the modification timestamp of the makefile when it determines whether to rebuild a target file. As a result, the target file gets rebuilt only when one or more of the dependency files is older than the target file.

The MAKE_FILE_DEP_CHK option first became available in release 4.00 of the ACI Make utility.

MAX_RADIX_TBL_SIZE Option

The MAX_RADIX_TBL_SIZE option specifies the size, in bytes, of memory allocated to hold each of the internal tables created by the ACI Make utility. This option overrides the TACL MAX-RADIX-TBL-SIZE parameter for a single run of the ACI Make utility. The default value if this option and the TACL parameter are not specified is 5 million bytes. Refer to the “Setting the Radix Table Size” topic earlier in this section for information about determining the proper Radix table size.

Syntax

-MAX_RADIX_TBL_SIZE=*size*

size — The size of memory, in bytes, allocated to hold each of the internal tables created by the ACI Make utility.

Example

-MAX_RADIX_TBL_SIZE=500000

The MAX_RADIX_TBL_SIZE option first became available in release 4.00 of the ACI Make utility.

NOBUILD Option

The NOBUILD option causes the ACI Make utility to check file dependency information without building or rebuilding target files.

A report generated with the NOBUILD and FORCE options together provides the exact order in which the ACI Make utility will build target files without actually building them. The following example shows the information provided when only the NOBUILD and FORCE options are used. The FORCE option is required to ensure that no target files are omitted.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

```
RUN CMD > \r.$sysaci.makev500.makev501 \r.$earl.at60auth.authm &  
> -NOBUILD -FORCE
```

Define =ALLMACS equals: \R.\$EARL.BA60MAKE.CUSTOMACS

```
*** (\R.$EARL.AT60AUTH.AUTHCSEE) needs rebuilding!  
*** (\R.$EARL.AT60AUTH.AUTHLIBE) needs rebuilding!
```

```
*** MAKE complete 6 JUN 2001 13:22:15.09 ***
```

Summary:

```
Files rebuilt : 2  
Build errors  : 0  
Build warnings : 0
```

Elapsed Time : 00:00:09.75 (h:m:s.hs)

Time before starting first build: 00:00:00.00 (h:m:s.hs)

Longest Line Length (bytes): 1753

Line Size Max value (bytes): 250000

Number of SQL macro hits: 0

Number of Memory macro hits: 563

Bytes in build steps: 21190

Build Steps Table Size: 120000000

Information on RADIX Tables:

```
2 Tables Allocated 10000000 bytes each.  
- Table 1 Used 1756996 (17.6%)  
- Table 2 Used 3869586 (38.7%)
```

NO_DELETE_DEFINES Option

The NO_DELETE_DEFINES option causes the ACI Make utility to accept any TACL defines that are in effect when the utility is started. With this option, the ACI Make utility also does not delete any of the TACL defines that it adds dynamically (e.g., target defines and dependency defines). Instead, the utility passes these TACL defines on to any processes that it starts. Without this option, the ACI Make utility ignores all of the TACL defines passed to it when it starts and deletes TACL defines between each set of targets that it builds. Regardless of the setting of this option, the ACI Make utility does not remove any TACL defines from the TACL environment.

NO_EXT_CHECK Option

The NO_EXT_CHECK option instructs the ACI Make utility to use the extended segment file specified in the EXT_USE option without first performing an extended segment file modification timestamp check. The EXT_USE option is required when the NO_EXT_CHECK option is used.

This option was available only in release 4.00 of the ACI Make utility. It was disabled in release 5.00 because, starting with release 5.00, the ACI Make utility always uses extended memory instead of temporary files.

NO_STOP_ON_ERRORS Option

The NO_STOP_ON_ERRORS option instructs the ACI Make utility to continue processing if it encounters an error (e.g., a compiler error) while building a target file. With this option, the ACI Make utility stops building the target file with the error, but continues building all target files that do not depend upon the target file that has an error. Without this option, the ACI Make utility stops immediately when it encounters a problem.

After all of the problems are corrected, you can restart the ACI Make utility, and it rereads the Make macro files and makefiles. Then the utility can automatically resume processing at the point of failure because the modification timestamps of the dependency files are older than the modification timestamps of target files that were built successfully. If the NO_STOP_ON_ERRORS option was used on the earlier runs, only target files with errors and target files that have other target files with errors as dependency files need to be rebuilt. If the NO_STOP_ON_ERRORS option was not used on the earlier runs, processing continues where the first error was encountered during the previous run.

The `NO_STOP_ON_ERRORS` option is particularly valuable when you run the ACI Make utility unattended (e.g., overnight or weekends). With this option, you take greater advantage of the available time because all target files other than those with errors or those that depend on the files with errors can be processed without operator intervention.

The `NO_STOP_ON_ERRORS` option first became available in release 4.00 of the ACI Make utility.

PURGE_TARGETS Option

The `PURGE_TARGETS` option causes the ACI Make utility to check dependency information and determine which target files must be built or rebuilt, then purge all target files that need to be rebuilt. The ACI Make utility does not rebuild targets with the `PURGE_TARGETS` option. The difference between the `PURGE_TARGETS` option and the `NOBUILD` option is the added step of purging the target files, which is done only by the `PURGE_TARGETS` option.

Adding the `FORCE` option to the `PURGE_TARGETS` option causes the ACI Make utility to purge all target files without checking dependency information.

When you must rebuild all targets, consider running the ACI Make utility one time with the `PURGE_TARGETS` and `FORCE` options to remove all targets, then running the utility without the `FORCE` or `PURGE_TARGETS` options as many times as required to build all targets. This combination provides an effective alternative to running the utility with the `FORCE` option to build the targets. The `FORCE` option rebuilds all targets each time the utility is run.

If a target identified for rebuilding is part of the DDL dictionary file set, the ACI Make utility purges all files in the set. If a target identified for rebuilding is a partial name (`DICT`), no dictionary files are purged. The DDL dictionary file set includes the following files: `DICTALT`, `DICTCDF`, `DICTDDF`, `DICTKDF`, `DICTMAP`, `DICTOBL`, `DICTODF`, `DICTOTF`, `DICTOUF`, `DICTOUK`, `DICTRDF`, `DICTTKN`, `DICTTYP`, and `DICTVER`.

The `PURGE_TARGETS` option first became available in release 5.00 of the ACI Make utility.

SYNTAX Option

The SYNTAX option instructs the ACI Make utility to flag any macro that appears to be in error in the report output. For example, assume that the CUSTMACS File contains the following macro:

```
classic_on          = $(TRXE)
```

The report generated by the ACI Make utility with the SYNTAX option contains the following message:

```
*** WARNING *** Possible bad macro definition: TRXE
```

Caution: Use the NOBUILD option whenever you use the SYNTAX option. The SYNTAX option without the NOBUILD option instructs the ACI Make utility to bypass any target errors and continue building target files, including target files with dependencies on the object that had an error.

Example

```
RUN $SYSACI.MAKEV500.MAKEV501 BA60MAKE.B24M &  
-NOBUILD -SYNTAX
```

The SYNTAX option first became available in release 4.00 of the ACI Make utility.

TEST Option

The TEST option causes the ACI Make utility to check your current Tandem volume and subvolume for files that match Make file macros. If the utility finds any, it updates the macros with the file location of your current volume and subvolume, and uses the updated macros to build the module. This option permits you to point to temporary files for testing purposes without modifying the associated Make file macros. It is particularly useful when you are building one module.

For example, if your current Tandem volume and subvolume contains a file named TESTFILE and the ACI Make utility locates any file definition macros that point to a file named TESTFILE, the utility revises the file definition macros to point to the file on your current volume and subvolume before building the module. The change is for the current run of the ACI Make utility only.

The ACI Make utility builds all target files on your current Tandem volume and subvolume. Before using the TEST option to update target files, use one of the following methods to determine whether the correct target files are built.

- The TESTCHK option of the ACI Make utility prompts you before building each target file.
- The TEST and NOBUILD options of the ACI Make utility, when used together, list all target files to be built without actually building them.

TESTCHK Option

The TESTCHK option causes the ACI Make utility to check your current Tandem volume and subvolume for files and replace the macros that correspond to those files. Like the TEST option, this option permits you to point to temporary files for testing purposes without modifying the associated Make file macros. However, the TESTCHK option prompts you for confirmation before it builds each target file.

TIMESTAMP_FILE Option

The TIMESTAMP_FILE option identifies a file from which the ACI Make utility obtains timestamp information to determine whether a target file must be built or rebuilt. This file is considered a dependency for every target file. The ACI Make utility builds or rebuilds every target file that has an older timestamp than the one on the file identified by this option.

By using the TIMESTAMP_FILE option, you can restart the ACI Make process when an error occurs without also having to rebuild the target files that were built successfully. For example, when you must rebuild all target files, consider creating a new file that contains a current modification timestamp, then running the utility with the TIMESTAMP_FILE option specifying this new file as many times as required to build all target files. This combination provides an effective alternative to running the utility with the FORCE option to build the target files. The FORCE option rebuilds all target files each time the utility is run. The TIMESTAMP_FILE option leaves the target files that have already been built successfully since the time when the new timestamp file was created or modified, and builds only the target files that still need to be built.

Create a new, empty file that has a current modification timestamp, then use the name of this file with the `TIMESTAMP_FILE` option. Do not use the `TIMESTAMP_FILE` option and `FORCE` option together because the `FORCE` option will cause the ACI Make utility to ignore the `TIMESTAMP_FILE` option.

Syntax

`-TIMESTAMP_FILE=file`

file — The fully qualified name of the file from which the ACI Make utility obtains the timestamp information.

Example

`-TIMESTAMP_FILE=$volume.subvolume.TIMEFILE`

The `TIMESTAMP_FILE` option first became available in release 5.00 of the ACI Make utility.

UPDATE Option

The `UPDATE` option uses the contents of the edit file created on the current subvolume by the ACI Make utility with the `EQUIV` option or `EQUIVALL` option. The default name for this edit file is `EQUIV`. However, if you specified a different file name for the `EQUIV` option or `EQUIVALL` option, you must also use that file name with the `UPDATE` option. The edit file must exist when this option is used.

With the `UPDATE` option, the makefile specified when starting the ACI Make utility matches the global level macro file specified in the `ALLMACS` define. This file name usually identifies the `ALLMACS` File or the `CUSTMACS` File, and can be fully qualified if necessary. The ACI Make utility preserves the position of comments or sequence numbers to the right of modified filenames to update the makefiles.

Note: ACI recommends always using the `DETAIL` option with the `UPDATE` option. It is very important to use this option for BASE24 Release 6.0 sequenced makefiles.

See the “Maintaining File Name References” topic for additional information about using this option.

The `UPDATE` option first became available in release 5.00 of the ACI Make utility.

Syntax

-UPDATE [=*file*]

file — The fully qualified name of the edit file from which the old and new file names are obtained. The edit file is named EQUIV if this parameter is not specified.

Examples

```
RUN $SYSACI.MAKEV500.MAKEV501 makefile -UPDATE -DETAIL
```

```
RUN $SYSACI.MAKEV500.MAKEV501 makefile &  
-UPDATE=$volume.subvolume.MYEQUIV -DETAIL
```

VERSION Option

The VERSION option causes the ACI Make utility to display the current version of the utility executable file.

Syntax

```
RUN $SYSACI.MAKEV500.MAKEV501 -VERSION
```

Note: The VERSION option does not use a makefile in the RUN command.

Running the ACI Make utility with the VERSION option displays the following information on your terminal.

```
ACI MAKE utility - Version: MAKEV501  May 23 2001 13:45:30
```

The VERSION option first became available in release 4.00 of the ACI Make utility.

Maintaining File Name References

The process of moving a Make tree from one system or volume location to another involves modifying all the file name references contained in the make files between those system locations.

With changes in the ACI Make utility to allow Software File Maintenance sequence numbers in BASE24 make files starting in release 6.0, it became necessary to perform more sophisticated manipulations of the file reference strings.

As a result, starting with release 5.00, the ACI Make utility has four command line options available to perform the file name reference maintenance functionality previously provided only by the Makescan utility. The EQUIV and EQUIVALL options are used to create the EQUIV File, the UPDATE option is used for reading an EQUIV File to update the file references in the new Make tree location, and the DETAIL option is used to provide additional reporting detail.

The ACI Make utility uses the actual path of the include files in the Make stream and the actual product flag settings. When creating the EQUIV File, the ACI Make utility includes only file macros or include lines that are within conditional text for activated products. However, if the product flag is turned on and the included file does not exist, the ACI Make utility adds the appropriate system, volume, and subvolume reference to the EQUIV File, displays error message 1660 for informational purposes, and continues processing, even though the included file is not found.

For this reason, consider setting all product flags in the CUSTMACS File on the BA60 subvolume to true when you produce the EQUIV File. The one exception to this recommendation is the RETAIL_ON flag in the CUSTMACS File on the BA60 subvolume, which is set to true only if you are installing one or more of the BASE24-retail products.

For example, assume the Make stream contains the following conditional text for the Really Cool Product, and this product is not on the current system.

```
! IF $(REALLY_COOL_PRODUCT_ON) = $(TRUE) | THEN |  
    !include \mysys.$data01.rc60src.rcsrcmm  
!ENDIF
```

If the REALLY_COOL_PRODUCT_ON flag is false, the ACI Make utility does not perform the INCLUDE statement for the Really Cool Product, and does not add an entry for \mysys.\$data01.rc60 to the EQUIV File.

If the `REALLY_COOL_PRODUCT_ON` flag is true, the ACI Make utility performs the `INCLUDE` statement for the Really Cool Product, and adds an entry for `\mysys.$data01.rc60` to the EQUIV File. If the file `\mysys.$data01.rc60src.rcsrcmm` exists, the ACI Make utility drills down through that portion of the Make tree. If an expected file does not exist, the ACI Make utility continues processing without drilling down through that portion of the Make tree. Because it is creating an EQUIV File, the ACI Make utility allows the nonexistent include file condition to occur, only raising a warning (error message 1660) in its reports.

Note: The recommendation to set all product flags in the CUSTMACS File on the BA60 subvolume to true applies only when you produce the EQUIV File, not when you are using the EQUIV File to update the file name references. With the `UPDATE` option, the ACI Make utility stops processing if it cannot locate a file that is required because the associated product flag in the CUSTMACS File is set to true.

EQUIV Report Examples

The `DETAIL` option controls the amount of information provided in the ACI Make utility report generated with the `EQUIV`, `EQUIVALL`, or `UPDATE` option.

Report Without DETAIL Option

The following example is a portion of the report generated by the ACI Make utility with the `EQUIV` option. The number at the beginning of each entry identifies the level of the file being opened. By using `INCLUDE` statements, you can nest files to a maximum depth of ten levels, indicated on this report with the numbers 0 through 9. The arrow following the level number indicates the relationship between the current level and the previous level. For example, the first three entries (`>`) show the current level number is greater than the previous level number, while the fourth entry (`<`) shows the current level number is less than the previous level number. This relationship is obvious in a report without the `DETAIL` option, but is very helpful with the `DETAIL` option. Following the list of all files opened, the report shows all entries placed in the new EQUIV File.

```
ACI MAKE utility - Version: MAKEV501   May 23 2001 13:45:30
```

```
RUN CMD > \r.$sysaci.makev500.makev501 &  
        > $volume.subvolume.custmacs -EQUIV
```

```
Define =ALLMACS equals: \R.$volume.subvolume.CUSTMACS
```

```
>>>> 0 > $volume.subvolume.CUSTMACS
>>>> 1 > \R.$BRIDGE.BA60MAKE.ALLMACS
>>>> 2 > \R.$BRIDGE.OK60SRC.ITSFM
>>>> 1 < \R.$BRIDGE.BA60MAKE.ALLMACS
>>>> 2 > \R.$BRIDGE.OK60SRC.ITSMM
>>>> 3 > \R.$BRIDGE.OK60SRC.KNLFM
>>>> 2 < \R.$BRIDGE.OK60SRC.ITSMM
>>>> 3 > \R.$BRIDGE.OK60SRC.KNLMM
>>>> 2 < \R.$BRIDGE.OK60SRC.ITSMM
>>>> 3 > \R.$BRIDGE.OK60AFM.KAFMMF
>>>> 2 < \R.$BRIDGE.OK60SRC.ITSMM
>>>> 3 > \R.$BRIDGE.OK60AFM.KAFMMM
>>>> 2 < \R.$BRIDGE.OK60SRC.ITSMM
>>>> 3 > \R.$BRIDGE.OK60DDL.KDDLFM
>>>> 2 < \R.$BRIDGE.OK60SRC.ITSMM
```

```
Creating Equiv File: \R.$volume.subvolume.EQUIV
at : 01-MAY-2001 07:35
```

Entries:

\r.\$volume.subvolume	\NEWSYSTEM.\$NEWVOLUME.subvolume
\r.\$dsv.zspidef	\NEWSYSTEM.\$NEWVOLUME.zspidef
\r.\$dsv.zsysdefs	\NEWSYSTEM.\$NEWVOLUME.zsysdefs
\r.\$bridge.acitpl	\NEWSYSTEM.\$NEWVOLUME.acitpl
\r.\$bridge.at60	\NEWSYSTEM.\$NEWVOLUME.at60
\r.\$bridge.ba60	\NEWSYSTEM.\$NEWVOLUME.ba60
\r.\$bridge.ok60	\NEWSYSTEM.\$NEWVOLUME.ok60
\r.\$sysaci.aciutils	\NEWSYSTEM.\$NEWVOLUME.aciutils
\r.\$sysaci.apmsobj	\NEWSYSTEM.\$NEWVOLUME.apmsobj
\r.\$sysaci.rdebug	\NEWSYSTEM.\$NEWVOLUME.rdebug
\r.\$sysaci.scribe	\NEWSYSTEM.\$NEWVOLUME.scribe
\r.\$sysaci.xnlib	\NEWSYSTEM.\$NEWVOLUME.xnlib
\r.\$sysaci.xpnet	\NEWSYSTEM.\$NEWVOLUME.xpnet
\r.\$system.sys00	\NEWSYSTEM.\$NEWVOLUME.sys00
\r.\$system.system	\NEWSYSTEM.\$NEWVOLUME.system
\$sysaci.acilibi	\$NEWVOLUME.acilibi
\$sysaci.makev500	\$NEWVOLUME.makev500
\$system.system	\$NEWVOLUME.system

Equiv File Creation Complete

*** MAKE complete 1 MAY 2001 7:35:27.18 ***

Report With DETAIL Option

The following example is a portion of the report generated by the ACI Make utility with the EQUIV option plus the DETAIL option. All information in the report generated without the DETAIL option also appears in this report. Arrows (→) identify additional information provided by using the DETAIL option. This example shows the additional information provided for just the first entry of the

report generated without the DETAIL option. There are six additional entries in this report between the first two entries of the previous report. Four of these entries identify macros from the CUSTMACS File. The *NEW SUBVOL* labels identify subvolumes that have entries in the EQUIV File.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

```
RUN CMD > \r.$sysaci.makev500.makev501 &
        > $volume.subvolume.custmacs -EQUIV -DETAIL
```

```
Define =ALLMACS equals: \R.$volume.subvolume.CUSTMACS
```

```
>>>> 0 > $volume.subvolume.CUSTMACS
        -> \R.$BRIDGE.BA60SRC.ZTMPMACS
*NEW SUBVOL* -> \r.$bridge.ba60
        -> \R.$BRIDGE.BA60CSE.ZTMPMACS
        -> \R.$volume.subvolume.POBJ
*NEW SUBVOL* -> \r.$volume.subvolume
        -> \R.$BRIDGE.BA60MAKE.ALLMACS
>>>> 1 > \R.$BRIDGE.BA60MAKE.ALLMACS
```

EQUIV File Example

The EQUIV option causes the ACI Make utility to capture the system, volume, and subvolume information used in file macros on a source system and write them into an edit file on the current volume and subvolume. That file, which has a default name of EQUIV, can be transported to the target system and used for mapping the various file names to their new locations. This is accomplished by updating the EQUIV File with the target system locations, then running the ACI Make utility with the UPDATE option.

The following table shows a portion of the EQUIV File located on the BA60MAKE subvolume on the ACI system named \r. The left column has the system, volume, and subvolume references used on the source system. Entries in the left column must not be altered because they must match the file names from the source system used to create the EQUIV File. The source system is an ACI system (i.e., \cfs, \global, \gr8fl, or \r) on tapes provided by ACI software management, and can be a customer system name when an existing customer system is used.

The right column has NEWSYSTEM, NEWVOLUME, and the same subvolume name as the left column. The subvolume names take two forms—the full subvolume name and the subvolume root name. The full subvolume name is used for unique subvolumes and the subvolume root name is used for product subvolumes. The root name is composed of a two-character product identifier

(e.g., BA for base) and a two-digit release number (e.g., 60 for release 6.0). The NEWSYSTEM and NEWVOLUME entries in the right column must be altered to reflect the file location of each subvolume or subvolume root on the new system.

Note: The left-column entries in this table, the Tandem system name in particular, must reflect the source system from which the Make macro files and makefiles are obtained. The EQUIV File created by using the EQUIV option of the ACI Make utility captures all of the system and volume information. New BASE24 software shipped from ACI includes a correct EQUIV File. Only the new system information (i.e., values for each of the \NEWSYSTEM and \$NEWVOLUME variables) must be provided to this EQUIV File.

```
#
# EQUIV File Created on 30-APR-2001 15:04
#
# =ALLMACS = \r.$sword.ba60make.custmacs
# MAKEFILE = \r.$sword.ba60make.custmacs
#

\r.$dsv.zspidef          \NEWSYSTEM.$NEWVOLUME.zspidef
\r.$dsv.zsysdefs         \NEWSYSTEM.$NEWVOLUME.zsysdefs
\r.$sword.acitpl         \NEWSYSTEM.$NEWVOLUME.acitpl
\r.$sword.at60           \NEWSYSTEM.$NEWVOLUME.at60
\r.$sword.ba60           \NEWSYSTEM.$NEWVOLUME.ba60
\r.$sword.bh60           \NEWSYSTEM.$NEWVOLUME.bh60
\r.$sword.bp60           \NEWSYSTEM.$NEWVOLUME.bp60
\r.$sword.fh60           \NEWSYSTEM.$NEWVOLUME.fh60
\r.$sword.ma60           \NEWSYSTEM.$NEWVOLUME.ma60
\r.$sword.oc60           \NEWSYSTEM.$NEWVOLUME.oc60
\r.$sword.oe60           \NEWSYSTEM.$NEWVOLUME.oe60
\r.$sword.ok60           \NEWSYSTEM.$NEWVOLUME.ok60
\r.$sword.ps60           \NEWSYSTEM.$NEWVOLUME.ps60
\r.$sword.sw60           \NEWSYSTEM.$NEWVOLUME.sw60
\r.$sword.tb60           \NEWSYSTEM.$NEWVOLUME.tb60
\r.$sword.tr60           \NEWSYSTEM.$NEWVOLUME.tr60
```


Installing BASE24 with the ACI Make Utility

Perform the following steps running ACI Make utility to install BASE24 software. These steps are a summary of the detailed installation steps presented in the *Installation of BASE24 Classic 6.0 Systems*.

Steps 1 through 4 are necessary only when installing a new BASE24 system. Steps 5 through 7 are necessary each time the ACI Make utility is used.

Note: Set the Radix table size and ALLMACS define before running the ACI Make utility. See the “Running the ACI Make Utility” topic for additional information.

1. Make sure that the following files are on your system in order to compile a BASE24 system using the ACI Make utility:

```
$volume.BA60MAKE.ALLMACS  
$volume.BA60MAKE.CUSTMACS  
$volume.BA60MAKE.EQUIV  
$volume.MAKEV500.MAKEV501  
$volume.MAKEV500.MAKERROR  
$volume.MAKEV500.MAKEHELP  
$volume.MAKEV500.TACLINIT  
$volume.MAKEV500.TACLMACS
```

2. Alter the right half of the entries in the EQUIV File on the BA60MAKE subvolume to reflect the file locations on your system.

Note: ACI provides the EQUIV File with a new BASE24 system. If you are preparing to move an existing BASE24 system to a new location, perform the following steps to create your own EQUIV File instead of using the one from ACI:

- a. Place a temporary copy of the CUSTMACS File from the BA60MAKE subvolume on another subvolume. This CUSTMACS File can still point to the ALLMACS File on the BA60MAKE subvolume.
- b. Set all product flags in the temporary CUSTMACS File to true.

Note: Set the RETAIL_ON flag in the temporary CUSTMACS File to true only if you are installing one or more of the BASE24-retail products.

- c. Run the ACI Make utility with the EQUIV option to create a new EQUIV File. Use the temporary CUSTMACS File for the ALLMACS define and the makefile name.

- d. Copy the new EQUIV File to the new location with the rest of the BASE24 software.
 - e. Alter the right half of the new EQUIV File entries to reflect the file locations on your new system.
3. Update the CUSTMACS File on the BA60MAKE subvolume to modify configuration defines necessary and set product flags for all installed products to true.
4. Run the ACI Make utility with the UPDATE option to update all Make macros.
5. In a test or development environment, create a customized global level macro file (e.g., XYZMACS) containing any overrides that need to be made to the CUSTMACS File that was updated in the previous step.

Note: Add a D command line option that redefines the MYMACS macro in the CUSTMACS File to point to the XYZMACS File that you created. Use the CUSTMACS File as the global level macro file in the ALLMACS define. With the MYMACS macro pointing at your XYZMACS File, the ACI Make utility reads your XYZMACS File as the first step of processing the CUSTMACS File.

6. Use an obey file to run the ACI Make utility with the NOBUILD option. Refer to “Using Obey Files to Start the ACI Make Utility” earlier in this section for information about creating these obey files.
 7. Use an obey file to run the ACI Make utility without the NOBUILD option and create all desired target files. Refer to “Using Obey Files to Start the ACI Make Utility” earlier in this section for information about creating these obey files.

If errors are present, correct the errors and repeat steps 6 and 7 as many times as necessary to achieve the desired results.

Performing Specific Tasks with the ACI Make Utility

Several specific tasks are performed regularly with the ACI Make utility. They include obtaining dependency lists, compiling a subset of the system, compiling DDLs, compiling Pathway requester processes, and using multiple makefiles. Each of these tasks is described in this subsection.

Note: The BASE24 ITS examples in the remainder of this section use release 4.00 of the ACI Make utility and release 1.3 of the BASE24 ITS software.

Obtaining Dependency Lists for a Specific Executable Module

You can use the ACI Make utility with the LISTDEP and NOBUILD options to obtain a dependency list for a specific BASE24 procedural or BASE24 ITS module, as shown in the following examples.

BASE24 Procedural Module

Finding all of the dependency files for the BASE24 procedural ISO Host Interface module involves the single step of running the ACI Make utility with the HISOM Makefile on the BA60HISO subvolume. A portion of the listing that results from this step is shown below. The total number of targets and dependencies depends on which flags are set to true in the ALLMACS File. The source files in this listing are the dependencies.

```
ACI MAKE utility - Version: MAKEV501  May 23 2001 13:45:30
```

```
RUN CMD > \r.$sysaci.makev500.makev501 \r.$earl.ba60hiso.hisom &
> -NOBUILD -LISTDEP -DMYMACS=$volume.subvolume.XYZMACS
```

```
Define =ALLMACS equals: \R.$EARL.BA60MAKE.CUSTMACS
```

```
*** TARGET: 17MAY2001 20:26:00.34 - \R.$EARL.BA60HISO.BAHISOE
*** SOURCE:  3MAY2001 10:17:05.26 - \R.$EARL.BA60HISO.BAHISOS
*** File (\R.$EARL.BA60HISO.BAHISOE) did not require rebuilding
*** TARGET: 17MAY2001 20:26:46.04 - \R.$EARL.BA60HISO.BAHISOO
*** SOURCE:  3JAN2001 22:17:44.44 - \R.$SYSACI.ACIUTILS.UTILDEFS
*** SOURCE:  3JAN2001 22:18:59.53 - \R.$SYSACI.ACIUTILS.UTILEXTS
*** SOURCE:  7MAY2001 11:20:22.42 - \R.$EARL.BA60DDL.DDLBCNST
*** SOURCE:  2JAN2001 17:11:18.04 - \R.$EARL.BA60DDL.DDLFCNF
*** SOURCE:  1NOV2000 09:59:19.72 - \R.$EARL.BA60DDL.DDLFEMF
```

```
*** SOURCE: 2JAN2001 17:00:57.95 - \R.$EARL.BA60DDL.DDLFHCF
*** SOURCE: 1NOV2000 09:59:32.64 - \R.$EARL.BA60DDL.DDLFKEYF
*** SOURCE: 1NOV2000 09:59:39.40 - \R.$EARL.BA60DDL.DDLFSAF
*** SOURCE: 1NOV2000 09:59:42.65 - \R.$EARL.BA60DDL.DDLFTKN
*** SOURCE: 1NOV2000 09:59:49.10 - \R.$EARL.BA60DDL.DDLGHLTH
*** SOURCE: 11MAY2001 14:34:37.27 - \R.$EARL.BA60DDL.DDLGISO
*** SOURCE: 2JAN2001 17:07:52.55 - \R.$EARL.BA60DDL.DDLGPTH
*** SOURCE: 1NOV2000 10:00:14.20 - \R.$EARL.BA60DDL.DDLGX25
*** SOURCE: 12APR2001 13:57:11.25 - \R.$EARL.BA60DEFS.DDLGDEFS
*** SOURCE: 14JAN2001 20:05:53.02 - \R.$EARL.BA60HISO.BAHISOD
*** SOURCE: 17MAY2001 20:26:00.34 - \R.$EARL.BA60HISO.BAHISOE
*** SOURCE: 3MAY2001 10:17:10.59 - \R.$EARL.BA60HISO.BAHISOG
*** SOURCE: 3MAY2001 10:17:05.26 - \R.$EARL.BA60HISO.BAHISOS
*** SOURCE: 9MAY2001 15:28:10.54 - \R.$EARL.BA60SRC.BATKNID
*** SOURCE: 17MAY2001 19:34:42.74 - \R.$EARL.BA60SRC.BAUTILE
*** SOURCE: 17MAY2001 19:38:16.10 - \R.$EARL.BA60SRC.HISWUTLE
*** SOURCE: 10NOV1992 16:39:48.29 - \R.$SYSACI.ACIUTILS.KEYUTILE
*** SOURCE: 26MAR1996 22:36:16.55 - \R.$SYSACI.RDEBUD.DUMPEXT
*** SOURCE: 4DEC1997 18:53:51.39 - \R.$SYSACI.XPNET.GLOBALS
*** SOURCE: 28MAY1997 19:05:57.96 - \R.$SYSACI.XPNET.INITLIBS
*** SOURCE: 31OCT1997 13:23:17.33 - \R.$SYSACI.XPNET.SKEL
*** SOURCE: 25SEP1997 11:47:09.54 - \R.$SYSACI.XPNET.SKELEXTS
*** SOURCE: 2JUL1998 11:40:23.89 - \R.$SYSTEM.SYSTEM.EXTDECS
*** File (\R.$EARL.BA60HISO.BAHIS00) did not require rebuilding
*** TARGET: 17MAY2001 20:26:52.56 - \R.$EARL.AT60HISO.ATHISOE
*** SOURCE: 1MAY2001 14:50:17.59 - \R.$EARL.AT60HISO.ATHISOS
*** File (\R.$EARL.AT60HISO.ATHISOE) did not require rebuilding
```

BASE24 ITS Module

Finding all of the dependency files for the BASE24 ITS CIEPLO executable module requires the following steps. In this example, the ACI Make utility is run with the NOBUILD and LISTDEP options each time.

1. Run the ACI Make utility with the CIEPLM Makefile on the OC60LINK subvolume. The output is shown below.

```
ACI MAKE utility - Version: MAKEV400 Jan 27 1999 19:33:26
Define =ALLMACS equals: \R.$volume.subvolume.CUSTOMACS
```

```
*** TARGET: 30AUG1999 16:57:09.96 - \R.$BRIDGE.OC130BJ.CIEPLO
*** SOURCE: 8JUL1999 09:11:46.25 - \R.$BRIDGE.OK130BJ.OKLIBTO
*** SOURCE: 30AUG1999 16:47:31.73 - \R.$BRIDGE.OE130BJ.IEPL0
*** SOURCE: 8JUL1999 09:12:01.92 - \R.$BRIDGE.OC130BJ.OCLIBTO
*** SOURCE: 30AUG1999 16:56:18.94 - \R.$BRIDGE.OC130BJ.VRUETO
*** SOURCE: 30AUG1999 16:52:01.47 - \R.$BRIDGE.OC130BJ.IMGRTO
*** SOURCE: 30AUG1999 16:56:43.16 - \R.$BRIDGE.OC130BJ.IMTETO
*** File (\R.$BRIDGE.OC130BJ.CIEPLO) did not require
rebuilding
```

```
*** MAKE complete  7 SEP 1999 10:02:14.85 ***
```

Summary:

```
Files rebuilt : 0
Build errors  : 0
Build warnings : 0
```

2. Using the dependency list created in step 1, run the ACI Make utility against the makefile for each source file in the list, as shown in the following table.

Source File	Use Makefile
oklibto	ok13lib.oklibm
ieplo	oe13link.ieplm
oclibto	oc13lib.oclibm
vructo	oc13vru.vreutm
imgrto	oc13imgr.imgrtm
imteto	oc13imt.imtetm

The output for each module shows the source files it uses. The following example is the ACI Make utility output from the OKLIBM Makefile. Only the target files are object files. Since none of the source files are object files, the utility does not need to be run again for this module.

```
ACI MAKE utility - Version: MAKEV400  Jan 27 1999 19:33:26
Define =ALLMACS equals: \R.$volume.subvolume.CUSTOMACS
```

```
*** TARGET:  8JUL1999 09:09:18.49 - \R.$BRIDGE.OK130BJ.OKLIBTE
*** SOURCE:  8JUL1999 08:52:14.80 - \R.$BRIDGE.OK13LIB.OKLIBTS
*** File (\R.$BRIDGE.OK130BJ.OKLIBTE) did not require
    rebuilding
*** TARGET:  8JUL1999 09:11:46.25 - \R.$BRIDGE.OK130BJ.OKLIBTO
*** SOURCE: 13MAY1999 17:24:28.46 - \R.$BRIDGE.OK13DDL.OKDT
*** SOURCE:  7DEC1998 16:05:03.90 - \R.$BRIDGE.OK13LIB.OKLIBTG
*** SOURCE:  8JUL1999 08:52:14.80 - \R.$BRIDGE.OK13LIB.OKLIBTS
*** SOURCE:  8JUL1999 09:09:18.49 - \R.$BRIDGE.OK130BJ.OKLIBTE
*** SOURCE:  7DEC1998 16:08:02.50 - \R.$BRIDGE.OK13SRC.OKTG
*** SOURCE:  2JUL1998 15:40:37.78 - \R.$SYSTEM.SYSTEM.EXTDECS0
*** File (\R.$BRIDGE.OK130BJ.OKLIBTO) did not require
    rebuilding

*** MAKE complete  7 SEP 1999 11:45:11.92 ***
```

Summary:

Files rebuilt : 0
Build errors : 0
Build warnings : 0

Output for each of the makefiles from the table above, except the IEPLM Makefile, contains only source files. The following example is the ACI Make utility output from the IEPLM Makefile. All of the source files listed in this report are object files (i.e., file names ending in *o*), meaning the ACI Make utility must also be run with additional makefiles. The output is shown below.

ACI MAKE utility - Version: MAKEV400 Jan 27 1999 19:33:26
Define =ALLMACS equals: \R.\$volume.subvolume.CUSTOMACS

```
*** TARGET: 30AUG1999 16:47:31.73 - \R.$BRIDGE.OE130BJ.IEPL0
*** SOURCE: 8JUL1999 09:43:47.36 - \R.$BRIDGE.OK130BJ.ANSTO
*** SOURCE: 8JUL1999 09:44:01.16 - \R.$BRIDGE.OK130BJ.CLASTO
*** SOURCE: 8JUL1999 09:45:04.21 - \R.$BRIDGE.OK130BJ.EVTLO
*** SOURCE: 8JUL1999 09:11:46.25 - \R.$BRIDGE.OK130BJ.OKLIBTO
*** SOURCE: 30AUG1999 16:44:06.16 - \R.$BRIDGE.OK130BJ.PCTLTO
*** SOURCE: 30AUG1999 16:44:25.84 - \R.$BRIDGE.OK130BJ.TRCTO
*** SOURCE: 30AUG1999 16:45:01.26 - \R.$BRIDGE.OK130BJ.XPNT0
*** SOURCE: 8JUL1999 09:58:16.10 - \R.$BRIDGE.OE130BJ.CTXMTO
*** SOURCE: 8JUL1999 09:58:41.58 - \R.$BRIDGE.OE130BJ.DESTTO
*** SOURCE: 30AUG1999 16:46:02.55 - \R.$BRIDGE.OE130BJ.EPCSTO
*** SOURCE: 30AUG1999 16:46:56.10 - \R.$BRIDGE.OE130BJ.GENETO
*** SOURCE: 8JUL1999 10:01:14.90 - \R.$BRIDGE.OE130BJ.MISOTO
*** File (\R.$BRIDGE.OE130BJ.IEPL0) did not require rebuilding

*** MAKE complete 7 SEP 1999 11:35:24.75 ***
```

Summary:

Files rebuilt : 0
Build errors : 0
Build warnings : 0

- Using the dependency list created for the IEPLM Makefile in step 2, run the ACI Make utility against the makefile for each source file in the list, as shown in the following table.

Source File	Use Makefile
ansto	ok13ans.anism
clasto	ok13clas.clasm
evtlo	ok13evt.evtm

Source File	Use Makefile
oklibto	ok13lib.oklibm
pctlto	ok13pctl.pctlm
trcto	ok13trc.trcm
xpnto	ok13xpn.xnpm
ctxmto	ok13ctxm.ctxmm
destto	ok13dest.destm
epcsto	ok13epcs.epcsm
geneto	ok13gene.genem
misoto	ok13miso.misom

Compiling a Subset of Your System

The B24M Makefile is used to run the ACI Make utility for a complete BASE24 procedural and BASE24 ITS system. However, during the software development cycle it may be necessary to compile subsets of your BASE24 system. Subsets exist at the levels shown in the following table. See “Makefiles” and associated topics in section 2 for additional information about the makefiles at each of these levels.

Level	BASE24 Procedural	BASE24 ITS
1	Master	Master
2	Product group	Object group
3	Product	Subsystem
4	Process	Object

Regardless of the makefile you specify in the RUN command, you must set an ALLMACS define value and set the Radix table size. Consider using an obey file for consistently starting the ACI Make utility. See “Using Obey Files to Start the ACI Make Utility” for a discussion of obey files.

BASE24 Procedural Subsets

To run the ACI Make utility for a BASE24 procedural subset, specify the appropriate product group (i.e., CLSCM Makefile), product (e.g., POSM Makefile), or process (e.g., HISOM Makefile) as the makefile in the RUN command.

The CLSCM Makefile on the BA60SRC subvolume contains instructions for the ACI Make utility to compile the entire BASE24 procedural product group.

The POSM Makefile on the PS60SRC subvolume contains instructions for the ACI utility to compile the BASE24-pos product.

The HISOM Makefile on the BA60HISO subvolume contains instructions for the ACI Make utility to compile the entire ISO Host Interface process. The ACI Make utility runs the hierarchy for the ISO Host Interface process and compiles, binds, and accelerates (if needed) the product modules (i.e., base, ATM, POS, teller, fhm, mail, SSB, and telebanking).

Also, there are makefiles on each of the product-specific ISO Host Interface subvolumes, (e.g., BA60HISO.BAHISOM, AT60HISO.ATHISOM, PS60HISO.PSHISOM). These makefiles contain instructions for the ACI Make utility to compile the product-specific module.

BASE24 ITS Subsets

To run the ACI Make utility for a BASE24 ITS subset, specify the appropriate object group (i.e., ITSM Makefile), subsystem (e.g., CORM Makefile), or object (e.g., CIEPLM Makefile) as the makefile in the RUN command.

The ITSM Makefile on the OK60SRC subvolume instructs the ACI Make utility to compile the entire BASE24 ITS product group.

The CORM Makefile on the OK60SRC subvolume instructs the ACI Make utility to compile the core subsystem.

The CIEPLM Makefile on the OC60LINK subvolume instructs the ACI Make utility to compile the Integrated End Point (CIEPLO) module. This makefile binds the CIEPLO module if necessary. However, it does not check the dependencies and cannot determine whether they must be built or updated.

Depending on the software changes, it is more reasonable to compile BASE24 ITS from the top level using the ITSM Makefile or from the subsystem level using either the KNLM, IEPM, or CORM makefiles. These input files are designed to have the ACI Make utility build a Make hierarchy, and compile and bind files as needed.

Note: The BASE24 procedural product must be compiled with the CLASSIC_ON and BASE_ON flags set to true before any of the BASE24 ITS products can be compiled, even if no other BASE24 procedural products have been installed.

Compiling DDLs

Makefiles are provided on the DDL subvolumes to compile the BASE24 procedural product DDLs and the BASE24 ITS product DDLs and SQL tables.

To run the ACI Make utility for all BASE24 procedural product DDLs, specify the CLSCDDL Makefile on the BA60SRC subvolume as the makefile in the RUN command. This makefile compiles the base DDLs, as well as the product-specific DDLs identified by flags in the CUSTMACS File.

To run the ACI Make utility for BASE24 ITS product DDLs and SQL tables, specify the appropriate makefile for the subsystem, as follows:

Subsystem	Subvolume and File
Kernel	OK60DDL.KDDL
Integrated Endpoint	OE60DDL.EDDL
Core	OC60DDL.CDDL

In addition to the subsystem makefiles shown above, each object of the BASE24 ITS product has two or three makefiles that can be used to compile DDLs and SQL tables. These makefiles, which can be used together or individually, are identified by the following suffixes:

M = Executes all makefiles on the subvolume, including the DDL and SQL table makefiles
DM = Compiles the DDLs
RM = Creates the SQL tables and indexes

Regardless of the makefile you specify in the RUN command, you must set an ALLMACS define value and set the Radix table size. Consider using an obey file for consistently starting the ACI Make utility. See “Using Obey Files to Start the ACI Make Utility” for a discussion of obey files.

Compiling Pathway Requester Processes

The ACI Make utility uses a special procedure to determine the modification timestamp for Pathway requester processes because the Tandem NonStop Kernel operating system sets the modification timestamp for these processes on a Screen COBOL object library (POBJ) file, instead of the individual requester files. Makefiles for Pathway requester processes create a pseudo target file for each compiled requester. The ACI Make utility then uses the modification timestamps from these pseudo target files when determining whether an object file needs to be rebuilt. Without these pseudo target files, the ACI Make utility must compile all requester processes every time it is run, regardless of whether their dependencies have changed, because it cannot find a modification timestamp for the target file.

A pseudo target file can be empty or contain a copy of the requester process source code, depending on whether the SYMBOLS option is included in the TAL Options macro (e.g., BASE_TAL_OPTS in the CUSTMACS File or BASEFM File). The requester process can be debugged with the Tandem INSPECT utility if the pseudo target file contains the source code.

The procedures and naming conventions vary between the BASE24 procedural and BASE24 ITS requester processes, as described in the following examples.

BASE24 Procedural Requesters

The following files are used for the Extract Configuration File (ECF) requester process. All of these files are on the product-specific AFT subvolume (e.g., BA60AFT subvolume for the ECF).

PECF	Temporary POBJ file for the requester process
RQECFM	Makefile for the requester process
RQECFO	Pseudo target file for the requester process
RQECFS	Source file for the requester process

SCRNECF Screen source file for the requester process

SCRNECFO Temporary screen object file for the requester process

The RQECFM Makefile is shown below, followed by a brief description of the processing performed by the inline code in the makefile.

```

!IF $(allmacs) = FALSE |THEN|                                     # 0061
#                                                                    # 0062
#   Define any macros you want to change from their standard      # 0063
#   values here                                                    # 0064
#                                                                    # 0065
#include =allmacs                                                  # 0066
!ENDIF                                                            # 0067
                                                                    # 0068
#-----                                                           # 0069
#   ECF Requester - RQECFS                                         # 0070
#-----                                                           # 0071
$(baaft_rqecfo) :{                                               \      # 0072
    $(dest_pobj) }[                                               \      # 0073
    $(basrc_baddlcob) ] $(baaft_rqecfs) \      # 0074
                        $(baaft_scrnecf) \      # 0075
                        $(baddl_ddlfecf) \      # 0076
                        $(baddl_ddlgpth) \      # 0077
                        $(basrc_bacoutls) \      # 0078
                        $(basrc_cobnames) \      # 0079
                        $(basrc_hlpscrn) \      # 0080
                                                                    # 0081
    $(TACL)/INLINE/                                              # 0082
= $(tacl_init) $(make_tacl_macros)                               # 0083
= #PUSH temp_pobj scn_ecfo                                       # 0084
                                                                    # 0085
= #SET  temp_pobj [expandssv $(baaft_rqecfo)].pecf               # 0086
= #SET  scn_ecfo  [expandssv $(baaft_rqecfo)].scnecfo            # 0087
                                                                    # 0088
= ADD DEFINE =baaft_scnecfo, FILE [scn_ecfo]                     # 0089
                                                                    # 0090
= $(ACIAID) / IN $(baaft_scrnecf), &                             # 0091
=          OUT $s.#ecf.scn / &                                   # 0092
=          [scn_ecfo]!                                           # 0093
= check_completion $(TARGET)                                     # 0094
= del_peruse_job $(PERUSE) $(baaft_purge_spooler_on) #ecf.scn   # 0095
                                                                    # 0096
= SINK [del [temp_pobj]*]                                         # 0097
                                                                    # 0098

```

```
!IF $(BAAFT_T3270_ON) = TRUE |THEN|                                # 0099
=   aci_scobolx_make $(SCOBOLX) $(TARGET) /IN $(baaft_rqecfs),    & # 0100
=                                     OUT $s.#rqecf.i            & # 0101
=                                     $(baaft_compile_opts) /&    # 0102
=                                     [temp_pobj]; settog 1; &    # 0103
=                                     $(baaft_scobolx_opts)       # 0104
=   check_scobol_completion $(TARGET)                             # 0105
=   del_peruse_job $(PERUSE) $(baaft_purge_spooler_on) #rqecf.i  # 0106
!ENDIF                                                            # 0107
                                                                    # 0108
!IF $(BAAFT_T6520_ON) = TRUE |THEN|                                # 0109
=   aci_scobolx_make $(SCOBOLX) $(TARGET) /IN $(baaft_rqecfs),    & # 0110
=                                     OUT $s.#rqecf.t            & # 0111
=                                     $(baaft_compile_opts) /&    # 0112
=                                     [temp_pobj]; settog 3; &    # 0113
=                                     $(baaft_scobolx_opts)       # 0114
=   check_scobol_completion $(TARGET)                             # 0115
=   del_peruse_job $(PERUSE) $(baaft_purge_spooler_on) #rqecf.t  # 0116
!ENDIF                                                            # 0117
                                                                    # 0118
= #PUSH :var                                                        # 0119
= #SET :var COPY [temp_pobj] (*),$(dest_pobj)                     # 0120
= $(SCUP)/inv :var/                                                # 0121
= check_completion $(TARGET)                                       # 0122
= #POP :var                                                         # 0123
                                                                    # 0124
= SINK [del [temp_pobj]*]                                          # 0125
= SINK [PURGE [scn_ecfo]]                                          # 0126
                                                                    # 0127
= #POP temp_pobj scn_ecfo                                          # 0128
= DELETE DEFINE =baaft_scnecfo                                    # 0129
```

The ACIAID utility compiles the screen source file and generates the temporary screen object file. The SCOBOLX compiler compiles the requester source file and creates the temporary POBJ file and the requester target file. The Screen COBOL Utility Program (SCUP) copies the temporary POBJ file to the central POBJ file used for the network. The temporary screen object file and temporary POBJ file are purged.

BASE24 ITS Requesters

The following files are used for the Account Type File (ATT) requester process. All files except the target file for the requester process and the subsystem POBJ file are located on the subsystem-specific AFM subvolume (e.g., OC60AFM). The requester process target file and the subsystem POBJ file are placed on the subsystem-specific object subvolume (e.g., OC60OBJ).

CPOBJ	POBJ file for all requester processes in the core subsystem
RQATTXM	Makefile for the requester process
RQATTXO	Target file for the requester process
RQATTXS	Source file for the requester process
RQATTSS	Screen source file for the requester process
RQATTSO	Screen object file for the requester process

The RQATTXM Makefile is shown below, followed by a brief description of the processing performed by the inline code in the makefile.

```

!IF $D(allmacs) = FALSE |THEN|
#
#  Define any macros you want to change from their standard
#  values here
#

!include =allmacs
!ENDIF

#-----
#  SCREEN                                -
#-----
$(ocafm_rqattso) : $(ocafm_rqattss)

    $(TACL)/INLINE/
= $(tacl_init) $(make_tacl_macros)
=
= $(ACIAID) /in $(ocafm_rqattss), out $s.#rqatts &
=      $(ocafm_compile_opts)/ $(ocafm_rqattso)!!;suppress
=
= check_completion $(TARGET)
= del_peruse_job $(PERUSE) $(ocafm_purge_spooler_on) #rqatts
=
= secure $(FUP) $(TARGET) $(ocafm_secure_opt)
= check_completion $(TARGET)

```

```
#-----
#  REQUESTER          -
#-----
$(ocafm_rqattxo)      :{                                \
$(ocafm_pobj)          }[                                \
$(basrc_baddlcob)      \                                \
$(ocddl_ocdx)          ] $(baddl_ddlgpth)                \
                        $(basrc_bacoutls)                \
                        $(basrc_hlpscrn)                  \
                        $(ocafm_rqattso)                  \
                        $(ocafm_rqattxs)                  \
                        $(ocddl_attds)

$(TACL)/INLINE/
= $(tacl_init) $(make_tacl_macros)
=
= [#IF $(ocafm_t3270_on) |THEN|
= aci_scobolx_make $(SCOBOLX) $(TARGET) /in $(ocafm_rqattxs), &
=                                     out $.#rqatt           &
=                                     $(ocafm_compile_opts)/   &
=                                     $(ocafm_pobj);           &
=                                     settog 1;                 &
=                                     $(ocafm_scobolx_opts)
= check_scobol_completion $(TARGET)
= del_peruse_job $(PERUSE) $(ocafm_purge_spooler_on) #rqatt
= ] == end if t3270
=
= [#IF $(ocafm_t6520_on) |THEN|
= aci_scobolx_make $(SCOBOLX) $(TARGET) /in $(ocafm_rqattxs), &
=                                     out $.#rqatt           &
=                                     $(ocafm_compile_opts)/   &
=                                     $(ocafm_pobj);           &
=                                     settog 3;                 &
=                                     $(ocafm_scobolx_opts)
= check_scobol_completion $(TARGET)
= del_peruse_job $(PERUSE) $(ocafm_purge_spooler_on) #rqatt
= ] == end if t6520
```

The ACIAID utility compiles the screen source file and generates the screen object file. The SCOBOLX compiler compiles the requester source file and places the output in the subsystem POBJ file and the requester target file.

The following differences can exist between the makefiles for BASE24 procedural requester processes and the makefiles for BASE24 ITS requester processes. However, some BASE24 ITS requester processes are being rewritten for release 6.0 to be handled like BASE24 procedural requester processes.

- The BASE24 ITS screen object files are not purged and BASE24 procedural screen object files are purged.
- The BASE24 ITS compiler output is placed in a subsystem-specific POBJ file instead of the central POBJ file that is used for the network. The Screen COBOL Utility program COPY command must be used to manually add the subsystem-specific POBJ file to the central POBJ file used for the network.

Using Multiple Makefiles

You can start the ACI Make utility with one makefile and then start the utility with another makefile while the utility is still processing the first makefile. The two issues that you must consider when processing multiple makefiles is the size of the internal tables for the file macros and the amount of disk space for the swap file used by these tables. You must set the MAX-RADIX-TABLE-SIZE parameter or MAX_RADIX_TBL_SIZE option large enough to hold internal tables for each run of the ACI Make utility.

If there is insufficient space for the swap file, indicated by ALLOCATE SEGMENT ERROR 43, you can use the TACL SWAPVOL parameter to direct the ACI Make utility to use a disk to which you have write access and that has sufficient space for the swap file, as follows:

```
param swapvol $volume
```

You can enter this command at a TACL prompt or add it to the TACLCSTM on your default volume and subvolume. You can use the following Tandem Disk Space Analysis Program (DSAP) command to display the amount of free space available on each disk of the Tandem system.

```
dsap *,short
```


Appendix A

ACI Make Utility Summary

This appendix summarizes the make files used by the ACI Make utility and provides an example of their use with the utility. The first part of this appendix reviews various make files from each level of the Make hierarchy. Example files are presented at each level for the BASE24 procedural architecture.

The second part of this appendix demonstrates the use of the ACI Make utility with the AUTHM Makefile to compile the BASE24-atm Authorization process. While this demonstration involves only the BASE24-atm Authorization module makefile, the concepts used in it apply to an entire BASE24 system or any subset of it.

Note: If the BASE24-atm product has been installed on your Tandem system and you have access to it, you can look at the files mentioned in this appendix to become familiar with the files and their locations. However, do **not** modify or execute the files presented in this appendix.

Viewing Files

You have the option of viewing each of the files mentioned in this appendix if they exist on your Tandem system. The BASE24 procedural examples are taken from the BASE24-atm product.

Use the text editor of your choice to view the files. Be sure to open the files as read only so that you cannot accidentally modify them. The examples throughout this appendix use the TEDIT command at a TACL prompt, as shown below.

```
>TEDIT file R
```

You will need to know where the following cataloged subvolumes are located on your system. Use the following table to record the location for your convenience when you want to view the files.

Volume	Subvolume
	MAKEV500
	BA60MAKE
	BA60SRC
	AT60SRC
	AT60AUTH

Note: If you cannot view the files with a text editor, portions of the macro files mentioned are presented as examples in section 2 and the makefiles are presented in this appendix.

CUSTMACS File

The CUSTMACS File is a global macro file at the top of the Make hierarchy. It contains customer-specific configuration information and identifies which BASE24 products to include in the system when the ACI Make utility runs.

The cataloged CUSTMACS File is located on the BA60MAKE subvolume, and can be displayed by entering the following command. A portion of this file is also presented in section 2.

```
TACL> TEDIT $volume.BA60MAKE.CUSTMACS R
```

As you review the CUSTMACS File, notice the following items:

- Global configuration macros *true* and *false*.
Note: These macros are ignored because they are defined within the ACI Make utility and cannot be changed. These macros are included in this CUSTMACS File because they were required with previous releases of the ACI Make utility.
- Logic that checks for the presence of personal override files.
- Logic that checks for the presence of channel-specific enhancement (CSE) macro files.
- Flags for BASE24 release 6.0. This example contains several groups of the flags contained in the CUSTMACS File. The ACI Make utility uses all of the flags in the CUSTMACS File to determine which BASE24 procedural (classic) products and BASE24 ITS objects are installed.
- Flags for channel-specific products and enhancements can be added to this portion of the CUSTMACS File.
- Flag for custom software modifications (CSMs). This flag is always set to false in the CUSTMACS File. When CSMs are involved, this flag is set to true in the CSMMACS File.
- Configuration options copied from the BASEFM File. These options must be uncommented if they are changed.
- Object file destinations. The MOVE_OBJ_SUBVOL_ON flag controls whether object files are moved from the subvolume that contains the Make files used to create them. This flag must be uncommented and set to true for any object files to be moved. The locations need to be uncommented to be changed.
- INCLUDE command that incorporates the ALLMACS File.

ALLMACS File

The ALLMACS File is another global macro file that follows the CUSTMACS File in the Make hierarchy, and contains file definition macros such as the location of the TAL compiler, NET24-XPNET system, and ACI utilities. The ALLMACS File also contains two architecture flags that indicate whether BASE24 procedural or ITS architectures are to be built.

The cataloged ALLMACS File is located on the BA60MAKE subvolume, and can be displayed by entering the following command. A portion of this file is also presented in section 2.

```
TACL> TEDIT $volume.BA60MAKE.ALLMACS R
```

As you review the ALLMACS File, notice the following items:

- The presence of the ALLMACS macro indicates the Make macro file hierarchy has been built.

The True and False macros are ignored because they are defined within the ACI Make utility and cannot be changed. These macros are included in this ALLMACS File because they were required with previous releases of the ACI Make utility.

The following macros are optional because the ACI Make utility assigns the default values shown below if these macros are not in the ALLMACS File.

Macro	Default Value
MAKE_TACL_MACROS	TACLMACS, located on the same subvolume as the Make utility.
TACL	\$SYSTEM.SYSTEM.TACL
TACL_INIT	TACLINIT, located on the same subvolume as the Make utility.

- Flags used to identify whether BASE24-retail products (RETAIL_ON) or BASE24 Classic products (CLASSIC_ON) are included during Make processing. When the ACI Make utility reads the CUSTMACS File first, the values assigned to these flags in the CUSTMACS File takes precedence over the values assigned in the ALLMACS File.
- Locations of some of the executables used during Make processing.
- Locations of some of the system declarations, definitions, and libraries used during Make processing.

- Locations of files containing configuration macros and file definition macros associated with channel-specific enhancements (CSEs). Because this logic is located ahead of the INCLUDE commands associated with the CLASSIC_ON flag, the CSE macro files take precedence over the classic macro files. See the “Channel-Specific Enhancements” topic in section 2 for more detailed information about using CSE macros to override the macros normally used to create the Make macro file hierarchy.
- Locations of files containing configuration macros and file definition macros associated with BASE24 Classic products.

BASE24 Procedural Product Make Files

The BASE24 procedural products use three levels of macro files that are located below the global macro files and master makefiles. These levels are called product group, product, and process, and contain configuration macros, file definition macros, and compiler instructions.

CLSCFM File

The CLSCFM File is a product group macro file that contains configuration macros for the BASE24 procedural (also known as classic) architecture.

The cataloged CLSCFM File is located on the BA60SRC subvolume, and can be displayed by entering the following command. A portion of this file is also presented in section 2.

```
TACL> TEDIT $volume.BA60SRC.CLSCFM R
```

As you review the CLSCFM File, notice the following items:

- The product flags (e.g., ATM_ON, POS_ON), and all of these flags are set to false. You can set flags in the CUSTMACS File for the appropriate BASE24 procedural products, and these flags override the flags in the CLSCFM File.
- The location of the product level macro files (e.g., BASEFM, BASEMM, ATMFM, ATMMM).
- The location of the product makefiles (e.g., BASEM, ATMM).

ATMFM File

The ATMFM File is a product macro file that describes the configuration for the ATM product and is configured in a slightly different manner than the CLSCFM File.

The cataloged ATMFM File is located on the AT60SRC subvolume, and can be displayed by entering the following command. A portion of this file is also presented in section 2.

```
TACL> TEDIT $volume.AT60SRC.ATMFM R
```

As you review the ATMFM File, notice the following items:

- Compiler option macros that are passed to the various compilers and can contain any options supported by the compiler. The default values for these macros are taken from the BASEFM File.
- Flags that identify which of the optional modules associated with a product is to be included in the system. For example, the ATM product is composed of various optional Device Handler processes. The ATMFM File contains flags that are used to specify which Device Handler processes to build. You can set flags in the CUSTMACS File to turn on the appropriate product modules, and these flags override the product module flags in the product level macro file.
- File defines that locate configuration macro files (file names ending with *FM*) and file definition macro files (file names ending with *MM*) for the modules associated with the BASE24-atm product. File defines for the AUTHFM File and the AUTHMM File are included in the ATMFM File.

AUTHFM File

The AUTHFM File is a process macro file that describes the compiler options for the BASE24-atm Authorization process. All of the information is inherited from the ATMFM File. Any changes that you make in the AUTHFM File apply only when the ACI Make utility compiles the BASE24-atm Authorization process. The AUTHFM File does not contain flags for optional modules or file defines.

The cataloged AUTHFM File is located on the AT60AUTH subvolume, and can be displayed by entering the following command. A portion of this file is also presented in section 2.

```
TACL> TEDIT $volume.AT60AUTH.AUTHFM R
```

AUTHMM File

The AUTHMM File is the file definition macro file (file name ending with *MM*) for the BASE24-atm Authorization process. The AUTHMM File contains definitions for other files used by the BASE24-atm Authorization process. For example, the define used for the AUTHG File is described in the AUTHMM File.

The cataloged AUTHMM File is located on the AT60AUTH subvolume, and can be displayed by entering the following command. This file is also presented in section 2.

```
TACL> TEDIT $volume.AT60AUTH.AUTHMM R
```

AUTHM File

The AUTHM File is used to build the BASE24-atm Authorization process.

The cataloged AUTHM File is located on the AT60AUTH subvolume, and can be displayed by entering the following command. This file is also presented in its entirety immediately following the list of items to look for as you review it.

```
TACL> TEDIT $volume.AT60AUTH.AUTHM R
```

As you review the AUTHM File, notice the following items:

- The AUTHM File builds only the BASE24-atm Authorization process.
- The AUTHM File tests for the ALLMACS define and provides a place for the entry of changes in macros; however, macro changes usually are set in a customized macros file such as XYZMACS.
- The AUTHM Makefile contains flags for turning off add-on modules; however, these flags usually are set in the CUSTMACS File or in a customized macros file such as XYZMACS.

- The AUTHM Makefile generates externals, compiles the AUTHLIBS and AUTHS modules, binds the modules to create the AUTH executable, and accelerates the code.
 - The AUTHM Makefile contains macros (e.g., CHECK_COMPLETION, DEL_PERUSE_JOB, and DUP_EXEC_FILE) that help with automation. These values are changeable in the makefile; however, macro changes usually are set in a customized macros file such as XYZMACS.
-

```
!IF $(allmacs) = FALSE |THEN|                                # 0171
#                                                            # 0172
#   Define any macros you want to change from their standard # 0173
#   values here                                              # 0174
#                                                            # 0175
#                                                            # 0176
#include =allmacs                                             # 0177
!ENDIF                                                        # 0178
#                                                            # 0179
#-----                                                    # 0180
#   AUTH Bindable Modules                                   # 0181
#-----                                                    # 0182
#                                                            # 0183
!IF $(ATM_EMV_ON) = TRUE |THEN|                               # 0184
    !include $(atiath_authemvm)                             # 0185
!ENDIF                                                        # 0186
#                                                            # 0187
!IF $(ATM_MCU_ON) = TRUE |THEN|                               # 0188
    !include $(atuath_authmcum)                             # 0189
!ENDIF                                                        # 0194
#                                                            # 0195
!IF $(NCD_ON) = TRUE |THEN|                                   # 0196
    !include $(atnath_authncdm)                             # 0197
!ENDIF                                                        # 0198
#                                                            # 0199
!IF $(SSB_BASE_ON) = TRUE |THEN|                              # 0200
    !include $(atbath_authssbm)                             # 0201
#                                                            # 0202
    !IF $(SSB_CHK_ON) = TRUE |THEN|                         # 0203
        !include $(atcath_authsscm)                         # 0204
    !ENDIF                                                    # 0205
!ENDIF                                                        # 0206
#                                                            # 0206A00
!IF $(PRM_ON) = TRUE |THEN|                                   # 0206A01
    !IF $(F$(atpath_authpim)) = TRUE |THEN|                 # 0206A02
        !include $(atpath_authpim)                         # 0206A03
    !ENDIF                                                    # 0206A04
!ENDIF                                                        # 0206A05
#                                                            # 0207A01
```



```

#----- # 0208
# AUTH Externals # 0209
#----- # 0210
# 0211
$(atauth_authcsee) : $(atauth_authcses) # 0212
$(GENFWD)/in $(atauth_authcses)/ ext $(atauth_authcsee) # 0213
# 0214
$(atauth_authlibe) : $(atauth_authlibs) # 0215
$(GENFWD)/in $(atauth_authlibs)/ ext $(atauth_authlibe) # 0216
# 0217
$(atauth_auth) : $(atauth_auths) # 0218
$(GENFWD)/in $(atauth_auths)/ ext $(atauth_auth) # 0219
# 0220
#----- # 0221
# AUTH Library Object # 0222
#----- # 0223
# 0224
$(atauth_authlibo) : [ \ # 0225
$(basrc_baddltal) \ # 0226
$(atsrc_atddltal) ] $(aciutils_utildefs) \ # 0227
$(aciutils_utilexts) \ # 0228
$(atauth_authcsee) \ # 0229
$(atauth_authcseg) \ # 0230
$(atauth_authcses) \ # 0231
# 0232
$(atauth_authemvd) \ # 0233
$(atauth_authg) \ # 0234
$(atauth_authlibe) \ # 0235
$(atauth_authlibs) \ # 0236
$(atauth_authmcud) \ # 0237
$(atauth_authpid) \ # 0237A01
$(atauth_authtbl) \ # 0238
$(atddl_ddlftlf) \ # 0239
$(bacse_attkcid) \ # 0240
$(bacse_batkccve) \ # 0241
$(bacse_batkcid) \ # 0242
$(bacse_ddlattkc) \ # 0243
$(bacse_ddlbatkc) \ # 0244
$(baddl_ddlattkn) \ # 0245
$(baddl_ddlbatkn) \ # 0246
$(baddl_ddlbcnst) \ # 0247
$(baddl_ddlfcf) \ # 0248
$(baddl_ddlfcnf) \ # 0249
$(baddl_ddlfcpf) \ # 0250
$(baddl_ddlfidf) \ # 0251
$(baddl_ddlfkeya) \ # 0252
$(baddl_ddlfneg) \ # 0253
$(baddl_ddlfpbf) \ # 0254
$(baddl_ddlfsurf) \ # 0255
$(baddl_ddlftkn) \ # 0256
$(baddl_ddlfuaf) \ # 0257
$(baddl_ddlgpth) \ # 0258

```

```

$(baddl_ddlgstm) \ # 0259
$(badefts_badeftal) \ # 0260
$(balib_balibte) \ # 0265
$(basrc_attknid) \ # 0266
$(basrc_batkncve) \ # 0267
$(basrc_batkaid) \ # 0268
$(basrc_bautile) \ # 0269
$(basrc_cvutile) \ # 0270
$(basrc_secutile) \ # 0271
$(okddl_okdt) \ # 0272
$(oklib_oklibte) \ # 0273
$(oklib_oklibtg) \ # 0274
$(oksrc_oktg) \ # 0275
$(rdebug_dumpext) \ # 0276
$(spannet_globals) \ # 0277
$(spannet_initlibs) \ # 0278
$(spannet_skel) \ # 0279
$(spannet_skelexts) \ # 0280
$(system_extdecs0) \ # 0281
# 0282
$(TACL)/INLINE/ # 0283
= $(tacl_init) $(make_tacl_macros) # 0284
# 0285
= $(TAL)/in $(atauth_authlibs), & # 0286
= out $s.#atauthl.t & # 0287
= $(atauth_compile_opts)/ & # 0288
= $(atauth_authlibo); & # 0289
= $(atauth_tal_opts) # 0290
= check_completion $(TARGET) # 0291
= del_peruse_job $(PERUSE) $(atauth_purge_spooler_on) #atauthl.t # 0292
# 0293
= secure $(FUP) $(atauth_authlibo) $(atauth_secure_opt) # 0294
= check_completion $(TARGET) # 0295
# 0296
#----- # 0297
# AUTH Object # 0298
#----- # 0299
# 0300
$(atauth_autho) : [ \ # 0301
$(basrc_baddltal) \ # 0302
$(atsrc_atddltal) ] $(aciutils_utildefs) \ # 0303
$(aciutils_utilexts) \ # 0304
$(atauth_authcsee) \ # 0305
$(atauth_authcseg) \ # 0306
$(atauth_authc) \ # 0307
# 0308
$(atauth_authemvd) \ # 0309
$(atauth_authg) \ # 0310
$(atauth_authlibe) \ # 0311
$(atauth_authmcud) \ # 0312
$(atauth_authpid) \ # 0312A01
$(atauth_auths) \ # 0313

```

```

$(atauth_authtbl)      \      # 0314
$(atddl_ddlftlf)      \      # 0315
$(bacse_attkcid)      \      # 0316
$(bacse_batkcid)      \      # 0317
$(bacse_ddlattkc)     \      # 0318
$(bacse_ddlbatkc)     \      # 0319
$(baddl_ddlattkn)     \      # 0324
$(baddl_ddlbatkn)     \      # 0325
$(baddl_ddlfcf)       \      # 0326
$(baddl_ddlfcnf)      \      # 0327
$(baddl_ddlfcpf)      \      # 0328
$(baddl_ddlfidf)      \      # 0329
$(baddl_ddlfkeya)     \      # 0330
$(baddl_ddlfneg)      \      # 0331
$(baddl_ddlfpbf)      \      # 0332
$(baddl_ddlfuaf)      \      # 0333
$(baddl_ddlgpth)      \      # 0334
$(baddl_ddlgstm)      \      # 0335
$(badefts_badeftal)   \      # 0336
$(basrc_attknid)      \      # 0337
$(basrc_batknid)      \      # 0338
$(basrc_bautile)      \      # 0339
$(basrc_cvutile)      \      # 0340
$(basrc_secutile)     \      # 0341
$(okddl_okdt)         \      # 0342
$(oklib_oklibtg)      \      # 0343
$(oksrc_oktg)         \      # 0344
$(rdebug_dumpext)     \      # 0345
$(spannet_globals)    \      # 0346
$(spannet_initlibs)   \      # 0347
$(spannet_skel)       \      # 0348
$(spannet_skelexts)   \      # 0349
$(system_extdecs0)    \      # 0350
                        \      # 0351
$(TACL)/INLINE/      \      # 0352
= $(tacl_init) $(make_tacl_macros) \      # 0353
                        \      # 0354
= $(TAL)/in $(atauth_auths),      &      # 0355
=      out $s.#atauth.t      &      # 0356
=      $(atauth_compile_opts)/ &      # 0357
=      $(atauth_autho);      &      # 0358
=      $(atauth_tal_opts)      # 0359
= check_completion $(TARGET)      # 0360
= del_peruse_job $(PERUSE) $(atauth_purge_spooler_on) #atauth.t      # 0361
                        # 0362
= secure $(FUP) $(atauth_autho) $(atauth_secure_opt)      # 0363
= check_completion $(TARGET)      # 0364
                        # 0365

```

```

#----- # 0366
# AUTH Executable - BIND # 0367
#----- # 0368
# 0369
$(atauth_auth) : $(aciutils_utilb) \ # 0370
# 0371
$(atauth_autho) \ # 0372
$(atauth_authlibo) \ # 0373
!IF $(atm_emv_on) = TRUE |THEN| \ # 0374
$(atiath_authemvo) \ # 0375
$(baiemv_camutilo) \ # 0376
!ENDIF \ # 0377
!IF $(atm_mcu_on) = TRUE |THEN| \ # 0378
$(atuath_authmcuo) \ # 0379
# 0379A00
$(atumcu_atmcutl) \ # 0379A01
$(baumcu_bamcutl) \ # 0379A02
# 0379A03
# 0380A00
# 0380A01
!ENDIF \ # 0381
# 0382
# 0383
# 0384
# 0385
!IF $(ncd_on) = TRUE |THEN| \ # 0386
$(atnath_authncdo) \ # 0387
!ENDIF \ # 0388
!IF $(ssb_base_on) = TRUE |THEN| \ # 0389
$(atbath_authssbo) \ # 0390
!IF $(ssb_chk_on) = TRUE |THEN| \ # 0391
$(atcath_authssco) \ # 0392
!ENDIF \ # 0393
!ENDIF \ # 0394
# 0394A00
!IF $(prm_on) = TRUE |THEN| \ # 0394A01
!IF $(atpath_authpio) = TRUE |THEN| \ # 0394A02
$(atpath_authpio) \ # 0394A03
!ENDIF \ # 0394A04
!ENDIF \ # 0394A05
# 0394A06
$(atlib_atliblo) \ # 0395
$(bacse_batkccvo) \ # 0396
$(basrc_bautil) \ # 0397
$(basrc_cvutil) \ # 0398
$(basrc_secutil) \ # 0399
$(basrc_batkncvo) \ # 0400
$(oklib_oklibto) \ # 0401
# 0402
$(TACL)/INLINE/ # 0403
= $(tacl_init) $(make_tacl_macros) # 0404
# 0405

```

```

= #PUSH      :var                                     # 0406
= #APPEND    :var add * from $(atauth_autho),         delete # 0407
= #APPEND    :var add code * from $(atauth_authlibo), delete # 0408
                                                    # 0409
                                                    # 0410
!IF $(atm_emv_on) = TRUE |THEN|                       # 0411
= #APPEND    :var add code * from $(atiath_authemvo), delete # 0412
!ENDIF                                              # 0413
                                                    # 0414
!IF $(atm_mcu_on) = TRUE |THEN|                       # 0415
= #APPEND    :var add code * from $(atuath_authmcuo), delete # 0416
!ENDIF                                              # 0417
                                                    # 0418
                                                    # 0419
                                                    # 0420
                                                    # 0421
!IF $(NCD_ON) = TRUE |THEN|                           # 0422
= #APPEND    :var add * from $(atnath_authncdo), delete # 0423
!ENDIF                                              # 0424
                                                    # 0425
!IF $(SSB_BASE_ON) = TRUE |THEN|                       # 0426
= #APPEND    :var add * from $(atbath_authssbo), delete # 0427
    !IF $(SSB_CHK_ON) = TRUE |THEN|                   # 0428
= #APPEND    :var add * from $(atcath_authssco), delete # 0429
    !ENDIF                                              # 0430
!ENDIF                                              # 0431
                                                    # 0432
                                                    # 0432A00
!IF $(PRM_ON) = TRUE |THEN|                           # 0432A01
    !IF $(atpath_authpio) = TRUE |THEN|               # 0432A02
= #APPEND    :var add code * from $(atpath_authpio), delete # 0432A03
    !ENDIF                                              # 0432A04
!ENDIF                                              # 0432A05
                                                    # 0432A06
                                                    # 0433
!IF $(atm_emv_on) = TRUE |THEN|                       # 0434
= #APPEND    :var add code * from $(baiemv_camutilo), delete # 0435
!ENDIF                                              # 0436
                                                    # 0437
!IF $(atm_mcu_on) = TRUE |THEN|                       # 0438
                                                    # 0438A00
= #APPEND    :var add code * from $(atumcu_atmcutl), delete # 0438A01
= #APPEND    :var add code * from $(baumcu_bamcutl), delete # 0438A02
                                                    # 0438A03
                                                    # 0439A00
                                                    # 0439A01
!ENDIF                                              # 0440
                                                    # 0441
= #APPEND    :var select search $(ATLIB_ATLIBLO)       # 0442
= #APPEND    :var select search $(OKLIB_OKLIBTO)       # 0443
= #APPEND    :var select search $(ACIUTILS_UTILB)      # 0444
= #APPEND    :var select search $(BASRC_BAUTIL)        # 0450

```

```
= #APPEND :var select search $(BASRC_TKNCV) # 0451A01
= #APPEND :var select search $(BASRC_CVUTIL) # 0452
= #APPEND :var select search $(BASRC_SECUTIL) # 0453
= #APPEND :var set data 64 # 0455
= #APPEND :var select list * off # 0456
= #APPEND :var $(atauth_bind_steps) # 0462
= #APPEND :var build $(atauth_auth) ! # 0463
# 0464
= $(BIND)/ inv :var, out $s.#atauth.b $(atauth_compile_opts)/ # 0465
= check_completion $(TARGET) # 0466
= del_peruse_job $(PERUSE) $(atauth_purge_spooler_on) #atauth.b # 0467
= #POP :var # 0468
# 0469
#----- # 0470
# AUTH Accelerate # 0471
#----- # 0472
# 0473
!IF $(atauth_axcel_on) = TRUE |THEN| # 0474
= #PUSH :var # 0475
= #EOF :var # 0476
= $(AXCEL)/inv :var, OUT $S.#atauth.axc & # 0477
= $(atauth_axcel_opts) / & # 0478
= $(atauth_auth), $(atauth_auth); & # 0479
= USERLIB $(spannet_hskelb) # 0480
= check_completion $(TARGET) # 0481
= del_peruse_job $(PERUSE) $(atauth_purge_spooler_on) #atauth.axc # 0482
= #POP :var # 0483
!ENDIF # 0484
# 0485
= dup_exec_file $(move_obj_subvol_on) $(atauth_secure_opt) & # 0486
= $(atauth_auth) $(atm_obj_loc) # 0487
```

Using the AUTHM Makefile

The remainder of this appendix contains a demonstration of running the ACI Make utility with the AUTHM Makefile to build the BASE24-atm Authorization process.

Obey files provide an efficient and accurate way to run the ACI Make utility. Throughout this demonstration, the RUNMKNB or RUNMAKE obey files are modified to contain the necessary commands.

Files Used

Several files are used to run the ACI Make utility for the AUTHM Makefile. Throughout the demonstration, the following variables are used:

\$EARL	= The volume that contains the BASE24 software for your system. The BASE24 software for this demonstration is located on the \$EARL volume.
\$SYSACI	= The volume that contains the ACI Make utility. The ACI Make utility for development systems at ACI are located on the \$SYSACI volume.
\R	= Your Tandem system. This demonstration is performed on \R.
\$BRIDGE	= Your test volume. This demonstration is performed on \$BRIDGE
XYZTEST	= Your test subvolume. XYZ can be your initials or other meaningful identifier.
XYZMACS	= Your personal override file. XYZ can be your initials or other meaningful identifier.

The following files were placed on the \$BRIDGE.XYZTEST subvolume prior to the demonstration.

AUTHG	Copied from \$EARL.AT60AUTH because it is defined in the \$EARL.AT60AUTH.AUTHMM and contains modifications making it necessary to rebuild the BASE24-atm Authorization process.
AUTHLIBS	Copied from \$EARL.AT60AUTH because it is defined in the \$EARL.AT60AUTH.AUTHMM and contains modifications making it necessary to rebuild the BASE24-atm Authorization process.

AUTHS	Copied from \$EARL.AT60AUTH because it is defined in the \$EARL.AT60AUTH.AUTHMM and contains modifications making it necessary to rebuild the BASE24-atm Authorization process.
RUNMAKE	Obey file created to run the ACI Make utility without the NOBUILD option.
RUNMKNB	Obey file created to run the ACI Make utility with the NOBUILD option.
XYZMACS	Personal override file that contains all of the macros that must be modified from the standard BASE24 system.

The AUTHM Makefile on \$EARL.AT60AUTH contains the following target files. Running the ACI Make utility with the AUTHM Makefile and the RUNMAKE obey file creates each of these files. You must create a personal override file with macros for each of these files so that you can control where the files are written.

AUTH	The bound Authorization module
AUTHCSEE	The Authorization channel-specific externals file
AUTHE	The Authorization externals file
AUTHLIBE	The Authorization library externals file
AUTHLIBO	The Authorization library object file
AUTHO	The Authorization object file

Personal Override File

The personal override file contains all of the macros that must be modified from the standard BASE24 system. The macros that must be changed include the file defines from the AUTHMM File for target files that are created on your subvolume and dependency files that you modify on your subvolume. File defines in the AUTHMM File for dependency files that you do not modify do not have to be placed in the personal override file. You must also provide configuration macros from the AUTHFM File that you want to override. In this demonstration, the personal override file is named XYZMACS.

The contents of the XYZMACS File for this demonstration are shown below.

```

atauth_authg          = \R.$bridge.xyztest.authg
atauth_auths          = \R.$bridge.xyztest.auths
atauth_authlibs       = \R.$bridge.xyztest.authlibs

atauth_auth           = \R.$bridge.xyztest.auth
atauth_authcsee       = \R.$bridge.xyztest.authcsee
atauth_authc          = \R.$bridge.xyztest.authc
atauth_autho          = \R.$bridge.xyztest.autho
atauth_authlibe       = \R.$bridge.xyztest.authlibe
atauth_authlibo       = \R.$bridge.xyztest.authlibo

atauth_axcel_on       = $(false)
atauth_tal_opts       = symbols, inspect

```

The XYZMACS File contains three groups of information, as shown in the following table.

Group	Description
1	Macros for each of the dependency files placed on the XYZTEST subvolume. The XYZMACS File in our example contains entries for the AUTHG, AUTHS, and AUTHLIBS files because these dependency files typically require modifications.
2	Macros for each of the target files to be created on the XYZTEST subvolume. Refer to “Identifying Target Files” later in this appendix for ways to obtain these macros.
3	Macros to turn off a flag so that the resulting object code is not accelerated and set the desired TAL compiler options.

Identifying Target Files

There are several ways to determine the target files. When you are using a single makefile like the AUTHM Makefile that does not have INCLUDE statements for additional makefiles, you can review the makefile and manually create a personal override file with macros for every target file contained in the makefile. However, as soon as you start working with multiple makefiles, it is best to use the tools provided by the ACI Make utility to create the personal overrides file.

Target and Dependency Report

Running the ACI Make utility using the AUTHM Makefile with the NOBUILD and LISTDEP options creates a report that contains all of the target files and dependency files (labeled as source files in the report) used to build the module. You can look through the report and use it when creating your personal override file. The report for the AUTHM Makefile contains six target files and approximately 75 dependency files. A portion of the report generated for the AUTHM Makefile is shown below. Four of the target files are shown. Following each target file, there is a list of dependency files required to build it and a status line that shows whether the target file must be rebuilt.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

```
RUN CMD > \r.$sysaci.makev500.makev501 \r.$earl.at60auth.authm &  
> -NOBUILD -LISTDEP
```

Define =ALLMACS equals: \R.\$EARL.BA60MAKE.CUSTOMACS

```
*** TARGET: 3APR2001 23:41:14.67 - \R.$EARL.AT60AUTH.AUTHCSEE  
*** SOURCE: 22MAR2001 08:59:01.80 - \R.$EARL.AT60AUTH.AUTHCSES  
*** File (\R.$EARL.AT60AUTH.AUTHCSEE) did not require rebuilding  
*** TARGET: 3APR2001 23:38:42.83 - \R.$EARL.AT60AUTH.AUTHLIBE  
*** SOURCE: 24MAR2001 11:18:11.60 - \R.$EARL.AT60AUTH.AUTHLIBS  
*** File (\R.$EARL.AT60AUTH.AUTHLIBE) did not require rebuilding  
*** TARGET: 3APR2001 23:38:36.59 - \R.$EARL.AT60AUTH.AUTHHE  
*** SOURCE: 24MAR2001 11:19:18.10 - \R.$EARL.AT60AUTH.AUTHS  
*** File (\R.$EARL.AT60AUTH.AUTHHE) did not require rebuilding  
*** TARGET: 21APR2001 14:54:47.58 - \R.$EARL.AT60AUTH.AUTH  
*** SOURCE: 3JAN2001 22:19:51.64 - \R.$SYSACI.ACIUTILS.UTILB  
*** SOURCE: 20APR2001 17:13:52.16 - \R.$EARL.AT60AUTH.AUTHO  
*** SOURCE: 20APR2001 17:13:09.08 - \R.$EARL.AT60AUTH.AUTHLIBO  
*** SOURCE: 23APR2001 19:29:10.41 - \R.$EARL.AT60LIB.ATLIBLO  
*** SOURCE: 3APR2001 20:15:12.50 - \R.$EARL.BA60CSE.BATKCCVO  
*** SOURCE: 3MAY2001 11:41:54.51 - \R.$EARL.BA60SRC.BAUTIL  
*** SOURCE: 20APR2001 15:50:44.80 - \R.$EARL.BA60SRC.CVUTIL  
*** SOURCE: 20APR2001 15:52:12.60 - \R.$EARL.BA60SRC.SECUTIL  
*** SOURCE: 3APR2001 20:15:54.44 - \R.$EARL.BA60SRC.BATKNCVO
```

```
*** SOURCE: 3APR2001 20:36:20.64 - \R.$EARL.OK600BJ.OKLIBTO
*** File (\R.$EARL.AT60AUTH.AUTH) did not require rebuilding
```

```
*** MAKE complete 3 MAY 2001 13:08:26.42 ***
```

Summary:

```
Files rebuilt : 0
Build errors  : 0
Build warnings : 0
```

```
Elapsed Time : 00:00:12.58 (h:m:s.hs)
Time before starting first build: 00:00:00.00 (h:m:s.hs)
Longest Line Length (bytes): 1721
Line Size Max value (bytes): 250000
Number of SQL macro hits: 0
Number of Memory macro hits: 557
Bytes in build steps: 19490
Build Steps Table Size: 120000000
```

Information on RADIX Tables:

```
2 Tables Allocated 10000000 bytes each.
- Table 1 Used 1753424 (17.6%)
- Table 2 Used 3858728 (38.6%)
```

The RUNMKNB obey file for running this report appears below. The XYZMACS File does not exist at this point.

```
PARAM MAX-RADIX-TBL-SIZE 10000000
```

```
delete define =allmacs
add define =allmacs, file $earl.ba60make.custmacs
```

```
run $sysaci.makev500.makev501 /out $s.#xyz,nowait/ &
\r.$earl.at60auth.authm -nobuild -listdep
```

```
delete define =allmacs
```

Generate the Personal Override File

Running the ACI Make utility using the AUTHM Makefile with the NOBUILD, EXCEPT_FILE and EXCEPT_LOC options automatically creates an edit file on your subvolume that contains a macro entry for each target file. This edit file becomes your personal override file. The EXCEPT_FILE and EXCEPT_LOC options became available with ACI Make utility release 4.0.

The report that results from running the ACI Make utility using the AUTHM Makefile with the EXCEPT_FILE, EXCEPT_LOC, FORCE, and NOBUILD options is shown below. This listing contains the six target file entries from the listing created by the LISTDEP and NOBUILD options, but does not contain the source file entries from that listing. The target file names in this listing are placed in an edit file identified by the EXCEPT_FILE and EXCEPT_LOC options.

Note: The FORCE option is required because only targets that must be rebuilt are shown on the report or placed in the edit file.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

```
RUN CMD > \r.$sysaci.makev500.makev501 \r.$earl.at60auth.authm
> -NOBUILD -FORCE -EXCEPT_FILE=XYZMACS &
> -EXCEPT_LOC=\R.$BRIDGE.XYZTEST
```

Define =ALLMACS equals: \R.\$EARL.BA60MAKE.CUSTMACS

```
*** (\R.$EARL.AT60AUTH.AUTHCSEE) needs rebuilding!
*** (\R.$EARL.AT60AUTH.AUTHLIBE) needs rebuilding!
*** (\R.$EARL.AT60AUTH.AUTHHE) needs rebuilding!
*** (\R.$EARL.AT60AUTH.AUTHLIBO) needs rebuilding!
*** (\R.$EARL.AT60AUTH.AUTHO) needs rebuilding!
*** (\R.$EARL.AT60AUTH.AUTH) needs rebuilding!
```

```
*** MAKE complete  3 MAY 2001 15:56:55.13 ***
```

Summary:

```
Files rebuilt : 6
Build errors  : 0
Build warnings : 0
```

```
Elapsed Time : 00:00:20.70 (h:m:s.hs)
Time before starting first build: 00:00:00.00 (h:m:s.hs)
Longest Line Length (bytes): 1721
Line Size Max value (bytes): 250000
Number of SQL macro hits: 0
Number of Memory macro hits: 557
Bytes in build steps: 19490
Build Steps Table Size: 120000000
```

Information on RADIX Tables:

```
2 Tables Allocated 10000000 bytes each.
- Table 1 Used      1753424 (17.5%)
- Table 2 Used      3858728 (38.6%)
```

The RUNMKNB obey file for running this report appears below.

```
PARAM MAX-RADIX-TBL-SIZE 10000000

delete define =allmacs
add define =allmacs, file $earl.ba60make.custmacs

run $sysaci.makev500.makev501 /out $s.#xyz,nowait/ &
\r.$earl.at60auth.authm -nobuild -force &
-except_file=XYZMACS -except_loc=\R.$bridge.xyztest

delete define =allmacs
```

The content of the XYZMACS File created on \$BRIDGE.XYZTEST by running the ACI Make utility is shown below.

ATAUTH_AUTHCSEE	= \R.\$BRIDGE.XYZTEST.AUTHCSEE
ATAUTH_AUTHLIBE	= \R.\$BRIDGE.XYZTEST.AUTHLIBE
ATAUTH_AUTHE	= \R.\$BRIDGE.XYZTEST.AUTHE
ATAUTH_AUTHLIBO	= \R.\$BRIDGE.XYZTEST.AUTHLIBO
ATAUTH_AUTHO	= \R.\$BRIDGE.XYZTEST.AUTHO
ATAUTH_AUTH	= \R.\$BRIDGE.XYZTEST.AUTH

You can use this XYZMACS File as the starting point and use a text editor to add macros for dependency files you plan to modify and any processing flags you want to override. Compare the entries shown here with the ones shown in the “Personal Override” topic at the start of this demonstration.

Testing the Personal Override File

Once you have added all of the necessary information to the personal override file (XYZMACS File), the next step is to run the ACI Make utility using the AUTHM Makefile with the NOBUILD option. This is to ensure that the XYZMACS File is accurate before you actually build any of the target files. The report from running the ACI Make utility with this option appears below.

```
ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30
```

```
RUN CMD > \r.$sysaci.makev500.makev501 \r.$earl.at60auth.authm
> -NOBUILD -DMYMACS=\R.$BRIDGE.XYZTEST.XYZMACS
```

```
Define =ALLMACS equals: \R.$EARL.BA60MAKE.CUSTMACS
```

```
*** (\R.$BRIDGE.XYZTEST.AUTHCSEE) needs rebuilding!
*** (\R.$BRIDGE.XYZTEST.AUTHLIBE) needs rebuilding!
*** (\R.$BRIDGE.XYZTEST.AUTHE) needs rebuilding!
*** (\R.$BRIDGE.XYZTEST.AUTHLIBO) needs rebuilding!
```

```
*** (\R.$BRIDGE.XYZTEST.AUTH0) needs rebuilding!
*** (\R.$BRIDGE.XYZTEST.AUTH) needs rebuilding!
```

```
*** MAKE complete  3 MAY 2001 16:23:22.12 ***
```

Summary:

```
Files rebuilt : 6
Build errors  : 0
Build warnings : 0
```

```
Elapsed Time : 00:00:10.98 (h:m:s.hs)
Time before starting first build: 00:00:00.00 (h:m:s.hs)
Longest Line Length (bytes): 1724
Line Size Max value (bytes): 250000
Number of SQL macro hits: 0
Number of Memory macro hits: 557
Bytes in build steps: 19490
Build Steps Table Size: 120000000
```

Information on RADIX Tables:

```
2 Tables Allocated 10000000 bytes each.
- Table 1 Used      1782664 (17.8%)
- Table 2 Used      3858728 (38.6%)
```

The RUNMKNB obey file for running this report appears below.

```
PARAM MAX-RADIX-TBL-SIZE 10000000
```

```
delete define =allmacs
add define =allmacs, file $earl.ba60make.custmacs
```

```
run $sysaci.makev500.makev501 /out $s.#xyz,nowait/ &
\r.$earl.at60auth.authm -nobuild &
-Dmymacs=\R.$bridge.xyztest.xyzmcs
```

```
delete define =allmacs
```

The obey file contains only the NOBUILD option and the D option to use the new XYZMACS File. The results of this run of the utility are the same with or without the FORCE option because none of the target files exist on the XYZTEST subvolume.

All six of the target files appear in the report as needing to be rebuilt.

Generating Target Files

Since the XYZMACS File test was successful, the next step is to generate the target files. The ACI Make utility is run using the AUTHM Makefile four times in this demonstration, as follows:

1. Run the utility to build the target files for the first time.
2. Run the utility when the dependency files are unchanged.
3. Run the utility after the AUTHLIBS has been modified.
4. Run the utility with the FORCE option.

The remainder of this appendix shows the following information for each run of the ACI Make utility.

- The report generated by the ACI Make utility
- The RUNMAKE obey file used
- A FUP INFO inquiry showing the contents of the XYZTEST subvolume

Building Target Files for the First Time

The ACI Make utility creates all of the target files during this run because none of them currently exist. The report from running the ACI Make utility using the AUTHM Makefile appears below.

As you review this report, notice the following items for each target file:

- Each file was built because it did not exist.
- Target and dependency defines were created.
- The first execution statement is provided.
- Security of NNU is assigned to the AUTH File.
- All six files appear in the list of files rebuilt and in the summary.
- No files appear in the list of files not rebuilt.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

RUN CMD > \r.\$sysaci.makev500.makev501 \r.\$earl.at60auth.authm &
> -DMYMACS=\$BRIDGE.XYZTEST.XYZMACS

Define =ALLMACS equals: \R.\$EARL.BA60MAKE.CUSTMACS

Starting the TACL Server...

TACL (T9205D46 - 30JUL1999), Operating System D45, Release D48.00
COPYRIGHT COMPAQ COMPUTER CORPORATION 1985,1987-1999
CPU 3, process has no backup
May 5, 2001 20:41:57
(Invoking \$SYSTEM.SYSTEM.TACLLOCL)

Loaded from \$SYSTEM.SYSTEM.RTACLDEF:

(Invoking \$BRIDGE.subvolume.TACLCSTM)
Current volume is \$BRIDGE.XYZTEST
\$SYSACI.MAKEV500.TACLINIT \$SYSACI.MAKEV500.TACLMACS

TACLINIT - Version 5.01

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD
CPATHEQ_CREATE	CPP_COMPILE
DDLTAL_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD

CPATHEQ_CREATE	CPP_COMPILE
DDLTA_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

TACL Server Started...

Build file:

\R.\$BRIDGE.XYZTEST.AUTHCSEE

Reason for Building File:

Target Does Not Exist

Target Define:

DEFINE ADDED: ATAUTH_AUTHCSEE

VALUE: \R.\$BRIDGE.XYZTEST.AUTHCSEE

Dependency Defines:

DEFINE ADDED: ATAUTH_AUTHCSES

VALUE: \R.\$EARL.AT60AUTH.AUTHCSES

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSACI.ACILIBI.GENFWD2/in \R.\$EARL.AT60AUTH.AUTHCSES/ ext \R.\$BRIDGE.XYZTEST.AUTHCSEE

Build file:

\R.\$BRIDGE.XYZTEST.AUTHLIBE

Reason for Building File:

Target Does Not Exist

Target Define:

DEFINE ADDED: ATAUTH_AUTHLIBE

VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE

Dependency Defines:

DEFINE ADDED: ATAUTH_AUTHLIBS

VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBS

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSACI.ACILIBI.GENFWD2/in \R.\$BRIDGE.XYZTEST.AUTHLIBS/ ext \R.\$BRIDGE.XYZTEST.AUTHLIBE

Build file:

\R.\$BRIDGE.XYZTEST.AUTHE

Reason for Building File:

Target Does Not Exist

Target Define:

DEFINE ADDED: ATAUTH_AUTHE VALUE: \R.\$BRIDGE.XYZTEST.AUTHE

Dependency Defines:

DEFINE ADDED: ATAUTH_AUTHS VALUE: \R.\$BRIDGE.XYZTEST.AUTHS

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSACI.ACILIBI.GENFWD2/in \R.\$BRIDGE.XYZTEST.AUTHS/ ext \R.\$BRIDGE.XYZTEST.AUTHE

Build file:

\R.\$BRIDGE.XYZTEST.AUTHLIBO

Reason for Building File:

Target Does Not Exist

Target Define:

DEFINE ADDED: ATAUTH_AUTHLIBO VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBO

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILDEFS	VALUE: \R.\$SYSACI.ACIUTILS.UTILDEFS
DEFINE ADDED: ACIUTILS_UTILEXTS	VALUE: \R.\$SYSACI.ACIUTILS.UTILEXTS
DEFINE ADDED: ATAUTH_AUTHCSEE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHCSEE
DEFINE ADDED: ATAUTH_AUTHCSEG	VALUE: \R.\$EARL.AT60AUTH.AUTHCSEG
DEFINE ADDED: ATAUTH_AUTHCSES	VALUE: \R.\$EARL.AT60AUTH.AUTHCSES
DEFINE ADDED: ATAUTH_AUTHMVD	VALUE: \R.\$EARL.AT60AUTH.AUTHMVD
DEFINE ADDED: ATAUTH_AUTHG	VALUE: \R.\$BRIDGE.XYZTEST.AUTHG
DEFINE ADDED: ATAUTH_AUTHLIBE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE
DEFINE ADDED: ATAUTH_AUTHLIBS	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBS
DEFINE ADDED: ATAUTH_AUTHMCUD	VALUE: \R.\$EARL.AT60AUTH.AUTHMCUD
DEFINE ADDED: ATAUTH_AUTHTBL	VALUE: \R.\$EARL.AT60AUTH.AUTHTBL
DEFINE ADDED: ATDDL_DDLFTLF	VALUE: \R.\$EARL.AT60DDL.DDLFTLF
DEFINE ADDED: BACSE_ATTCKID	VALUE: \R.\$EARL.BA60CSE.ATTCKID
DEFINE ADDED: BACSE_BATKCCVE	VALUE: \R.\$EARL.BA60CSE.BAKCCVE
DEFINE ADDED: BACSE_BATKCID	VALUE: \R.\$EARL.BA60CSE.BATKCID
DEFINE ADDED: BACSE_DDLATTKC	VALUE: \R.\$EARL.BA60CSE.DDLATTKC
DEFINE ADDED: BACSE_DDLBATKC	VALUE: \R.\$EARL.BA60CSE.DDLBATKC

DEFINE ADDED: BADDL_DDLATTKN	VALUE: \R.\$EARL.BA60DDL.DDLATTKN
DEFINE ADDED: BADDL_DDLBATKN	VALUE: \R.\$EARL.BA60DDL.DDLBATKN
DEFINE ADDED: BADDL_DDLBCNST	VALUE: \R.\$EARL.BA60DDL.DDLBCNST
DEFINE ADDED: BADDL_DDLFCAF	VALUE: \R.\$EARL.BA60DDL.DDLFCAF
DEFINE ADDED: BADDL_DDLFCONF	VALUE: \R.\$EARL.BA60DDL.DDLFCONF
DEFINE ADDED: BADDL_DDLFCPF	VALUE: \R.\$EARL.BA60DDL.DDLFCPF
DEFINE ADDED: BADDL_DDLFIDF	VALUE: \R.\$EARL.BA60DDL.DDLFIDF
DEFINE ADDED: BADDL_DDLFKEYA	VALUE: \R.\$EARL.BA60DDL.DDLFKEYA
DEFINE ADDED: BADDL_DDLFNEG	VALUE: \R.\$EARL.BA60DDL.DDLFNEG
DEFINE ADDED: BADDL_DDLFPBF	VALUE: \R.\$EARL.BA60DDL.DDLFPBF
DEFINE ADDED: BADDL_DDLFSURF	VALUE: \R.\$EARL.BA60DDL.DDLFSURF
DEFINE ADDED: BADDL_DDLFTKN	VALUE: \R.\$EARL.BA60DDL.DDLFTKN
DEFINE ADDED: BADDL_DDLFUAF	VALUE: \R.\$EARL.BA60DDL.DDLFUAF
DEFINE ADDED: BADDL_DDLGPTH	VALUE: \R.\$EARL.BA60DDL.DDLGPTH
DEFINE ADDED: BADDL_DDLGSTM	VALUE: \R.\$EARL.BA60DDL.DDLGSTM
DEFINE ADDED: BADEFS_BADEFTAL	VALUE: \R.\$EARL.BA60DEFS.BADEFTAL
DEFINE ADDED: BALIB_BALIBTE	VALUE: \R.\$EARL.BA60LIB.BALIBTE
DEFINE ADDED: BASRC_ATTKNID	VALUE: \R.\$EARL.BA60SRC.ATTKNID
DEFINE ADDED: BASRC_BATKNCVE	VALUE: \R.\$EARL.BA60SRC.BATKNCVE
DEFINE ADDED: BASRC_BATKNID	VALUE: \R.\$EARL.BA60SRC.BATKNID
DEFINE ADDED: BASRC_BAUTILE	VALUE: \R.\$EARL.BA60SRC.BAUTILE
DEFINE ADDED: BASRC_CVUTILE	VALUE: \R.\$EARL.BA60SRC.CVUTILE
DEFINE ADDED: BASRC_SECUTILE	VALUE: \R.\$EARL.BA60SRC.SECUTILE
DEFINE ADDED: OKDDL_OKDT	VALUE: \R.\$EARL.OK60DDL.OKDT
DEFINE ADDED: OKLIB_OKLIBTE	VALUE: \R.\$EARL.OK60OBJ.OKLIBTE
DEFINE ADDED: OKLIB_OKLIBTG	VALUE: \R.\$EARL.OK60LIB.OKLIBTG
DEFINE ADDED: OKSRC_OKTG	VALUE: \R.\$EARL.OK60SRC.OKTG
DEFINE ADDED: RDEBUG_DUMPEXT	VALUE: \R.\$SYSACI.RDEBUG.DUMPEXT
DEFINE ADDED: SPANNET_GLOBALS	VALUE: \R.\$SYSACI.XPNET.GLOBALS
DEFINE ADDED: SPANNET_INITLIBS	VALUE: \R.\$SYSACI.XPNET.INITLIBS
DEFINE ADDED: SPANNET_SKEL	VALUE: \R.\$SYSACI.XPNET.SKEL
DEFINE ADDED: SPANNET_SKELEXTS	VALUE: \R.\$SYSACI.XPNET.SKELEXTS
DEFINE ADDED: SYSTEM_EXTDECS0	VALUE: \R.\$SYSTEM.SYSTEM.EXTDECS0

Additional Non-Dependent Defines:

DEFINE ADDED: BASRC_BADDLTAL	VALUE: \R.\$EARL.BA60SRC.BADDLTAL
DEFINE ADDED: ATSRC_ATDDLTAL	VALUE: \R.\$EARL.AT60SRC.ATDDLTAL

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

Build file:

\R.\$BRIDGE.XYZTEST.AUTHO

Reason for Building File:

Target Does Not Exist

Target Define:

DEFINE ADDED: ATAUTH_AUTHO

VALUE: \R.\$BRIDGE.XYZTEST.AUTHO

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILDEFS	VALUE: \R.\$SYSACI.ACIUTILS.UTILDEFS
DEFINE ADDED: ACIUTILS_UTILEXTS	VALUE: \R.\$SYSACI.ACIUTILS.UTILEXTS
DEFINE ADDED: ATAUTH_AUTHCSEE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHCSEE
DEFINE ADDED: ATAUTH_AUTHCSEG	VALUE: \R.\$EARL.AT60AUTH.AUTHCSEG
DEFINE ADDED: ATAUTH_AUTHE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHE
DEFINE ADDED: ATAUTH_AUTHEMVD	VALUE: \R.\$EARL.AT60AUTH.AUTHEMVD
DEFINE ADDED: ATAUTH_AUTHG	VALUE: \R.\$BRIDGE.XYZTEST.AUTHG
DEFINE ADDED: ATAUTH_AUTHLIBE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE
DEFINE ADDED: ATAUTH_AUTHMCUD	VALUE: \R.\$EARL.AT60AUTH.AUTHMCUD
DEFINE ADDED: ATAUTH_AUTHS	VALUE: \R.\$BRIDGE.XYZTEST.AUTHS
DEFINE ADDED: ATAUTH_AUTHHTBL	VALUE: \R.\$EARL.AT60AUTH.AUTHHTBL
DEFINE ADDED: ATDDL_DDLFTLF	VALUE: \R.\$EARL.AT60DDL.DDLFTLF
DEFINE ADDED: BACSE_ATTCKID	VALUE: \R.\$EARL.BA60CSE.ATTCKID
DEFINE ADDED: BACSE_BATKID	VALUE: \R.\$EARL.BA60CSE.BATKID
DEFINE ADDED: BACSE_DDLATTKC	VALUE: \R.\$EARL.BA60CSE.DDLATTKC
DEFINE ADDED: BACSE_DDLBATKC	VALUE: \R.\$EARL.BA60CSE.DDLBATKC
DEFINE ADDED: BADDL_DDLATTKN	VALUE: \R.\$EARL.BA60DDL.DDLATTKN
DEFINE ADDED: BADDL_DDLBATKN	VALUE: \R.\$EARL.BA60DDL.DDLBATKN
DEFINE ADDED: BADDL_DDLFCAF	VALUE: \R.\$EARL.BA60DDL.DDLFCAF
DEFINE ADDED: BADDL_DDLFCNF	VALUE: \R.\$EARL.BA60DDL.DDLFCNF
DEFINE ADDED: BADDL_DDLFCPF	VALUE: \R.\$EARL.BA60DDL.DDLFCPF
DEFINE ADDED: BADDL_DDLFIDF	VALUE: \R.\$EARL.BA60DDL.DDLFIDF
DEFINE ADDED: BADDL_DDLFKEYA	VALUE: \R.\$EARL.BA60DDL.DDLFKEYA
DEFINE ADDED: BADDL_DDLFNEG	VALUE: \R.\$EARL.BA60DDL.DDLFNEG
DEFINE ADDED: BADDL_DDLFPBF	VALUE: \R.\$EARL.BA60DDL.DDLFPBF
DEFINE ADDED: BADDL_DDLFUAF	VALUE: \R.\$EARL.BA60DDL.DDLFUAF
DEFINE ADDED: BADDL_DDLGPTH	VALUE: \R.\$EARL.BA60DDL.DDLGPTH
DEFINE ADDED: BADDL_DDLGSTM	VALUE: \R.\$EARL.BA60DDL.DDLGSTM
DEFINE ADDED: BADEFS_BADEFTAL	VALUE: \R.\$EARL.BA60DEFS.BADEFTAL
DEFINE ADDED: BASRC_ATTCKNID	VALUE: \R.\$EARL.BA60SRC.ATTCKNID
DEFINE ADDED: BASRC_BATKNID	VALUE: \R.\$EARL.BA60SRC.BATKNID
DEFINE ADDED: BASRC_BAUTILE	VALUE: \R.\$EARL.BA60SRC.BAUTILE
DEFINE ADDED: BASRC_CVUTILE	VALUE: \R.\$EARL.BA60SRC.CVUTILE
DEFINE ADDED: BASRC_SECUTILE	VALUE: \R.\$EARL.BA60SRC.SECUTILE
DEFINE ADDED: OKDDL_OKDT	VALUE: \R.\$EARL.OK60DDL.OKDT
DEFINE ADDED: OKLIB_OKLIBTG	VALUE: \R.\$EARL.OK60LIB.OKLIBTG
DEFINE ADDED: OKSRC_OKTG	VALUE: \R.\$EARL.OK60SRC.OKTG
DEFINE ADDED: RDEBUG_DUMPEXT	VALUE: \R.\$SYSACI.RDEBUG.DUMPEXT
DEFINE ADDED: SPANNET_GLOBALS	VALUE: \R.\$SYSACI.XPNET.GLOBALS
DEFINE ADDED: SPANNET_INITLIBS	VALUE: \R.\$SYSACI.XPNET.INITLIBS
DEFINE ADDED: SPANNET_SKEL	VALUE: \R.\$SYSACI.XPNET.SKEL
DEFINE ADDED: SPANNET_SKELEXTS	VALUE: \R.\$SYSACI.XPNET.SKELEXTS
DEFINE ADDED: SYSTEM_EXTDECS0	VALUE: \R.\$SYSTEM.SYSTEM.EXTDECS0

Additional Non-Dependent Defines:

DEFINE ADDED: BASRC_BADDLTAL

VALUE: \R.\$EARL.BA60SRC.BADDLTAL

DEFINE ADDED: ATSRC_ATDDLTLAL

VALUE: \R.\$EARL.AT60SRC.ATDDLTLAL

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

Build file:

\R.\$BRIDGE.XYZTEST.AUTH

Reason for Building File:

Target Does Not Exist

Target Define:

DEFINE ADDED: ATAUTH_AUTH

VALUE: \R.\$BRIDGE.XYZTEST.AUTH

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILB

VALUE: \R.\$SYSACI.ACIUTILS.UTILB

DEFINE ADDED: ATAUTH_AUTHO

VALUE: \R.\$BRIDGE.XYZTEST.AUTHO

DEFINE ADDED: ATAUTH_AUTHLIBO

VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBO

DEFINE ADDED: ATLIB_ATLIBLO

VALUE: \R.\$EARL.AT60LIB.ATLIBLO

DEFINE ADDED: BACSE_BATKCCVO

VALUE: \R.\$EARL.BA60CSE.BATKCCVO

DEFINE ADDED: BASRC_BAUTIL

VALUE: \R.\$EARL.BA60SRC.BAUTIL

DEFINE ADDED: BASRC_CVUTIL

VALUE: \R.\$EARL.BA60SRC.CVUTIL

DEFINE ADDED: BASRC_SECUTIL

VALUE: \R.\$EARL.BA60SRC.SECUTIL

DEFINE ADDED: BASRC_BATKNCVO

VALUE: \R.\$EARL.BA60SRC.BATKNCVO

DEFINE ADDED: OKLIB_OKLIBTO

VALUE: \R.\$EARL.OK60OBJ.OKLIBTO

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

File Utility Program - T6553D45 - (09NOV98) System \R

Copyright Tandem Computers Incorporated 1981, 1983, 1985-1998

SECURE \R.\$BRIDGE.XYZTEST.AUTH, "NNNU"

*** MAKE complete 5 MAY 2001 20:44:07.43 ***

Files Rebuilt:

\R.\$BRIDGE.XYZTEST.AUTHCSEE

\R.\$BRIDGE.XYZTEST.AUTHLIBE

\R.\$BRIDGE.XYZTEST.AUTHE

\R.\$BRIDGE.XYZTEST.AUTHLIBO

\R.\$BRIDGE.XYZTEST.AUTHO

\R.\$BRIDGE.XYZTEST.AUTH

Files NOT Rebuilt:

NONE

Summary:

Files rebuilt : 6
Build errors : 0
Build warnings : 0

Elapsed Time : 00:02:19.11 (h:m:s.hs)
Time before starting first build: 00:00:11.08 (h:m:s.hs)
Longest Line Length (bytes): 1726
Line Size Max value (bytes): 250000
Number of SQL macro hits: 0
Number of Memory macro hits: 651
Bytes in build steps: 19490
Build Steps Table Size: 120000000

Information on RADIX Tables:

2 Tables Allocated 10000000 bytes each.
- Table 1 Used 1780514 (17.8%)
- Table 2 Used 3858728 (38.6%)

The RUNMAKE obey file for running this report appears below.

PARAM MAX-RADIX-TBL-SIZE 10000000

```
delete define =allmacs
add define =allmacs, file $earl.ba60make.custmacs

run $sysaci.makev500.makev501 /out $s.#xyz,nowait/ &
\r.$earl.at60auth.authm &
-Dmymacs=$bridge.xyztest.xyzmacs

delete define =allmacs
```

The contents of the XYZTEST subvolume before the target files are built appear below. Only the source files specified in the XYZMACS File now exist.

	CODE	EOF	LAST MODIF	OWNER	RWEP
\$BRIDGE.XYZTEST					
AUTHG	101	75332	2May2001 08:00	37,5	NCNC
AUTHLIBS	101	792010	2May2001 08:00	37,5	NCNC
AUTHS	101	904834	2May2001 08:00	37,5	NCNC
RUNMAKE	101	2384	19:58	37,5	NCNC
RUNMKNB	101	2396	3May2001 16:22	37,5	NCNC
XYZMACS	101	2620	20:20	37,5	NCNC

The contents of the XYZTEST subvolume after the target files are built appear below. All of the source and target files specified in the XYZMACS File now exist. The target files appear in bold.

	CODE	EOF	LAST	MODIF	OWNER	RWEP
\$BRIDGE.XYZTEST						
AUTH	100	1784746		20:44	37,5	NNNU
AUTHCSEE	101	2048		20:41	37,5	NCNC
AUTHE	101	81746		20:42	37,5	NCNC
AUTHG	101	75332	2May2001	08:00	37,5	NCNC
AUTHLIBE	101	81764		20:42	37,5	NCNC
AUTHLIBO	100	689554		20:42	37,5	NNNU
AUTHLIBS	101	792010	2May2001	08:00	37,5	NCNC
AUTHO	100	758272		20:43	37,5	NNNU
AUTHS	101	904834	2May2001	08:00	37,5	NCNC
RUNMAKE	101	2384		19:58	37,5	NCNC
RUNMKNB	101	2396	3May2001	16:22	37,5	NCNC
XYZMACS	101	2620		20:20	37,5	NCNC

Dependency Files are Unchanged

The ACI Make utility does not update any of the target files during this run because all of them currently exist and have newer modification timestamps than the dependency files from which they were built. The report from running the ACI Make utility with the AUTHM Makefile appears below.

As you review this report, notice that no target files are built and the report totals show that no target files are rebuilt. The list of files not rebuilt in the totals includes only files that should be rebuilt but cannot be rebuilt because of errors, not files that do not need to be rebuilt.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

RUN CMD > \r.\$sysaci.makev500.makev501 \r.\$earl.at60auth.authm &
> -DMYMACS=\$BRIDGE.XYZTEST.XYZMACS

Define =ALLMACS equals: \R.\$EARL.BA60MAKE.CUSTMACS

Starting the TACL Server...

TACL (T9205D46 - 30JUL1999), Operating System D45, Release D48.00
COPYRIGHT COMPAQ COMPUTER CORPORATION 1985,1987-1999
CPU 3, process has no backup
May 5, 2001 20:55:48
(Invoking \$SYSTEM.SYSTEM.TACLLOCL)
Loaded from \$SYSTEM.SYSTEM.RTACLDEF:

(Invoking \$BRIDGE.subvolume.TACLCSTM)
Current volume is \$BRIDGE.XYZTEST
\$SYSACI.MAKEV500.TACLINIT \$SYSACI.MAKEV500.TACLMACS

TACLINIT - Version 5.01

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD
CPATHEQ_CREATE	CPP_COMPILE
DDLTAL_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD
CPATHEQ_CREATE	CPP_COMPILE
DDLTAL_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE


```
PRODUCE_TIME_STR      PURGE_DICT
SCUP_ROUTINE          SECURE
TEMPLMAC              TOTAL_SIZE
UPDATE_DICT_ALT_FILES
```

TACL Server Started...

*** MAKE complete 5 MAY 2001 20:55:49.85 ***

Files Rebuilt:

NONE

Files NOT Rebuilt:

NONE

Summary:

Files rebuilt : 0
Build errors : 0
Build warnings : 0

Elapsed Time : 00:00:11.33 (h:m:s.hs)
Time before starting first build: 00:00:08.59 (h:m:s.hs)
Longest Line Length (bytes): 1726
Line Size Max value (bytes): 250000
Number of SQL macro hits: 0
Number of Memory macro hits: 554
Bytes in build steps: 19490
Build Steps Table Size: 120000000

Information on RADIX Tables:

2 Tables Allocated 10000000 bytes each.
- Table 1 Used 1780514 (17.8%)
- Table 2 Used 3858728 (38.6%)

The RUNMAKE obey file for running this report appears below.

PARAM MAX-RADIX-TBL-SIZE 10000000

delete define =allmacs
add define =allmacs, file \$earl.ba60make.custmacs

run \$sysaci.makev500.makev501 /out \$s.#xyz,nowait/ &
\r.\$earl.at60auth.authm &
-Dmymacs=\$bridge.xyztest.XYZMACS

delete define =allmacs

The contents of the XYZTEST subvolume after this run of the ACI Make utility appear below. There are no changes in modification timestamps since the first run of the utility.

	CODE	EOF	LAST	MODIF	OWNER	RWEP
\$BRIDGE.XYZTEST						
AUTH	100	1784746		20:44	37,5	NNNU
AUTHCSEE	101	2048		20:41	37,5	NCNC
AUTHE	101	81746		20:42	37,5	NCNC
AUTHG	101	75332	2May2001	08:00	37,5	NCNC
AUTHLIBE	101	81764		20:42	37,5	NCNC
AUTHLIBO	100	689554		20:42	37,5	NNNU
AUTHLIBS	101	792010	2May2001	08:00	37,5	NCNC
AUTHO	100	758272		20:43	37,5	NNNU
AUTHS	101	904834	2May2001	08:00	37,5	NCNC
RUNMAKE	101	2384		19:58	37,5	NCNC
RUNMKNB	101	2396	3May2001	16:22	37,5	NCNC
XYZMACS	101	2620		20:20	37,5	NCNC

The AUTHLIBS File Has Been Modified

In this run, the ACI Make utility recreates four of the six target files because the AUTHLIBS File has been modified and it is a dependency for four of the target files. The report from running the ACI Make utility with the AUTHM Makefile appears below.

As you review this report, notice the following items for each target file that was rebuilt:

- Modification timestamps are provided for the existing target file and the updated dependency file so you can see why the target file was rebuilt.
- Target and dependency file defines were created.
- The first execution statement is provided.
- Security of NNNU is assigned to the AUTH.
- Just four files appear in the list of files rebuilt and in the summary.
- No files appear in the list of files not rebuilt.

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

```
RUN CMD > \r.$sysaci.makev500.makev501 \r.$earl.at60auth.authm &
> -DMYMACS=$BRIDGE.XYZTEST.XYZMACS
```

Define =ALLMACS equals: \R.\$EARL.BA60MAKE.CUSTMACS

Starting the TACL Server...

TACL (T9205D46 - 30JUL1999), Operating System D45, Release D48.00
 COPYRIGHT COMPAQ COMPUTER CORPORATION 1985,1987-1999
 CPU 1, process has no backup
 May 5, 2001 21:07:31
 (Invoking \$SYSTEM.SYSTEM.TACLLOCL)

Loaded from \$SYSTEM.SYSTEM.RTACLDEF:

(Invoking \$BRIDGE.subvolume.TACLCSTM)
 Current volume is \$BRIDGE.XYZTEST
 \$SYSACI.MAKEV500.TACLINIT \$SYSACI.MAKEV500.TACLMACS

TACLINIT - Version 5.01

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD
CPATHEQ_CREATE	CPP_COMPILE
DDLTAL_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD

CPATHEQ_CREATE	CPP_COMPILE
DDLTA_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

TACL Server Started...

Build file:

\R.\$BRIDGE.XYZTEST.AUTHLIBE

Reason for Building File:

Timestamp: 2001 05:05:20:42:006:013 Target: \R.\$BRIDGE.XYZTEST.AUTHLIBE
 Timestamp: 2001 05:05:21:04:052:007 Dependency: \R.\$BRIDGE.XYZTEST.AUTHLIBS

Target Define:

DEFINE ADDED: ATAUTH_AUTHLIBE VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE

Dependency Defines:

DEFINE ADDED: ATAUTH_AUTHLIBS VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBS

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSACI.ACILIBI.GENFWD2/in \R.\$BRIDGE.XYZTEST.AUTHLIBS/ ext \R.\$BRIDGE.XYZTEST.AUTHLIBE

Build file:

\R.\$BRIDGE.XYZTEST.AUTHLIBO

Reason for Building File:

Timestamp: 2001 05:05:20:42:057:075 Target: \R.\$BRIDGE.XYZTEST.AUTHLIBO
 Timestamp: 2001 05:05:21:07:039:006 Dependency: \R.\$BRIDGE.XYZTEST.AUTHLIBE

Target Define:

DEFINE ADDED: ATAUTH_AUTHLIBO VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBO

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILDEFS	VALUE: \R.\$SYSACI.ACIUTILS.UTILDEFS
DEFINE ADDED: ACIUTILS_UTILEXTS	VALUE: \R.\$SYSACI.ACIUTILS.UTILEXTS
DEFINE ADDED: ATAUTH_AUTHCSEE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHCSEE
DEFINE ADDED: ATAUTH_AUTHCSEG	VALUE: \R.\$EARL.AT60AUTH.AUTHCSEG
DEFINE ADDED: ATAUTH_AUTHCSES	VALUE: \R.\$EARL.AT60AUTH.AUTHCSES

DEFINE ADDED: ATAUTH_AUTHMVD	VALUE: \R.\$EARL.AT60AUTH.AUTHMVD
DEFINE ADDED: ATAUTH_AUTHG	VALUE: \R.\$BRIDGE.XYZTEST.AUTHG
DEFINE ADDED: ATAUTH_AUTHLIBE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE
DEFINE ADDED: ATAUTH_AUTHLIBS	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBS
DEFINE ADDED: ATAUTH_AUTHMCUD	VALUE: \R.\$EARL.AT60AUTH.AUTHMCUD
DEFINE ADDED: ATAUTH_AUTHHTBL	VALUE: \R.\$EARL.AT60AUTH.AUTHHTBL
DEFINE ADDED: ATDDL_DDLFTLF	VALUE: \R.\$EARL.AT60DDL.DDLFTLF
DEFINE ADDED: BACSE_ATTKCID	VALUE: \R.\$EARL.BA60CSE.ATTKCID
DEFINE ADDED: BACSE_BATKCCVE	VALUE: \R.\$EARL.BA60CSE.BAKCCVE
DEFINE ADDED: BACSE_BATKCID	VALUE: \R.\$EARL.BA60CSE.BATKCID
DEFINE ADDED: BACSE_DDLATTKC	VALUE: \R.\$EARL.BA60CSE.DDLATTKC
DEFINE ADDED: BACSE_DDLBATKC	VALUE: \R.\$EARL.BA60CSE.DDLBATKC
DEFINE ADDED: BADDL_DDLATTKN	VALUE: \R.\$EARL.BA60DDL.DDLATTKN
DEFINE ADDED: BADDL_DDLBATKN	VALUE: \R.\$EARL.BA60DDL.DDLBATKN
DEFINE ADDED: BADDL_DDLBCNST	VALUE: \R.\$EARL.BA60DDL.DDLBCNST
DEFINE ADDED: BADDL_DDLFCAF	VALUE: \R.\$EARL.BA60DDL.DDLFCAF
DEFINE ADDED: BADDL_DDLFCONF	VALUE: \R.\$EARL.BA60DDL.DDLFCONF
DEFINE ADDED: BADDL_DDLFCPF	VALUE: \R.\$EARL.BA60DDL.DDLFCPF
DEFINE ADDED: BADDL_DDLFIDF	VALUE: \R.\$EARL.BA60DDL.DDLFIDF
DEFINE ADDED: BADDL_DDLFKEYA	VALUE: \R.\$EARL.BA60DDL.DDLFKEYA
DEFINE ADDED: BADDL_DDLFNEG	VALUE: \R.\$EARL.BA60DDL.DDLFNEG
DEFINE ADDED: BADDL_DDLFPBF	VALUE: \R.\$EARL.BA60DDL.DDLFPBF
DEFINE ADDED: BADDL_DDLFSURF	VALUE: \R.\$EARL.BA60DDL.DDLFSURF
DEFINE ADDED: BADDL_DDLFTKN	VALUE: \R.\$EARL.BA60DDL.DDLFTKN
DEFINE ADDED: BADDL_DDLFUAF	VALUE: \R.\$EARL.BA60DDL.DDLFUAF
DEFINE ADDED: BADDL_DDLGPTH	VALUE: \R.\$EARL.BA60DDL.DDLGPTH
DEFINE ADDED: BADDL_DDLGSTM	VALUE: \R.\$EARL.BA60DDL.DDLGSTM
DEFINE ADDED: BADEFS_BADEFTAL	VALUE: \R.\$EARL.BA60DEFS.BADEFTAL
DEFINE ADDED: BALIB_BALIBTE	VALUE: \R.\$EARL.BA60LIB.BALIBTE
DEFINE ADDED: BASRC_ATTKNID	VALUE: \R.\$EARL.BA60SRC.ATTKNID
DEFINE ADDED: BASRC_BATKNCVE	VALUE: \R.\$EARL.BA60SRC.BATKNCVE
DEFINE ADDED: BASRC_BATKNID	VALUE: \R.\$EARL.BA60SRC.BATKNID
DEFINE ADDED: BASRC_BAUTILE	VALUE: \R.\$EARL.BA60SRC.BAUTILE
DEFINE ADDED: BASRC_CVUTILE	VALUE: \R.\$EARL.BA60SRC.CVUTILE
DEFINE ADDED: BASRC_SECUTILE	VALUE: \R.\$EARL.BA60SRC.SECUTILE
DEFINE ADDED: OKDDL_OKDT	VALUE: \R.\$EARL.OK60DDL.OKDT
DEFINE ADDED: OKLIB_OKLIBTE	VALUE: \R.\$EARL.OK60OBJ.OKLIBTE
DEFINE ADDED: OKLIB_OKLIBTG	VALUE: \R.\$EARL.OK60LIB.OKLIBTG
DEFINE ADDED: OKSRC_OKTG	VALUE: \R.\$EARL.OK60SRC.OKTG
DEFINE ADDED: RDEBUG_DUMPEXT	VALUE: \R.\$SYSACI.RDEBUG.DUMPEXT
DEFINE ADDED: SPANNET_GLOBALS	VALUE: \R.\$SYSACI.XPNET.GLOBALS
DEFINE ADDED: SPANNET_INITLIBS	VALUE: \R.\$SYSACI.XPNET.INITLIBS
DEFINE ADDED: SPANNET_SKEL	VALUE: \R.\$SYSACI.XPNET.SKEL
DEFINE ADDED: SPANNET_SKELEXTS	VALUE: \R.\$SYSACI.XPNET.SKELEXTS
DEFINE ADDED: SYSTEM_EXTDECS0	VALUE: \R.\$SYSTEM.SYSTEM.EXTDECS0

Additional Non-Dependent Defines:

DEFINE ADDED: BASRC_BADDLTAL	VALUE: \R.\$EARL.BA60SRC.BADDLTAL
DEFINE ADDED: ATSRC_ATDDLTLAL	VALUE: \R.\$EARL.AT60SRC.ATDDLTLAL

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

Build file:

\R.\$BRIDGE.XYZTEST.AUTH0

Reason for Building File:

Timestamp: 2001 05:05:20:43:043:019 Target: \R.\$BRIDGE.XYZTEST.AUTH0
 Timestamp: 2001 05:05:21:07:039:006 Dependency: \R.\$BRIDGE.XYZTEST.AUTHLIBE

Target Define:

DEFINE ADDED: ATAUTH_AUTH0 VALUE: \R.\$BRIDGE.XYZTEST.AUTH0

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILDEFS	VALUE: \R.\$SYSACI.ACIUTILS.UTILDEFS
DEFINE ADDED: ACIUTILS_UTILEXTS	VALUE: \R.\$SYSACI.ACIUTILS.UTILEXTS
DEFINE ADDED: ATAUTH_AUTHCSEE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHCSEE
DEFINE ADDED: ATAUTH_AUTHCSEG	VALUE: \R.\$EARL.AT60AUTH.AUTHCSEG
DEFINE ADDED: ATAUTH_AUTHE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHE
DEFINE ADDED: ATAUTH_AUTHEMVD	VALUE: \R.\$EARL.AT60AUTH.AUTHEMVD
DEFINE ADDED: ATAUTH_AUTHG	VALUE: \R.\$BRIDGE.XYZTEST.AUTHG
DEFINE ADDED: ATAUTH_AUTHLIBE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE
DEFINE ADDED: ATAUTH_AUTHMCUD	VALUE: \R.\$EARL.AT60AUTH.AUTHMCUD
DEFINE ADDED: ATAUTH_AUTHS	VALUE: \R.\$BRIDGE.XYZTEST.AUTHS
DEFINE ADDED: ATAUTH_AUTHTBL	VALUE: \R.\$EARL.AT60AUTH.AUTHTBL
DEFINE ADDED: ATDDL_DDLFTLF	VALUE: \R.\$EARL.AT60DDL.DDLFTLF
DEFINE ADDED: BACSE_ATTCKID	VALUE: \R.\$EARL.BA60CSE.ATTCKID
DEFINE ADDED: BACSE_BATKID	VALUE: \R.\$EARL.BA60CSE.BATKID
DEFINE ADDED: BACSE_DDLATTKC	VALUE: \R.\$EARL.BA60CSE.DDLATTKC
DEFINE ADDED: BACSE_DDLBATKC	VALUE: \R.\$EARL.BA60CSE.DDLBATKC
DEFINE ADDED: BADDL_DDLATTKN	VALUE: \R.\$EARL.BA60DDL.DDLATTKN
DEFINE ADDED: BADDL_DDLBATKN	VALUE: \R.\$EARL.BA60DDL.DDLBATKN
DEFINE ADDED: BADDL_DDLFCAF	VALUE: \R.\$EARL.BA60DDL.DDLFCAF
DEFINE ADDED: BADDL_DDLFCNF	VALUE: \R.\$EARL.BA60DDL.DDLFCNF
DEFINE ADDED: BADDL_DDLFCPF	VALUE: \R.\$EARL.BA60DDL.DDLFCPF
DEFINE ADDED: BADDL_DDLFIDF	VALUE: \R.\$EARL.BA60DDL.DDLFIDF
DEFINE ADDED: BADDL_DDLFKEYA	VALUE: \R.\$EARL.BA60DDL.DDLFKEYA
DEFINE ADDED: BADDL_DDLFNEG	VALUE: \R.\$EARL.BA60DDL.DDLFNEG
DEFINE ADDED: BADDL_DDLFPBF	VALUE: \R.\$EARL.BA60DDL.DDLFPBF
DEFINE ADDED: BADDL_DDLFUAF	VALUE: \R.\$EARL.BA60DDL.DDLFUAF
DEFINE ADDED: BADDL_DDLGPTH	VALUE: \R.\$EARL.BA60DDL.DDLGPTH
DEFINE ADDED: BADDL_DDLGSTM	VALUE: \R.\$EARL.BA60DDL.DDLGSTM
DEFINE ADDED: BADEFS_BADEFTAL	VALUE: \R.\$EARL.BA60DEFS.BADEFTAL
DEFINE ADDED: BASRC_ATTCKNID	VALUE: \R.\$EARL.BA60SRC.ATTCKNID
DEFINE ADDED: BASRC_BATKNID	VALUE: \R.\$EARL.BA60SRC.BATKNID
DEFINE ADDED: BASRC_BAUTILE	VALUE: \R.\$EARL.BA60SRC.BAUTILE
DEFINE ADDED: BASRC_CVUTILE	VALUE: \R.\$EARL.BA60SRC.CVUTILE
DEFINE ADDED: BASRC_SECUTILE	VALUE: \R.\$EARL.BA60SRC.SECUTILE
DEFINE ADDED: OKDDL_OKDT	VALUE: \R.\$EARL.OK60DDL.OKDT
DEFINE ADDED: OKLIB_OKLIBTG	VALUE: \R.\$EARL.OK60LIB.OKLIBTG

DEFINE ADDED: OKSRC_OKTG	VALUE: \R.\$EARL.OK60SRC.OKTG
DEFINE ADDED: RDEBUG_DUMPEXT	VALUE: \R.\$SYSACI.RDEBUG.DUMPEXT
DEFINE ADDED: SPANNET_GLOBALS	VALUE: \R.\$SYSACI.XPNET.GLOBALS
DEFINE ADDED: SPANNET_INITLIBS	VALUE: \R.\$SYSACI.XPNET.INITLIBS
DEFINE ADDED: SPANNET_SKEL	VALUE: \R.\$SYSACI.XPNET.SKEL
DEFINE ADDED: SPANNET_SKELEXTS	VALUE: \R.\$SYSACI.XPNET.SKELEXTS
DEFINE ADDED: SYSTEM_EXTDECS0	VALUE: \R.\$SYSTEM.SYSTEM.EXTDECS0

Additional Non-Dependent Defines:

DEFINE ADDED: BASRC_BADDLTAL	VALUE: \R.\$EARL.BA60SRC.BADDLTAL
DEFINE ADDED: ATSRC_ATDDLAL	VALUE: \R.\$EARL.AT60SRC.ATDDLAL

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

Build file:

\R.\$BRIDGE.XYZTEST.AUTH

Reason for Building File:

Timestamp: 2001 05:05:20:44:006:008	Target: \R.\$BRIDGE.XYZTEST.AUTH
Timestamp: 2001 05:05:21:09:009:084	Dependency: \R.\$BRIDGE.XYZTEST.AUTH0

Target Define:

DEFINE ADDED: ATAUTH_AUTH	VALUE: \R.\$BRIDGE.XYZTEST.AUTH
---------------------------	---------------------------------

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILB	VALUE: \R.\$SYSACI.ACIUTILS.UTILB
DEFINE ADDED: ATAUTH_AUTH0	VALUE: \R.\$BRIDGE.XYZTEST.AUTH0
DEFINE ADDED: ATAUTH_AUTHLIB0	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIB0
DEFINE ADDED: ATLIB_ATLIBLO	VALUE: \R.\$EARL.AT60LIB.ATLIBLO
DEFINE ADDED: BACSE_BATKCCVO	VALUE: \R.\$EARL.BA60CSE.BATKCCVO
DEFINE ADDED: BASRC_BAUTIL	VALUE: \R.\$EARL.BA60SRC.BAUTIL
DEFINE ADDED: BASRC_CVUTIL	VALUE: \R.\$EARL.BA60SRC.CVUTIL
DEFINE ADDED: BASRC_SECUTIL	VALUE: \R.\$EARL.BA60SRC.SECUTIL
DEFINE ADDED: BASRC_BATKNCVO	VALUE: \R.\$EARL.BA60SRC.BATKNCVO
DEFINE ADDED: OKLIB_OKLIBTO	VALUE: \R.\$EARL.OK60OBJ.OKLIBTO

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

File Utility Program - T6553D45 - (09NOV98) System \R
 Copyright Tandem Computers Incorporated 1981, 1983, 1985-1998
 SECURE \R.\$BRIDGE.XYZTEST.AUTH, "NNNU"

*** MAKE complete 5 MAY 2001 21:09:34.79 ***

Files Rebuilt:

 \R.\$BRIDGE.XYZTEST.AUTHLIBE
 \R.\$BRIDGE.XYZTEST.AUTHLIBO
 \R.\$BRIDGE.XYZTEST.AUTHO
 \R.\$BRIDGE.XYZTEST.AUTH

Files NOT Rebuilt:

 NONE

Summary:

 Files rebuilt : 4
 Build errors : 0
 Build warnings : 0

Elapsed Time : 00:02:13.38 (h:m:s.hs)
Time before starting first build: 00:00:11.47 (h:m:s.hs)
Longest Line Length (bytes): 1726
Line Size Max value (bytes): 250000
Number of SQL macro hits: 0
Number of Memory macro hits: 645
Bytes in build steps: 19490
Build Steps Table Size: 120000000

Information on RADIX Tables:

2 Tables Allocated 10000000 bytes each.
 - Table 1 Used 1780514 (17.8%)
 - Table 2 Used 3858728 (38.6%)

The RUNMAKE obey file for running this report appears below.

PARAM MAX-RADIX-TBL-SIZE 10000000

delete define =allmacs
add define =allmacs, file \$earl.ba60make.custmacs

run \$sysaci.makev500.makev501 /out \$s.#xyz,nowait/ &
 \r.\$earl.at60auth.authm &
 -Dmymacs=\$bridge.xyztest.XYZMACS

delete define =allmacs

The contents of the XYZTEST subvolume after the AUTHLIBS is modified but before the ACI Make utility is run appears below. The AUTHLIBS has the newest modification timestamp (appearing in bold).

	CODE	EOF	LAST	MODIF	OWNER	RWEP
\$BRIDGE.XYZTEST						
AUTH	100	1784746		20:44	37,5	NNNU
AUTHCSEE	101	2048		20:41	37,5	NCNC
AUTHE	101	81746		20:42	37,5	NCNC
AUTHG	101	75332	2May2001	08:00	37,5	NCNC
AUTHLIBE	101	81764		20:42	37,5	NCNC
AUTHLIBO	100	689554		20:42	37,5	NNNU
AUTHLIBS	101	792010		21:04	37,5	NCNC
AUTHO	100	758272		20:43	37,5	NNNU
AUTHS	101	904834	2May2001	08:00	37,5	NCNC
RUNMAKE	101	2384		19:58	37,5	NCNC
RUNMKNB	101	2396	3May2001	16:22	37,5	NCNC
XYZMACS	101	2620		20:20	37,5	NCNC

The contents of the XYZTEST subvolume after the ACI Make utility is run appears below. The modification timestamps of the four rebuilt files have changed and appear in bold.

	CODE	EOF	LAST	MODIF	OWNER	RWEP
\$BRIDGE.XYZTEST						
AUTH	100	1784746		21:09	37,5	NNNU
AUTHCSEE	101	2048		20:41	37,5	NCNC
AUTHE	101	81746		20:42	37,5	NCNC
AUTHG	101	75332	2May2001	08:00	37,5	NCNC
AUTHLIBE	101	81764		21:07	37,5	NCNC
AUTHLIBO	100	689554		21:08	37,5	NNNU
AUTHLIBS	101	792010		21:04	37,5	NCNC
AUTHO	100	758272		21:09	37,5	NNNU
AUTHS	101	904834	2May2001	08:00	37,5	NCNC
RUNMAKE	101	2384		19:58	37,5	NCNC
RUNMKNB	101	2396	3May2001	16:22	37,5	NCNC
XYZMACS	101	2620		20:20	37,5	NCNC

The FORCE Option is Used

In this run, the ACI Make utility recreates all of the target files because the FORCE option is used, even though none of the dependency files have changed since the previous run. If the utility is run now without the FORCE option, no target files are updated. The report from running the ACI Make utility with the AUTHM Makefile appears below.

As you review this report, notice the following items for each target file:

- Each file was built because the FORCE option is being used.
 - Target and dependency file defines were created.
 - The first execution statement is provided.
 - Security of NNNU is assigned to the AUTH.
 - All six files appear in the list of files rebuilt and in the summary.
 - No files appear in the list of files not rebuilt.
-

ACI MAKE utility - Version: MAKEV501 May 23 2001 13:45:30

```
RUN CMD > \r.$sysaci.makev500.makev501 \r.$earl.at60auth.authm &  
> -DMYMACS=$BRIDGE.XYZTEST.XYZMACS -FORCE
```

Define =ALLMACS equals: \R.\$EARL.BA60MAKE.CUSTMACS

Starting the TACL Server...

TACL (T9205D46 - 30JUL1999), Operating System D45, Release D48.00
COPYRIGHT COMPAQ COMPUTER CORPORATION 1985,1987-1999
CPU 1, process has no backup
May 5, 2001 21:17:14
(Invoking \$SYSTEM.SYSTEM.TACLLOCL)

Loaded from \$SYSTEM.SYSTEM.RTACLDEF:

(Invoking \$BRIDGE.subvolume.TACLCSTM)
Current volume is \$BRIDGE.XYZTEST
\$SYSACI.MAKEV500.TACLINIT \$SYSACI.MAKEV500.TACLMACS

TACLINIT - Version 5.01

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD
CPATHEQ_CREATE	CPP_COMPILE
DDLTLAL_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE

EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

Loaded from \$SYSACI.MAKEV500.TACLMACS:

TACLMACS_VERSION_501	ACI_SCOBOLX
ACI_SCOBOLX_MAKE	CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION	CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE	CPATHEQ_BUILD
CPATHEQ_CREATE	CPP_COMPILE
DDLTAL_UNDERBAR_CONV	DEL
DEL_PERUSE_JOB	DISPLAY_COMPLETION_INFO
DUP_DICT	DUP_EXEC_FILE
EXPANDDEF	EXPANDSSV
EXPANDSSV_DICT_NOSAVE	GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE	PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR	PURGE_DICT
SCUP_ROUTINE	SECURE
TEMPLMAC	TOTAL_SIZE
UPDATE_DICT_ALT_FILES	

TACL Server Started...

Build file:

\R.\$BRIDGE.XYZTEST.AUTHCSEE

Reason for Building File:

-FORCE option is on

Target Define:

DEFINE ADDED: ATAUTH_AUTHCSEE

VALUE: \R.\$BRIDGE.XYZTEST.AUTHCSEE

Dependency Defines:

DEFINE ADDED: ATAUTH_AUTHCSES

VALUE: \R.\$EARL.AT60AUTH.AUTHCSES

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSACI.ACILIBI.GENFWD2/in \R.\$EARL.AT60AUTH.AUTHCSES/ ext \R.\$BRIDGE.XYZTEST.
AUTHCSEE

Build file:

\R.\$BRIDGE.XYZTEST.AUTHLIBE

Reason for Building File:

-FORCE option is on

Target Define:

DEFINE ADDED: ATAUTH_AUTHLIBE VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE

Dependency Defines:

DEFINE ADDED: ATAUTH_AUTHLIBS VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBS

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSACI.ACILIBI.GENFWD2/in \R.\$BRIDGE.XYZTEST.AUTHLIBS/ ext \R.\$BRIDGE.XYZTEST.AUTHLIBE

Build file:

\R.\$BRIDGE.XYZTEST.AUTHE

Reason for Building File:

-FORCE option is on

Target Define:

DEFINE ADDED: ATAUTH_AUTHE VALUE: \R.\$BRIDGE.XYZTEST.AUTHE

Dependency Defines:

DEFINE ADDED: ATAUTH_AUTHS VALUE: \R.\$BRIDGE.XYZTEST.AUTHS

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSACI.ACILIBI.GENFWD2/in \R.\$BRIDGE.XYZTEST.AUTHS/ ext \R.\$BRIDGE.XYZTEST.AUTHE

Build file:

\R.\$BRIDGE.XYZTEST.AUTHLIBO

Reason for Building File:

-FORCE option is on

Target Define:

DEFINE ADDED: ATAUTH_AUTHLIBO VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBO

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILDEFS VALUE: \R.\$SYSACI.ACIUTILS.UTILDEFS
DEFINE ADDED: ACIUTILS_UTILEXTS VALUE: \R.\$SYSACI.ACIUTILS.UTILEXTS

DEFINE ADDED: ATAUTH_AUTHCSEE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHCSEE
DEFINE ADDED: ATAUTH_AUTHCSEG	VALUE: \R.\$EARL.AT60AUTH.AUTHCSEG
DEFINE ADDED: ATAUTH_AUTHCSES	VALUE: \R.\$EARL.AT60AUTH.AUTHCSES
DEFINE ADDED: ATAUTH_AUTHMVD	VALUE: \R.\$EARL.AT60AUTH.AUTHMVD
DEFINE ADDED: ATAUTH_AUTHG	VALUE: \R.\$BRIDGE.XYZTEST.AUTHG
DEFINE ADDED: ATAUTH_AUTHLIBE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE
DEFINE ADDED: ATAUTH_AUTHLIBS	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBS
DEFINE ADDED: ATAUTH_AUTHMCUD	VALUE: \R.\$EARL.AT60AUTH.AUTHMCUD
DEFINE ADDED: ATAUTH_AUTHHTBL	VALUE: \R.\$EARL.AT60AUTH.AUTHHTBL
DEFINE ADDED: ATDDL_DDLFTLF	VALUE: \R.\$EARL.AT60DDL.DDLFTLF
DEFINE ADDED: BACSE_ATTCKID	VALUE: \R.\$EARL.BA60CSE.ATTCKID
DEFINE ADDED: BACSE_BATKCCVE	VALUE: \R.\$EARL.BA60CSE.BAKCCVE
DEFINE ADDED: BACSE_BATKCID	VALUE: \R.\$EARL.BA60CSE.BATKCID
DEFINE ADDED: BACSE_DDLATTKC	VALUE: \R.\$EARL.BA60CSE.DDLATTKC
DEFINE ADDED: BACSE_DDLBATKC	VALUE: \R.\$EARL.BA60CSE.DDLBATKC
DEFINE ADDED: BADDL_DDLATTKN	VALUE: \R.\$EARL.BA60DDL.DDLATTKN
DEFINE ADDED: BADDL_DDLBATKN	VALUE: \R.\$EARL.BA60DDL.DDLBATKN
DEFINE ADDED: BADDL_DDLBCNST	VALUE: \R.\$EARL.BA60DDL.DDLBCNST
DEFINE ADDED: BADDL_DDLFCAF	VALUE: \R.\$EARL.BA60DDL.DDLFCAF
DEFINE ADDED: BADDL_DDLFCONF	VALUE: \R.\$EARL.BA60DDL.DDLFCONF
DEFINE ADDED: BADDL_DDLFCPF	VALUE: \R.\$EARL.BA60DDL.DDLFCPF
DEFINE ADDED: BADDL_DDLFIDF	VALUE: \R.\$EARL.BA60DDL.DDLFIDF
DEFINE ADDED: BADDL_DDLFKEYA	VALUE: \R.\$EARL.BA60DDL.DDLFKEYA
DEFINE ADDED: BADDL_DDLFNEG	VALUE: \R.\$EARL.BA60DDL.DDLFNEG
DEFINE ADDED: BADDL_DDLFPBF	VALUE: \R.\$EARL.BA60DDL.DDLFPBF
DEFINE ADDED: BADDL_DDLFSURF	VALUE: \R.\$EARL.BA60DDL.DDLFSURF
DEFINE ADDED: BADDL_DDLFTKN	VALUE: \R.\$EARL.BA60DDL.DDLFTKN
DEFINE ADDED: BADDL_DDLFUAF	VALUE: \R.\$EARL.BA60DDL.DDLFUAF
DEFINE ADDED: BADDL_DDLGPTH	VALUE: \R.\$EARL.BA60DDL.DDLGPTH
DEFINE ADDED: BADDL_DDLGSTM	VALUE: \R.\$EARL.BA60DDL.DDLGSTM
DEFINE ADDED: BADEFS_BADEFTAL	VALUE: \R.\$EARL.BA60DEFS.BADEFTAL
DEFINE ADDED: BALIB_BALIBTE	VALUE: \R.\$EARL.BA60LIB.BALIBTE
DEFINE ADDED: BASRC_ATTCKNID	VALUE: \R.\$EARL.BA60SRC.ATTCKNID
DEFINE ADDED: BASRC_BATKNCVE	VALUE: \R.\$EARL.BA60SRC.BATKNCVE
DEFINE ADDED: BASRC_BATKNID	VALUE: \R.\$EARL.BA60SRC.BATKNID
DEFINE ADDED: BASRC_BAUTILE	VALUE: \R.\$EARL.BA60SRC.BAUTILE
DEFINE ADDED: BASRC_CVUTILE	VALUE: \R.\$EARL.BA60SRC.CVUTILE
DEFINE ADDED: BASRC_SECUTILE	VALUE: \R.\$EARL.BA60SRC.SECUTILE
DEFINE ADDED: OKDDL_OKDT	VALUE: \R.\$EARL.OK60DDL.OKDT
DEFINE ADDED: OKLIB_OKLIBTE	VALUE: \R.\$EARL.OK60OBJ.OKLIBTE
DEFINE ADDED: OKLIB_OKLIBTG	VALUE: \R.\$EARL.OK60LIB.OKLIBTG
DEFINE ADDED: OKSRC_OKTG	VALUE: \R.\$EARL.OK60SRC.OKTG
DEFINE ADDED: RDEBUG_DUMPEXT	VALUE: \R.\$SYSACI.RDEBUG.DUMPEXT
DEFINE ADDED: SPANNET_GLOBALS	VALUE: \R.\$SYSACI.XPNET.GLOBALS
DEFINE ADDED: SPANNET_INITLIBS	VALUE: \R.\$SYSACI.XPNET.INITLIBS
DEFINE ADDED: SPANNET_SKEL	VALUE: \R.\$SYSACI.XPNET.SKEL
DEFINE ADDED: SPANNET_SKELEXTS	VALUE: \R.\$SYSACI.XPNET.SKELEXTS
DEFINE ADDED: SYSTEM_EXTDECS0	VALUE: \R.\$SYSTEM.SYSTEM.EXTDECS0

Additional Non-Dependent Defines:

DEFINE ADDED: BASRC_BADDLTAL	VALUE: \R.\$EARL.BA60SRC.BADDLTAL
DEFINE ADDED: ATSRC_ATDDLTLAL	VALUE: \R.\$EARL.AT60SRC.ATDDLTLAL

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

Build file:

\R.\$BRIDGE.XYZTEST.AUTHO

Reason for Building File:

-FORCE option is on

Target Define:

DEFINE ADDED: ATAUTH_AUTHO

VALUE: \R.\$BRIDGE.XYZTEST.AUTHO

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILDEFS	VALUE: \R.\$SYSACI.ACIUTILS.UTILDEFS
DEFINE ADDED: ACIUTILS_UTILEXTS	VALUE: \R.\$SYSACI.ACIUTILS.UTILEXTS
DEFINE ADDED: ATAUTH_AUTHCSEE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHCSEE
DEFINE ADDED: ATAUTH_AUTHCSEG	VALUE: \R.\$EARL.AT60AUTH.AUTHCSEG
DEFINE ADDED: ATAUTH_AUTHE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHE
DEFINE ADDED: ATAUTH_AUTHEMVD	VALUE: \R.\$EARL.AT60AUTH.AUTHEMVD
DEFINE ADDED: ATAUTH_AUTHG	VALUE: \R.\$BRIDGE.XYZTEST.AUTHG
DEFINE ADDED: ATAUTH_AUTHLIBE	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBE
DEFINE ADDED: ATAUTH_AUTHMCUD	VALUE: \R.\$EARL.AT60AUTH.AUTHMCUD
DEFINE ADDED: ATAUTH_AUTHS	VALUE: \R.\$BRIDGE.XYZTEST.AUTHS
DEFINE ADDED: ATAUTH_AUTHHTBL	VALUE: \R.\$EARL.AT60AUTH.AUTHHTBL
DEFINE ADDED: ATDDL_DDLFTLF	VALUE: \R.\$EARL.AT60DDL.DDLFTLF
DEFINE ADDED: BACSE_ATTCKID	VALUE: \R.\$EARL.BA60CSE.ATTCKID
DEFINE ADDED: BACSE_BATKID	VALUE: \R.\$EARL.BA60CSE.BATKID
DEFINE ADDED: BACSE_DDLATTKC	VALUE: \R.\$EARL.BA60CSE.DDLATTKC
DEFINE ADDED: BACSE_DDLBATKC	VALUE: \R.\$EARL.BA60CSE.DDLBATKC
DEFINE ADDED: BADDL_DDLATTKN	VALUE: \R.\$EARL.BA60DDL.DDLATTKN
DEFINE ADDED: BADDL_DDLBATKN	VALUE: \R.\$EARL.BA60DDL.DDLBATKN
DEFINE ADDED: BADDL_DDLFCAF	VALUE: \R.\$EARL.BA60DDL.DDLFCAF
DEFINE ADDED: BADDL_DDLFCNF	VALUE: \R.\$EARL.BA60DDL.DDLFCNF
DEFINE ADDED: BADDL_DDLFCPF	VALUE: \R.\$EARL.BA60DDL.DDLFCPF
DEFINE ADDED: BADDL_DDLFIDF	VALUE: \R.\$EARL.BA60DDL.DDLFIDF
DEFINE ADDED: BADDL_DDLFKEYA	VALUE: \R.\$EARL.BA60DDL.DDLFKEYA
DEFINE ADDED: BADDL_DDLFNEG	VALUE: \R.\$EARL.BA60DDL.DDLFNEG
DEFINE ADDED: BADDL_DDLFPBF	VALUE: \R.\$EARL.BA60DDL.DDLFPBF
DEFINE ADDED: BADDL_DDLFUAF	VALUE: \R.\$EARL.BA60DDL.DDLFUAF
DEFINE ADDED: BADDL_DDLGPTH	VALUE: \R.\$EARL.BA60DDL.DDLGPTH
DEFINE ADDED: BADDL_DDLGSTM	VALUE: \R.\$EARL.BA60DDL.DDLGSTM
DEFINE ADDED: BADEFS_BADEFTAL	VALUE: \R.\$EARL.BA60DEFS.BADEFTAL
DEFINE ADDED: BASRC_ATTCKNID	VALUE: \R.\$EARL.BA60SRC.ATTCKNID
DEFINE ADDED: BASRC_BATKNID	VALUE: \R.\$EARL.BA60SRC.BATKNID
DEFINE ADDED: BASRC_BAUTILE	VALUE: \R.\$EARL.BA60SRC.BAUTILE
DEFINE ADDED: BASRC_CVUTILE	VALUE: \R.\$EARL.BA60SRC.CVUTILE
DEFINE ADDED: BASRC_SECUTILE	VALUE: \R.\$EARL.BA60SRC.SECUTILE

DEFINE ADDED: OKDDL_OKDT	VALUE: \R.\$EARL.OK60DDL.OKDT
DEFINE ADDED: OKLIB_OKLIBTG	VALUE: \R.\$EARL.OK60LIB.OKLIBTG
DEFINE ADDED: OKSRC_OKTG	VALUE: \R.\$EARL.OK60SRC.OKTG
DEFINE ADDED: RDEBUG_DUMPEXT	VALUE: \R.\$SYSACI.RDEBUG.DUMPEXT
DEFINE ADDED: SPANNET_GLOBALS	VALUE: \R.\$SYSACI.XPNET.GLOBALS
DEFINE ADDED: SPANNET_INITLIBS	VALUE: \R.\$SYSACI.XPNET.INITLIBS
DEFINE ADDED: SPANNET_SKEL	VALUE: \R.\$SYSACI.XPNET.SKEL
DEFINE ADDED: SPANNET_SKELEXTS	VALUE: \R.\$SYSACI.XPNET.SKELEXTS
DEFINE ADDED: SYSTEM_EXTDECS0	VALUE: \R.\$SYSTEM.SYSTEM.EXTDECS0

Additional Non-Dependent Defines:

DEFINE ADDED: BASRC_BADDLTAL	VALUE: \R.\$EARL.BA60SRC.BADDLTAL
DEFINE ADDED: ATSRC_ATDDLTAL	VALUE: \R.\$EARL.AT60SRC.ATDDLTAL

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

Build file:

\R.\$BRIDGE.XYZTEST.AUTH

Reason for Building File:

-FORCE option is on

Target Define:

DEFINE ADDED: ATAUTH_AUTH	VALUE: \R.\$BRIDGE.XYZTEST.AUTH
---------------------------	---------------------------------

Dependency Defines:

DEFINE ADDED: ACIUTILS_UTILB	VALUE: \R.\$SYSACI.ACIUTILS.UTILB
DEFINE ADDED: ATAUTH_AUTHO	VALUE: \R.\$BRIDGE.XYZTEST.AUTHO
DEFINE ADDED: ATAUTH_AUTHLIBO	VALUE: \R.\$BRIDGE.XYZTEST.AUTHLIBO
DEFINE ADDED: ATLIB_ATLIBLO	VALUE: \R.\$EARL.AT60LIB.ATLIBLO
DEFINE ADDED: BACSE_BATKCCVO	VALUE: \R.\$EARL.BA60CSE.BATKCCVO
DEFINE ADDED: BASRC_BAUTIL	VALUE: \R.\$EARL.BA60SRC.BAUTIL
DEFINE ADDED: BASRC_CVUTIL	VALUE: \R.\$EARL.BA60SRC.CVUTIL
DEFINE ADDED: BASRC_SECUTIL	VALUE: \R.\$EARL.BA60SRC.SECUTIL
DEFINE ADDED: BASRC_BATKNCVO	VALUE: \R.\$EARL.BA60SRC.BATKNCVO
DEFINE ADDED: OKLIB_OKLIBTO	VALUE: \R.\$EARL.OK60OBJ.OKLIBTO

Make File:

\R.\$EARL.AT60AUTH.AUTHM

Execution Statement:

\$SYSTEM.SYSTEM.TACL/INLINE/

File Utility Program - T6553D45 - (09NOV98) System \R
 Copyright Tandem Computers Incorporated 1981, 1983, 1985-1998
 SECURE \R.\$BRIDGE.XYZTEST.AUTH, "NNNU"

*** MAKE complete 5 MAY 2001 21:19:26.68 ***

Files Rebuilt:

```
-----  
  \R.$BRIDGE.XYZTEST.AUTHCSEE  
  \R.$BRIDGE.XYZTEST.AUTHLIBE  
  \R.$BRIDGE.XYZTEST.AUTHE  
  \R.$BRIDGE.XYZTEST.AUTHLIBO  
  \R.$BRIDGE.XYZTEST.AUTHO  
  \R.$BRIDGE.XYZTEST.AUTH
```

Files NOT Rebuilt:

```
-----  
  NONE
```

Summary:

```
-----  
  Files rebuilt : 6  
  Build errors  : 0  
  Build warnings : 0
```

```
Elapsed Time : 00:02:21.91 (h:m:s.hs)  
Time before starting first build: 00:00:11.46 (h:m:s.hs)  
Longest Line Length (bytes): 1726  
Line Size Max value (bytes): 250000  
Number of SQL macro hits: 0  
Number of Memory macro hits: 651  
Bytes in build steps: 19490  
Build Steps Table Size: 120000000
```

Information on RADIX Tables:

```
  2 Tables Allocated 10000000 bytes each.  
  - Table 1 Used 1780514 (17.8%)  
  - Table 2 Used 3858728 (38.6%)
```

The RUNMAKE obey file for running this report appears below. The only change from the previous three examples is the addition of the FORCE option.

```
PARAM MAX-RADIX-TBL-SIZE 10000000
```

```
delete define =allmacs  
add define =allmacs, file $earl.ba60make.custmacs
```

```
run $sysaci.makev500.makev501 /out $s.#xyz,nowait/ &  
\r.$earl.at60auth.authm &  
-Dmymacs=$bridge.xyztest.XYZMACS -force
```


The contents of the XYZTEST subvolume after this run of the utility appear below. The RUNMAKE obey file and all six of the target files have new modification timestamps (appearing in bold).

	CODE	EOF	LAST	MODIF	OWNER	RWEP
\$BRIDGE.XYZTEST						
AUTH	100	1784746		21:19	37,5	NNNU
AUTHCSEE	101	2048		21:17	37,5	NCNC
AUTHE	101	81746		21:17	37,5	NCNC
AUTHG	101	75332	2May2001	08:00	37,5	NCNC
AUTHLIBE	101	81764		21:17	37,5	NCNC
AUTHLIBO	100	689554		21:18	37,5	NNNU
AUTHLIBS	101	792010		21:04	37,5	NCNC
AUTHO	100	758272		21:19	37,5	NNNU
AUTHS	101	904834	2May2001	08:00	37,5	NCNC
RUNMAKE	101	2390		21:16	37,5	NCNC
RUNMKNB	101	2396	3May2001	16:22	37,5	NCNC
XYZMACS	101	2620		20:20	37,5	NCNC

ACI Worldwide Inc.

Appendix B

Implementation of Make in BASE24

BASE24 release 5.1 was the first ACI product release to use the ACI Make utility to compile source code. This appendix identifies the changes made to BASE24 source code in release 5.1 to support the use of the ACI Make utility.

Common Source File Modifications

The following modifications were made to all source files, regardless of code type.

External File References

All external file references (e.g., TAL ?SOURCE statements, COBOL COPY statements) were modified to reference the appropriate TACL define instead of the actual file name. This change eliminated the use of the search subvolume (SSV) feature of the various compilers. SSVs are not officially supported by Tandem and may be eliminated in a future release of the NonStop Kernel.

The following examples illustrate the code modifications made to retrieve DDLs from base DDLs:

Language	BASE24 Release	Code
TAL	5.0	?source baddltal(...)
	5.1	?source =basrc_baddltal(...)
COBOL	5.0	COPY <i>sect-name</i> OF “=BADDLCOB”.
	5.1	COPY <i>sect-name</i> OF “=BASRC_BADDLCOB”.
Screen COBOL	5.0	COPY <i>sect-name</i> OF “BA50SRC.BADDLCOB”.
	5.1	COPY <i>sect-name</i> OF “=BASRC_BADDLCOB”.

Compile and Bind TACL Command Files

The TACL command files used to compile and, optionally, bind a given module were replaced by a makefile. Certain subvolumes contain multiple modules (e.g., all the base requesters and servers are located on the BAnnAFT subvolume). In these cases, an individual makefile for each requester and server was created.

Additionally, a subvolume level makefile was created. This makefile references the individual makefiles for that subvolume to build all of the modules.

Product Level DDL Modifications

The following changes were made to product level DDLs.

Base Global Definitions

The base global definitions file, DDLGDEFS, was moved to its own subvolume, BA51DEFS, and compiled there. The resulting dictionary is saved, along with the TAL and COBOL language output files (BADEFTAL and BADEFCOB).

The ACI Make utility, using statements in the product DDLs makefile, can copy the dictionary and language files needed when compiling product DDLs. This copying eliminates unnecessary compilation of these definitions. For example, in release 5.0, the BASE24-pos product (including the CRT Administration and Authorization modules) recompiled DDLGDEFS three times.

DDL Command File

The DDL commands that were located in the DDLS file for previous releases were moved into the various DDL makefiles, making the DDLS and DDLC files obsolete.

DDL Dictionary Command

In prior releases, the ?DICT command was used in the DDLS file to open or create the DDL dictionary. This command caused the dictionary to be protected by the Transaction Monitoring Facility (TMF). For BASE24, TMF protection of the DDL dictionary is unnecessary and resulted in longer DDL compile times. In release 5.1, the ?DICTN command was used to open the DDL dictionary. This command instructs the DDL compiler to open the dictionary without TMF protection.

Language Output Files

In prior releases, the language output files (BADDLTAL and BADDLCOB) were created on the DDL subvolume (e.g., BA50DDL, AT50DDL, and PS50DDL), then manually copied to the SRC subvolume (e.g., BA50SRC, AT50SRC, and PS50SRC) for use by other modules. The files were left on the DDL subvolumes, even though they were no longer needed.

In release 5.1, the language output files are created as temporary files in the same subvolume as the makefile. At the end of a successful DDL compilation, the language output files are copied to the SRC subvolume (e.g., BA51SRC, AT51SRC, PS51SRC), and the temporary files are then purged.

TAL Output File ITS Support

To aid migration to the ITS architecture, the base, ATM, Base POS, and POS TAL output files must undergo additional processing. The BASE24 ITS source code uses the underbar character (_) as a separator instead of the circumflex character (^) used by BASE24 procedural files. These files are processed by the DDLTAL_UNDERBAR_CONV TACL macro, which changes the circumflex characters (^) to underbars (_). See the “DDL TAL Underbar Conversion Macro” topic in section 3 for details regarding this macro.

Example DDL Makefile

The following is an example makefile used with product level DDLs. Comments added to clarify the content of the makefile are shown in bold. If a comment refers to text in the makefile, that text is also shown in bold.

```
Use additional output mechanism
45  $(atsrc_atddlтал) :{                                \
46      $(atsrc_atddlсob)                                \
47      $(atsrc_atddlфup)                                \
48      $(atsrc_attplt) } $(badeфs_ddlgdefs) \
49      $(atddl_ddlфrcpt) \
50      $(atddl_ddlфrpt) \
51      $(atddl_ddlфтdf) \
52      $(atddl_ddlфтlf) \
53      $(atddl_ddlgpem) \
54      $(atddl_ddlgsem) \
55      $(atddl_ddlgтlfх)
56      $(TACL)/INLINE/
57  = $(tacl_init) $(make_tacl_macros)
58  #
59  #   Create our temporary language files based upon the location
60  #   of our MAKEFILE (so the -TEST option will work)
61  #
62  = #PUSH    temp_tal_fname
63  = #PUSH    temp_cob_fname
64  = #PUSH    temp_fup_fname
65  = #PUSH    temp_tplt_fname
```

```

66
67 = #SET temp_tal_fname [expandssv $(atddl_atddlm)].atmptal
68 = #SET temp_cob_fname [expandssv $(atddl_atddlm)].atmpcob
69 = #SET temp_fup_fname [expandssv $(atddl_atddlm)].atmpfup
70 = #SET temp_tplt_fname [expandssv $(atddl_atddlm)].atmptplt
71
72 = ADD DEFINE =atddl_atddlta, FILE [temp_tal_fname]
73 = ADD DEFINE =atddl_atddlcob, FILE [temp_cob_fname]
74 = ADD DEFINE =atddl_atddlufup, FILE [temp_fup_fname]
75 = ADD DEFINE =atddl_atdplt, FILE [temp_tplt_fname]
76
77 #
78 #      Copy the Global Definitions DDL Dictionary and language
79 #      output files
80 #
81 = #PUSH :var
82 = #APPEND :var DUP $(badevs_badeftal), [temp_tal_fname], PURGE
83 = #APPEND :var DUP $(badevs_badevcob), [temp_cob_fname], PURGE
84 = SINK [$(FUP)/inv :var/]
85
86 = dup_dict [expandssv $(badevs_defsdict)] [expandssv $(atddl_atddlm)]
87
Open dictionary as non-audited
88 = #SET :var ?DICTN [expandssv $(atddl_atddlm) ]
89 = #APPEND :var ?nocomments
90 = #APPEND :var ?noclistout
91 = #APPEND :var ?talbound 0
92 = #APPEND :var ?deflist
93 = #APPEND :var ?COBOL [temp_cob_fname]
94 = #APPEND :var ?TAL [temp_tal_fname]
95 = #APPEND :var ?FUP [temp_fup_fname] !
96
97 = #APPEND :var ?NOFUP
98
99 = #APPEND :var ?nocomments
100 = #APPEND :var ?source $(atddl_ddlfrpt)

```

Other DDL commands removed for brevity

```

128 = #APPEND :var output record RPT5TTL.
129
Start DDL Compiler using the TACL variable :var as input. Variable contains the
above DDL commands
130 = $(DDL)/inv :var, out $s.#atddl $(atddl_compile_opts)/      &
131 =      $(atddl_ddl_opts)
132 = check_completion $(TARGET)
133 = del_peruse_job $(PERUSE) $(atddl_purge_spooler_on) #atddl
134
Add underbar version of sections
135 = ddlta_underbar_conv [temp_tal_fname]
136
137 = #SET :var CAB/TLYYMD0/T0YYMMDD/A

```

```
138 = #APPEND :var E
139 = $(EDIT) /inv :var/ [temp_tplt_fname]
140
Duplicate new DDL language output files to the SRC subvolume
141 = #SET :var DUP [temp_tal_fname], $(atsrc_atddlta), PURGE, SOURCEDATE
142 = #APPEND :var DUP [temp_cob_fname], $(atsrc_atddlcob), PURGE, SOURCEDATE
143 = #APPEND :var DUP [temp_fup_fname], $(atsrc_atddlfup), PURGE, SOURCEDATE
144 = #APPEND :var DUP [temp_tplt_fname], $(atsrc_attplt), PURGE, SOURCEDATE
145 = SINK [$(FUP)/inv :var/]
146 = check_completion $(TARGET)
147
Secure the SRC language files
148 = secure $(FUP) $(atsrc_atddlta) $(atddl_secure_opt)
149 = secure $(FUP) $(atsrc_atddlcob) $(atddl_secure_opt)
150 = secure $(FUP) $(atsrc_atddlfup) $(atddl_secure_opt)
151 = secure $(FUP) $(atsrc_attplt) $(atddl_secure_opt)
152
Clean up the subvolume and the TACL environment
153 = SINK [PURGE [temp_tal_fname]]
154 = SINK [PURGE [temp_cob_fname]]
155 = SINK [PURGE [temp_fup_fname]]
156 = SINK [PURGE [temp_tplt_fname]]
157
158 = #POP temp_tal_fname
159 = #POP temp_cob_fname
160 = #POP temp_fup_fname
161 = #POP temp_tplt_fname
162
163 = DELETE DEFINE =atddl_atddlta
164 = DELETE DEFINE =atddl_atddlcob
165 = DELETE DEFINE =atddl_atddlfup
166 = DELETE DEFINE =atddl_attplt
167
168 = #POP :var
```


Pathway Requester Modifications

The following changes were made to Pathway requester processes.

Pathway Screen Source

In release 5.0, all the Pathway screen source was contained in one product level file (e.g., BA50AFT.BASCRNS). In release 5.1, new requester-specific screen source files were created. This allows a dependency to be placed on the screen source, so the ACI Make utility can determine if the given requester process must be compiled.

POBJ Management

In release 5.0, the requester source files on a given subvolume were compiled into a Screen COBOL Object Library (POBJ) file located on that subvolume (e.g., AT50AFT.POBJ). The resulting POBJ code was then manually copied into a POBJ file used for testing (e.g., AT50OBJ.POBJ). In release 5.1, the makefile includes this copy step.

Example Requester Makefile

The following is an example makefile used with a requester process. Comments added to clarify the content of the makefile are shown in bold. If a comment refers to text in the makefile, that text is also shown in bold.

```
Use additional input and output mechanisms
42  $(ataft_rqtlfo) :{                                \
43      $(dest_pobj) }[                               \
44      $(atsrc_atddlco)                               \
45      $(basrc_baddlco) ] $(atsrc_atcoutls)          \
46                          $(atddl_ddlftdf)            \
47                          $(baddl_ddlfsec)             \
48                          $(baddl_ddlgpth)             \
49                          $(basrc_bacoutls)            \
50                          $(basrc_cobnames)            \
51                          $(ataft_rqtlfs)              \
Requester screens are in their own file
52      $(ataft_scrntlf)
53
```

```
54    $(TACL)/INLINE/
55 = $(tacl_init) $(make_tacl_macros)
56 = #PUSH temp_pobj scn_tlfo
57
Dynamically create temporary pobj and screen output file names
58 = #SET temp_pobj [expandssv $(ataft_rqtlfm)].ptlf
59 = #SET scn_tlfo [expandssv $(ataft_rqtlfm)].scntlfo
60
61 = ADD DEFINE =ataft_scntlfo, FILE [scn_tlfo]
62
Compile screens
63 = $(ACIAID) /in $(ataft_scrntlf), out $s.#scntlf / &
64 = [scn_tlfo]!
65 = check_completion $(TARGET)
66 = del_peruse_job $(PERUSE) $(ataft_purge_spooler_on) #scntlf
67
Purge left-over temporary pobj files
68 = SINK [del [temp_pobj]*]
69
Compile in IBM 3270 mode (if applicable)
70 !IF $(ataft_t3270_on) = TRUE |THEN|
71 = aci_scobolx_make $(SCOBOLX) $(TARGET) /in $(ataft_rqtlfs), &
72 = out $s.#rqtlfi/ [temp_pobj]; settog 1; &
73 = $(ataft_scobolx_opts)
74 = check_scobol_completion $(TARGET)
75 = del_peruse_job $(PERUSE) $(ataft_purge_spooler_on) #rqtlfi
76 !ENDIF
77
Compile in TANDEM 6520 mode (if applicable)
78 !IF $(ataft_t6520_on) = TRUE |THEN|
79 = aci_scobolx_make $(SCOBOLX) $(TARGET) /in $(ataft_rqtlfs), &
80 = out $s.#rqtlft/ [temp_pobj]; settog 3; &
81 = $(ataft_scobolx_opts)
82 = check_scobol_completion $(TARGET)
83 = del_peruse_job $(PERUSE) $(ataft_purge_spooler_on) #rqtlft
84 !ENDIF
85
Use SCUP to copy new requesters to destination POBJ
86 = #PUSH :var
87 = #SET :var COPY [temp_pobj] (*),$(dest_pobj)
88 = $(SCUP)/inv :var/
89 = check_completion $(TARGET)
90 = #POP :var
91
Clean up subvolume and TACL environment
92 = SINK [del [temp_pobj]*]
93 = SINK [PURGE [scn_tlfo]]
94
95 = #POP temp_pobj scn_tlfo
96 = DELETE DEFINE =ataft_scntlfo
```

COBOL Modifications

There were no language-specific modifications necessary for COBOL modules.

Example COBOL Makefile

The following is an example makefile used with a COBOL module. Comments added to clarify the content of the makefile are shown in bold. If a comment refers to text in the makefile, that text is also shown in bold.

```

Use additional input mechanisms
39     $(ataft_svtlf) : [                                \
40     $(atsrc_atddlcb)                                \
41     $(basrc_baddlcb)] $(system_cobollib)            \
42     $(aciutils_utilb)                                \
43     $(baddl_ddlattkn)                                \
44     $(baddl_ddlfomf)                                 \
45     $(baddl_ddlglnf)                                 \
46     $(baddl_ddlgpth)                                 \
47     $(basrc_bacoutls)                                \
48     $(basrc_bautil)                                  \
49     $(basrc_cobnames)                                \
50     $(atddl_ddlftlf)                                 \
51     $(atsrc_atcoutls)                                \
52     $(ataft_svtlfs)
53
54
55     $(TACL)/INLINE/
56 = $(tacl_init) $(make_tacl_macros)
57
58 = $(COBOL)/in $(ataft_svtlfs),                        &
59 =         out $S.#svtlf $(ataft_compile_opts) /      &
60 =         $(ataft_svtlf);$(ataft_cobol_opts)
61
62 = check_completion $(TARGET)
63 = del_peruse_job $(PERUSE) $(ataft_purge_spooler_on) #svtlf
64
65 = dup_exec_file $(move_obj_subvol_on) $(ataft_secure_opt) &
66 =         $(ataft_svtlf) $(atm_obj_loc)

```

TAL Modifications

The following TAL modifications were made.

General Modifications

General modifications were made to the global data files and the ?SEARCH statements were removed.

Global Data

If global data for a module (e.g., ATM Device Handlers, ATM Authorization) was contained inline in release 5.0, the global declarations were moved into their own file (with a suffix of G).

This change was done for customized code and bindable add-on products because the base module global definitions were required. With the global data inline, the dependency is set on the whole base module source file. If a change to a procedure in the base module is made, the customized code or add-on product is compiled, when it only needs to be bound with the new base object file.

?SEARCH Statements

?SEARCH statements were removed and a BIND step was added to the makefile.

BASE24-atm Device Handler Processes

Because of the way the release 5.0 code was structured, additional modifications were made to the BASE24-atm Device Handler processes.

DH NAMES File

In release 5.0, the BASE24-atm Device Handler processes obtained certain text displayed or printed from tables contained in AT50SRC.DH NAMES. Originally, it was envisioned that these tables could be shared between Device Handler processes; but in fact, due to device dependencies, no text tables have been shared.

In release 5.1, the text tables for each Device Handler process were moved from the DHNAMES File and placed in new files on each of the Device Handler subvolumes. The naming convention established for these new files is `xxxxNAMS`, where `xxxx` is derived from the Device Handler name (e.g., `N50NAMS`, `C910NAMS`, `C100NAMS`).

The only table remaining from the release 5.0 DHNAMES File was the configurable receipt literal table. This table was placed in a new file called `AT51SRC.RCPTDEFS`.

DDLDDDL File

In release 5.0, the device dependent data DDL definition for each Device Handler process was combined in one file named DDLDDDL. That file was then included in the BASE24-atm product DDL output and used by the Device Control Terminal (DCT) Server process. The DCT Server process used these DDLs to determine if a given ATM device had a fault. (The DDLs were used by only two procedures.)

In release 5.1, there is no longer a DDLDDDL File. The device dependent data DDL definition for each Device Handler process was placed in new DDL source files located on each of the Device Handler subvolumes. A step to compile the new DDL was included in each Device Handler makefile.

The naming convention for these DDL source files is `xxxxDDLs`, where `xxxx` is derived from the Device Handler process name (e.g., `N50DDLs`, `C910DDLs`, `C100DDLs`). The language output files are called `xxxxTAL`, `xxxxCOB`, etc.

Example Makefile

The following is an example makefile used with a BASE24-atm 910/911 Device Handler process.

```
#-----
#   Diebold 910 DDLs
#-----

$(atc910_c910tal) : $(atc910_c910ddl)

    $(TACL)/INLINE/
= $(tacl_init) $(make_tacl_macros)

= #PUSH      :var
```

```
#
#      Copy the Global Definitions DDL Dictionary and language
#      output files
#
= #APPEND :var DUP $(badeufs_badeftal), $(atc910_c910tal), PURGE
= SINK [$(FUP)/inv :var/]

= dup_dict [expandssv $(badeufs_defsdict)] &
=      [expandssv $(atc910_c910tal)]

= #SET      :var ?DICTN [expandssv $(atc910_c910tal) ]
= #APPEND :var ?nocomments
= #APPEND :var ?noclistout
= #APPEND :var ?talbound 0
= #APPEND :var ?deflist
= #APPEND :var ?nofup
= #APPEND :var ?nocobol
= #APPEND :var ?tal $(atc910_c910tal)
= #APPEND :var ?list, source $(atc910_c910ddls) (d910data)
= $(DDL) /inv :var, out $.#c910ddl $(c910_compile_opts) / &
=      $(c910_ddl_opts)
= check_completion $(TARGET)
= del_peruse_job $(PERUSE) $(c910_purge_spooler_on) #c910ddl
= #POP      :var

#-----
#      Diebold 910 Externals
#-----
$(atc910_c910csee)      :      $(atc910_c910cses)
      $(GENFWD)/in $(atc910_c910cses)      / ext $(atc910_c910csee)

$(atc910_c910e)      :      $(atc910_c910s)
      $(GENFWD)/in $(atc910_c910s)      / ext $(atc910_c910e)

#-----
#      Diebold 910 Compile
#-----
$(atc910_c910o)      :[      \
      $(basrc_baddltal)      \
      $(atc910_c910tal)      \
      $(atsrc_atddltal) ] \
      $(system_extdecs)      \
      $(spannet_globals)      \
      $(spannet_initlibs)      \
      $(spannet_skel)      \
      $(spannet_skelexts)      \
      $(rdebug_dumpext)      \
      $(aciutils_utilexts)      \
      $(aciutils_utildefs)      \
      $(baddl_ddlattkn)      \
      $(baddl_ddlbatkn)      \
      $(baddl_ddlfcnf)      \
```

```

$(baddl_ddlfidf) \
$(baddl_ddlgpth) \
$(baddl_ddlgdna) \
$(baddl_ddlgstm) \
$(badefts_ddlgdefs) \
$(basrc_attnid) \
$(basrc_batkid) \
$(basrc_bautile) \
$(basrc_secutile) \
$(atddl_ddlfrcpt) \
$(atddl_ddlftdf) \
$(atddl_ddlftlf) \
$(atsrc_atutile) \
$(atsrc_rcptdefs) \
$(atsrc_snadiebs) \
$(atc910_c910csee) \
$(atc910_c910cses) \
$(atc910_c910ddls) \
$(atc910_c910e) \
$(atc910_c910g) \
$(atc910_c910nams) \
$(atc910_c910s)

$(TACL)/INLINE/
= $(tacl_init) $(make_tacl_macros)

= $(TAL)/in $(atc910_c910s), out $s.#c910s $(c910_compile_opts) / &
= $(atc910_c910o);$(c910_tal_opts)
= check_completion $(TARGET)
= del_peruse_job $(PERUSE) $(c910_purge_spooler_on) #c910s

= secure $(FUP) $(atc910_c910o) $(c910_secure_opt)

#-----
# Build Diebold 910 DH executable
#-----
$(atc910_c910) : \
$(aciutils_utilb) \
$(basrc_bautil) \
$(basrc_secutil) \
$(atsrc_atutil) \
$(atc910_c910o)

$(TACL)/INLINE/
= $(tacl_init) $(make_tacl_macros)

= #PUSH :var
= #APPEND :var add * from $(atc910_c910o), delete
= #APPEND :var select search $(atsrc_atutil)
= #APPEND :var select search $(basrc_secutil)
= #APPEND :var select search $(basrc_bautil)
= #APPEND :var select search $(aciutils_utilb)
= #APPEND :var set data 64

```

```
= #APPEND :var select list * off
= #APPEND :var $(c910_bind_steps)
= #APPEND :var build $(atc910_c910) !

= $(BIND)/ inv :var, out $s.#c910 $(c910_compile_opts)/

= check_completion $(TARGET)
= #POP      :var
= del_peruse_job $(PERUSE) $(c910_purge_spooler_on) #c910

#-----
#   Accelerate Diebold 910 DH executable
#-----

= [#IF $(c910_axcel_on) = $(TRUE) |THEN|
=   #PUSH      :var
=   #EOF       :var
=   $(AXCEL)/inv :var, OUT $S.#c910x                                &
=   $(c910_axcel_opts)/                                           &
=   $(atc910_c910),$(atc910_c910);                                &
=   USERLIB $(spannet_hskelb)
=   #POP      :var
=   check_completion $(TARGET)
=   del_peruse_job $(PERUSE) $(c910_purge_spooler_on) #c910x
= ] == end if AXCEL On

= dup_exec_file $(move_obj_subvol_on) $(c910_secure_opt) &
=   $(atc910_c910) $(atm_obj_loc)
= check_completion $(TARGET)
```


ATM DCT Server

As a result of splitting the DDLDDDL File, the Device Control Terminal (DCT) Server process was reworked to contain dummy procedures for each device type. The device-specific DCT code was relocated to the subvolume for each Device Handler process. The DCT Server Makefile (ATDCTM) contains a step to bind the device-specific procedures into the DCT Server executable. By using ACI Make conditional support (!IF), only those Device Handler modules purchased by the customer are bound into the DCT Server process.

The name established for the device-specific DCT source files (on each of the Device Handler subvolumes) is SVDxxxxS, where xxxx is derived from the Device Handler process name (e.g., SVDN50S, SVDC910S, SVD1000S).

ACI Worldwide Inc.

Appendix C

Make for Personalized BASE24 Software

This appendix describes the relationship between the ACI Make utility and customized BASE24 software. See the informal ***ACI Make CSM Standards Manual*** for additional information about using the ACI Make utility with customized BASE24 software.

Code Location and Naming Conventions

The standards for the location of customized modules are the same for customized BASE24 procedural modules and customized ITS modules. The standards for ITS customized modules have been changed to follow the standards for BASE24 procedural customized modules.

In most cases, the last four characters of the customized software subvolume names are taken from the first four characters of the standard subvolume name (e.g., OC60, BA60, AT60, FH60). The exception to this rule is in the case of customized interchange modules, where the last four characters are taken from the last four characters of the standard subvolume (e.g., STRI, FDRI).

For standard subvolumes, the naming convention is *xxnnyyyy*, as follows:

xx = Product ID (e.g., AT, BA, OC, OE, OK, PS, FH, TR)
nn = Release level
yyyy = Code category (e.g., AUTH, DDL, ISMT, LEX, OBJ, SRC)

The following table illustrates the subvolume containing standard software and the corresponding subvolume containing customized software for *customer*, where *customer* is the acronym (maximum of four characters) that uniquely identifies the customer for which the customizations are made.

Standard Software Subvolume	Personalized Software Subvolume
OC60VRU	<i>customer</i> OC60
BA60DDL	<i>customer</i> BA60
BA60HISO	<i>customer</i> BA60
PS60RTAU	<i>customer</i> PS60

The naming convention for customized files is to add a Q prior to the last one or two characters in the module name (depending on whether a one or two character suffix identifies the type of file).

For example, when customizations are made to files on the OC60HIST subvolume and the PS60RTAU subvolume, the customized files are named as follows:

Standard File Names	Customized File Names
OC60HIST.HISTDS	<i>customerOC60.HISTQDS</i>
OC60HIST.HISTPS	<i>customerOC60.HISTQPS</i>
OC60HIST.HISTTG	<i>customerOC60.HISTQTG</i>
OC60HIST.HISTTS	<i>customerOC60.HISTQTS</i>
PS60RTAU.AUTHS	<i>customerPS60.AUTHQS</i>
PS60RTAU.ROUTERS	<i>customerPS60.ROUTERQS</i>
PS60RTAU.RTAUG	<i>customerPS60.RTAUQG</i>

For standard file names that are eight characters long, the naming convention for customized files is to replace the sixth or seventh character with a Q (depending on whether a one or two character suffix identifies the type of file). The following examples fall in that category:

Standard File Names	Customized File Names
AT60AFT.RQTDIEBS	<i>customerAT60.RQTDIEQS</i>
OC60AFM.RQCUSTXS	<i>customerOC60.RQCUSQXS</i>
OC60VRU.SVVRUTTS	<i>customerOC60.SVVRUQTS</i>
PS60RTAU.AUTHLIBS	<i>customerPS60.AUTHLBQS*</i>

* This is a slight variation of the standard where the I in AUTHLIBS has been dropped.

In addition to the customer product and release subvolumes, a customized customer object subvolume and a customer Make subvolume is created (*customerOBJ* and *customerMAKE*, respectively). In the customized makefiles, customized object files for all products are directed to the *customerOBJ* subvolume. Customized makefiles are placed on the customer product and release subvolumes or the *customerMAKE* subvolume, depending on the hierarchy level of the makefile being modified. The only makefiles on the product and release

subvolumes are those makefiles normally found on the standard product and release subvolumes. All other makefiles are placed on the *customerMAKE* subvolume.

For example, customized makefiles for *customer* would be placed on the following product and release subvolumes:

Standard Software Subvolume	Customized Software Subvolume
OC60ACCT.ACCTM	<i>customer</i> OC60.ACCTQM
BA60AFT.RQPBFM	<i>customer</i> BA60.RQPBFQM
BA60AFT.BAAFTM	<i>customer</i> BA60.BAAFTQM
PS60RTAU.AUTHM	<i>customer</i> PS60.AUTHQM

Customized obey files, makefiles, and macro files for *customer* are placed on the *customerMAKE* subvolume, as follows:

Standard Software Make Files	Customized Software Make Files
BA60SRC.B24C	<i>customerMAKE</i> .CSMC
BA60SRC.B24M	<i>customerMAKE</i> .CSMM
No equivalent file	<i>customerMAKE</i> .LOADMACS*
BA60MAKE.CUSTMACS	<i>customerMAKE</i> .CSMMACS

* The LOADMACS File contains the TACL DELETE DEFINE and ADD DEFINE commands to set the ALLMACS macro to point to the CSMMACS.

Makefile Modifications for Customized Code

All standard makefiles use the ALLMACS File, which incorporates a Make macro for each file name in the BASE24 system. Therefore, when files are customized and the file names change, changes are required with the Make macros for the names of the customized files.

File Name Macros

Previously, the standard approach for handling customized files within the Make methodology was to create a new file name macro for every new or modified file. While this approach guaranteed that customized files are not unwittingly used in the creation of standard targets, it also created some unnecessary overhead by customizing a source file solely because it copied in a customized file.

A compromise approach is to create a new file name macro for a file only if the file is **not** logically equivalent to an existing standard file. For example, it is common when modifying the BASE24-atm Authorization process to use a skeleton file that usually sources in the majority of the code from the standard Authorization process. Since this skeleton file is not the logical equivalent of the original Authorization source file, it requires a new customized Make file name macro. The opposite example is a modification to Authorization process global data. In this case, the entire source present in the original Authorization global data file is also present in the customized version. Therefore, a customized Authorization global data file is logically equivalent to the standard global data file, and a new file name macro is unnecessary.

CSMM Makefile

While using existing file macro names whenever possible has a number of advantages, it has the undesirable side effect of propagating customized code into the compilation of a standard system unless another procedural change occurs in the way customized modules and standard modules are compiled. Previously, the compilation of customized modules with the ACI Make utility was designed to be included in the compilation of the overall system. That is, the path for compiling all of the customized modules was part of the existing make hierarchy because the CSMM Makefile was included in the B24M Makefile. Using this method, a standard file name macro that pointed at a customized file results in that file being used in the compilation of any standard modules as well as customized modules. The CSMM Makefile is still included in the current B24M Makefile. However, to

completely isolate the compilation of the customized components, the flag for the conditional INCLUDE statement for the CSMM Makefile is always set to false in the B24M Makefile.

CSMMACS File

Breaking out the CSMM Makefile from the standard run of the ACI Make utility is only half of the solution. In addition, another global macro file must be placed ahead of the existing CUSTMACS File and ALLMACS File. All changes related to customized software are placed in the CSMMACS File, and the CUSTMACS File and ALLMACS File remains unchanged.

Previously, the CUSTMACS File provided all of the customization for a customer environment, then included the ALLMACS File. This customization actually fell into one of two categories: changes that impacted the Make run specific to the customized components, and changes that impacted the Make run specific to the standard components. Since the CSMM Makefile is now executed by itself using the CSMC File, those changes that previously were included in the CUSTMACS File are now placed in the CSMMACS File. This CSMMACS File then includes the CUSTMACS File, which includes the ALLMACS File.

The CSMMACS File adheres to the following guidelines.

- The CSMMACS File is added as a global file ahead of the standard CUSTMACS File, and contains an INCLUDE command for the standard CUSTMACS File (which in turn contains an INCLUDE command for the standard ALLMACS File).
- The CSMMACS File contains an CSM_ON flag set to false to ensure that the CSMM Makefile is not executed by the B24M Makefile. The standard CUSTMACS File also has the CSM_ON flag set to false.
- The CSMMACS File contains only those macros (file name defines or configuration flags) that are required for the customized code. This can be accomplished by either beginning with a complete CUSTMACS File as a base and paring it back or by beginning with an empty file and adding the required macros as you develop the customized code.
 - If a customized file is not a complete replacement for an existing standard file, a new file name macro is created to point at the new physical file, and this declaration is added to the CSMMACS File.

- If a customized file is a complete replacement for an existing standard file, a new file name macro is **not** created. The standard macro name that corresponds with the existing standard physical file for which the custom software file is a replacement is added to the CSMMACS File and altered to point at the customized physical file.

ALLMACS Define

For customers with standard software, the ALLMACS define (which identifies where the top level global macro file is located) is loaded with the following commands:

```
delete define =allmacs
add define =allmacs, file $volume.BA60MAKE.CUSTMACS
```

See the “Setting the ALLMACS Define” topic in section 4 for the options available for setting this define.

For customers with customized software, the ALLMACS define must be modified to point to the CSMMACS File that is located on the *customerMAKE* subvolume. The first time ACI provides customized software involving makefiles to a customer, the software delivery includes a LOADMACS File on the *customerMAKE* subvolume. The LOADMACS File contains the following two lines:

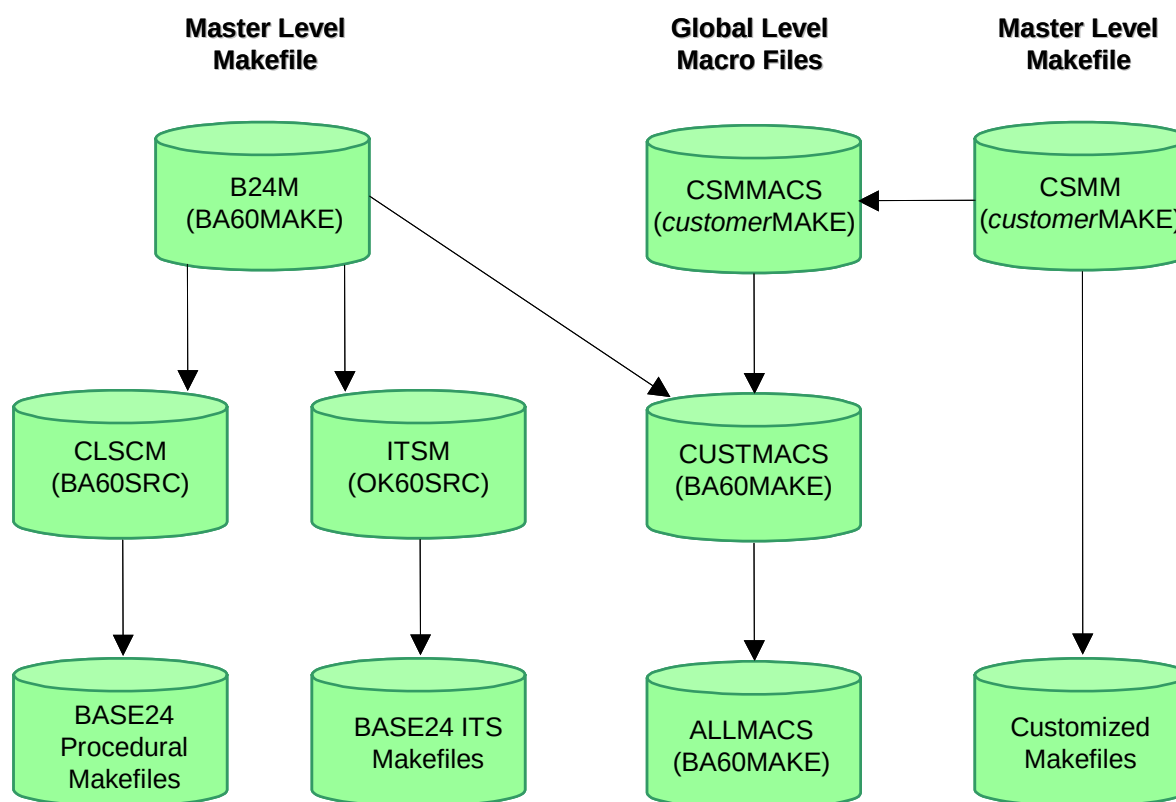
```
delete define =allmacs
add define =allmacs, file $volume.customerMAKE.CSMMACS
```

Users who need to run any makefiles can obey the LOADMACS File to get the correct ALLMACS define or they can place the ALLMACS define line for the CSMMACS File in their TACLCSTM, replacing the original define. The CSMC File also executes the LOADMACS File.

Makefile Hierarchy

The following chart illustrates the Make hierarchy when a configuration includes customized software modules. The first level includes the global level macro files CSMMACS, CUSTMACS and ALLMACS, and the master level makefiles B24M and CSMM. The macro files and makefiles at this level are used by the BASE24 procedural products and the BASE24 ITS products. See the “Hierarchy Charts” topic in section 2 for charts illustrating the standard BASE24 Make hierarchy.

The subvolume on which each file is located is shown in parentheses below the file name. The CUSTMACS File, ALLMACS File, and B24M Makefile are located on the BA60MAKE subvolume, and the CSMMACS File and CSMM File are located on the *customerMAKE* subvolume.



The B24M Makefile is the master level makefile for the standard system files and the CSMM Makefile is the master level makefile for customized software modules. The CLSCM Makefile and ITSM Makefile apply to standard system

files. The B24M Makefile still contains an INCLUDE command for the CSMM Makefile; however, the CSMM Makefile is no longer executed from the B24M Makefile.

To build an entire system (standard and customized), a customer must first run the ACI Make utility with the B24M Makefile for the standard portion of the system, then run the ACI Make utility with the CSMM Makefile for the customized portion of the system. The B24M Makefile reads the macro files, starting with the CUSTMACS File, then reads the makefiles, starting with the CSLCM and ITSM. The CSMM Makefile reads the macro files, starting with the CSMMACS File, then reads the customized makefiles.

ACI Worldwide Inc.

Appendix D

Upgrading BASE24 Sync Point 2 Products

This appendix contains the procedures needed to upgrade BASE24 sync point 2 products from release 1.02 of the Make utilities. Only minor changes are required to use release 5.01 of the ACI Make utility with BASE24 sync point 2 products, each of which is presented in this appendix.

Tandem Operating System Requirements

Release 5.01 of the ACI Make utility is available in two versions.

- The MAKEV501 process is a native object (file type 700) for Tandem NonStop Reduced Instruction Set Computing (RISC) processors, also called TNS/R processors. It requires a NonStop Kernel operating system of D45 or greater.
- The MAKET501 process is a regular object (file type 100) for Tandem NonStop processors, also called TNS processors, that do not support native object files, as follows:
 - Processors without Reduced Instruction Set Computing must have a NonStop Kernel operating system of at least D38.
 - Processors with Reduced Instruction Set Computing and a NonStop Kernel operating system of at least D38 but less than D45.

The MAKET501 process can be used on TNS/R processors with a NonStop Kernel operating system of D45 or greater. However, the MAKEV501 process operates considerably faster than the MAKET501 process on these operating systems.

Modifying the ALLMACS File

The TACLINIT File, which contains the TACLINIT macro, and the TACLMACS File, which contains the TACL macros used with the ACI Make utility, have been updated for release 5.01 of the utility. The current release of both of these files must be used to obtain the most reliable results from the ACI Make utility.

Two ALLMACS entries that contain the locations of these files must be modified to point to the new files. Use a text editor to edit the ALLMACS on the BA51MAKE subvolume and modify the MAKE_TACL_MACROS and TACL_INIT macros to point to the new files on *\$volume*.MAKEV500, where *\$volume* is the volume to which the MAKEV500 subvolume was restored.

```
tacl_init           = $volume.makev500.taclinit
make_tacl_macros    = $volume.makev500.taclmacs
```

TACLINIT File

A new version of the TACLINIT File is used with release 5.01 of the ACI Make utility, regardless of which BASE24 product sync point is being used. This new version is required because it creates a more stable TACL environment for the TACL Server process used by the ACI Make utility.

When this new version is executed, it displays the following line for version verification:

```
TACLINIT - Version 5.01
```

This version of the TACLINIT File is located on the subvolume named *\$volume*.MAKEV500, where *\$volume* is the volume to which the MAKEV500 subvolume was restored.

Note: If necessary, you can use version 4.00 of the TACLINIT File, which was distributed with release 4.0 of the Make utilities, with release 5.01 of the ACI Make utility. Version 4.0 is the minimum TACLINIT File required.

TACLMACS File

A new version of the TACLMACS File is used with release 5.01 of the ACI Make utility, regardless of which BASE24 product sync point is being used. Since release 5.01 of the ACI Make utility uses one TACL Server process, it is important that all of the TACL macros in the TACLMACS File work properly. This version of the TACLMACS File is located on the subvolume named *\$volume*. MAKEV500, where *\$volume* is the volume to which the MAKEV500 subvolume was restored.

One macro in the TACLMACS File lets you determine if you are using the current version. To check the version number, enter the following command at a TACL prompt:

```
LOAD / KEEP 0 / $volume.MAKEV500.TACLMACS
```

A list of the TACL macros contained in the TACLMACS File is displayed on the screen, as follows:

```
LOADED FROM $volume.MAKEV500.TACLMACS:
TACLMACS_VERSION_501      ACI_SCOBOLX
ACI_SCOBOLX_MAKE          CHECK_COMPLETION
CHECK_SCOBOL_COMPLETION   CHK_PERUSE_JOB
CONVERT_SCOBOL_FILE       CPATHEQ_BUILD
CPATHEQ_CREATE            CPP_COMPILE
DDLTLAL_UNDERBAR_CONV     DEL
DEL_PERUSE_JOB            DISPLAY_COMPLETION_INFO
DUP_DICT                  DUP_EXEC_FILE
EXPANDDEF                 EXPANDSSV
EXPANDSSV_DICT_NOSAVE     GET_VOL_SUBVOL
PRCSS_SCOBOL_CMD_LINE     PRCSS_SCOBOL_CMD_LINE_MAKE
PRODUCE_TIME_STR          PURGE_DICT
SCUP_ROUTINE              SECURE
TEMPLMAC                  TOTAL_SIZE
UPDATE_DICT_ALT_FILES
```

The first TACL macro listed is TACLMACS_VERSION_###, where ### is the current release of the macros in the TACLMACS File.

Note: If necessary, you can use version 4.00 of the TACLMACS File, which was distributed with release 4.0 of the Make utilities, with release 5.01 of the ACI Make utility. However, version 5.01 or greater of the TACLMACS File is required for ACI Make utility output to contain completion information that would otherwise be lost.

Modifying the B24C File

The B24C File on your present BA51SRC subvolume can be used to start the ACI Make utility. If you use this file to start the ACI Make utility, you need to modify it so that it uses the new release of the ACI Make utility. If you have other edit files that you use (e.g., the B24C File) to start the ACI Make utility, be sure to modify them as well to use the new release of the ACI Make utility.

The RUN MAKE command in your current B24C File probably looks like the example below. Use a text editor to edit the B24C File and change the boldface text shown in the following example to the location of the release 5.01 ACI Make utility (\$volume.MAKEV500.MAKEV501).

```

==SYNC.03 08/26/97 B24C 5100 tacl BA51SRC BA03000 @ ***
Comment *****
Comment * This file compiles all the products and modules that *
Comment * comprise your BASE24 system. *
Comment * *
Comment * Usage: TACL/ IN B24C/ *
Comment *****

Comment Set TACL Defines for MAKE

param max-radix-tbl-size 20000000

delete define =allmacs
add define =allmacs, file $<vol>.ba51make.custmacs

Comment *****
Comment * The syntax to MAKE the required modules is: *
Comment * *
Comment * MAKEVxxx / out $s.#b24m,nowait/ b24m -<opt1> -<opt2> ... *
Comment * *
Comment * where: xxx is the version of MAKE *
Comment * <opt1> is any option supported by MAKE *
Comment * <opt2> is any option supported by MAKE *
Comment * (see the MAKE manual for supported options) *
Comment * *
Comment * THE FORCE OPTION IS REQUIRED FOR FIRST TIME INSTALLS. *
Comment *****

run $sysaci.acicmake.makev102 /out $s.#b24m,name $b24,nowait/ b24m -force &
-listreplace
delete define =allmacs

```

Modifying the BASE24-mail Makefile

The SVMBAPTM located on the MA51POS subvolume requires a change in the way that it initializes TACL. The new version of the TACLINIT takes care of this problem, but the initialization can be done in a much better way by replacing two existing lines in the SVMBAPTM with one new line.

If you have the BASE24-mail product, perform one of the following steps.

- Contact HELP24 and secure a copy of SCN MA03002. Apply the fix, which is a replacement for the SVMBAPTM on the MA51POS subvolume.
- Use a text editor and make the required changes to the SVMBAPTM. The text in the file before the change is made appears below.

```
$(TACL)/INLINE/  
= load /keep 1/ $(make_tacl_macros)  
= #set #informat tacl
```

Delete the second and third lines shown above and replace them with the second line shown below. The first line does not change.

```
$(TACL)/INLINE/  
= $(tacl_init) $(make_tacl_macros)
```

Modifying the BASE24-telebanking Customer Service Representative Interface Makefile

The CSRLM on the OC11CSI subvolume requires a change to replace a missing line of text. The missing text must be added because the single TACL Server process used by the ACI Make utility starting with release 4.00 interprets the commands differently.

If you have the BASE24-telebanking Customer Service Representative Interface product, perform one of the following steps.

- Contact HELP24 and secure a copy of SCN OC03140. Apply the fix, which is a replacement for the CSRLM on the OC11CSI subvolume.
- Use a text editor and make the required changes to the CSRLM. Seven lines of text from the file before the change is made appear below. These lines are build statements for the target file identified with the `occsi_csrlm` macro.

```
= chk_peruse_job $(TARGET)                &
=                $(TRUE)                  &
=                $(PERUSE)                 &
=                #mkcsi.svcsr              &
=                $(bind_warning_text)      &
=
= del_peruse_job $(PERUSE) $(occsi_purge_spooler_on) #mkcsi.svcsr
```

Add the text `$(bind_alwd_warning_1)` to the sixth line of text, as shown below.

```
= chk_peruse_job $(TARGET)                &
=                $(TRUE)                  &
=                $(PERUSE)                 &
=                #mkcsi.svcsr              &
=                $(bind_warning_text)      &
=                $(bind_alwd_warning_1)
= del_peruse_job $(PERUSE) $(occsi_purge_spooler_on) #mkcsi.svcsr
```

ACI Worldwide Inc.

Glossary

The following terms are used to describe the ACI Make utility.

Additional define — A file that is required to create or update a target file. The ACI Make utility does not use the modification timestamp of this file when determining whether the target file needs to be created or updated. If this file is not present, the ACI Make utility does not process the target file.

Additional target — A file to be created or updated by the ACI Make utility. The ACI Make utility does not use the modification timestamp of this file when determining whether the target file needs to be created or updated. If the ACI Make utility processes the target file, it builds this file also.

ALLMACS define — A TACL define (=ALLMACS) that identifies which global level macro file the ACI Make utility should load first. This define must be established in the TACL environment before the ACI Make utility can be started.

ALLMACS File — A global level macro file that contains configuration macros and file definition macros such as the location of the TAL compiler, NET24-XPNET system, and ACI utilities.

ALLMACS macro — A macro that is created when the ALLMACS File is loaded. The first statement in each makefile performs a test for the presence of the ALLMACS macro to determine whether the Make macro file hierarchy has been built.

Conditional text — Statements that can be included or omitted from a macro file or a makefile based on the value of a flag. Conditional text can include additional defines, additional targets, macros, target files, dependency files, or inline code.

CSMMACS File — A global level macro file that contains configuration information for customized code, and usually serves as the top level global macro file when a BASE24 installation includes customized code.

The CSMMACS File contains only the macros that are required for the customized code. The last statement of the CSMMACS File incorporates the CUSTMACS File, which contains configuration information for the standard BASE24 code, by reference.

CUSTMACS File — A global level macro file that provides customers a single location to define which BASE24 products to include in their system, as well as define customer-specific changes to BASE24. The flags in the CUSTMACS File appear in other files throughout the Make hierarchy. However, by having all flags contained in the CUSTMACS File, customers can set all flags, as well as some of the configuration options, in a single location instead of modifying multiple files located throughout the Make hierarchy.

Dependency file — A file from which the target file is created or updated, including source code and DDLs. The ACI Make utility uses the modification timestamp of this file, which is assigned by the Tandem NonStop Kernel operating system, when determining whether the target file needs to be created or updated.

Flag — A value defined in one of the macro files to identify a product or a specific module maintained with the ACI Make utility. Each product (e.g., BASE24-atm, BASE24-pos, BASE24-teller, BASE24-telebanking) and each module (e.g., Refresh, Super Extract) has its own flag. See the FLGSDOC File on the BA60MAKE subvolume for a complete list of BASE24 flags.

Included file — An external source file that is incorporated by reference.

Inline code — A series of commands intended for the TACL Server process. These commands are imbedded within a makefile instead of being maintained in a separate file.

Macro file — A file used by the ACI Make utility to build the Make hierarchy. Macro files contain two types of information that must be established so that it can be used in the makefiles. At the global or system level, files such as the CUSTMACS File, ALLMACS File, and CSMMACS File contain configuration and file definition macros. Below the global or system level, files with names ending with *FM* contain configuration macros, and files with names ending with *MM* contain file definition macros.

Make file — The generic term for macro files and makefiles.

Makefile — A file that contains the commands, macros, dependency information, and inline code needed to create a target file. Makefile names include suffixes that identify the type of information in the makefile, as shown in the following table.

M	=	Makefile
DM	=	DDL makefile
LM	=	Link makefile
RM	=	Relational makefile
TM	=	TAL makefile
XM	=	SCOBOLX makefile

RPQMACS File — The obsolete name for a global level macro file that contains configuration information for customized code, and usually serves as the top level global macro file when a BASE24 installation includes customized code. This global level file is now named the CSMMACS File. See CSMMACS File for additional information.

Target file — A file to be created or updated by the ACI Make utility. If the file exists, the ACI Make utility uses the modification timestamp of this file, which is assigned by the Tandem NonStop Kernel operating system, when determining whether the file needs to be updated. If the file does not exist, the ACI Make utility determines that it should be built.

Target specification — The parameters that identify specific target and dependencies, which are used most often for deciding whether to execute build commands. Target specifications can also contain additional defines and additional targets. One makefile can contain multiple target specifications.

ACI Worldwide Inc.

Index

A

ABEND_ON_ERROR option, overview, [4-33](#)

ACI Make utility

- command line options, [4-33](#)
- demonstration of, [A-15](#)
- implementation in BASE24, [B-1](#)
- installing BASE24, [4-57](#)
- introduction, [1-2](#)
- location of, [2-2](#)
- printing options, [4-19](#)
- procedure for running the utility, [4-2](#)
- procedure for starting the utility, [4-6](#)
- relationship to specific products, [2-2](#)
- releases of, [1-4](#)
- summary of, [A-1](#)
- upgrading BASE24 sync point 2 products, [D-1](#)
- used with customized BASE24 software, [C-1](#)
- using obey files to start, [4-7](#)

Additional define

- definition of, [Glossary-1](#)
- overview, [3-21](#)
- software example, [3-21](#)

Additional input file

- see Additional define

Additional output file

- see Additional target

Additional target

- definition of, [Glossary-1](#)
- overview, [3-23](#)
- software example, [3-23](#)

ALLMACS define

- definition of, [Glossary-1](#)
- overview, [2-7](#)
- procedure for setting, [4-2](#)
- used with customized BASE24 software, [C-7](#)

ALLMACS File

- definition of, [Glossary-1](#)
- overview, [2-18](#)
- software example, [2-18](#)
- summary of, [A-4](#)

ALLMACS macro

- definition of, [Glossary-1](#)
- overview, [2-53](#)
- testing for, [2-53](#)

ATMM File

- software example, [2-28](#)
- summary of, [A-6](#)

ATMMM File, software example, [2-31](#)

AUTHFM File

- software example, [2-33](#)
- summary of, [A-6](#)

AUTHM Makefile

- demonstration of, [A-15](#)
- software example, [A-7](#)
- summary of, [A-7](#)

AUTHMM File

- software example, [2-34](#)
- summary of, [A-7](#)

B

B24C File, [4-7](#)

B24M Makefile, software example, [2-54](#), [3-3](#)

BASE24 ITS products, [1-4](#)

BASE24 procedural products, [1-4](#)

BASE24 sync point 2 products, upgrading, [D-1](#)

BASEFM File, options contained in CUSTMACS File, [2-9](#)

C

Charts of Make hierarchy, [2-59](#), [C-8](#)

CLSCFM File

- software example, [2-25](#)
- summary of, [A-5](#)

CLSCM Makefile, definition of, [2-56](#)

CLSCMM File, software example, [2-26](#)

Command line options, ACI Make utility, [4-33](#)

Comments, overview of, [3-13](#)

Compiling a system subset, procedure for, [4-63](#)

Compiling DDLs, procedure for, [4-65](#)

Compiling Pathway requester processes, procedure for, [4-66](#)

Conditional text

- definition of, [Glossary-1](#)
- overview, [3-12](#)
- software example, [3-12](#)

Configuration macros

- contained in ALLMACS File, [2-18](#)
- definition of, [2-3](#)

CSM

- see Customized BASE24 software

CSM Makefile, definition of, [C-5](#)
CSMC File, [C-7](#)
CSMM Makefile
 contained in the B24M Makefile, [2-56](#)
 definition of, [2-55](#)
CSMMACS File
 overview, [2-24](#), [C-6](#)
CUSTMACS File
 definition of, [Glossary-2](#)
 options from BASEFM File, [2-9](#)
 overview, [2-7](#)
 software example, [2-7](#)
 summary of, [A-3](#)
Customized BASE24 software
 chart of Make hierarchy, [C-8](#)
 code locations, [C-2](#)
 makefile modifications, [C-5](#)
 naming conventions, [C-2](#)

D

D option, overview, [4-33](#)
Definitions, [Glossary-1](#)
Dependency file
 as a target file, [3-7](#)
 definition of, [Glossary-2](#)
Dependency lists, procedure for obtaining, [4-59](#)
Dependent file
 see Dependency file
DETAIL option, overview, [4-23](#)
DUMPMACS option, overview, [4-34](#)

E

EQUIV File
 ACI Make software example, [4-55](#)
EQUIV option, overview, [4-37](#)
EQUIVALL option, overview, [4-38](#)
ERROR option, overview, [4-35](#)
EXCEPT_FILE option
 overview, [4-39](#)
 summary of, [A-19](#)
EXCEPT_LOC option
 overview, [4-39](#)
 summary of, [A-19](#)
EXT_BUILD option, overview, [4-40](#)
EXT_USE option, overview, [4-40](#)

F

File definition macros, definition of, [2-3](#)
File name reference maintenance, [4-52](#)
Flag Documentation File
 see FLGSDOC File

Flags
 contained in CUSTMACS File, [2-7](#)
 definition of, [Glossary-2](#)
FLGSDOC File
 definition of, [2-15](#)
 software example, [2-15](#)
FORCE option, overview, [4-41](#)

G

Global macro file
 ALLMACS File, [2-18](#)
 CSMMACS File, [2-24](#)
 CUSTMACS File, [2-7](#)
 definition of, [2-7](#)
 personal override file, [2-22](#)

H

HELP option, overview, [4-41](#)
Hierarchy
 see Make hierarchy
HIGHPIN_OFF option, overview, [4-43](#)
HISOM Makefile, software example, [3-26](#)

I

Included file
 definition of, [Glossary-2](#)
 INCLUDE statement, syntax of, [3-11](#)
Inline code
 definition of, [Glossary-2](#)
 overview, [3-25](#)
 software example, [3-25](#)
ITSFM File
 software example, [2-35](#)
ITSM Makefile, definition of, [2-56](#)
ITSMM File, software example, [2-40](#)

K

KISMTFM File, software example, [2-48](#)
KISMTMM File, software example, [2-49](#)
KNLFM File
 software example, [2-45](#)
KNLMM File
 software example, [2-47](#)

L

LINE_SIZE_MAX option, overview, [4-43](#)
LIST option, overview, [4-24](#)
LISTBS option, overview, [4-25](#)
LISTDEP option
 overview, [4-26](#)
 summary of, [A-18](#)

LISTINLINE option, overview, [4-27](#)
LISTREPLACE option, overview, [4-30](#)
LOADMACS File, software example, [C-7](#)

M

Macro file
 configuration macros, definition of, [2-3](#)
 definition of, [Glossary-2](#)
 file definition macros, definition of, [2-3](#)
 global macro file, [2-7](#)
 location of, [2-2](#)
 object, [2-47](#)
 object group, [2-35](#)
 ordering of, [2-3](#)
 process, [2-32](#)
 product, [2-27](#)
 product group, [2-25](#)
 subsystem, [2-44](#)

Macros, Make
 see Make macros

Macros, TACL
 see TACL macros

Make file
 definition of, [Glossary-2](#)
 see also Makefile

Make hierarchy
 BASE24 ITS levels, [2-3](#)
 BASE24 procedural levels, [2-3](#)
 charts of, [2-59](#), [C-8](#)
 introduction, [1-2](#)

Make macros
 definition syntax, [3-17](#)
 overview, [3-17](#)
 predefined macros, [3-18](#)
 usage syntax, [3-18](#)

Make utility
 see ACI Make utility

Makefile
 BASE24 ITS levels, [2-50](#)
 BASE24 procedural levels, [2-50](#)
 components of, [3-2](#)
 definition of, [Glossary-3](#)
 dependency file as a target file, [3-7](#)
 dependency lists, [3-10](#)
 location of, [2-2](#)
 object makefiles, overview, [2-58](#)
 overview, [2-50](#)
 process makefiles, overview, [2-58](#)
 product makefiles, list of, [2-57](#)
 subsystem makefiles, list of, [2-57](#)
 target specification syntax, [3-5](#)
 timestamps, use of, [3-6](#)
 see also Make file

MAKE_FILE_DEP_CHK option, overview, [4-44](#)

Makescan utility
 replacement for, [4-52](#)

Master makefile
 B24M Makefile, [2-54](#)
 CSMM Makefile, [2-55](#), [C-5](#)
 overview, [2-54](#)

MAX_RADIX_TBL_SIZE option
 overview, [4-44](#)
 use of, [4-3](#)

MAX-RADIX-TBL-SIZE parameter, use of, [4-3](#)

N

Naming conventions
 configuration macro files, [2-4](#)
 file defines, [2-5](#)
 file definition macro files, [2-4](#)
 makefiles, [2-50](#)

NOBUILD option, overview, [4-45](#)

NO_DELETE_DEFINES option, overview, [4-46](#)

NO_EXT_CHECK option, overview, [4-46](#)

NOLIST option, overview, [4-31](#)

NO_STOP_ON_ERRORS option, overview, [4-46](#)

O

Obey files
 B24C File, [4-7](#)
 RUNMAKE File, [4-14](#), [A-30](#)
 RUNMKNB File, [4-13](#)
 using to start the ACI Make utility, [4-7](#)

Obtaining dependency lists, procedure for, [4-59](#)

Override file
 see Personal override file

P

Pathway requester processes, procedure for
 compiling, [4-66](#)

Personal override file
 overview, [2-22](#)
 software example, [A-16](#)
 summary of, [A-16](#)

Precedence of macros, [2-3](#)

Predefined macros, [3-18](#)

Printing options, ACI Make utility, [4-19](#)

Procedures
 compiling a system subset, [4-63](#)
 compiling DDLs, [4-65](#)
 compiling Pathway requester processes, [4-66](#)
 identifying target files, [A-18](#)
 obtaining dependency lists, [4-59](#)
 running the ACI Make utility, [4-2](#)
 setting the ALLMACS define, [4-2](#)
 setting the Radix table size, [4-3](#)
 starting the ACI Make utility, [4-6](#)
 using multiple makefiles, [4-71](#)

Pseudo target file, [3-6](#)

PURGE_TARGETS option, overview, [4-47](#)

R

Radix table size, procedure for setting, [4-3](#)

Releases of ACI Make utility, [1-4](#)

Reports

ACI Make utility, [4-19](#)

RPQ

see Customized BASE24 software

RPQMACS File, definition of, [Glossary-3](#)

RQATTXM Makefile, software example, [4-69](#)

RQECFM Makefile, software example, [4-67](#)

RUNMAKE File

overview, [4-14](#)

software example, [A-30](#)

summary of, [A-30](#)

RUNMKNB File

overview, [4-13](#)

software example, [A-19](#)

summary of, [A-19](#)

S

Screen COBOL object library (POBJ), [3-6](#), [3-23](#), [4-66](#)

Software examples

additional define, [3-21](#)

additional target, [3-23](#)

ALLMACS File, [2-18](#)

ATMFM File, [2-28](#)

ATMMM File, [2-31](#)

AUTHFM File, [2-33](#)

AUTHM Makefile, [A-7](#)

AUTHMM File, [2-34](#)

B24C File, [4-7](#)

B24M Makefile, [2-54](#), [3-3](#)

CLSCFM File, [2-25](#)

CLSCMM File, [2-26](#)

conditional text, [3-12](#)

CUSTMACS File, [2-7](#)

EQUIV File, [4-55](#)

FLGSDOC File, [2-15](#)

HISOM Makefile, [3-26](#)

inline code, [3-25](#)

ITSFM File, [2-35](#)

ITSMM File, [2-40](#)

KISMTFM File, [2-48](#)

KISMTMM File, [2-49](#)

KNLFM File, [2-45](#)

KNLMM File, [2-47](#)

LOADMACS File, [C-7](#)

personal override file, [A-16](#)

RQATTXM Makefile, [4-69](#)

RQECFM Makefile, [4-67](#)

RUNMAKE File, [4-14](#), [A-30](#)

RUNMKNB File, [4-13](#), [A-19](#)

SUPPRESS_COMMENTS option, overview, [4-31](#)

SUPPRESS_UNTIL_SERVER option, overview, [4-31](#)

SYNTAX option, overview, [4-48](#)

T

TACL defines, [3-20](#)

TACL macros, [3-29](#)

TACL server process, [3-25](#)

TACLMACS File, [3-29](#)

Target file

definition of, [Glossary-3](#)

identification of, [A-18](#)

Target specification

definition of, [Glossary-3](#)

order of appearance in makefiles, [3-10](#)

syntax of, [3-5](#)

Terminology, [Glossary-1](#)

TEST option, overview, [4-48](#)

TESTCHK option, overview, [4-49](#)

TIMESTAMP_FILE option, overview, [4-49](#)

Timestamps, use of, [3-6](#)

U

UPDATE option, overview, [4-50](#)

Using multiple makefiles, procedure for, [4-71](#)

V

VERSION option, overview, [4-51](#)